

CS 598 Practical Statistical Learning

Alexandra Chronopoulou

2026-09-11

Contents

Course Information	7
1 Introduction to Statistical Learning	9
1.1 Examples of Statistical Learning Problems	10
1.2 Supervised Learning Framework	12
1.3 Why Statistical Learning is Challenging	13
1.4 Bias-Variance Trade-Off	14
1.5 Two Toy Examples: k -NN vs. Linear Regression	16
2 Linear Regression Models	23
2.1 Multiple Linear Regression (MLR) Model	24
2.2 MLR Model Fitting	25
2.3 Least-Squares & Normal Equations	29
2.4 Goodness-Of-Fit: R -Square	31
2.5 Linear Transformations on X	32
2.6 Rank deficiency	33
2.7 Hypothesis Testing in MLR	33
2.8 Categorical Variables in MLR	35
2.9 Collinearity	36
2.10 Model Diagnostics	37
2.11 The Birthweight Example	38
3 Variable Selection & Regularization	39
3.1 Training vs. Testing Errors	39
3.2 Subset Selection	42
3.3 Shrinkage Methods	45
3.4 The Student Performance Example	54
4 NonLinear Regression	57
4.1 Polynomial Regression	58
4.2 Splines Regression	76
4.3 Smoothing Splines	87
4.4 Fitting Smoothing Splines	89

4.5	The Birthrates Example	91
5	Introduction to Neural Networks for Regression	93
5.1	Learning by Fitting a Model	93
5.2	A Neuron: Two ways to think about it	95
5.3	Illustration: Predicting House Prices	97
5.4	Same Loss, New f_β	98
5.5	Stacking Units is Learning ϕ	99
5.6	Activations, Depth, and Approximation	101
5.7	Backpropagation	102
5.8	Capacity, Nonconvexity, and Penalty	105
5.9	When to specify ϕ instead of learning it!	106
6	Regression Trees	107
6.1	Single-Tree Building	108
6.2	Regression Forests	113
6.3	Tree-Based Regression Models in R	116
6.4	Conclusion	134
7	Introduction to Classification	137
7.1	Discriminant Analysis	139
7.2	Fisher Discriminant Analysis (FDA)	143
7.3	Naive Bayes Classifiers	144
7.4	The Digits Recognition Example	145
8	Logistic Regression	153
8.1	Fitting a Logistic Regression Model	153
8.2	Logistic Regression with Separable Data	157
8.3	Retrospective Sampling	158
8.4	Logistic Regression for the Heart Data Set in R	159
9	Support Vector Machines	169
9.1	Linear SVM in the Separable Case: <i>Separation Margin</i>	169
9.2	Linear SVM in the Non-Separable Case: <i>Slack Variables</i>	177
9.3	Nonlinear SVM: <i>The Kernel Trick</i>	181
9.4	SVM as a Penalization Method	184
9.5	SVM Examples in R	187
10	Classification Trees & Ensemble Methods	197
10.1	Splitting Rules in Classification Trees	199
10.2	Random Forests	206
10.3	Boosting	210
10.4	A Controlled Comparison	224
11	Recommender Systems	229
11.1	Content-Based Methods	230
11.2	Collaborative Filtering	231

11.3 Baseline Predictors	235
11.4 Latent Factor Model	236
11.5 Evaluating Recommender Systems	237
11.6 Challenges & Strategies	238
12 Clustering Analysis	241
12.1 Introduction to Clustering	242
12.2 K -means Clustering	243
12.3 Hierarchical Clustering	250
12.4 Self-Organizing Maps	253
12.5 Spectral Clustering	254
13 Model-Based Clustering: Latent Structure Models	263
13.1 Gaussian Mixtures	263
13.2 EM Algorithm	264
13.3 EM Algorithm: General Framework	267
13.4 The MM Algorithm	268
13.5 Hidden Markov Models	270
13.6 Inference on the Hidden States Z	275

Course Information

Course Description

This course provides an introduction to modern techniques for statistical analysis of complex and massive data. Examples of these include model selection for regression, classification, nonparametric models such as splines and kernel models, regularization, dimension reduction, and clustering analysis. Applications are discussed as well as computation and theoretical foundations.

Course Prerequisites

Knowledge of basic multivariate calculus, statistical inference, and linear algebra. You should be comfortable with the following concepts: probability distribution functions, expectations, conditional distributions, likelihood functions, random samples, estimators, and linear regression models.

Course Learning Outcomes

By the end of the course, you will be able to:

- Use a broad range of methods and techniques in machine learning.
- Have a deeper understanding of major algorithms and techniques in machine learning.
- Build analytics pipelines for regression problems.
- Build analytics pipelines for classification problems.
- Build analytics pipelines for recommendation problems.

Textbook and Readings

There is no required textbook for this course.

Recommended resources for a statistical approach to machine learning are:

- *An Introduction to Statistical Learning with Applications in R* (basic)
- *An Introduction to Statistical Learning with Applications in Python* (basic)

- *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (more advanced)

You may also want to view the Data School YouTube videos associated with the first book as an additional resource.

The detailed syllabus for the course can be found on Coursera.

Chapter 1

Introduction to Statistical Learning

Statistical Learning is a field at the intersection of statistics, computer science, and data science that focuses on developing and understanding models that can *learn* from data. While statistics has long dealt with model building, validation, and prediction, the dramatic growth in the volume, complexity, and dimensionality of data has strained many classical techniques. The introduction of sophisticated algorithms, data-management tools, and optimization methods from computer science has enabled more powerful approaches suited to modern data regimes.

In the supervised setting, learning from data typically means that, given an outcome (quantitative or categorical) and a (usually large) set of features, we construct a **prediction model** (also called a *learner*) that can accurately forecast the outcome for new, previously unseen observations.

We can separate statistical learning problems in two types:

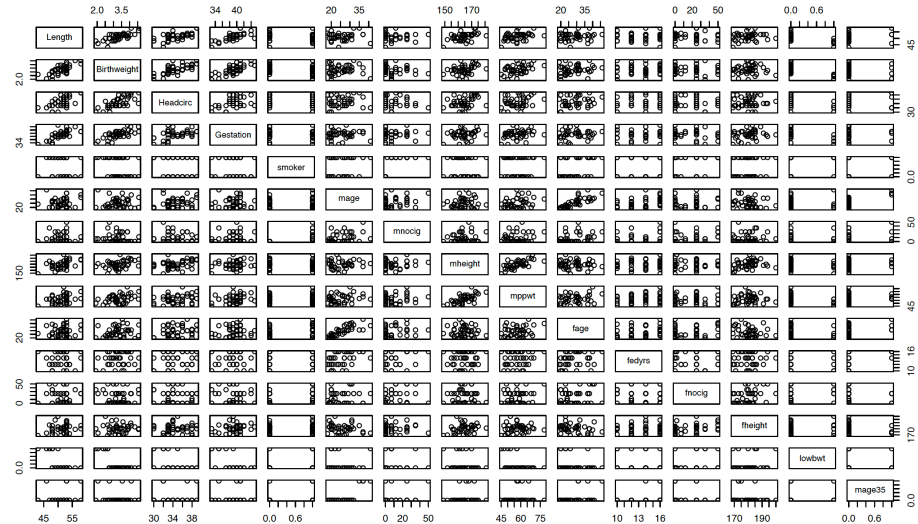
- **Supervised Learning:** when an outcome variable is present to guide the learning process. The two primary examples are:
 - *Regression:* Response is quantitative (continuous or discrete numeric).
 - *Classification:* Response is categorical (binary or multi-class).
- **Unsupervised Learning:** when an outcome variable is not present to guide the learning process. The goal is to discover latent structure or patterns in the data (e.g., clustering, association rules, hidden Markov models, etc.).

1.1 Examples of Statistical Learning Problems

Birthweight Data: A Regression Example

This dataset comes from a study whose goal is to predict the birth weight of a newborn from a collection of baby and parental characteristics (gestation length, length, head circumference, mother's and father's height, mother's weight, parental smoking status, etc.). The study focuses particularly on babies born prematurely (before 40 weeks of gestation).

The scatterplot matrix below shows the pairwise relationships among the variables in the dataset:



Note that three of the predictors (`smoker`, `lowbwt`, and `mage35`) are categorical.

This is a classic example of a supervised regression problem and a natural setting in which a multiple linear regression (MLR) model can be considered. If the MLR model fits well and the key assumptions are reasonably satisfied, it can provide useful predictions of birth weight for new observations.

Trees and Shrubs Data: A Binary Classification Example

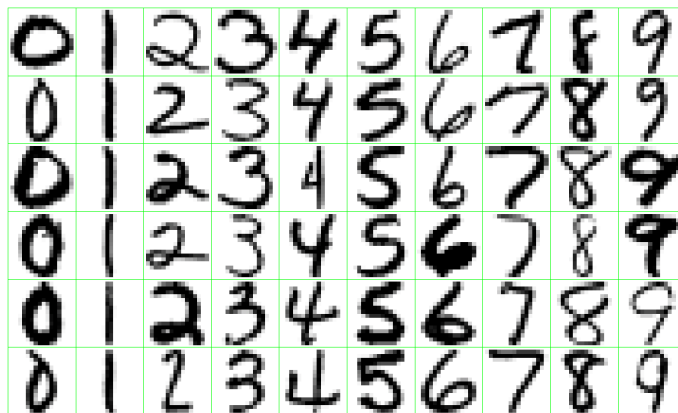
In this problem, the goal is to predict whether a woody plant common in the Black Forest region of southwestern Germany is a tree or a shrub (Lederer). A sample of the data is shown below:

Name	Height	Diameter	GrowthRate	Longevity	Type
Silver Fir	55.0	1.50	0.5	500	Tree
European Beech	40.0	1.00	0.5	200	Tree
Common Hazel	5.0	0.10	0.5	70	Shrub
Red Raspberry	1.5	0.01	0.4	10	Shrub
European Spruce	50.0	1.20	0.5	600	Tree

This is a binary classification problem in which we want to use a set of characteristics to predict the class label (tree or shrub).

Handwritten Digits: A Multiclass Classification Example

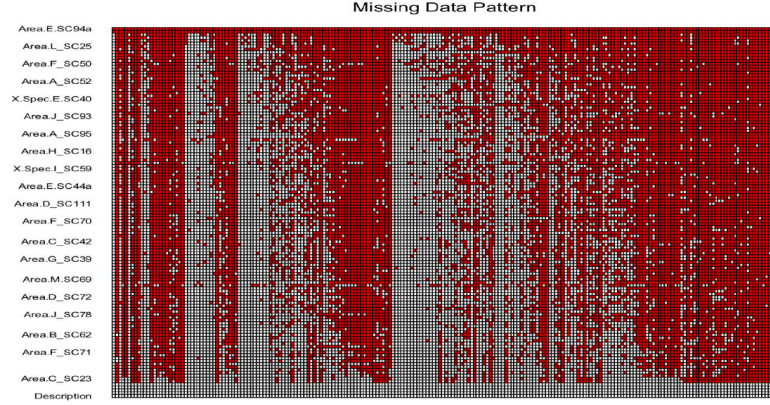
This is a classic multiclass classification problem in which the goal is to predict the handwritten digit (0, ..., 9) appearing on a postal envelope, based on a 16×16 eight-bit grayscale image (ESL). The practical challenge is to keep the error rate sufficiently low so that mail is not misdirected.



Proteomics Data: A Clustering Example

Proteomics is the large-scale study of proteins. Data of this type are typically generated through laboratory processes such as protein purification or mass spectrometry (a technique that measures the mass-to-charge ratio of ions).

The figure below shows an expression matrix for 437 proteins, of which only 101 are of primary interest (Romanova et al.). A common challenge with such data is the large amount of missing values (shown as grey pixels). The goal is to cluster the proteins into coherent groups for further biological analysis.



These examples illustrate the variety of problems that arise in statistical learning. The birthweight data is a typical supervised regression problem in which we aim to predict a quantitative outcome. The trees-and-shrubs data is a binary classification task, while the handwritten-digit data is a multiclass classification problem. Finally, the proteomics data is an unsupervised learning example whose goal is to uncover latent structure through clustering. Together they show how statistical learning methods can be applied across different types of outcomes and data structures, whether the aim is *prediction* or the *discovery of patterns*.

1.2 Supervised Learning Framework

In this class, our main focus is **supervised learning**. This means that we observe **target** (*outcome, response*) variable Y that we wish to predict using a set of *features* (*predictors*), X . We assume that the relationship between the response Y and the features is given by a function f that is known up to a parameter vector w :

$$\mathbf{Y} = f(\mathbf{X}, w) + \text{varepsilon}$$

When f is fully known except for the value of w (for example when f is linear), then learning problem reduces to **estimating** w . To do so, we collect a **training sample** $\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^n$ and minimize an **objective** function with respect to w :

$$F(w) = \sum_{i=1}^n \text{loss}(y_i, f(\mathbf{x}_i, w))$$

The **loss** function measures the discrepancy between the model prediction $f(\mathbf{x}_i, w)$ and the observed outcome $\mathbf{y}_i$. The choice of loss is made by the practitioner; the most common examples are:

- **Squared Error Loss**

$$\text{loss} = \left[y_i - f(x_i; w) \right]^2$$

- **Absolute Error Loss**

$$\text{loss} = \left| y_i - f(x_i; w) \right|$$

- **Log Loss** (binary classification)

$$\text{loss} = -y_i \log f(x_i; w) - (1 - y_i) \log(1 - f(x_i; w))$$

- **0-1 Loss**

$$\text{loss} = \mathbf{1}_{\{y_i \neq \hat{y}_i\}} \quad \text{where} \quad \hat{y}_i = f(x_i; w)$$

Depending on the chosen loss, the minimizer of $F(w)$ may or may not admit a closed-form expression. When a closed form is unavailable we resort to numerical optimization algorithms. In many cases gradient-based methods (in particular gradient descent and its variants) can be used to obtain a good solution.

1.3 Why Statistical Learning is Challenging

In the framework of the previous section we reduced the learning problem to the estimation of a parameter vector w , under the assumption that the true function f is known up to this parameter. In practice, however, we rarely know the exact form of f . We must therefore *approximate* it by some function $\hat{f}$. This approximation introduces an *additional source of error*.

Moreover, the objective function we minimize is computed on a *training* sample. While this quantifies how well the model fits the observed data, the real goal of statistical learning is good performance on **new, unseen** data. In other words, we aim to minimize the **test error** (also called generalization error), not the training error.

Statistical learning is therefore challenging for several reasons:

- The training error typically underestimates the test/generalization error.
- A model may perform well on the training data yet poorly on future data because of overfitting. The number of parameters required to approximate f can be large—sometimes larger than the number of available observations.
- As the number of model parameters grows, the test error often increases substantially.

Two simple illustrations make these points concrete:

1. **Classification: 1-Nearest Neighbour**

The one-nearest-neighbour classifier achieves zero training error (it predicts every training point perfectly) but typically performs poorly on test data. An instructive demonstration of how increasing dimensionality affects the performance of linear classifiers can be found here.

2. **Regression: the p versus n problem**

Consider the linear model

$$\mathbf{y}_{n \times 1} = \mathbf{X}_{n \times p} \mathbf{w}_{p \times 1} + \varepsilon,$$

where $\mathbf{y}$ is the response vector, $\mathbf{X}$ is the design matrix with p predictors, and $\mathbf{w}$ contains the coefficients to be estimated.

- When $p < n$ we have more equations than parameters; classical regression methods usually work well.
- When $p = n$ we have exactly as many equations as parameters and can achieve a perfect fit on the training data.
- When $p > n$ we have fewer equations than parameters. In this high-dimensional regime classical methods break down and specialized techniques are required.

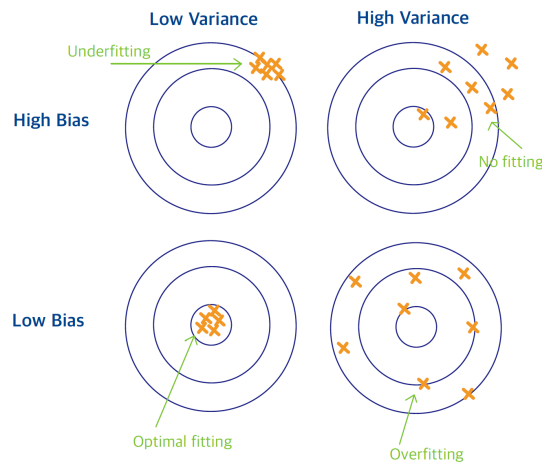
In summary, statistical learning is challenging because we must approximate an unknown function f , control the gap between training and test error, and often operate in regimes where the number of parameters is large relative to the sample size. Successfully addressing these issues is essential for building models that generalize well to new data.

1.4 Bias-Variance Trade-Off

We have already noted that *two sources of error* affect the quality of our predictions. In statistical learning we must balance:

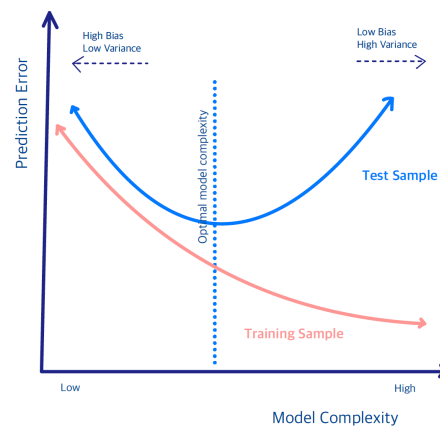
- **Bias:** the systematic difference between the average of our estimated function $\hat{f}$ and the true underlying function f . High bias typically leads to under-fitting and an inaccurate model.
- **Variance:** the amount by which $\hat{f}$ would change if we estimated it using a different training sample. High variance typically leads to over-fitting and an unreliable model.

A helpful analogy is to think of the learning problem as a **target** whose *center represents the true model*.



How close the fitted values lie to the center is measured by bias. If the points systematically miss the center, the model is a poor approximation of f . Increasing the sample size will not correct a large bias. The spread of the fitted values around their average is measured by variance. When the points are tightly clustered the variance is low; when they are widely scattered the variance is high.

Ideally we would like to reduce both bias and variance simultaneously. In practice this is not possible, so we must trade one against the other. Remembering that our ultimate goal is to minimize the generalization (test) error, not the training error, we observe the following typical behavior:



- When the model is highly complex (many parameters), the training error tends to decrease. However, the model often over-fits the training data and therefore exhibits high variance on new data, resulting in poor generalization.

- When the model is very simple (few parameters), it tends to under-fit the data. This produces large bias and, again, poor generalization performance.

In this class we will discuss:

- (i) flexible modeling techniques that help reduce bias, and
- (ii) practical strategies for achieving a good bias-variance trade-off.

Two important approaches that we will study in detail are:

- **Regularization:** constrain the parameters to a lower-dimensional space that is determined adaptively by the data.
- **Ensemble:** average many low-bias, high-variance models; the averaging step reduces variance while largely preserving the low bias.

1.5 Two Toy Examples: k -NN vs. Linear Regression

Before we conclude this introductory chapter, we examine two simple supervised learning methods:

- (i) k -Nearest Neighbors (k -NN)
- (ii) Linear regression

We will compare their behavior on simple examples and use them to illustrate the bias-variance trade-off.

1.5.1 k -Nearest Neighbors

In the k -nearest neighbors (k -NN) method we predict the response at a point $\mathbf{x}$ by using the k training observations that are closest to $\mathbf{x}$. The k -NN fit is

$$\hat{y}(\mathbf{x}) = \frac{1}{k} \sum_{x_i \in N_k(\mathbf{x})} y_i$$

where $N_k(\mathbf{x})$ denotes the neighborhood of $\mathbf{x}$ consisting of the k nearest training points.

- In **regression**, $\hat{y}(\mathbf{x})$ is a local average of the neighboring responses.

- In **classification**, $\hat{y}(\mathbf{x})$ is the majority class (or the class probability) within the neighborhood.

Two practical challenges that arise are choosing the neighborhood size k and choosing the distance metric that defines the neighborhood.

The value of k directly controls model complexity, which is roughly proportional to n/k .

- When $k = 1$ the prediction at each training point x_i is exactly y_i , yielding zero training error.
- When $k = n$ every neighborhood contains the entire training sample, so the prediction is constant for all $\mathbf{x}$.

The “default” metric is Euclidean distance

$$d(\mathbf{x}, \tilde{\mathbf{x}}) = \sum_{j=1}^p w_j (x_j - \tilde{x}_j)^2,$$

where the weights w_j may be learned from the data.

1.5.2 Linear Regression

Given an input vector $\mathbf{x}^\top = (x_1, \dots, x_p)$, we approximate the response by a linear function

$$f(\mathbf{x}) \approx \beta_0 + \sum_{j=1}^p x_j \beta_j$$

The coefficients β_j are estimated by ordinary least squares (OLS), i.e., by minimizing the residual sum of squares (*objective function*)

$$\min_{\beta_0, \dots, \beta_p} \sum_{i=1}^n \left(y_i - \beta_0 - x_{i1}\beta_1 - \dots - x_{ip}\beta_p \right)^2$$

Under standard conditions the solution has a closed form, and the fitted value at the i -th observation is

$$\hat{y}_i = \hat{y}(x_i) = x_i^T \hat{\beta}$$

We postpone the algebraic details until Week 2.

Linear Regression in a Classification Setting

Linear regression can also be applied to binary classification problems in which $Y \in \{0, 1\}$. One simply predicts `class 1` when the fitted value $\hat{f}(\mathbf{x})$ exceeds 0.5 and `class 0` otherwise.

This approach has well-known drawbacks: the squared-error loss is not a natural measure for binary outcomes, and a linear function can produce fitted values outside the interval $[0, 1]$. For these reasons logistic regression is the preferred linear method for classification. It models the log-odds

$$\log \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} \approx \beta_0 + \sum_{j=1}^p x_j \beta_j$$

We will return to logistic regression in Week 8.

1.5.3 A Simulated (Binary) Classification Example

Consider a response variable G that takes two values (0 = BLUE or 1 = ORANGE). We generate 200 observations (100 in each class) and compare linear regression and k -nearest neighbors **as classifiers**.

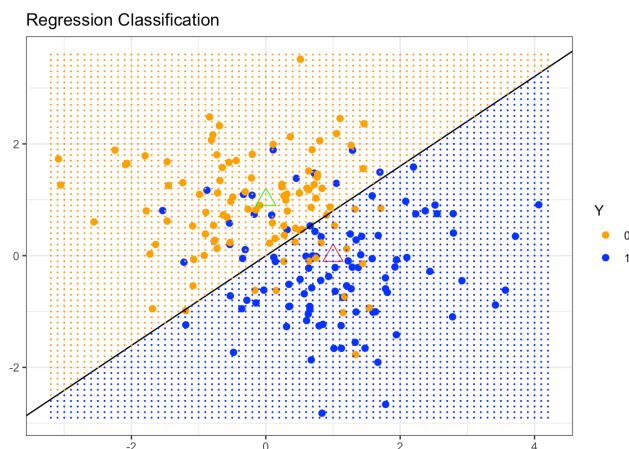
The code used to produce the simulation and all figures is available in R and Python.

Linear Regression Classifier

We first fit a linear regression model, treating the binary response as if it were continuous. The continuous fitted values $\hat{Y}$ are then converted to class predictions $\hat{G}$ by the simple rule

$$\hat{G} = \begin{cases} \text{Blue,} & \text{if } \hat{Y} > 0.5 \\ \text{Orange,} & \text{otherwise} \end{cases}$$

Because the problem is two-dimensional, the decision boundary is a *straight line*.



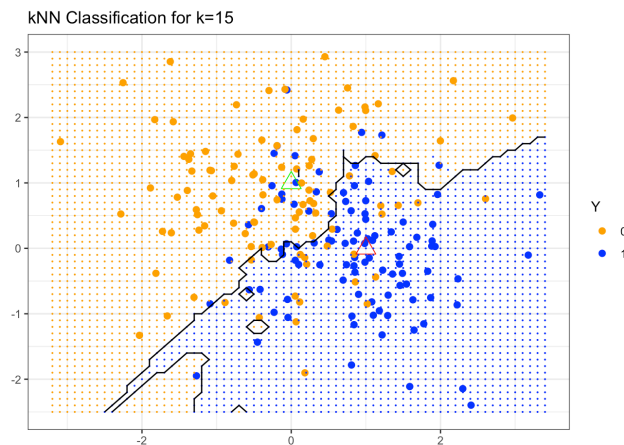
The fitted boundary (black line in the figure below) is the set of points satisfying $\mathbf{x}^\top \hat{\beta} = 0.5$. In the simulation the true blue region lies above this line and the orange region below it. Many points on both sides of the boundary are nevertheless misclassified. The linear decision boundary is smooth but rigid; it cannot adapt to local structure in the data.

k -Nearest-Neighbor Classifier

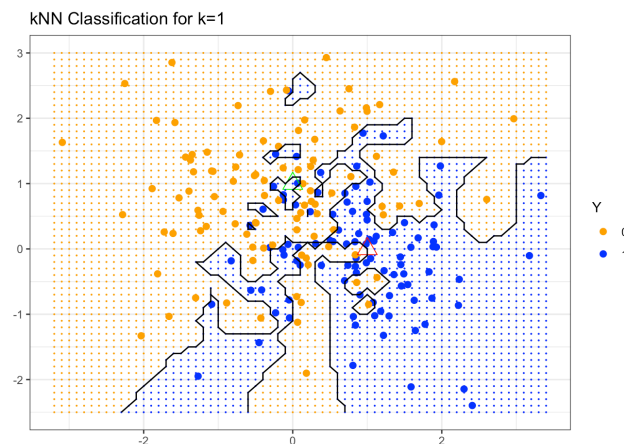
We now apply k -nearest neighbors to the same data. Using $k = 15$, $\hat{Y}(\mathbf{x})$ is the proportion of blue observations among the 15 nearest neighbors of $\mathbf{x}$. Class labels are again assigned by the 0.5 threshold:

$$\hat{G} = \begin{cases} \text{Blue, if } \hat{Y} > 0.5 \\ \text{Orange, otherwise} \end{cases}$$

(The prediction is therefore a majority vote among the 15 neighbors.)



The resulting decision boundary is far more irregular and follows local clusters of blue and orange points. Consequently the number of misclassifications on the training data is smaller than with linear regression. If we take the extreme case $k = 1$, each point is classified by its single nearest neighbor:

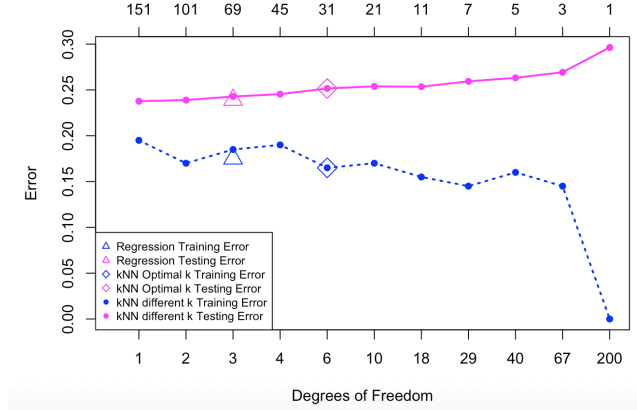


The boundary becomes extremely rough and the training misclassification rate drops nearly to zero. Whether this is desirable, however, can be judged only by examining the *generalization error*.

Training versus Test Error

Up to this point we have evaluated both methods on the same data that were used to fit them. A method such as 1-NN therefore appears to have zero error. Fair comparison requires an independent test set. The figure below shows misclassification error on a test set of 10,000 observations as a function of model degrees of freedom. Several k -NN fits (varying k) are compared with the linear

regression fit.



The magenta curve is the test error and the blue curve is the training error of k -NN. A 5-fold cross-validation procedure selected the optimal k (marked by a diamond). Linear regression results appear as the magenta and blue triangles at 3 degrees of freedom (the dimension of the fitted linear model). Linear regression has only a few parameters and therefore relatively low variance, but the linearity assumption is restrictive for this problem and produces high bias.

In contrast, k -NN makes almost no assumption about the shape of the decision boundary (apart from a mild local-smoothness condition). This flexibility yields low bias but high variance; the extreme case $k = 1$ corresponds to roughly n parameters and severely overfits. It can be shown that k -NN is consistent when $k, n \rightarrow \infty$ with $k/n \rightarrow 0$. The price of that consistency is the high variance that appears when the effective number of parameters becomes large.

Chapter 2

Linear Regression Models

A regression model is a model for the **conditional expectation** of the response/outcome variable given a set of predictors/features. A linear regression model specifically assumes that $E(Y | \mathbf{X} = \mathbf{x})$ is a linear function of the inputs $X_1, X_2, \dots, X_p$, i.e.,

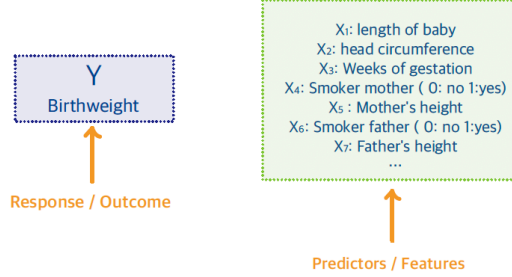
$$E(Y | \mathbf{X} = \mathbf{x}) \sim \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$$

To describe the properties and characteristics of the multiple linear regression (MLR) model, we introduce the **Birthweight** example.

Introduction to the Birthweight Example

The goal of the birthweight study is to **predict** the birthweight (*response/outcome*) of a newborn baby given a set of *predictors/features*, specifically for babies born prematurely.

The features include baby-related characteristics, such as length, weight, head circumference, gestation period, and parent characteristics, such as mother's/father's height, smoking habits, mother's pre-pregnancy weight, mother's/father's age, etc.



The **predictors** we consider may be:

- quantitative, such as length, weight, or gestation period;
- qualitative/categorical, such as smoking habits (yes/no); or
- transformations of quantitative inputs. Examples include power transformations (e.g., X_3^3), continuous functions of variables (e.g., $\log X_5$), polynomial expansions (e.g., $\sum_{k=1}^n c_k X_7^k$), and interactions (e.g., $X_1 \cdot X_2$).

2.1 Multiple Linear Regression (MLR) Model

The model formulation is given by:

Multiple Linear Regression Model

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \varepsilon$$

where

- β_0 is the intercept
- β_j is the regression coefficient associated with predictor X_j
- ε is the error term. The usual assumptions for the error terms are $\varepsilon \sim IID(0, \sigma^2 \mathbf{I})$

Given the **training data** $\{x_{i1}, x_{i2}, \dots, x_{ip}; y_i\}_{i=1}^n$, we can rewrite the model as

$$y_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} + \varepsilon_i, \quad i = 1, \dots, n$$

where n is the sample size. Using the following matrix representation for the

response and the features:

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix} \quad \mathbf{X} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1p} \\ 1 & x_{21} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{pmatrix}$$

the model equation can be written in as

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1p} \\ 1 & x_{21} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix}$$

which leads to the concise form

$$\begin{array}{ccccccc} \mathbf{y}_{n \times 1} & = & \mathbf{X}_{n \times (p+1)} & \begin{matrix} (p+1) \times 1 \\ \uparrow \\ \text{Coefficients} \end{matrix} & + & \begin{matrix} n \times 1 \\ \uparrow \\ \text{Error Term} \end{matrix} \\ \uparrow & & \uparrow & & & \uparrow \\ \text{Response} & & \text{Design Matrix} & & & \end{array}$$

where n is the sample size, and $p+1$ is number of columns of $\mathbf{X}$ (the p predictors plus the intercept).

2.2 MLR Model Fitting

Given the training data $\{x_{i1}, x_{i2}, \dots, x_{ip}; y_i\}_{i=1}^n$, we want to estimate β , i.e. express:

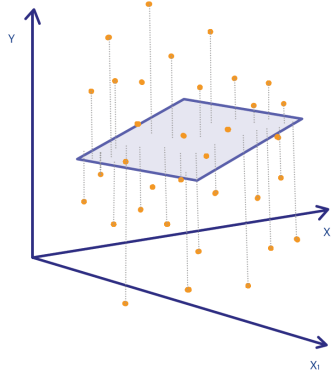
$$\hat{\beta} = (\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p)^T$$

as a **function of the data**.

The estimated regression coefficients $\hat{\beta}$ are obtained by minimizing the *Residual Sum of Squares* (RSS):

$$RSS = \|\mathbf{y} - \mathbf{X}\beta\|^2 = (\mathbf{y} - \mathbf{X}\beta)^T(\mathbf{y} - \mathbf{X}\beta)$$

The RSS minimizes the (Euclidean) distance of the points from the regression surface:



This approach makes minimal assumptions, since it only requires that the pairs (x_i, y_i) are *random draws from their population*. Specifically, it makes **no assumptions** about the underlying distribution of the response. It simply finds the **best linear fit**, without assuming that the model is valid. In fact, the true underlying model does not even need to be linear for this method to produce estimators. Therefore, it often works well regardless of how the data were obtained. If a linear model provides a good approximation to a non-linear truth, then the regression surface can also be viewed as a measure **lack-of-fit**.

Least-Squares Method

We want to estimate the **vector** of coefficients by minimizing the *sum of squared residuals*:

$$RSS = \|y - \mathbf{X}\beta\|^2 = (y - \mathbf{X}\beta)^T(y - \mathbf{X}\beta)$$

We take derivatives with respect to β and set them equal to zero:

$$\begin{aligned} \frac{\partial RSS}{\partial \beta} &= \mathbf{0}_{p \times 1} \Leftrightarrow \\ -2 \mathbf{X}_{p \times n}^T (y - \mathbf{X}\beta)_{n \times 1} &= \mathbf{0}_{p \times 1} \end{aligned}$$

This leads to the so-called **Normal Equations**

$$\mathbf{X}^T(y - \mathbf{X}\beta) = \mathbf{0}$$

Solving the Normal Equations

$$(\mathbf{X}^T \mathbf{X}) \beta = \mathbf{X}^T y$$

leads to the

Least Squares Estimators in MLR

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T y$$

We assume that **the rank of $\mathbf{X}$ is $p + 1$** , i.e. no columns of $\mathbf{X}$ are a linear combinations of the other columns. *Since $\mathbf{X}$ has full column rank, the inverse of $(\mathbf{X}^T \mathbf{X})$ exists.*

Fitted, Predicted Values & Residuals

The **fitted value** of y_i at $x_i = (x_{i1}, x_{i2}, \dots, x_{ip})$ is computed as:

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2} + \dots + \hat{\beta}_p x_{ip}$$

More generally, using matrix formulation, the **fitted values** of $\mathbf{y}$ are given by

$$\begin{aligned} \hat{\mathbf{y}}_{n \times 1} &= \mathbf{X} \hat{\beta} \\ &= \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T y \\ &= \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T y := \mathbf{H}_{n \times n} y_{n \times 1} \end{aligned}$$

where we define the *hat matrix*

The Hat Matrix

$$\mathbf{H}_{n \times n} = \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$$

It is called the **hat matrix** because it produces the “*y-hat*” values: $\hat{y} = \mathbf{H}t$.

The **predicted value** of y at a new point $x_i^* = (x_{i1}^*, x_{i2}^*, \dots, x_{ip}^*)$ is

$$\hat{y}^* = \hat{\beta}_0 + \hat{\beta}_1 x_{i1}^* + \hat{\beta}_2 x_{i2}^* + \dots + \hat{\beta}_p x_{ip}^*$$

In matrix form, the **predicted values** are

$$\hat{\mathbf{y}}^* = \mathbf{X}^* \hat{\beta}$$

Note: The key difference between the fitted and predicted values is that the vector $\mathbf{x}$ belongs to the training data (it has already been observed), whereas $\mathbf{x}^*$ is an **unobserved** feature vector that is **independent** of the training data.

The **residual** at observation i is

$$\begin{aligned} r_i &= y_i - \hat{y}_i \\ &= y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - \hat{\beta}_2 x_{i2} - \dots - \hat{\beta}_p x_{ip} \end{aligned}$$

In matrix form the residual vector is

$$\begin{aligned}\mathbf{r}_{n \times 1} &= \mathbf{y} - \hat{\mathbf{y}} \\ &= \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{y} - \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \\ &= \mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I} - \mathbf{H})\mathbf{y}\end{aligned}$$

The residuals $\mathbf{r}$ are used to estimate the **error variance**:

$$\hat{\sigma}^2 = \frac{1}{n - p - 1} \sum_i r_i^2 = \frac{RSS}{n - p - 1}$$

The denominator $n - p - 1$ is the *degrees of freedom of the residuals*.

Properties of the Residuals

The least-squares estimator $\hat{\boldsymbol{\beta}}$ satisfies the **normal equations**

$$\mathbf{X}^T(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{X}^T(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = \mathbf{0}$$

From this we obtain the following properties of the residual vector $\mathbf{r}$:

- The residual vector is orthogonal to every column of $\mathbf{X}$:

$$\begin{aligned}\mathbf{X}^T \mathbf{r} &= \mathbf{X}^T(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) \\ &= \mathbf{X}^T \mathbf{y} - \mathbf{X}^T \mathbf{X} \hat{\boldsymbol{\beta}} \\ &= \mathbf{X}^T \mathbf{y} - (\mathbf{X}^T \mathbf{X})(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \mathbf{0}\end{aligned}$$

- The residual vector is also orthogonal to the vector of fitted values:

$$\hat{\mathbf{y}}^T \mathbf{r} = \hat{\boldsymbol{\beta}}^T \mathbf{X}^T \mathbf{r} = \mathbf{0}$$

Thus, the residual vector $\mathbf{r}$ is **orthogonal to each column of $\mathbf{X}$ and to $\hat{\mathbf{y}}$** .

When the model is used for prediction at a new point $\mathbf{x}^*$, two different interval estimates are commonly reported: (i) a confidence interval for the mean response $E(Y | \mathbf{x}^*)$, and (ii) a prediction interval for a new individual observation Y^* . The prediction interval is wider because it must also account for the irreducible error variance σ^2 .

2.3 Least-Squares & Normal Equations

In this section, we focus on the system of linear equations that we solve to obtain the least-squares estimators for β . Recall, that we want to find a vector $\hat{\beta}$ that minimizes

$$\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|^2$$

Any vector that provides a minimum value of this expression is called a *least-squares solution*.

The set of all least squares solutions is precisely the **set of solutions** to the normal equations

$$(\mathbf{X}^T \mathbf{X})\beta = \mathbf{X}^T \mathbf{y}$$

There is a *unique* solution **if and only if** $\text{rank}(\mathbf{X}) = p + 1$ in which case $(\mathbf{X}^T \mathbf{X})$ is invertible.

System of Linear Equations

Linear Algebra Review Let us review a few facts from linear algebra related to solutions of a system of linear equations. For simplicity of notation, we use the *generic* system of equations:

$$\mathbf{A}z = c$$

where

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1k} \\ a_{21} & a_{22} & \cdots & a_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mk} \end{pmatrix}, \quad z = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_m \end{pmatrix}, \quad c = \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_m \end{pmatrix}$$

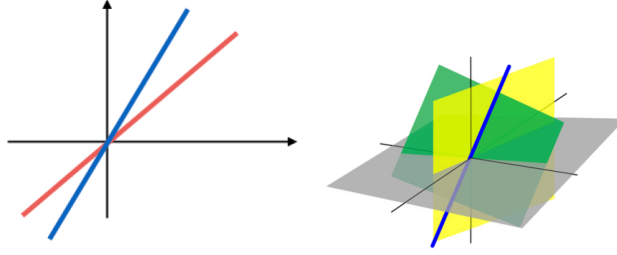
Denote by $\mathbf{A}_i = (a_{1i}, a_{2i}, \dots, a_{mi})^T$ the vector representing the *i*th *column* of $\mathbf{A}$. Then, $\mathcal{C}(\mathbf{A})$ is the space generated by the columns of $\mathbf{A}$, i.e. the span of $\{\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_k\}$.

Definition

The **span** of a collection of vectors $(\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_k)$ is the set of all linear combinations of these vectors:

$$\begin{aligned} & \text{span}(\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_k) \\ &= \left\{ d_1 \mathbf{A}_1 + d_2 \mathbf{A}_2 + \dots + d_k \mathbf{A}_k, \text{ for any constants } d_1, \dots, d_k \in \mathbb{R} \right\} \end{aligned}$$

The column space of $\mathbf{A}$, $\mathcal{C}(\mathbf{A})$, is a **subspace** in $\mathbb{R}^m$. A linear subspace of $\mathbb{R}^m$ can be thought of as a “flat” surface within $\mathbb{R}^m$; it is a collection of vectors that is closed under linear combinations. Illustrations of subspaces of $\mathbb{R}^n$ are shown below:



Solving the System of Equations $\mathbf{A}z = c$

For the system $\mathbf{A}z = c$ to have a solution, c must lie in the column space of $\mathbf{A}$, i.e. $c \in \mathcal{C}(\mathbf{A})$. This follows directly from the definition of matrix multiplication:

$$\mathbf{A}z = c \Leftrightarrow c = z_1\mathbf{A}_1 + \dots + z_k\mathbf{A}_k$$

When $c \notin \mathcal{C}(\mathbf{A})$, we instead seek the vector $\hat{c} \in \mathcal{C}(\mathbf{A})$ that is closest to c . In that case, the equation $\mathbf{A}z = \hat{c}$ has a solution, and $\hat{c}$ comes as close as possible to the original vector c . To obtain $\hat{c}$ we **project c orthogonally** onto $\mathcal{C}(\mathbf{A})$ by multiplying both sides by $\mathbf{A}^T$:

$$\mathbf{A}^T\mathbf{A}z = \mathbf{A}^Tc$$

* These are the normal equations.

In the sketch to the right, $\mathcal{C}(\mathbf{A})$, the space spanned by the columns of $\mathbf{A}$, is a *flat* surface, and c is a point lying off that flat surface. The **shortest distance** from the point c to the plane $\mathcal{C}(\mathbf{A})$ is the line segment *orthogonal* to the plane.

The *normal equations* locate the point in $\mathcal{C}(\mathbf{A})$ that is closest to c by means of an orthogonal projection.

Geometric Representation of LS

Translating the preceding discussion into the linear-regression setting:

- $\mathcal{C}(\mathbf{X})$ is the subspace *spanned* by the columns of the design matrix, i.e. by the predictors $X_1, X_2, \dots, X_p$. It is a *flat* subspace of $\mathbb{R}^n$.

- The response $\mathbf{y}$ is a vector lying **off** this subspace, and the fitted value $\hat{\mathbf{y}}$ is the orthogonal projection of $\mathbf{y}$ onto $\mathcal{C}(\mathbf{X})$.
- The residual vector $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$ is **orthogonal** both to $\hat{\mathbf{y}}$ and to the entire subspace $\mathcal{C}(\mathbf{X})$.

In other words, the least-squares method decomposes the data vector $\mathbf{y}$ into two orthogonal components:

$$\mathbf{y}_{n \times 1} = \hat{\mathbf{y}}_{n \times 1} + \mathbf{r}_{n \times 1}$$

2.4 Goodness-Of-Fit: *R*-Square

A measure of how well the model fits the data is the ***R*-square** (also called **coefficient of determination** or the *percentage of variance explained*). It is defined as

$$\begin{aligned} R^2 &= \frac{\sum_i (\hat{y}_i - \bar{y})^2}{\sum_i (y_i - \bar{y})^2} = \frac{\text{distance of model from grand mean}}{\text{distance of observations from grand mean}} \\ &= \frac{\|\hat{\mathbf{y}} - \bar{\mathbf{y}}\|^2}{\|\mathbf{y} - \bar{\mathbf{y}}\|^2} \\ &= \frac{\|\mathbf{y} - \bar{\mathbf{y}}\|^2 - \|\hat{\mathbf{y}} - \mathbf{y}\|^2}{\|\mathbf{y} - \bar{\mathbf{y}}\|^2} \\ &= 1 - \frac{\|\hat{\mathbf{y}} - \mathbf{y}\|^2}{\|\mathbf{y} - \bar{\mathbf{y}}\|^2} := 1 - \frac{RSS}{TSS} \end{aligned}$$

The second-to-last equality follows from the orthogonal decomposition

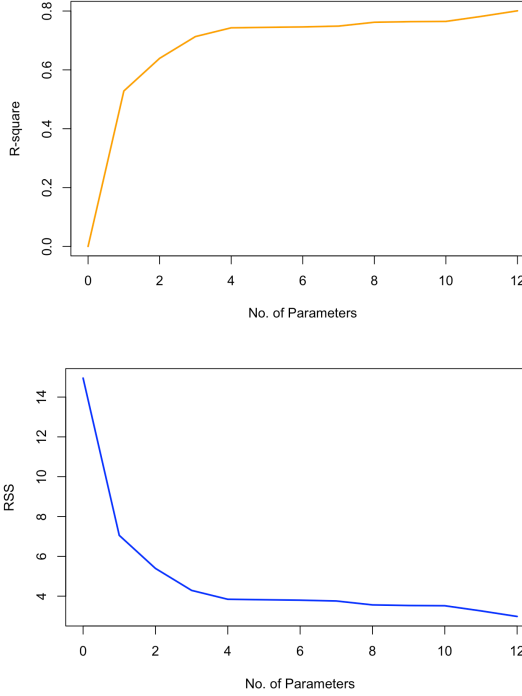
$$\underbrace{\|\mathbf{y} - \bar{\mathbf{y}}\|^2}_{\text{total variation}} = \underbrace{\|\hat{\mathbf{y}} - \bar{\mathbf{y}}\|^2}_{\text{model variation from mean}} + \underbrace{\|\hat{\mathbf{y}} - \mathbf{y}\|^2}_{\text{error}}$$

Properties of R^2

- $0 \leq R^2 \leq 1$
- R^2 is invariant of any location or scale change of Y or of the predictors X .
- R^2 alone does not tell us much about the quality of the LS fit.
- A small R^2 does not necessarily imply that the LS model is poor.

- Adding a new predictor, even if it is randomly generated and completely unrelated to Y , decreases the RSS and therefore increases R^2 .

These relationships are illustrated in the two plots below:



2.5 Linear Transformations on X

Suppose we have a linear regression model of Y on the predictors X . If we **scale** or **shift** a predictor, the fitted values $\hat{y}$ and R^2 remain unchanged, but the LS estimators $\hat{\beta}$ change.

Some Examples

- Scaling a predictor, i.e. $\tilde{x}_{i2} = 2x_{i2}$ or $\tilde{x}_{i3} = x_{i3}/4$, does not affect $\hat{y}$ or R^2 .
- In the **birthweight** example, the length of a baby is recorded in cm. Converting the length column to inches by the transformation $\tilde{X}_1 = X_1/2.54$ leaves the predicted values unchanged.

These statements remain true, if we apply any linear transformation of the p predictors **as long as the transformation does not change the rank of X** .

2.6 Rank deficiency

The design matrix $\mathbf{X}$ is an $n \times (p + 1)$ matrix. If this matrix is **not of full rank** (i.e., its columns are not linearly independent), the matrix $\mathbf{X}^T \mathbf{X}$ cannot be inverted. This means that there is redundancy among the columns of $\mathbf{X}$: at least one column can be written as a linear combination of other columns. Consequently, $\mathbf{X}^T \mathbf{X}$ is *singular* and the LS solution is not unique (an *identifiability problem*).

Nevertheless, the column space $\mathcal{C}(\mathbf{X})$ remains well-defined, so the fitted vector $\hat{y}$ is still well-defined and can be computed. **R** and **Python** handle this situation, and the resulting model can still be used for prediction.

2.7 Hypothesis Testing in MLR

When fitting a regression model, we often want to understand which predictors are statistically significant, or which of two models (a larger one or a smaller one) is more appropriate –either for prediction or for estimation.

Testing for the statistical significance of one or more predictors can be formulated either as a test that the corresponding coefficient(s) equal zero, or as a model-comparison test. In this course we summarize all such testing questions with the following hypothesis test:

$$\begin{cases} H_0 : \text{Reduced Model with } p_0 \text{ coefficients} \\ H_\alpha : \text{Full (larger) Model with } p_\alpha \text{ coefficients} \end{cases}$$

Obviously, $p_\alpha > p_0$, since the reduced model is a special case of the full model. Consequently we expect the reduced model to have a larger residual sum of squares: $RSS_0 \geq RSS_\alpha$. The main tool for this comparison is the **partial F test**:

Partial F test

Hypothesis Test:

$$\begin{cases} H_0 : \text{Reduced Model with } p_0 \text{ coefficients} \\ H_\alpha : \text{Full/ Larger Model with } p_\alpha \text{ coefficients} \end{cases}$$

Test Statistic:

$$F = \frac{(RSS_0 - RSS_\alpha)/(p_\alpha - p_0)}{RSS_\alpha/(n - p_\alpha)} \sim F_{p_\alpha - p_0, n - p_\alpha}$$

Decision Rule: Reject H_0 , if the F test statistic is large (or larger than $F_{p_\alpha - p_0, n - p_\alpha}$), i.e. the variation missed by the reduced model, when being compared with the error variance, is significantly large.

- The **numerator** in the partial F test measures the extra variation in the data not explained by the reduced model, but explained by the full model.
- The **denominator** is the variation in the data not explained by the full model (i.e., not explained by either model), which is used to estimate the error variance.

Birthweight Example (cont'd)

We wish to test whether the group of predictors that describe the father's characteristics (i.e. variables $X_9 - X_{12}$) is jointly significant. The hypotheses are

$$\begin{cases} H_0 : Y \sim 1 + \beta_1 X_1 + \dots + \beta_8 X_8 \\ H_\alpha : Y \sim 1 + \beta_1 X_1 + \dots + \beta_8 X_8 + \dots + \beta_{12} X_{12} \end{cases}$$

Here $p_0 = 8 + 1$ (8 predictors plus intercept) and $p_\alpha = 12 + 1$ is the number of β 's in the full model. Extracting RSS_0 and RSS_α in R/Python and computing the partial F test statistic yields $F = 0.7225$. Because this value is smaller than the critical value $F_{4, n-13} \approx 2.7$ we do not reject H_0 and prefer the reduced model.

Partial F Test: Special Cases

- **Test for a Single Predictor β_j**

$$\begin{cases} H_0 : Y \sim 1 + \beta_1 X_1 + \dots + \beta_{j-1} X_{j-1} + \beta_{j+1} X_{j+1} + \dots + \beta_p X_p \\ H_\alpha : Y \sim 1 + \beta_1 X_1 + \dots + \beta_{j-1} X_{j-1} + \beta_j X_j + \beta_{j+1} X_{j+1} + \dots + \beta_p X_p \end{cases}$$

The partial F test for a single coefficient β_j is exactly equivalent to the usual t -test

$$t_j = \frac{\hat{\beta}_j}{\widehat{\text{se}}(\hat{\beta}_j)} \sim t_{n-p-1}$$

under $H_0 : \beta_j = 0$. In practice most software reports the t -statistic and its p -value directly.

- **Test for all Predictors**

$$\begin{cases} H_0 : Y \sim 1 & [\text{intercept-only model}] \\ H_a : Y \sim 1 + \beta_1 X_1 + \dots + \beta_{j-1} X_{j-1} + \beta_j X_j + \beta_{j+1} X_{j+1} + \dots + \beta_p X_p \end{cases}$$

This is the **overall** F test, which can be viewed as a *goodness-of-fit test* for the regression model.

2.8 Categorical Variables in MLR

As we discussed, some of the predictors may be *categorical*. Consider a categorical predictor with k levels. When it is included in a linear regression model it is represented by binary indicator variables. Two common coding schemes are used:

One-hot encoding creates k binary indicators, one for each level:

$$D_i = \begin{cases} 1 & \text{if the observation belongs to level } i, \\ 0 & \text{otherwise.} \end{cases}$$

Dummy coding (the default in classical linear regression) creates only $k-1$ binary indicators and absorbs one level into the intercept. The indicators take the form

$$D_i = \begin{cases} 0 & \text{if the observation is not at level } i, \\ 1 & \text{if the observation is at level } i, \end{cases}$$

where the omitted level is called the reference level.

An Example with Categorical Variables

Consider a categorical predictor with three levels (a , b , c).

One-hot encoding produces $k = 3$ columns:

$$\begin{pmatrix} a \\ a \\ b \\ b \\ b \\ c \\ c \end{pmatrix} \Rightarrow \begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

Dummy coding (with level c as the reference) produces $k - 1 = 2$ columns:

$$\begin{pmatrix} a \\ a \\ b \\ b \\ b \\ c \\ c \end{pmatrix} \Rightarrow \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

In the dummy-coding matrix, Column 1 is the indicator for level a and Column 2 is the indicator for level b . Level c is absorbed into the intercept.

In general any level may be chosen as the reference level; the choice is usually driven by subject-matter context rather than by statistical considerations.

Dummy coding vs. One-hot encoding

- Dummy coding uses $k - 1$ indicators and keeps the design matrix full rank when an intercept is present. This is the standard approach in classical linear regression.
- One-hot encoding uses k indicators. When an intercept is also included, the k columns sum to the constant vector $\mathbf{1}$, creating perfect multicollinearity; $\mathbf{X}^T \mathbf{X}$ becomes singular and the ordinary least-squares estimator is no longer unique.
- Fitted and predicted values are identical under both codings. *Only the interpretation of the individual coefficients changes.* In practice, any level may be chosen as the reference; the choice is usually guided by subject-matter context rather than by statistical considerations.

2.9 Collinearity

In practice we often encounter data sets in which many of the predictors are highly correlated with one another. In such cases the fitted values and the sampling variances of the regression coefficients can depend heavily on which particular predictors are included in the model. The resulting estimators become sensitive and unreliable.

Possible *symptoms* of collinearity include :

- (i) high pairwise (sample) correlation among the predictors,
- (ii) a relatively large R^2 ,

- (iii) a statistically significant overall F test even though non of the individual predictors is statistically significant.

Specific diagnostic tests and metrics exist for quantifying the severity of collinearity, but they lie beyond the scope of this course.

The simplest remedy is to remove one or more predictors from each highly correlated group. More advanced approaches include principal-component analysis (PCA) and regularization methods based on penalized least squares.

2.10 Model Diagnostics

We made several assumptions when defining the linear regression model

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon, \text{ where } \varepsilon \sim IID(0, \sigma^2)$$

These assumptions are:

- *Linearity*: the conditional mean of Y is a linear function of the predictors.
- *Constant Variance*(homoscedasticity): all responses share a common, constant variance σ^2 .
- *Uncorrelated errors*: the errors ε_i and ε_j (and therefore Y_i and Y_j) are uncorrelated for $i \neq j$.
- *(Normality)*: this is not required for the derivation of the least-squares estimators themselves. It becomes necessary, however, when we want exact finite-sample distributional results for the estimators, fitted values, and residuals, and it is especially important for hypothesis testing.

If the goal is to interpret the effects of the predictors on the response (rather than pure prediction), all of the above assumptions should be checked with diagnostic plots and tests, and appropriate remedial measures should be taken when they are violated.

Because the primary focus of this course is prediction, we refer students to a dedicated regression course or textbook for a thorough treatment of diagnostics and remedial measures in multiple linear regression.

Outliers

Outliers can be a concern in both large and small data sets. When the sample size is modest, it is advisable to test formally for outliers—for example, with tests based on the leave-one-out prediction error. In very large data sets such

tests lose power and are less useful; one should instead adjust for multiple comparisons or adopt alternative strategies.

If outliers appear to be problematic, useful practical steps include:

- (i) examining the range of each variable,
- (ii) applying a log, square-root, or other suitable transformation to right-skewed predictors or to the response Y , and
- (iii) using winsorization to limit the influence of extreme values.

2.11 The Birthweight Example

The `birthweight.csv` data set contains observations on 42 babies. The variables are:

- **ID**: Unique Identification number of a baby
- **Length**: Length of the baby at time of birth in cm (X_1)
- **Birthweight**: Weight of the baby at time of birth in kg (Y)
- **Headcirc**: Head circumference of the baby at time of birth in cm (X_2)
- **Gestation**: Completed weeks of gestation (X_3)
- **smoker**: Mother is/is not a smoker {0: No, 1: Yes} (X_4)
- **mage**: Mother's age at time of birth (X_5)
- **mnocig**: Mother's number of cigarettes smoked per day (X_6)
- **mheight**: Mother's height in cm (X_7)
- **mppwt**: Mother's pre-pregnancy weight (X_8)
- **fage**: Father's age at time of birth (X_9)
- **fedys**: Father's years of education (X_{10})
- **fnocig**: Father's number of cigarettes smoked per day (X_{11})
- **fheight**: Father's height (X_{12})

You can download the data file here: [Birthweight.csv](#) .

The analysis of this data set is available in R and Python.

The data analysis example also includes a highly correlated data set (`seatpos`) from Faraway's book on "Linear Models in R". You can download this data here: [seatpos.csv](#)

Chapter 3

Variable Selection & Regularization

Consider the multiple linear regression model with p predictors **plus** an intercept:

$$Y \sim 1 + X_1 + X_2 + \dots + X_p$$

In many applications the number of explanatory variables p is large; in some cases we even have $p \gg n$. This does not mean that every variable is relevant to the response Y . In fact, *only a small fraction* of the p variables is typically believed to be relevant.

Our **goal** in this chapter is to develop methods that efficiently identify the predictors that are useful for estimating or predicting the response. Because the least-squares estimator $\hat{\beta}$ is unbiased, the coefficients of irrelevant variables tend to be close to zero. Nevertheless, if our primary objective is good prediction, we must still ask whether it is beneficial to remove those unnecessary variables from the model.

To understand the consequences of including superfluous predictors, we first examine the **training** and **testing** errors more carefully.

3.1 Training vs. Testing Errors

Consider that we *split our data in two parts*:

- **Training data** $\{\mathbf{x}_i, y_i\}_{i=1}^n$: used to fit the model

- **Testing data** $\{\mathbf{x}_i, y_i^*\}_{i=1}^n$: an **independent** data set collected at the same locations $\mathbf{x}_i$

Remark: In practice we are given a single data set that we randomly partition into a training set and a testing set (typically allocating roughly 80 % of the observations to training).

We assume that both *testing* and *training* observations come from the same population (or are observed at the same design points $\mathbf{x}_i$). Statistically we may therefore write

$$\mathbf{y}_{n \times 1}, \mathbf{y}_{n \times 1}^* \sim^{iid} N_n(\boldsymbol{\mu}, \sigma^2 \mathbf{I}_n) \text{ and } \mathbf{X} = \mathbf{X}$$

or equivalently,

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim^{iid} \mathcal{N}_n(\mathbf{0}, \sigma^2 \mathbf{I}_n) \quad \text{and} \quad \mathbf{y}^* = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}^*, \quad \boldsymbol{\epsilon}^* \sim^{iid} \mathcal{N}_n(\mathbf{0}, \sigma^2 \mathbf{I}_n)$$

with $\boldsymbol{\epsilon}, \boldsymbol{\epsilon}^*$ independent.

Having these models in mind, we compute the **MSE for train and testing data**:

For simplicity we suppress the intercept in the calculations below (so the model dimension is p rather than $p + 1$). The expected training and testing mean squared errors are then:

$$\begin{aligned} \mathbb{E}(\text{Train Error}) &= \mathbb{E} \left\| \mathbf{y} - \hat{\mathbf{y}} \right\|^2 = \mathbb{E} \left\| (\mathbf{I} - \mathbf{H})\mathbf{y} \right\|^2 \\ &= \text{tr} \left((\mathbf{I} - \mathbf{H}) \text{Cov}(\mathbf{y}) (\mathbf{I} - \mathbf{H})^T \right) \\ &= \sigma^2 \text{tr} \left((\mathbf{I} - \mathbf{H}) \right) = (n - p) \sigma^2 \\ &= n\sigma^2 - p\sigma^2 \end{aligned}$$

$$\begin{aligned}
\mathbb{E}(\text{Test Error}) &= \mathbb{E} \left\| \mathbf{y}^* - \hat{\mathbf{X}} \right\|^2 \\
&= \mathbb{E} \left\| (\mathbf{y}^* - \mathbf{X}) + (\mathbf{X} - \hat{\mathbf{X}}) \right\|^2 \\
&= \mathbb{E} \left\| \mathbf{y}^* - \right\|^2 + \mathbb{E} \left\| \mathbf{X} - \hat{\mathbf{X}} \right\|^2 \\
&= \mathbb{E} \left\| \right\|^2 + \text{tr}(\mathbf{X} \text{Cov}(\hat{\mathbf{X}}) \mathbf{X}^T) \\
&= n\sigma^2 + \sigma^2 \text{tr} \mathbf{H} \\
&= n\sigma^2 + p\sigma^2
\end{aligned}$$

From these expressions we see that

- the *training error decreases with p* .
- the *testing error increases with p* .

Consequently, if the sole objective is pure prediction, simply adding more columns to $\mathbf{X}$ is not automatically beneficial. Does this imply that the intercept-only model ($p = 0$)—the model with the smallest expected test error under the calculation above—is optimal? **No.**

The preceding analysis rests on two important **assumptions**:

1. The conditional mean $E(\mathbf{Y}|\mathbf{X})$ lies in the column space of $\mathbf{X}$; that is, there exists a coefficient vector β such that $\mathbb{E}(\mathbf{Y} | \mathbf{X}) = \mathbf{X}\beta$.
2. The design matrix $\mathbf{X}$ contains *all* available predictors. When we fit a linear model that uses only a **subset** of the columns of $\mathbf{X}$, an additional **bias** term appears.

More generally we may face a model of the form

$$Y = f(X) + \varepsilon$$

in which f is not linear.

Approximating f by a linear function f^* introduces bias. Even when the true relationship is linear, omitting relevant predictors likewise produces bias.

In the presence of bias the expected errors become:

$$\begin{aligned}\mathbb{E}(\text{Test Error})^2 &= n\sigma^2 + p\sigma^2 + \text{Bias} \\ \mathbb{E}(\text{Training Error})^2 &= n\sigma^2 - p\sigma^2 + \text{Bias}\end{aligned}$$

where $p\sigma^2$ is the unavoidable error – the ε – and Bias is the model error.

- Larger model (p large) $\rightarrow$ small Bias, large Variance ($p\sigma^2$).
- Smaller model (p small) $\rightarrow$ large Bias, small Variance ($p\sigma^2$).

Reducing the test (prediction) error therefore requires finding a good **trade-off between bias and variance**.

3.2 Subset Selection

The basic idea of *subset selection* is to score every candidate model by an information criterion that balances

$$\text{Training Error} + \text{Complexity Penalty}$$

and then to search for the model with the best score. For linear regression the complexity of a model increases with the number of predictors p .

- **Training Error:** an increasing function of RSS .
- **Complexity Penalty:** an increasing function of p .

Why not simply use R^2 or RSS ? Both R^2 and RSS improve (or at least never worsen) whenever a new variable is added, so they do not penalize the inclusion of irrelevant predictors.

3.2.1 Information Criteria

Akaike Information Criterion & Bayesian Information Criterion

AIC and BIC are defined as

$$\begin{aligned}AIC &= -2 \cdot \text{loglik} + 2p \\ BIC &= -2 \cdot \text{loglik} + \log(n)p\end{aligned}$$

where p is the number of predictors in the model under consideration and $\log \text{lik}$ is the maximized log-likelihood. For the normal-error linear model the first term reduces to

$$-2 \cdot \log \text{lik} = n \log \frac{RSS}{n}$$

(up to an additive constant that does not affect model comparison). Consequently

$$\begin{aligned} AIC &= n \log \frac{RSS}{n} + 2p \\ BIC &= n \log \frac{RSS}{n} + \log(n)p \end{aligned}$$

Lower values of AIC or BIC are preferred. When n is large, the penalty for an extra predictor is substantially heavier under BIC than under AIC; thus AIC tends to select larger models than BIC.

Adjusted- R^2 for model with p predictors

$$\begin{aligned} R_a^2 &= 1 - \frac{RSS/(n-p-1)}{TSS/(n-1)} \\ &= 1 - (1 - R^2) \left(\frac{n-1}{n-p-1} \right) \\ &= 1 - \frac{\hat{\sigma}^2}{\hat{\sigma}_0^2} \end{aligned}$$

where $\hat{\sigma}^2$ is the residual variance estimate for the current model and $\hat{\sigma}_0^2$ is the residual variance estimate for the intercept-only model.

Higher values of R_a^2 are preferred.

Mallow's C_p

$$C_p = \frac{RSS_{\mathbf{p}}}{\hat{\sigma}_0^2} + 2p - n$$

where $\hat{\sigma}_0^2$ is the residual variance estimate obtained from the *full* model. Mallows' C_p behaves similarly to AIC; **lower** values are preferred.

Illustration of Complexity Criteria vs. p

The figure below shows how the adjusted R^2 , Mallows' C_p , AIC and BIC change with the number of predictors p for the `student-grades` data set introduced in the previous section.

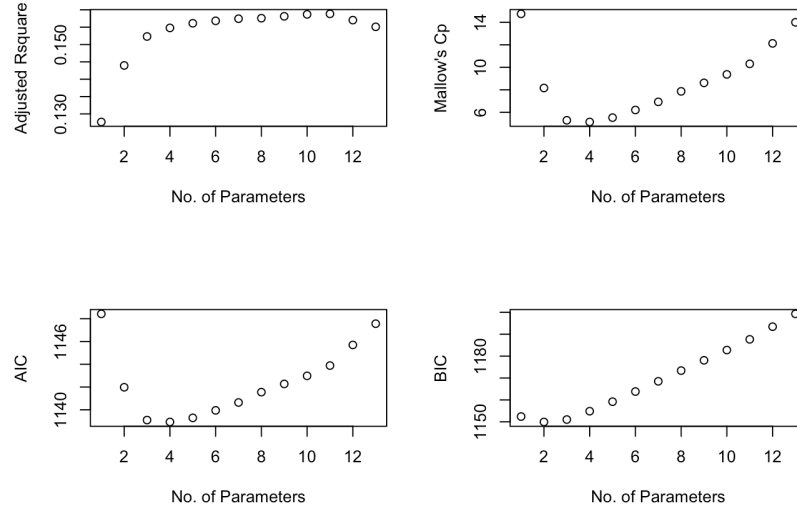


Figure 3.1: Subset selection criteria vs. p .

3.2.2 Search Algorithms

With p available predictors there are 2^p possible models. When p is modest we can evaluate all of them; when p is large we need systematic search strategies.

Level-wise search algorithms: These procedures return a globally optimal solution among all possible models and are practical for up to roughly 40 variables.

The idea is that they identify the m different models (default $m = 8$ in `R`) that have the smallest residual sum of squares, then score those models with the chosen information criterion. The algorithm need not examine every subset: if, for example, $RSS(X_1, X_2) < RSS(X_3, X_4, X_5, X_6)$, then every size-2 or size-3 sub-model of X_3, X_4, X_5, X_6 can be safely skipped (“leaped over”).

Greedy algorithms: These algorithms add or remove predictors according to an information criterion (AIC, BIC, ...). Three common variants are:

- *Backward elimination:* start with the full model and successively delete the predictor whose removal most improves the criterion, until no further improvement is possible;
- *Forward selection:* start with the null model and successively add the predictor that most improves the criterion;
- *Stepwise selection:* at each step consider both adding and deleting a single predictor.

Greedy methods are computationally efficient but return only a *locally optimal* model; in practice this is usually satisfactory.

Some practical considerations

When $p \gg n$ it is infeasible to begin with the full model. A common screening strategy is therefore:

- Rank the p predictors by the absolute value of their marginal correlation with Y .
- Retain only the top K predictors (e.g., $K = n/3$).
- Apply a stepwise procedure that is still allowed to re-introduce previously discarded variables.

3.3 Shrinkage Methods

The variable-selection methods discussed earlier work well when the number of predictors is moderate (for example, level-wise algorithms are practical for $p < 40$). In this section we study two shrinkage methods that can handle larger p by continuously shrinking coefficient estimates toward zero, thereby *balancing the bias-variance trade-off*.

We can re-frame the earlier approaches as a single penalized least-squares problem:

Regression with Penalization

$$\begin{aligned}\hat{\beta} &= \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|^2 + \underbrace{\lambda \sum_{j=1}^p \mathbf{1}_{\beta_j \neq 0}}_{\text{penalty}} \\ &= \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|^2 + \underbrace{\lambda \|\beta\|_0}_{\text{penalty}}\end{aligned}$$

where $\|\beta\|_0 = \sum_{j=1}^p \mathbf{1}_{\beta_j \neq 0}$ simply counts the number of non-zero coefficients. Different choices of the penalty parameter λ recover the information criteria we studied previously.

Replacing the ℓ_0 “norm” by a convex surrogate yields two of the most widely used *penalized regression* methods:

- **Ridge Regression**

$$\hat{\beta} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|^2 + \underbrace{\lambda \|\beta\|^2}_{\text{penalty}}, \text{ where } \|\beta\|^2 = \sum_{j=1}^p \beta_j^2$$

- **Lasso Regression**

$$\hat{\beta} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|^2 + \underbrace{\lambda \|\beta\|_1}_{\text{penalty}}, \text{ where } \|\beta\|_1 = \sum_{j=1}^p |\beta_j|$$

Before examining the mathematics and implementation of these methods, we note two important practical considerations.

Implementation Considerations

Neither the ℓ_1 nor the ℓ_2 penalty is invariant to location or scale. Consequently the penalized estimators are sensitive to the units in which the predictors are measured. It is therefore standard practice to **center and scale** each column of the design matrix:

$$\tilde{\mathbf{x}}_j = \frac{\mathbf{x}_j - \bar{\mathbf{x}}_j}{sd_{\mathbf{x}_j}}$$

and to center the response,

$$\tilde{\mathbf{y}}_i = \mathbf{y}_i - \bar{\mathbf{y}}$$

After model fitting the coefficients can be transformed back to the original scale for interpretation and prediction. The intercept is recovered by

$$\hat{\beta}_0 = \bar{y} - \sum_{j=1}^n \hat{\beta}_j \bar{\mathbf{x}}_j$$

The R package `glmnet` performs the centering, scaling, and back-transformation automatically.)

In Python, `scikit-learn`'s Ridge, Lasso, and ElasticNet do not automatically center and scale the predictors. You should do this yourself—most commonly by placing a `StandardScaler` inside a `Pipeline`. The fitted coefficients are then returned on the scaled scale; you can transform them back to the original scale if needed. There is also a Python port of the original `glmnet` Fortran code that mirrors the R package's automatic standardization and back-transformation behavior.

3.3.1 Ridge Regression

Ridge regression shrinks the coefficients of a linear model by adding an ℓ_2 penalty. It is especially useful when predictors are highly collinear and ordinary least-squares estimates become unstable. The method was originally introduced by Tikhonov as a way to stabilize the normal equations by adding a non-negative constant to the diagonal of $\mathbf{X}^\top \mathbf{X}$.

Ridge Regression

Solve

$$\text{minimize}_{\beta} (y - X\beta)^T (y - X\beta) + \lambda \sum_j \beta_j^2$$

for a fixed $\lambda \geq 0$.

After the predictors have been standardized and the response centered, the ridge estimator admits the **closed-form** expression

$$\hat{\beta}_{\text{Ridge}} = (X^T X + \underbrace{\lambda I}_{\text{ridge}})^{-1} X^T y$$

When $\lambda = 0$ the estimator reduces to ordinary least squares; as $\lambda \rightarrow \infty$ the coefficients shrink to zero. In practice λ is chosen by cross-validation (or generalized cross-validation). A well-known drawback is that the ridge estimator is biased: $\mathbb{E}(\hat{\beta}_{\text{ridge}}) = \beta + \text{bias}$.

To gain intuition, first suppose the columns of $\mathbf{X}$ are orthonormal, so that $\mathbf{X}^\top \mathbf{X} = \mathbf{I}_p$. Then

$$\hat{\beta}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y} = \frac{1}{1 + \lambda} \mathbf{X}^\top \mathbf{y}$$

and the fitted values satisfy

$$\hat{\mathbf{y}}_{ridge} = \mathbf{X}\hat{\boldsymbol{\beta}}_{ridge} = \frac{1}{1 + \lambda} \hat{\mathbf{y}}_{LS}$$

When the columns are not orthogonal the same geometric picture can be obtained via the singular-value decomposition (SVD).

Singular Value Decomposition (SVD)

The SVD is a fundamental matrix factorization used throughout applied mathematics, statistics, and machine learning. Principal-component analysis is a special case of the SVD.

SVD of $\mathbf{X}_{n \times p}$

Any matrix $\mathbf{X}_{n \times p}$ admits the factorization

$$\mathbf{X} = \mathbf{U}_{n \times p} \mathbf{D}_{p \times p} \mathbf{V}_{p \times p}^T$$

where

- $\mathbf{U}_{n \times p}$ has orthonormal columns that span $\mathcal{C}(\mathbf{X})$,
- $\mathbf{V}_{p \times p}$ is an orthogonal matrix whose columns span $\mathbb{R}^p$,
- $\mathbf{D}_{p \times p}$ contains the singular values $d_1 \geq \dots \geq d_p \geq 0$.

If any $d_j = 0$, then $\mathbf{X}$ is rank-deficient.

This factorization of the matrix $\mathbf{X}$ is called the **singular value decomposition of $\mathbf{X}$** , and the columns of $\mathbf{U}$ and $\mathbf{V}$ are called the left- and right-hand **singular vectors** of $\mathbf{X}$.

For the remainder of the discussion we assume $n > p$ and $\text{rank}(\mathbf{X}) = p$ (so $d_p > 0$).

Useful Properties of the SVD

1. The right singular vectors (columns of $\mathbf{V}$) are orthonormal eigenvectors of $\mathbf{X}^T \mathbf{X}$.
2. The left singular vectors (columns of $\mathbf{U}$) are orthonormal eigenvectors of $\mathbf{X} \mathbf{X}^T$.
3. The columns of $\mathbf{V}$ are the *principal-component directions* of the columns of $\mathbf{X}$.
4. The first principal-component direction $\mathbf{v}_1$ maximizes the sample variance of the linear combination $\mathbf{z}_1 = \mathbf{X} \mathbf{v}_1 = \mathbf{u}_1 d_1$ among all unit-norm coefficient vectors.

5. The singular values are the square roots of the eigenvalues of both $\mathbf{X}^\top \mathbf{X}$ and $\mathbf{X} \mathbf{X}^\top$.
6. The largest singular value satisfies

$$d_1 = \max_{\|\mathbf{x}\|=1} \|\mathbf{X}\mathbf{x}\|_2.$$

The principal-component *scores* are the columns of the matrix $\mathbf{S} = \mathbf{U}\mathbf{D}$; they give the coordinates of the observations in the principal-component basis:

$$\mathbf{X} = \mathbf{S}\mathbf{V}^\top.$$

SVD of Fitted Values: LS vs. Ridge

Using the SVD we can write the ordinary-least-squares fitted values as

$$\begin{aligned} \hat{\mathbf{y}}_{LS} &= \mathbf{X}\hat{\beta}_{LS} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \\ &= \mathbf{U} \mathbf{D} \mathbf{V}^\top (\mathbf{V} \mathbf{D}^2 \mathbf{V}^\top)^{-1} \mathbf{V} \mathbf{D} \mathbf{U}^\top \mathbf{y} \\ &= \mathbf{U} \mathbf{D} \mathbf{V}^\top (\mathbf{V}^\top)^{-1} \mathbf{D}^{-2} \mathbf{V}^{-1} \mathbf{V} \mathbf{D} \mathbf{U}^\top \mathbf{y} \\ &= \mathbf{U} \mathbf{D} \mathbf{D}^{-2} \mathbf{D} \mathbf{U}^\top \mathbf{y} \\ &= \mathbf{U} \mathbf{U}^\top \mathbf{y} \\ &= \sum_{j=1}^p (\mathbf{u}_j^\top \mathbf{y}) \mathbf{u}_j \end{aligned}$$

(The algebraic steps rely on $\mathbf{X}^\top \mathbf{X} = \mathbf{V} \mathbf{D}^2 \mathbf{V}^\top$ and the orthonormality of $\mathbf{U}$ and $\mathbf{V}$.)

For ridge regression the same calculation yields

$$\begin{aligned} \hat{\mathbf{y}}_{ridge} &= \mathbf{X}\hat{\beta}_{ridge} \\ &= \mathbf{X}(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y} \\ &= \mathbf{U} \mathbf{D} \mathbf{V}^\top (\mathbf{V} \mathbf{D}^2 \mathbf{V}^\top + \lambda \mathbf{I})^{-1} \mathbf{V} \mathbf{D} \mathbf{U}^\top \mathbf{y} \\ &= \mathbf{U} \mathbf{D} \mathbf{V}^\top (\mathbf{V} \mathbf{D}^2 \mathbf{V}^\top + \lambda \mathbf{V} \mathbf{V}^\top)^{-1} \mathbf{V} \mathbf{D} \mathbf{U}^\top \mathbf{y} \\ &= \mathbf{U} \mathbf{D} \mathbf{V}^\top (\mathbf{V} (\mathbf{D}^2 + \lambda \mathbf{I}) \mathbf{V}^\top)^{-1} \mathbf{V} \mathbf{D} \mathbf{U}^\top \mathbf{y} \\ &= \mathbf{U} \mathbf{D} \mathbf{V}^\top (\mathbf{V}^\top)^{-1} (\mathbf{D}^2 + \lambda \mathbf{I})^{-1} \mathbf{V}^{-1} \mathbf{V} \mathbf{D} \mathbf{U}^\top \mathbf{y} \\ &= \mathbf{U} \mathbf{D} (\mathbf{D}^2 + \lambda \mathbf{I})^{-1} \mathbf{D} \mathbf{U}^\top \mathbf{y} \end{aligned}$$

Thus ridge regression projects $\mathbf{y}$ onto the principal-component basis and then multiplies the j -th coordinate by the shrinkage factor $d_j^2/(d_j^2 + \lambda)$. Components associated with small singular values (low-variance directions) are shrunk more aggressively.

In this last expression note that

- $\mathbf{D}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}$ is a diagonal matrix whose j -th diagonal entry is $\frac{d_j^2}{d_j^2 + \lambda}$,
- the vector $\mathbf{U}^\top \mathbf{y}$ contains the coordinates of $\mathbf{y}$ in the orthonormal basis spanned by the columns of $\mathbf{U}$.

Consequently the ridge fitted values simplify to

$$\hat{\mathbf{y}}_{\text{ridge}} = \sum_{j=1}^p \mathbf{u}_j \cdot \frac{d_j^2}{d_j^2 + \lambda} \cdot (\mathbf{u}_j^\top \mathbf{y}).$$

In other words, ridge regression first projects $\mathbf{y}$ onto the principal-component basis and then multiplies each coordinate by the shrinkage factor $\frac{d_j^2}{d_j^2 + \lambda}$. Equivalently, the ridge estimator shrinks the ordinary-least-squares coefficients by these same factors: the smaller the singular value d_j (i.e., the lower the sample variance of the corresponding principal component), the stronger the shrinkage.

Complexity of Ridge Regression

To quantify the complexity of a model, heuristically we need to understand the number of coefficients that need to be estimated. Ordinary least squares with p predictors has p degrees of freedom. Because ridge regression shrinks coefficients, its effective degrees of freedom lie strictly between 0 and p . If λ is extremely large, there will be no effective covariates left in the model, meaning that they should all be close to zero. On the other hand, if λ is 0, then we go back to linear regression with p covariates and p degrees of freedom.

A convenient definition of **degrees of freedom** is

$$df = \sum_{i=1}^n \text{Cor}(y_i, \hat{y}_i)$$

- For **ordinary least squares** $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ with $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$, so

$$df = \sum_{i=1}^n \text{Cor}(y_i, \hat{y}_i) = \sum_{i=1}^n H_{ii} = \text{tr}(\mathbf{H}) = p$$

- For **ridge regression** $\hat{\mathbf{y}} = \mathbf{S}_\lambda \mathbf{y}$ where

$$\mathbf{S}_\lambda = \mathbf{X}(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top.$$

Hence

$$\begin{aligned} df(\lambda) &= \sum_{i=1}^n \text{Cor}(y_i, \hat{y}_i) = \sum_{i=1}^n [S_\lambda]_{ii} \\ &= \text{tr}(\mathbf{S}_\lambda) \\ &= \text{tr}(\mathbf{X}(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top) \\ &= \text{tr}\left(\sum_{j=1}^p \frac{d_j^2}{d_j^2 + \lambda} \mathbf{u}_j \mathbf{u}_j^\top\right) = \sum_{j=1}^p \frac{d_j^2}{d_j^2 + \lambda} \end{aligned}$$

The effective degrees of freedom decrease monotonically with λ and equal p when $\lambda = 0$. This expression is often used to choose a grid of λ values that correspond to integer degrees of freedom $k = 1, \dots, p$.

3.3.2 Lasso Regression

Lasso regression replaces the ℓ_2 penalty by an ℓ_1 penalty.

Lasso Regression

Solve

$$\text{minimize } (y - X\beta)^\top (y - X\beta) + \lambda \sum_j |\beta_j|$$

for a fixed $\lambda \geq 0$.

In two-dimensions the lasso constraint defines a square, while in higher dimensions it defines a polytope.

Lasso is useful when the response can be explained by *few* predictors with zero effect on the remaining predictors (Lasso is similar to a variable selection method). When $\beta_j = 0$ the corresponding predictor is eliminated which is not the case for ridge regression. Therefore, we use lasso when the effect of predictors is **sparse**. This means that only few predictors will have an effect on the response (e.g. gene expression data) or when number of predictors is large ($p > n$).

Unlike ridge regression, the lasso estimator *does not admit a closed-form expression in general*. When the columns of $\mathbf{X}$ are orthonormal, however, the problem separates coordinate-wise and the solution is given by soft-thresholding.

The lasso solution is defined as

$$\hat{\beta}_{lasso} = \arg \min_{\beta \in \mathbb{R}^p} \left((y - X\beta)^\top (y - X\beta) + \lambda \sum_j |\beta_j| \right)$$

and does not have a closed form expression.

If we assume that $\mathbf{X}^T \mathbf{X} = \mathbf{I}_p$, then

$$\begin{aligned} \|\mathbf{y} - \mathbf{X}\beta\|^2 &= \|\mathbf{y} - \mathbf{X}\hat{\beta}_{LS} + \mathbf{X}\hat{\beta}_{LS} - \mathbf{X}\beta\|^2 \\ &= \|\mathbf{y} - \mathbf{X}\hat{\beta}_{LS}\|^2 + \|\mathbf{X}\hat{\beta}_{LS} - \mathbf{X}\beta\|^2 \end{aligned}$$

where

$$2(\mathbf{y} - \mathbf{X}\hat{\beta}_{LS})^T (\mathbf{X}\hat{\beta}_{LS} - \mathbf{X}\beta) = 2 \mathbf{r}^T (\mathbf{X}\hat{\beta}_{LS} - \mathbf{X}\beta) = 0$$

since the second term is a linear combination of columns of $\mathbf{X}$ no matter what value β takes, and thus is in $\mathcal{C}(\mathbf{X})$, therefore orthogonal to the residual vector $\mathbf{r}$.

Obtaining the Lasso Solution

Under the orthonormality assumption $\mathbf{X}^T \mathbf{X} = \mathbf{I}_p$ the lasso criterion reduces to a sum of p independent one-dimensional problems of the form

$$\arg \min_x (x - a)^2 + \lambda |x|, \lambda > 0$$

The lasso solution can be expressed as:

$$\begin{aligned} \hat{\beta}_{lasso} &= \arg \min_{\beta \in \mathbb{R}^p} \left(\|\mathbf{y} - \mathbf{X}\beta\|^2 + \lambda |\beta| \right) \\ &= \arg \min_{\beta \in \mathbb{R}^p} \left(\|\mathbf{X}\hat{\beta}_{LS} - \mathbf{X}\beta\|^2 + \lambda |\beta| \right) \\ &= \arg \min_{\beta \in \mathbb{R}^p} \left((\hat{\beta}_{LS} - \beta)^T \mathbf{X}^T \mathbf{X} (\hat{\beta}_{LS} - \beta) + \lambda |\beta| \right) \\ &= \arg \min_{\beta \in \mathbb{R}^p} \left((\hat{\beta}_{LS} - \beta)^T (\hat{\beta}_{LS} - \beta) + \lambda |\beta| \right) \\ &= \arg \min_{\beta_1, \dots, \beta_p} \sum_{i=1}^p \left((\beta_i - \hat{\beta}_{(LS)i})^2 + \lambda |\beta_i| \right) \end{aligned}$$

Therefore, to solve the one-dim lasso above, define

$$f(x) = (x - a)^2 + \lambda |x|, \quad a \in \mathbb{R}, \quad \lambda > 0$$

The value x^* that minimizes $f(x)$ must satisfy:

$$\begin{aligned}\frac{\partial}{\partial x}(x^* - a)^2 + \lambda \frac{\partial}{\partial x}|x^*| &= 0 \\ 2(x^* - a) + \lambda z^* &= 0\end{aligned}$$

where z^* is the *sub-gradient* of the absolute value function evaluated at x^* , which equals to $\text{sign}(x^*)$ if $x^* \neq 0$ and any number in $[-1, 1]$, if $x^* = 0$.

The unique minimizer is the **soft-thresholding operator**

$$x^* = S_{\lambda/2}(a) = \text{sign}(a)(|a| - \lambda/2)_+ = \begin{cases} a - \lambda/2 & \text{if } a > \lambda/2, \\ 0 & \text{if } |a| \leq \lambda/2, \\ a + \lambda/2 & \text{if } a < -\lambda/2. \end{cases}$$

Consequently the *orthonormal lasso solution* is simply

$$\hat{\beta}_j^{\text{lasso}} = S_{\lambda/2}(\hat{\beta}_j^{\text{LS}}).$$

A sufficiently large λ forces some coefficients to zero, so the lasso simultaneously selects variables and shrinks the retained coefficients.

Remarks

- Lasso is especially attractive when the underlying signal is *sparse*, since lasso will “make” some of the β coefficients zero keeping the coefficients that will have an effect on $\mathbf{y}$. This is the reason why this method also works when the number of predictors is larger than the sample size ($p > n$). These are scenarios often encountered in genomic or proteomic data where the design matrices tend to have a lot of zeros and too many predictors.
- In lasso as in ridge regression, we can select λ using Cross-Validation (CV). When λ increases, the number of predictors decreases.

Comparing Ridge Regression and Lasso

Ridge regression performs proportional shrinkage and tends to work well when many predictors have moderate effects of similar size. Lasso performs both shrinkage and selection and is preferable when only a few predictors have large effects and the rest are essentially zero. Because the true sparsity pattern is rarely known in advance, cross-validation is the practical tool for deciding which method is better for a given data set.

Geometrically, the residual-sum-of-squares contours are ellipses centered at the ordinary-least-squares estimate. Ridge regression finds the first point where these ellipses touch the disk $\beta^\top \beta \leq t$; lasso finds the first point where they

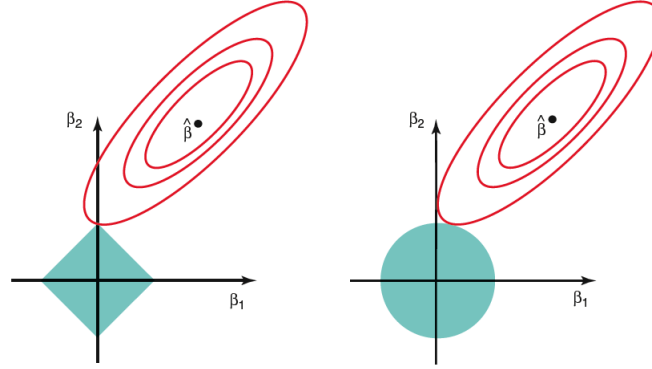


Figure 3.2: Contour of the optimization for Lasso (left) and Ridge (right).

touch the diamond $\|\beta\|_1 \leq t$. The corners of the diamond make exact zeros possible.

In the picture below, we can see the difference in the contours of optimization for lasso (left) and Ridge (right) when there are only two parameters. The residual sum of squares has elliptical contours, centered at the full least squares estimate, $\hat{\beta}_{LS}$. The constraint region for ridge regression is the disk $\beta_1^2 + \beta_2^2 \leq t$, while that for lasso is the diamond $|\beta_1| + |\beta_2| \leq t$. Both methods find the first point where the elliptical contours hit the constraint region. Unlike the disk, the diamond has corners which means that if the solution occurs at a corner, then it has one parameter β_j equal to zero. When $p > 2$, the diamond becomes a rhomboid, and has many corners, flat edges and faces; there are many more opportunities for the estimated parameters to be zero.

Lasso with $p > n$

- When $\mathbf{X}$ has full column rank the lasso criterion is strictly convex, so the minimizer is unique.
- When $\mathbf{X}$ is rank-deficient (in particular when $p > n$) the criterion remains convex but is no longer strictly convex; consequently the solution need not be unique. In the $p > n$ regime the lasso selects at most n variables before saturating—an intrinsic limitation of the convex optimization problem.

3.4 The Student Performance Example

Up to now, we studied two complementary approaches to controlling model complexity in linear regression. Classical subset-selection methods (best-subset,

forward, backward, and stepwise) explicitly search over candidate models and penalize complexity via information criteria such as AIC, BIC, adjusted R^2 , or Mallows' C_p .

When the number of predictors is large, continuous shrinkage methods become preferable. Ridge regression (ℓ_2 penalty) produces a closed-form estimator that shrinks all coefficients toward zero, with stronger shrinkage applied to low-variance principal-component directions; it is especially useful under multicollinearity but does not perform variable selection. Lasso regression (ℓ_1 penalty) simultaneously shrinks coefficients and sets some of them exactly to zero, thereby combining estimation and variable selection; it is particularly effective when the underlying signal is sparse or when $p > n$. Both ridge and lasso require the predictors to be centered and scaled, and the tuning parameter λ is typically chosen by cross-validation.

The choice between these methods ultimately depends on the *bias-variance trade-off* and on whether sparsity is expected in the true model.

In this final section we illustrate the practical implementation of the variable-selection and shrinkage methods developed earlier, using a real data example.

The analysis is based on the `student-mat.csv` data set, which contains information on student achievement in secondary education at two Portuguese schools. The original study is described in:

“Using data mining to predict secondary school student performance” By P. Cortez, A. M. G. Silva. 2008. Published in Proceedings of 5th Annual Future Business Technology Conference.

The data are publicly available from the UCI Machine Learning Repository: [here](#)

The analysis of this data set is available in R and Python.

Chapter 4

NonLinear Regression

As we discussed in the previous sections, multiple linear regression relies on a set of assumptions about the error terms (normality, homoscedasticity, and independence) together with the fundamental assumption that the mean response is a linear function of the predictors. When one or more of these assumptions are violated, the applicability of the linear model is limited. In this section we focus on **relaxing the linearity assumption** and introduce three of the most widely used approaches to nonlinear regression:

- (i) Polynomial regression
- (ii) Regression splines
- (iii) Smoothing splines

A Note on Nonlinearity

Assume that the true underlying model takes the form

$$Y = f(\mathbf{X}) + \varepsilon$$

where ε satisfies the usual assumptions. So far we have **approximated** the *unknown* function f with a multiple linear regression model:

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \dots + \beta_p X_{ip} + \varepsilon_i$$

This model is linear both in the predictors X_{ij} and in the coefficients β_j .

Now consider the following two models:

$$Y_i = \beta_1 X_{i1} + \beta_2 X_{i1}^2 + \beta_3 X_{i2} + \beta_4 X_{i2}^2 + \beta_5 X_{i1} X_{i2} + \varepsilon_i$$

or

$$\log_{10} Y_i = \beta_1 X_{i1} + \beta_2 \sqrt{X_{i1}} + \beta_3 e^{X_{i3}} + \varepsilon_i$$

Both are nonlinear in the predictors but remain linear in the coefficients β_j . In contrast, the model

$$Y_i = \frac{\gamma_0}{1 + \gamma_1 e^{\gamma_2 X_i}} + \varepsilon_i$$

is nonlinear in both the parameters *and* the predictors.

In this chapter we approximate the general nonlinear relationship f using models that are nonlinear in the predictors (so that we can capture nonlinear associations between the response and the features) yet linear in the coefficients. Equivalently, these models can be viewed as ordinary linear regressions in which the original predictors have been replaced by nonlinear transformations of those predictors.

The main advantage of this approach is that we can often describe nonlinear relationships effectively while still estimating the parameters easily. Because the models remain linear in the parameters, the coefficients continue to be obtained by ordinary least squares, which again reduces to solving a system of linear equations.

4.1 Polynomial Regression

The simplest form of nonlinear regression is polynomial regression, which extends the linear model by including higher-order powers of the predictor(s). To study this approach we first introduce polynomial basis functions.

4.1.1 Polynomial Basis Functions

If $b_j(x)$ denotes the j th basis function, then any function f that lies in the span of these basis functions can be written

$$f(x) = \sum_{j=0}^d b_j(x) \beta_j$$

for suitable coefficients β_j . Consequently the nonlinear model $y_i = f(x_i) + \varepsilon_i$ becomes a linear model in the coefficients:

$$y_i = \beta_0 + \sum_{j=1}^d b_j(x_i) \beta_j + \varepsilon_i$$

Suppose we believe that f is a polynomial of degree at most 4. A natural basis for the space of all such polynomials is

$$\begin{aligned}
 b_0(x) &= 1 \\
 b_1(x) &= x \\
 b_2(x) &= x^2 \\
 b_3(x) &= x^3 \\
 b_4(x) &= x^4
 \end{aligned}$$

The corresponding model is therefore

$$y_i = \underbrace{\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \beta_4 x_i^4}_{=\beta_0 + \sum_{j=1}^d b_j(x_i)\beta_j} + \varepsilon_i$$

Illustration of the Polynomial Basis Functions

The following code plots the individual polynomial basis functions of degree 0 through 4 and a linear combination of them.

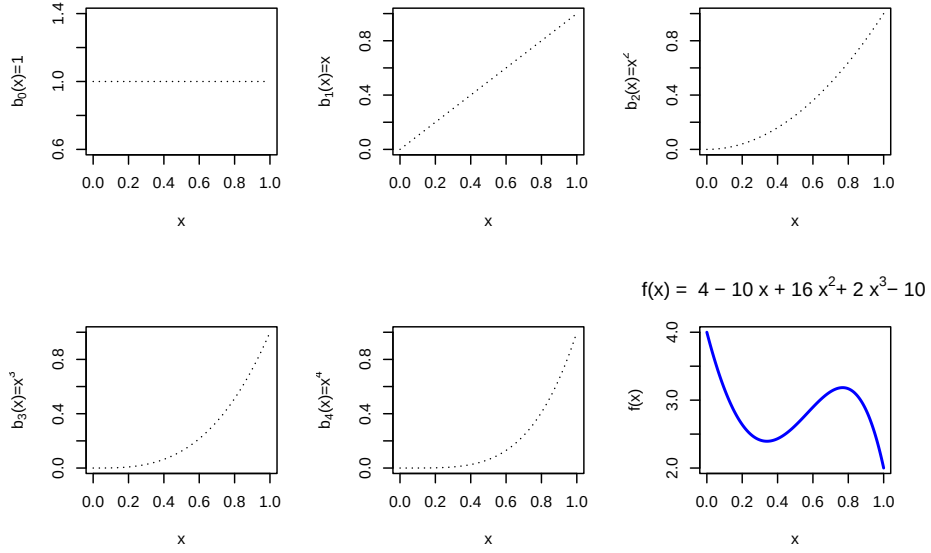
```

x = seq(0, 1, by = 0.001)
b0 = rep(1, length(x))
b1 = x
b2 = x^2
b3 = x^3
b4 = x^4

fun1 = 4 * b0 - 10 * b1 + 16 * b2 + 2 * b3 - 10 * b4

par(mfrow = c(2, 3))
plot(x, b0, type = "l", lty = 3, ylab = expression("b"[0] * "(x)=1"))
plot(x, b1, type = "l", lty = 3, ylab = expression("b"[1] * "(x)=x"))
plot(x, b2, type = "l", lty = 3, ylab = expression("b"[2] * "(x)=x"^2))
plot(x, b3, type = "l", lty = 3, ylab = expression("b"[3] * "(x)=x"^3))
plot(x, b4, type = "l", lty = 3, ylab = expression("b"[4] * "(x)=x"^4))
plot(x, fun1, type = "l", ylab = "f(x)", main = expression("f(x) = 4 - 10 x + 16 x^2 *
    "+ 2 x^3 * "- 10 x^4"), col = "blue", lwd = 2)

```



4.1.2 Polynomial Regression

From now on we assume that the predictor x is one-dimensional; the multivariate case will be discussed later. For $x_i \in \mathbb{R}$ the polynomial regression model of degree d is

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_d x_i^d + \varepsilon_i$$

We simply create the new variables $X_2 = X^2, \dots, X_d = X^d$ and treat the problem as ordinary multiple linear regression:

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}_{n \times 1} = \begin{pmatrix} 1 & x_1 & x_1^2 & \dots & x_1^d \\ 1 & x_2 & x_2^2 & \dots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^d \end{pmatrix}_{n \times (d+1)} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_d \end{pmatrix}_{(d+1) \times 1} + \varepsilon$$

Polynomial Regression Model

A *non-linear* mean function can be represented with a polynomial basis as

$$y_i = f(x_i) + \varepsilon_i \longrightarrow y_i = \beta_0 + \sum_{j=1}^d b_j(x_i) \beta_j + \varepsilon_i$$

where d is the degree of the polynomial.

How do we choose d ?

1. *Forward Approach*: Begin with a linear model and successively add higher-order terms until the most recently added term is no longer statistically significant.
2. *Backward Approach*: Start with a high-degree polynomial and successively remove the highest-order term that is not statistically significant.

Once a degree d has been selected, we ordinarily do not test the significance of the lower-order terms. In other words, a polynomial of degree d is understood to contain all powers up to d .

Reasoning

In regression we prefer results that are invariant to simple location or scale changes of the predictor. Consider the following illustration. Suppose the data $\{(y_i, x_i)\}_{i=1}^n$ are generated by

$$y_i = x_i^2 + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

If the predictor is recorded instead as $z_i = x_i + 2$, the same relationship becomes

$$y_i = (z_i - 2)^2 + \varepsilon_i = 4 - 4z_i + z_i^2 + \varepsilon_i.$$

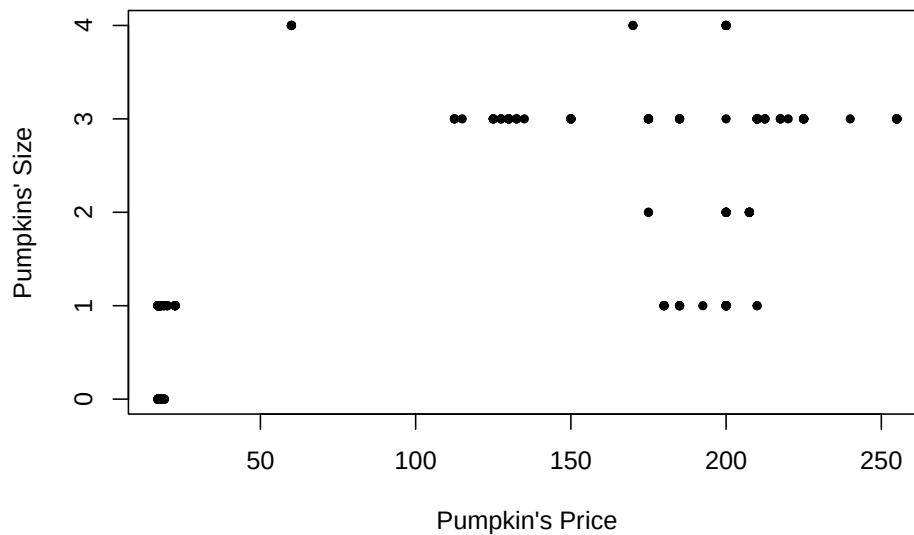
A linear term that was absent in the original scale now appears. Consequently, once we decide that a degree- d polynomial is appropriate we retain every lower-order term.

Exception: If scientific knowledge dictates a specific functional form (for example, a pure quadratic relationship $Y \approx X^2 + C$ arising from a physical law), then it is legitimate to test whether the lower-order terms may be omitted.

The Chicago Pumpkins Example

The file `pumpkins.csv` contains information regarding the `size` and `price` of pumpkins sold in the Chicago area (data can be found [here](#)). Our goal is to *predict* the pumpkin size from price.

```
pumpkins = read.csv("data/week4/chicagopumpkins.csv", header = TRUE)
plot(pumpkins$price, pumpkins$size, pch = 20, xlab = "Pumpkin's Price",
     ylab = "Pumpkins' Size")
```



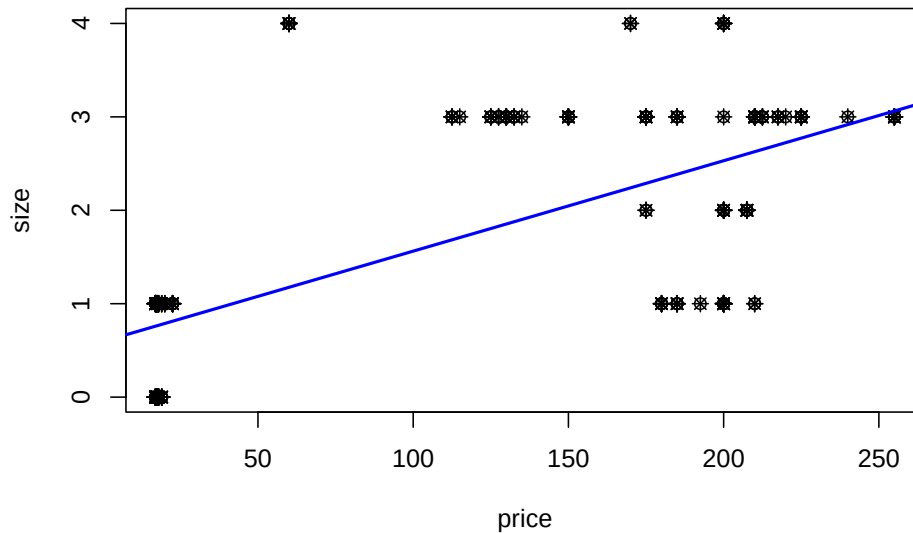
A linear fit is visibly inadequate:

```
lm.pumpkins = lm(size ~ price, data = pumpkins)
summary(lm.pumpkins)
```

```
##
## Call:
## lm(formula = size ~ price, data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.6272 -0.7594  0.2244  0.3728  2.8245
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.5948315  0.0988339   6.018 6.35e-09 ***
## price        0.0096781  0.0006856  14.117 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9477 on 246 degrees of freedom
## Multiple R-squared:  0.4475, Adjusted R-squared:  0.4453
## F-statistic: 199.3 on 1 and 246 DF, p-value: < 2.2e-16
```

Although price is statistically significant, R^2 is modest and the scatter plot does not support a straight-line relationship.

```
plot(size ~ price, data = pumpkins)
points(size ~ price, data = pumpkins, pch = 8)
abline(lm.pumpkins, col = "blue", lwd = 2)
```



We therefore consider higher-order polynomials, first by forward selection and then by backward elimination.

(1) Forward selection. We start with a linear model and add successive powers until the newest term becomes insignificant.

```
# Forward Selection
```

```
lm.pumpkins = lm(size ~ price, data = pumpkins)
```

```
summary(lm.pumpkins)
```

```
##
## Call:
## lm(formula = size ~ price, data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.6272 -0.7594  0.2244  0.3728  2.8245
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.5948315  0.0988339   6.018 6.35e-09 ***
## price        0.0096781  0.0006856  14.117 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9477 on 246 degrees of freedom
## Multiple R-squared:  0.4475, Adjusted R-squared:  0.4453
## F-statistic: 199.3 on 1 and 246 DF, p-value: < 2.2e-16
```

```
lm.pumpkins2 = lm(size ~ price + I(price^2), data = pumpkins)
summary(lm.pumpkins2)
```

```
##
## Call:
## lm(formula = size ~ price + I(price^2), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.6438 -0.6029  0.3695  0.4895  2.3941
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.093e-01  1.174e-01   0.932   0.353
## price        3.037e-02  3.208e-03   9.469 < 2e-16 ***
## I(price^2)   -9.051e-05  1.375e-05  -6.582  2.8e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8754 on 245 degrees of freedom
## Multiple R-squared:  0.5305, Adjusted R-squared:  0.5267
## F-statistic: 138.4 on 2 and 245 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins3 = lm(size ~ price + I(price^2) + I(price^3), data = pumpkins)
summary(lm.pumpkins3)
```

```
##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.2786 -0.4937 -0.1368  0.5531  1.8633
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.414e+00  1.691e-01  -8.36 4.83e-15 ***
## price        1.256e-01  9.079e-03  13.83 < 2e-16 ***
## I(price^2)   -9.845e-04  8.239e-05 -11.95 < 2e-16 ***
## I(price^3)    2.228e-06  2.034e-07  10.95 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7182 on 244 degrees of freedom
## Multiple R-squared:  0.6853, Adjusted R-squared:  0.6814
## F-statistic: 177.1 on 3 and 244 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins4 = lm(size ~ price + I(price^2) + I(price^3) + I(price^4),
  data = pumpkins)
summary(lm.pumpkins4)
```

```
##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3) + I(price^4),
##     data = pumpkins)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
	-1.3200	-0.4497	-0.1241	0.5539	1.7925

```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-2.871e+00	3.782e-01	-7.590	6.83e-13 ***
price	2.314e-01	2.630e-02	8.800	2.60e-16 ***
I(price^2)	-2.565e-03	3.785e-04	-6.776	9.25e-11 ***
I(price^3)	1.061e-05	1.973e-06	5.378	1.76e-07 ***
I(price^4)	-1.470e-08	3.443e-09	-4.271	2.80e-05 ***

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6941 on 243 degrees of freedom
## Multiple R-squared:  0.7073, Adjusted R-squared:  0.7025
## F-statistic: 146.8 on 4 and 243 DF,  p-value: < 2.2e-16

lm.pumpkins5 = lm(size ~ price + I(price^2) + I(price^3) + I(price^4) +
  I(price^5), data = pumpkins)
summary(lm.pumpkins5)
```

```
##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3) + I(price^4) +
##     I(price^5), data = pumpkins)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
	-1.38082	-0.46537	-0.08211	0.53463	1.91954

```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-1.691e+00	7.112e-01	-2.378	0.0182 *
price	1.305e-01	5.791e-02	2.253	0.0251 *
I(price^2)	-2.479e-04	1.244e-03	-0.199	0.8422
I(price^3)	-1.078e-05	1.113e-05	-0.969	0.3333

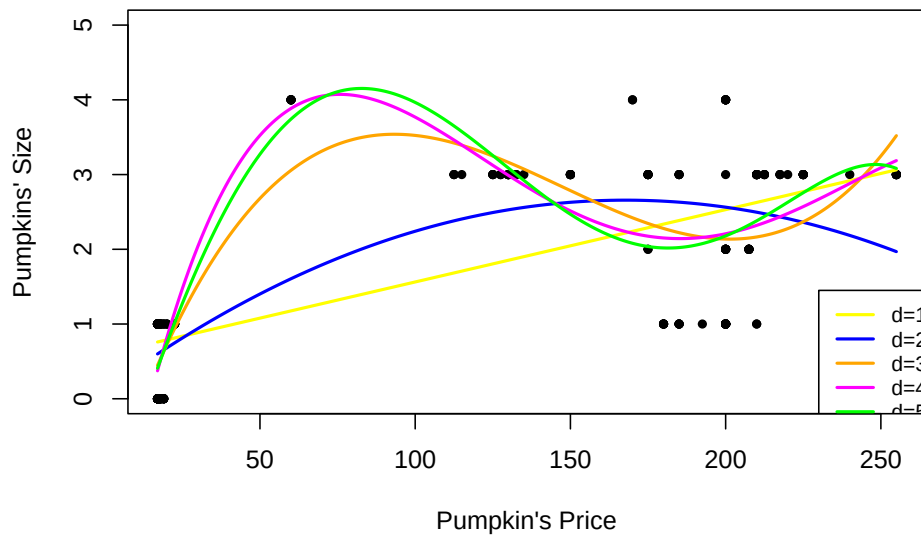
```
## I(price^4)    7.118e-08  4.409e-08   1.615   0.1077
## I(price^5)  -1.248e-10  6.386e-11  -1.954   0.0519 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6901 on 242 degrees of freedom
## Multiple R-squared:  0.7118, Adjusted R-squared:  0.7059
## F-statistic: 119.6 on 5 and 242 DF,  p-value: < 2.2e-16
```

The fifth-order term has a p -value of 0.0519, which exceeds the conventional 5% threshold. We therefore stop at degree 4. The fitted model is

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x + \hat{\beta}_2 x^2 + \hat{\beta}_3 x^3 + \hat{\beta}_4 x^4$$

```
newprice = data.frame(price = seq(17, 255, 1))
plot(pumpkins$price, pumpkins$size, pch = 20, ylim = c(0, 5),
     xlab = "Pumpkin's Price", ylab = "Pumpkins' Size", main = "Forward Selection Model")
lines(newprice$price, predict(lm.pumpkins, newprice), col = "yellow",
      lty = 1, lwd = 2)
lines(newprice$price, predict(lm.pumpkins2, newprice), col = "blue",
      lty = 1, lwd = 2)
lines(newprice$price, predict(lm.pumpkins3, newprice), col = "orange",
      lty = 1, lwd = 2)
lines(newprice$price, predict(lm.pumpkins4, newprice), col = "magenta",
      lty = 1, lwd = 2)
lines(newprice$price, predict(lm.pumpkins5, newprice), col = "green",
      lty = 1, lwd = 2)
legend(230, 1.45, legend = c("d=1", "d=2", "d=3", "d=4", "d=5"),
      col = c("yellow", "blue", "orange", "magenta", "green"),
      lty = c(1, 1, 1, 1, 1), cex = 0.8, lwd = c(2, 2, 2, 2, 2))
```

Forward Selection Models



The magenta curve corresponds to the selected degree-4 model.

(2) Backward elimination. We begin with a high-degree polynomial and remove terms until the highest-order coefficient is significant.

```
lm.pumpkins10 = lm(size ~ price + I(price^2) + I(price^3) + I(price^4) +
  I(price^5) + I(price^6) + I(price^7) + I(price^8) + I(price^9) +
  I(price^10), data = pumpkins)
summary(lm.pumpkins10)
```

```
##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3) + I(price^4) +
##      I(price^5) + I(price^6) + I(price^7) + I(price^8) + I(price^9) +
##      I(price^10), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.44082 -0.45530 -0.01853  0.53535  2.12546
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.331e+01  2.671e+01   0.498   0.619
## price        -2.191e+00  4.274e+00  -0.513   0.609
## I(price^2)    1.396e-01  2.625e-01   0.532   0.595
## I(price^3)   -4.389e-03  8.140e-03  -0.539   0.590
```

```
## I(price^4) 8.198e-05 1.458e-04 0.562 0.575
## I(price^5) -9.818e-07 1.624e-06 -0.605 0.546
## I(price^6) 7.731e-09 1.163e-08 0.664 0.507
## I(price^7) -3.970e-11 5.374e-11 -0.739 0.461
## I(price^8) 1.276e-13 1.549e-13 0.824 0.411
## I(price^9) -2.321e-16 2.535e-16 -0.916 0.361
## I(price^10) 1.821e-19 1.800e-19 1.012 0.313
##
## Residual standard error: 0.6483 on 237 degrees of freedom
## Multiple R-squared: 0.7509, Adjusted R-squared: 0.7404
## F-statistic: 71.45 on 10 and 237 DF, p-value: < 2.2e-16

lm.pumpkins9 = lm(size ~ price + I(price^2) + I(price^3) + I(price^4) +
  I(price^5) + I(price^6) + I(price^7) + I(price^8) + I(price^9),
  data = pumpkins)
summary(lm.pumpkins9)

##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3) + I(price^4) +
##     I(price^5) + I(price^6) + I(price^7) + I(price^8) + I(price^9),
##     data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.4514 -0.4230 -0.0091  0.5177  2.1072
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.191e+01  9.589e+00  -1.242  0.2154
## price        1.877e+00  1.450e+00   1.295  0.1967
## I(price^2)   -1.125e-01  8.271e-02  -1.360  0.1751
## I(price^3)    3.506e-03  2.316e-03   1.514  0.1314
## I(price^4)   -6.094e-05  3.623e-05  -1.682  0.0939 .
## I(price^5)    6.252e-07  3.391e-07   1.844  0.0664 .
## I(price^6)   -3.875e-09  1.945e-09  -1.993  0.0475 *
## I(price^7)    1.425e-11  6.704e-12   2.125  0.0346 *
## I(price^8)   -2.859e-14  1.276e-14  -2.241  0.0259 *
## I(price^9)    2.411e-17  1.030e-17   2.340  0.0201 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6483 on 238 degrees of freedom
## Multiple R-squared: 0.7499, Adjusted R-squared: 0.7404
## F-statistic: 79.27 on 9 and 238 DF, p-value: < 2.2e-16
```

```

lm.pumpkins8 = lm(size ~ price + I(price^2) + I(price^3) + I(price^4) +
  I(price^5) + I(price^6) + I(price^7) + I(price^8), data = pumpkins)
summary(lm.pumpkins8)

##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3) + I(price^4) +
##     I(price^5) + I(price^6) + I(price^7) + I(price^8), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.39988 -0.45678 -0.05827  0.53309  2.02119
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  9.140e+00  3.347e+00   2.731 0.006791 **
## price       -1.368e+00  4.257e-01  -3.215 0.001486 **
## I(price^2)    7.522e-02  2.032e-02   3.702 0.000266 ***
## I(price^3)   -1.798e-03  4.783e-04  -3.759 0.000214 ***
## I(price^4)    2.262e-05  6.154e-06   3.675 0.000293 ***
## I(price^5)   -1.611e-07  4.541e-08  -3.549 0.000466 ***
## I(price^6)    6.535e-10  1.918e-10   3.407 0.000771 ***
## I(price^7)   -1.406e-12  4.316e-13  -3.259 0.001282 **
## I(price^8)    1.246e-15  4.008e-16   3.108 0.002112 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6544 on 239 degrees of freedom
## Multiple R-squared:  0.7441, Adjusted R-squared:  0.7355
## F-statistic: 86.87 on 8 and 239 DF,  p-value: < 2.2e-16

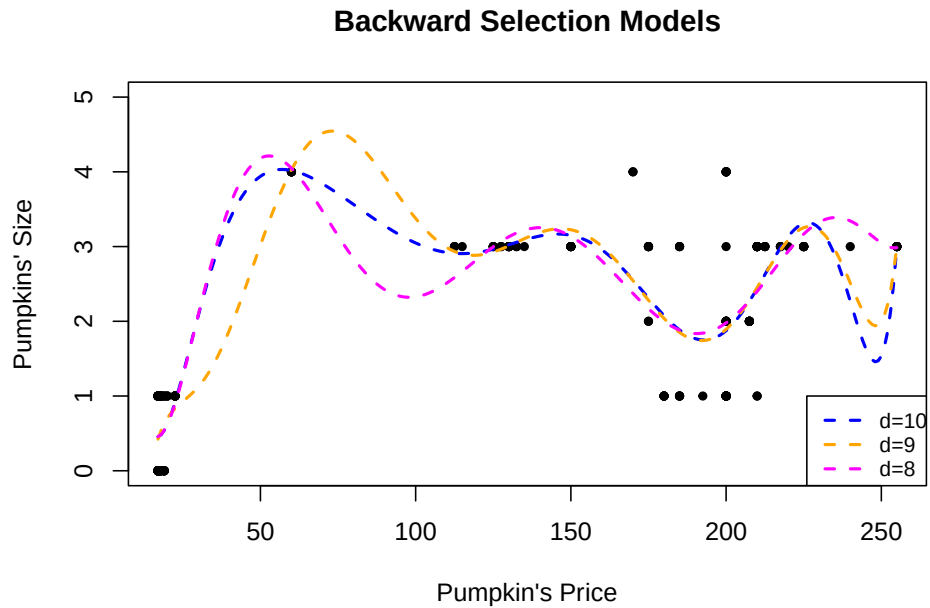
```

Starting from degree 10 we arrive at a degree-9 model. The corresponding fits are shown below.

```

newprice = data.frame(price = seq(17, 255, 1))
plot(pumpkins$price, pumpkins$size, ylim = c(0, 5), pch = 20,
     xlab = "Pumpkin's Price", ylab = "Pumpkins' Size", main = "Backward Selection Models")
lines(newprice$price, predict(lm.pumpkins10, newprice), col = "blue",
      lty = 2, lwd = 2)
lines(newprice$price, predict(lm.pumpkins9, newprice), col = "orange",
      lty = 2, lwd = 2)
lines(newprice$price, predict(lm.pumpkins8, newprice), col = "magenta",
      lty = 2, lwd = 2)
legend(226, 1, legend = c("d=10", "d=9", "d=8"), col = c("blue",
  "orange", "magenta"), lty = c(2, 2, 2), cex = 0.8, lwd = 2)

```



The magenta curve is the degree-9 fit selected by backward elimination.

As always, the final model should be subjected to the usual residual diagnostics.

4.1.3 Orthogonal Polynomials

High-degree polynomials are generally not recommended: they tend to be numerically unstable, hard to interpret, and the successive powers $x, x^2, \dots, x^d$ are highly correlated, producing severe multicollinearity. One remedy is to work with orthogonal polynomials

$$y_i = \beta_0 + \beta_1 z_1 + \dots + \beta_d z_d + \varepsilon_i$$

where each z_j is a polynomial of exact degree j constructed so that

$$z_j^\top z_k = 0 \quad \text{for all } j \neq k.$$

The Chicago Pumpkins Example In R the function `poly` generates orthogonal polynomial bases. The forward- and backward-selection procedures can be repeated with orthogonal polynomials as follows.

```
# Forward Selection
lm.pumpkins02 = lm(size ~ poly(price, 2), data = pumpkins)
summary(lm.pumpkins02)
```

```
##
## Call:
## lm(formula = size ~ poly(price, 2), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.6438 -0.6029  0.3695  0.4895  2.3941
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.70161    0.05559  30.612 < 2e-16 ***
## poly(price, 2)1 13.37846    0.87538  15.283 < 2e-16 ***
## poly(price, 2)2 -5.76139    0.87538  -6.582 2.8e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8754 on 245 degrees of freedom
## Multiple R-squared:  0.5305, Adjusted R-squared:  0.5267
## F-statistic: 138.4 on 2 and 245 DF,  p-value: < 2.2e-16
lm.pumpkins03 = lm(size ~ poly(price, 3), data = pumpkins)
summary(lm.pumpkins03)
```

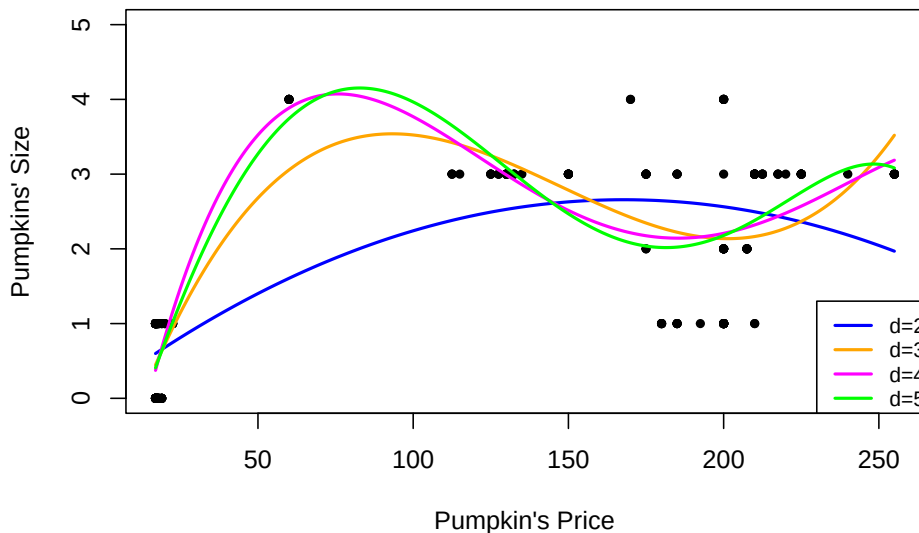
```
##
## Call:
## lm(formula = size ~ poly(price, 3), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.2786 -0.4937 -0.1368  0.5531  1.8633
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.7016    0.0456  37.313 < 2e-16 ***
## poly(price, 3)1 13.3785    0.7182  18.628 < 2e-16 ***
## poly(price, 3)2 -5.7614    0.7182  -8.022 4.35e-14 ***
## poly(price, 3)3  7.8672    0.7182  10.954 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7182 on 244 degrees of freedom
## Multiple R-squared:  0.6853, Adjusted R-squared:  0.6814
## F-statistic: 177.1 on 3 and 244 DF,  p-value: < 2.2e-16
lm.pumpkins04 = lm(size ~ poly(price, 4), data = pumpkins)
summary(lm.pumpkins04)
```

```
##
## Call:
## lm(formula = size ~ poly(price, 4), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.3200 -0.4497 -0.1241  0.5539  1.7925
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.70161    0.04407  38.608 < 2e-16 ***
## poly(price, 4)1 13.37846    0.69408  19.275 < 2e-16 ***
## poly(price, 4)2 -5.76139    0.69408  -8.301 7.22e-15 ***
## poly(price, 4)3  7.86718    0.69408  11.335 < 2e-16 ***
## poly(price, 4)4 -2.96423    0.69408  -4.271 2.80e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6941 on 243 degrees of freedom
## Multiple R-squared:  0.7073, Adjusted R-squared:  0.7025
## F-statistic: 146.8 on 4 and 243 DF, p-value: < 2.2e-16
lm.pumpkins05 = lm(size ~ poly(price, 5), data = pumpkins)
summary(lm.pumpkins05)

##
## Call:
## lm(formula = size ~ poly(price, 5), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.38082 -0.46537 -0.08211  0.53463  1.91954
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.70161    0.04382  38.831 < 2e-16 ***
## poly(price, 5)1 13.37846    0.69009  19.387 < 2e-16 ***
## poly(price, 5)2 -5.76139    0.69009  -8.349 5.35e-15 ***
## poly(price, 5)3  7.86718    0.69009  11.400 < 2e-16 ***
## poly(price, 5)4 -2.96423    0.69009  -4.295 2.53e-05 ***
## poly(price, 5)5 -1.34841    0.69009  -1.954  0.0519 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6901 on 242 degrees of freedom
## Multiple R-squared:  0.7118, Adjusted R-squared:  0.7059
```

```
## F-statistic: 119.6 on 5 and 242 DF, p-value: < 2.2e-16
newprice = data.frame(price = seq(17, 255, 1))
plot(pumpkins$price, pumpkins$size, pch = 20, ylim = c(0, 5),
     xlab = "Pumpkin's Price", ylab = "Pumpkins' Size", main = "Forward Selection Models: Orthogonal",
     lines(newprice$price, predict(lm.pumpkins02, newprice), col = "blue",
        lty = 1, lwd = 2)
     lines(newprice$price, predict(lm.pumpkins03, newprice), col = "orange",
        lty = 1, lwd = 2)
     lines(newprice$price, predict(lm.pumpkins04, newprice), col = "magenta",
        lty = 1, lwd = 2)
     lines(newprice$price, predict(lm.pumpkins05, newprice), col = "green",
        lty = 1, lwd = 2)
     legend(230, 1.3, legend = c("d=2", "d=3", "d=4", "d=5"), col = c("blue",
        "orange", "magenta", "green"), lty = c(1, 1, 1, 1), cex = 0.8,
        lwd = c(2, 2, 2, 2))
```

Forward Selection Models: Orthogonal Polynomials



```
# Backward Selection
lm.pumpkins010 = lm(size ~ poly(price, 10), data = pumpkins)
summary(lm.pumpkins010)
```

```
##
## Call:
## lm(formula = size ~ poly(price, 10), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
```

```
## -1.44082 -0.45530 -0.01853 0.53535 2.12546
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      1.70161    0.04117  41.335 < 2e-16 ***
## poly(price, 10)1  13.37846    0.64829  20.636 < 2e-16 ***
## poly(price, 10)2  -5.76139    0.64829  -8.887 < 2e-16 ***
## poly(price, 10)3   7.86718    0.64829  12.135 < 2e-16 ***
## poly(price, 10)4  -2.96423    0.64829  -4.572 7.77e-06 ***
## poly(price, 10)5  -1.34841    0.64829  -2.080 0.03861 *
## poly(price, 10)6  -1.45456    0.64829  -2.244 0.02578 *
## poly(price, 10)7  -2.57955    0.64829  -3.979 9.20e-05 ***
## poly(price, 10)8   2.03378    0.64829   3.137 0.00192 **
## poly(price, 10)9   1.51698    0.64829   2.340 0.02012 *
## poly(price, 10)10  0.65591    0.64829   1.012 0.31269
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6483 on 237 degrees of freedom
## Multiple R-squared:  0.7509, Adjusted R-squared:  0.7404
## F-statistic: 71.45 on 10 and 237 DF, p-value: < 2.2e-16

lm.pumpkins09 = lm(size ~ poly(price, 9), data = pumpkins)
summary(lm.pumpkins09)

##
## Call:
## lm(formula = size ~ poly(price, 9), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.4514 -0.4230 -0.0091  0.5177  2.1072
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      1.70161    0.04117  41.333 < 2e-16 ***
## poly(price, 9)1  13.37846    0.64833  20.635 < 2e-16 ***
## poly(price, 9)2  -5.76139    0.64833  -8.887 < 2e-16 ***
## poly(price, 9)3   7.86718    0.64833  12.135 < 2e-16 ***
## poly(price, 9)4  -2.96423    0.64833  -4.572 7.76e-06 ***
## poly(price, 9)5  -1.34841    0.64833  -2.080 0.03861 *
## poly(price, 9)6  -1.45456    0.64833  -2.244 0.02578 *
## poly(price, 9)7  -2.57955    0.64833  -3.979 9.20e-05 ***
## poly(price, 9)8   2.03378    0.64833   3.137 0.00192 **
## poly(price, 9)9   1.51698    0.64833   2.340 0.02012 *
## ---
```

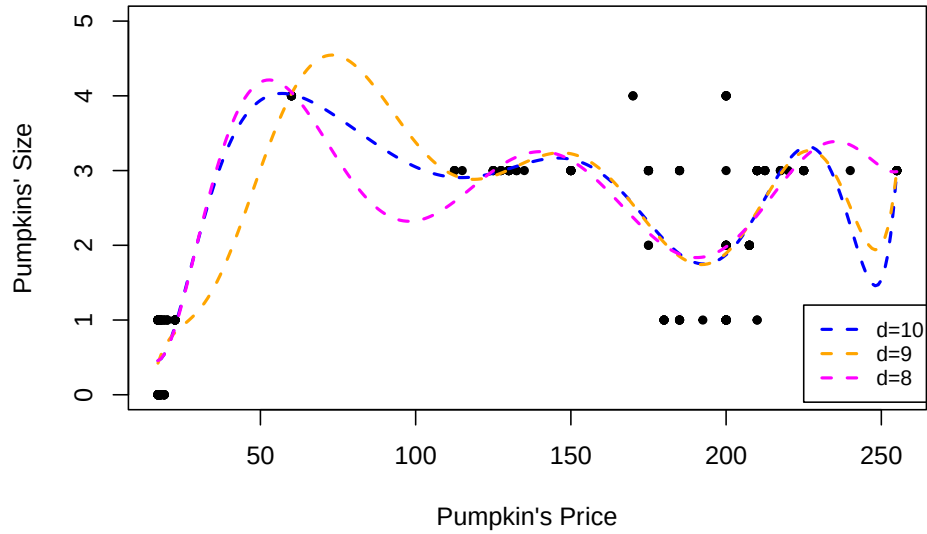
```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6483 on 238 degrees of freedom
## Multiple R-squared:  0.7499, Adjusted R-squared:  0.7404
## F-statistic: 79.27 on 9 and 238 DF,  p-value: < 2.2e-16

lm.pumpkins08 = lm(size ~ poly(price, 8), data = pumpkins)
summary(lm.pumpkins08)
```

```
##
## Call:
## lm(formula = size ~ poly(price, 8), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.39988 -0.45678 -0.05827  0.53309  2.02119
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.70161    0.04155  40.951 < 2e-16 ***
## poly(price, 8)1 13.37846    0.65437  20.445 < 2e-16 ***
## poly(price, 8)2 -5.76139    0.65437  -8.805 2.71e-16 ***
## poly(price, 8)3  7.86718    0.65437  12.023 < 2e-16 ***
## poly(price, 8)4 -2.96423    0.65437  -4.530 9.32e-06 ***
## poly(price, 8)5 -1.34841    0.65437  -2.061 0.040421 *
## poly(price, 8)6 -1.45456    0.65437  -2.223 0.027162 *
## poly(price, 8)7 -2.57955    0.65437  -3.942 0.000106 ***
## poly(price, 8)8  2.03378    0.65437   3.108 0.002112 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6544 on 239 degrees of freedom
## Multiple R-squared:  0.7441, Adjusted R-squared:  0.7355
## F-statistic: 86.87 on 8 and 239 DF,  p-value: < 2.2e-16
```

```
plot(pumpkins$price, pumpkins$size, pch = 20, ylim = c(0, 5),
     xlab = "Pumpkin's Price", ylab = "Pumpkins' Size", main = "Backward Selection Models: Orthogonal",
     lines(newprice$price, predict(lm.pumpkins010, newprice), col = "blue",
           lty = 2, lwd = 2)
lines(newprice$price, predict(lm.pumpkins09, newprice), col = "orange",
      lty = 2, lwd = 2)
lines(newprice$price, predict(lm.pumpkins08, newprice), col = "magenta",
      lty = 2, lwd = 2)
legend(225, 1.2, legend = c("d=10", "d=9", "d=8"), col = c("blue",
  "orange", "magenta"), lty = c(2, 2, 2), cex = 0.8, lwd = 2)
```

Backward Selection Models: Orthogonal Polynomials



4.1.4 Piece-wise Polynomials

When the true mean function $\mathbb{E}(Y \mid X = x) = f(x)$ is highly oscillatory, a single high-degree polynomial may be required. Such polynomials are often undesirable. An attractive alternative is to use piecewise polynomials:

1. Partition the range of x into contiguous intervals.
2. Inside each interval approximate f by a low-degree polynomial (typically quadratic or cubic) whose coefficients are allowed to differ across intervals.
3. Impose continuity (and possibly continuity of derivatives) at the knots that separate the intervals.

The resulting fit is sometimes called *broken-stick* regression when the pieces are linear. The principal advantage is that each observation influences the fit only locally; the principal disadvantage is that the overall curve is usually less smooth than a single global polynomial of comparable complexity.

4.2 Splines Regression

Splines are piecewise polynomials that are constructed so that they can be both sensitive and smooth, but also capture local features of the data.

A **Cubic Spline** is a curve constructed from sections of cubic polynomials, joined together so that the curve is *continuous up to second derivative*. The points at which the sections join are called the **knots** of the spline. For a conventional spline, the knots occur wherever there is a datum, but for the regression splines the locations of the knots must be chosen. Typically, the knots would either be *evenly spaced* through the range of observed x values, or placed at the *quantiles* of the distribution of unique x values. Each section of cubic has different coefficients, but at the knots *it will match its neighboring sections in value and first two derivatives*.

Cubic Splines (Mathematical) Definition

A function g defined on $[a, b]$ is a **cubic spline** with respect to **knots** $\{\xi_i\}_{i=1}^m$ (specifically $a < \xi_1 < \xi_2 < \dots < \xi_m < b$) if:

- g is a cubic polynomial in each of the $m + 1$ intervals, i.e.

$$g(x) = d_i x^3 + c_i x^2 + b_i x + a_i, \quad x \in [\xi_i, \xi_{i+1}]$$

where $i = 0, \dots, m$, $\xi_0 = a$ and $\xi_{m+1} = b$.

- g is continuous up to the 2nd derivative. Since g is continuous up to the 2nd derivative *for any point inside an interval*, it suffices to check the following conditions:

$$g^{(0,1,2)}(\xi_i^+) = g^{(0,1,2)}(\xi_i^-), \quad i = 1, \dots, m$$

This expression indicates that the function and the first and second order derivatives are continuous at the knots.

How many free parameters do we need to represent a cubic spline?

(+) 4 parameters (d_i, c_i, b_i, a_i) for each of the $(m + 1)$ intervals.

(-) 3 constraints at each of the m knots (continuity constraints).

The total number of free parameters (similar to the number of degrees of freedom) is:

$$4(m + 1) - 3m = m + 4$$

A property of the cubic splines

Suppose the knots $\{\xi_i\}_{i=1}^m$ are given.

- If $g_1(x)$ and $g_2(x)$ are cubic splines, the linear combination

$$a_1 g_1(x) + a_2 g_2(x)$$

is also a cubic spline, where a_1 and a_2 are known constants.

That is, for a set of given knots, the corresponding *cubic splines form a linear space (of functions) with dim $(m + 4)$* .

4.2.1 Examples of Cubic Splines Basis

1. A set of basis functions for cubic splines (w.r.t knots $\{\xi_i\}_{i=1}^m$) is given by:

$$\begin{aligned} h_0(x) &= 1 \\ h_1(x) &= x \\ h_2(x) &= x^2 \\ h_3(x) &= x^3; \\ h_{i+3}(x) &= (x - \xi_i)_+^3, \quad i = 1, 2, \dots, m \end{aligned}$$

That is, any cubic spline can be uniquely expressed as:

$$\beta_0 + \sum_{j=1}^{m+3} \beta_j h_j(x)$$

2. Given knot locations, there are many alternative, but equivalent ways of writing down a basis for cubic splines. For example, *another* basis for cubic splines can be written as:

$$\begin{aligned} h_0(x) &= 1 \\ h_1(x) &= x \\ h_{i+1}(x) &= R(x, \xi_i^*), \quad i = 1, \dots, q - 1 \end{aligned}$$

where

$$\begin{aligned} R(x, z) &= [(z - 1/2)^2 - 1/12] [(x - 1/2)^2 - 1/12] / 4 \\ &\quad - [(|x - z| - 1/2)^4 - 1/2(|x - z| - 1/2)^2 + 7/240] / 24 \end{aligned}$$

An Example of a Cubic Splines Basis

We first define the function $R(x, z)$ as above in R:

```
R_xz <- function(x, z) {
  ((z - 0.5)^2 - 1/12) * ((x - 0.5)^2 - 1/12)/4 - ((abs(x -
    z) - 0.5)^4 - (abs(x - z) - 0.5)^2/2 + 7/240)/24
}
```

We then need to define the knots

```
new.knots = c(1/6, 3/6, 5/6)
```

In regression examples the x variable will be the predictor (there is no need to generate any x values). Here, we need a “generic” x for illustration purposes.

```
x = seq(0, 1, by = 0.01)
```

Finally, we defined the splines functions, based on the definition given above:

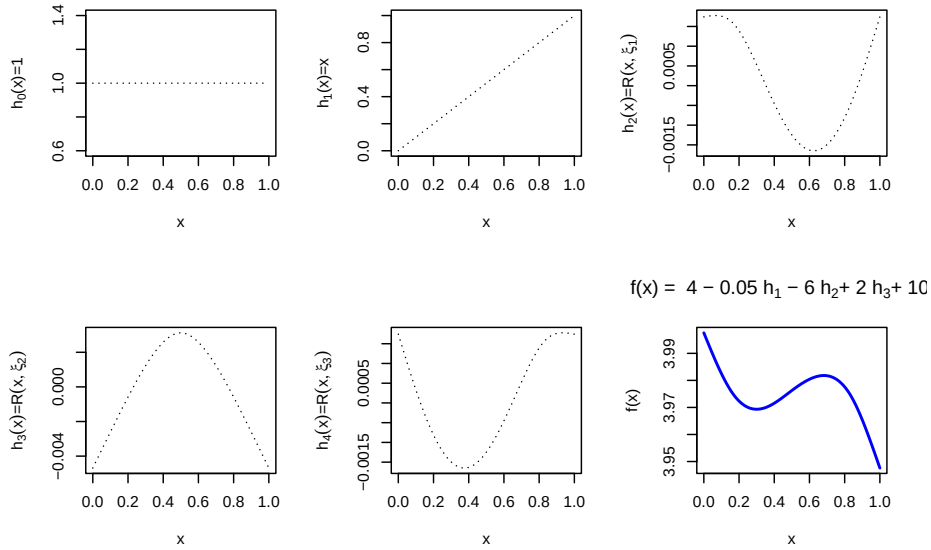
```
spline1 = rep(1, length(x)) # First basis function
spline2 = x # Second basis function
spline3 = R_xz(x, new.knots[1]) # Third basis function
spline4 = R_xz(x, new.knots[2]) # Fourth basis function
spline5 = R_xz(x, new.knots[3]) # Fifth basis function
```

If we want to represent a function using the basis above, we have:

```
fun1s = 4 * spline1 - 0.05 * spline2 - 6 * spline3 + 2 * spline4 +
        10 * spline5
```

For illustration purposes, we plot the basis functions defined above as well as the function f :

```
par(mfrow = c(2, 3))
plot(x, spline1, type = "l", lty = 3, ylab = expression("h"[0] *
  "(x)=1"))
plot(x, spline2, type = "l", lty = 3, ylab = expression("h"[1] *
  "(x)=x"))
plot(x, spline3, type = "l", lty = 3, ylab = expression(paste(h[2](x)) *
  "=" * paste(R(x, xi[1]))))
plot(x, spline4, type = "l", lty = 3, ylab = expression(paste(h[3](x)) *
  "=" * paste(R(x, xi[2]))))
plot(x, spline5, type = "l", lty = 3, ylab = expression(paste(h[4](x)) *
  "=" * paste(R(x, xi[3]))))
plot(x, fun1s, type = "l", ylab = "f(x)", main = expression("f(x) = 4 - 0.05 h"[1] *
  " - 6 h"[2] * "+ 2 h"[3] * "+ 10 h"[4]), col = "blue", lwd = 2)
```



4.2.2 B-Splines Basis Functions in R

In this course one of the cubic spline bases we will use is the **B-spline** basis.

Cubic B Splines Definition

The cubic B-spline basis on an interval $[a, b]$ with interior knots $\{i\}_{i=1}^m$ satisfies the following properties:

1. Each basis function is nonzero only on an interval spanned by four successive knots (and is zero elsewhere).
2. On every sub-interval between consecutive knots the basis function is a cubic polynomial.
3. The basis function is continuous and has continuous first and second derivatives at every knot.
4. The basis function integrates to 1 over its support.
5. The two boundary basis functions are modified so that the required continuity conditions hold at the endpoints a and b .

These bases are provided by the `splines` library in R; the generating function is `bs()`.

Its **main arguments** are

- **x**: the predictor vector.
- **df**: desired degrees of freedom (number of columns of the returned basis matrix).

- **knots**: vector of interior knot locations.
- **degree**: degree of the piecewise polynomial (default = 3, i.e., cubic).
- **intercept**: if TRUE an intercept column is included in the basis; default is FALSE. (This is not the intercept of the eventual regression model.)
- **Boundary.knots**: the two boundary knots at which the basis is anchored; by default they are set to the range of the non-missing values of **x**

(The full documentation is available [here](#).)

In practice we supply either the vector **knots** or the integer **df**, but not both. All other arguments are left at their defaults.

The value returned by **bs()** is a matrix with **length(x)** rows.

- If **df** is supplied the matrix has **df** columns.
- If **knots** is supplied the number of columns equals

$$\text{length}(\text{knots}) + \text{degree} + \text{as.integer}(\text{intercept}).$$

With the default **intercept = FALSE** this reduces to $\text{length}(\text{knots}) + 3$.

Specifying knots versus degrees of freedom**

If the knot locations are known, pass them via the **knots** argument:

Defining knots location in **bs()**

```
library(splines)
x = seq(0, 1, by = 0.01) # generic `x` for illustration purposes
new.knots = c(1/6, 3/6, 5/6) # define three knots at locations 1/6, 3/6, 5/6.
Bsplines.basis1 = bs(x, knots = new.knots)
head(Bsplines.basis1)
```

```
##           1           2           3 4 5 6
## [1,] 0.000000 0.000000 0.000000 0 0 0
## [2,] 0.165912 0.0034896 0.0000144 0 0 0
## [3,] 0.304896 0.0135168 0.0001152 0 0 0
## [4,] 0.418824 0.0294192 0.0003888 0 0 0
## [5,] 0.509568 0.0505344 0.0009216 0 0 0
## [6,] 0.579000 0.0762000 0.0018000 0 0 0
```

In this case, R sets

$$\text{df} = \text{length}(\text{knots}) + \text{degree}$$

(or one more if **if intercept=TRUE**).

- If a target degrees of freedom is preferred, pass `df`. Then `bs()` places `df - degree` interior knots at suitable quantiles of `x` (or `df - degree - 1` knots when `intercept = TRUE`).

Specifying degrees of freedom with `bs()`

```
x = seq(0, 1, by = 0.01) # generic `x` for illustration purposes
Bsplines.basis2 = bs(x, df = 4)
head(Bsplines.basis2)
```

```
##           1           2           3 4
## [1,] 0.000000 0.000000 0.000000 0
## [2,] 0.058214 0.000592 0.000002 0
## [3,] 0.112912 0.002336 0.000016 0
## [4,] 0.164178 0.005184 0.000054 0
## [5,] 0.212096 0.009088 0.000128 0
## [6,] 0.256750 0.014000 0.000250 0
```

The following code illustrates the first six B-spline basis functions obtained from the three knots defined above.

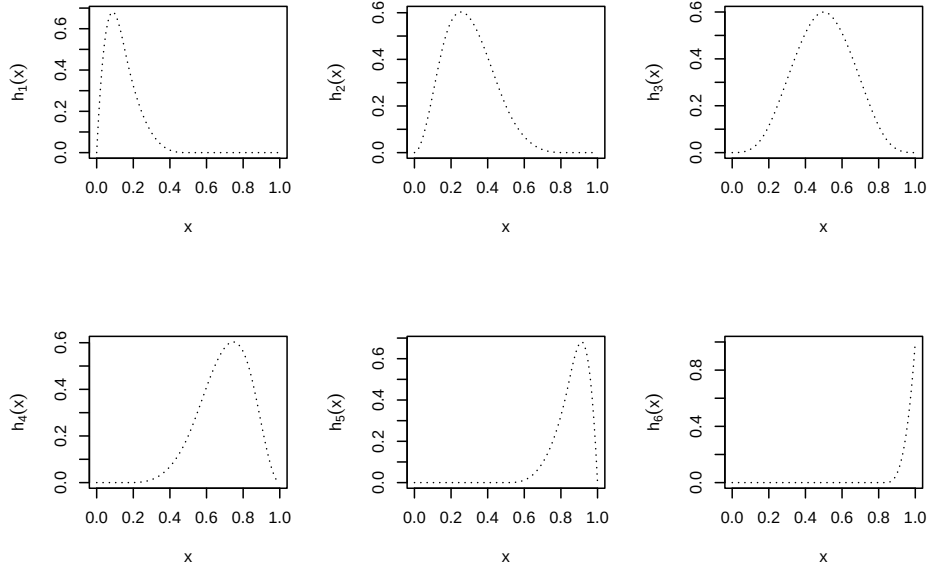
Illustration of B Splines Basis Function

```
Bsplines.basis = bs(x, knots = new.knots)
dim(Bsplines.basis)
```

```
## [1] 101 6
head(Bsplines.basis)
```

```
##           1           2           3 4 5 6
## [1,] 0.000000 0.0000000 0.0000000 0 0 0
## [2,] 0.165912 0.0034896 0.0000144 0 0 0
## [3,] 0.304896 0.0135168 0.0001152 0 0 0
## [4,] 0.418824 0.0294192 0.0003888 0 0 0
## [5,] 0.509568 0.0505344 0.0009216 0 0 0
## [6,] 0.579000 0.0762000 0.0018000 0 0 0
```

```
par(mfrow = c(2, 3))
plot(x, Bsplines.basis[, 1], type = "l", lty = 3, ylab = expression(paste(h[1](x))))
plot(x, Bsplines.basis[, 2], type = "l", lty = 3, ylab = expression(paste(h[2](x))))
plot(x, Bsplines.basis[, 3], type = "l", lty = 3, ylab = expression(paste(h[3](x))))
plot(x, Bsplines.basis[, 4], type = "l", lty = 3, ylab = expression(paste(h[4](x))))
plot(x, Bsplines.basis[, 5], type = "l", lty = 3, ylab = expression(paste(h[5](x))))
plot(x, Bsplines.basis[, 6], type = "l", lty = 3, ylab = expression(paste(h[6](x))))
```



4.2.3 Natural Cubic Splines (NCS)

A cubic spline on $[a, b]$ is called a **natural cubic spline** if its second and third derivatives vanish at the endpoints a and b . Consequently the spline is linear on the two extreme intervals $[a, \xi_1]$ and $[\xi_m, b]$. The linearity constraints reduce the dimension of the space: a natural cubic spline with m interior knots has only m degrees of freedom (the usual textbook count).

Remark

If we place a knot at every distinct observation

$$x_i$$

and use a natural cubic spline, the resulting interpolant is a smooth curve that passes through every data point

$$(x_i, y_i)$$

.

Natural Cubic Spline

One convenient basis for the space of natural cubic splines with m interior knots is

$$\begin{aligned} N_1(x) &= 1 \\ N_2(x) &= x \\ N_{k+2}(x) &= d_k(x) - d_{k-1}(x) \end{aligned}$$

where

$$d_k(x) = \frac{(x - \xi_k)_+^3 - (x - \xi_m)_+^3}{\xi_m - \xi_k}$$

Each of these functions has vanishing second and third derivatives for $x \geq \xi_m$.

In R natural cubic splines are generated by the function `ns()` from the same `splines` package:

```
ns(x, df, knots, intercept = FALSE, Boundary.knots)
```

(Note that the default value of `intercept` is `FALSE`, exactly as for `bs()`.)

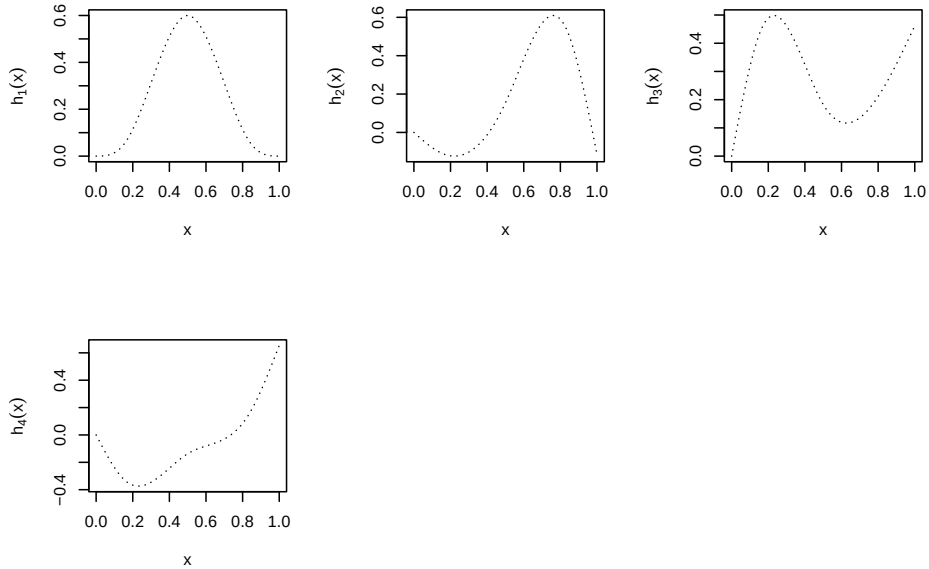
Natural Cubic Splines in R

```
NCS.basis = ns(x, knots = new.knots, Boundary.knots = c(0, 1))
dim(NCS.basis)
```

```
## [1] 101 4
```

```
par(mfrow = c(2, 3))
```

```
plot(x, NCS.basis[, 1], type = "l", lty = 3, ylab = expression(paste(h[1](x))))
plot(x, NCS.basis[, 2], type = "l", lty = 3, ylab = expression(paste(h[2](x))))
plot(x, NCS.basis[, 3], type = "l", lty = 3, ylab = expression(paste(h[3](x))))
plot(x, NCS.basis[, 4], type = "l", lty = 3, ylab = expression(paste(h[4](x))))
```



Because the spline is forced to be linear outside the boundary knots, observations that fall outside those knots do not influence the fit. By default, therefore,

R places the boundary knots at the minimum and maximum of the observed

$$x_i$$

Knots versus degrees of freedom for `ns()`

- If interior knot locations are supplied, the number of columns of the basis matrix is

$$\text{length(knots)} + 1 + \text{as.integer(intercept)}$$

- If a target `df` is supplied, R places

$$m = \text{df} - 1 - \text{intercept}$$

interior knots at quantiles of x .

4.2.4 Regression Splines

For a fixed set of knots the corresponding cubic splines form a linear space of dimension

$$m + 4$$

• **Regression splines** simply expand the mean function in this basis:

$$g(x) = \beta_1 h_1(x) + \beta_2 h_2(x) + \dots + \beta_p h_p(x)$$

- Ordinary Cubic Splines: $p = m + 4$.
- Natural Cubic Splines (NCS): $p = m$.

On the observed data the model may be written in matrix form

$$\begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{pmatrix}_{n \times 1} = \begin{pmatrix} h_1(x_1) & \dots & h_p(x_1) \\ h_1(x_2) & \dots & h_p(x_2) \\ \dots & \dots & \dots \\ h_1(x_n) & \dots & h_p(x_n) \end{pmatrix}_{n \times p} \begin{pmatrix} \beta_1 \\ \beta_2 \\ \dots \\ \beta_p \end{pmatrix}_{p \times 1}$$

where the $n \times p$ matrix $\mathbf{F}$ contains the basis functions evaluated at the x_i . The coefficient vector is obtained by ordinary least squares: Therefore, we can estimate the coefficients by solving the following Least-Squares problem:

$$\hat{\beta} = \arg \min ||\mathbf{y} - \mathbf{F}\beta||^2$$

4.2.5 K-Fold Cross-Validation

A practical difficulty with regression splines is the choice of the number (and location) of knots, or equivalently the degrees of freedom. One systematic approach is

$$K$$

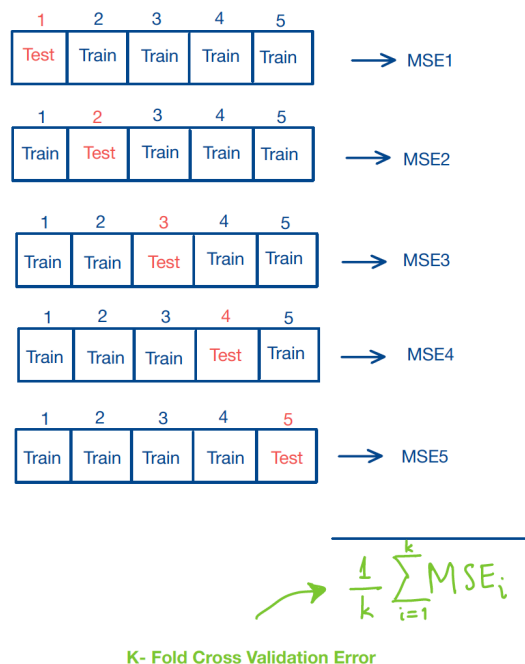
-fold cross-validation:

1. Fix a candidate number of knots (or degrees of freedom).
2. Randomly partition the data into K roughly equal folds.
3. For each fold $k = 1, \dots, K$:
 - hold fold k out as a validation set,
 - fit the regression spline on the remaining $K - 1$ folds,
 - compute the mean squared error MSE_k on the held-out fold.
4. Average the K errors:

$$\text{CV}(K) = \frac{1}{K} \sum_{k=1}^K \text{MSE}_k.$$

5. Repeat the whole procedure for every candidate number of knots / degrees of freedom.
6. Select the value that minimises $\text{CV}(K)$.

A schematic illustration of the procedure appears below.



4.3 Smoothing Splines

In regression splines we must choose both the number and the locations of the knots (or, equivalently, the degrees of freedom). As we have seen, B-splines and natural cubic splines both construct an $n \times p$ basis matrix $\mathbf{F}$ and then fit a linear model on that basis. Inevitably this requires decisions about the order of the spline, the number of knots (often guided by AIC, BIC or cross-validation), and even the placement of those knots—tasks that can be quite challenging. It is therefore natural to ask whether there exists an alternative approach that selects the effective complexity of the fit automatically.

A smoothing spline is designed precisely to balance fidelity to the data against smoothness of the fitted curve. The construction begins with an extreme idea: place a knot at every distinct observed value $x_1, \dots, x_n$. This produces a natural-cubic-spline basis of dimension n ,

$$\mathbf{y}_{n \times 1} = \mathbf{F}_{n \times p} \beta_{p \times 1}$$

Such a basis can interpolate the data perfectly and therefore overfits. To control the overfitting we borrow the idea of penalization introduced earlier and minimize a ridge-type objective

$$\min_{\beta} \left\{ \|\mathbf{y} - \mathbf{F}\beta\|^2 + \lambda \beta^T \Omega \beta \right\}$$

where the tuning parameter $\lambda \geq 0$ is typically chosen by cross-validation and the penalty matrix Ω will be defined shortly. We now examine whether the solution of this penalized problem possesses attractive statistical properties.

4.3.1 The Roughness Penalty Approach

Let $S[a, b]$ be the space of all “smooth” functions defined on $[a, b]$. This is a second order Sobolev space where global polynomial functions and cubic splines functions live ($S[a, b]$ is an infinite-dimensional function space). Our goal is to find the best function in $S[a, b]$ to approximate f .

Let us consider solving the following Penalized Residual Sum of Squares problem:

$$RSS_\lambda(g) = \sum_{i=1}^n [y_i - g(x_i)]^2 + \lambda \int_a^b [g''(x)]^2 dx.$$

where λ is a smoothing parameter. The first term measures the closeness of the model to the data, while the second term penalizes the roughness/curvature of the function. We solve this problem on $[a, b] = [\min x_i, \max x_i]$ for functions with finite roughness penalty $\int_a^b [g''(x)]^2 dx < \infty$. This is known as a second order Sobolev space.

Note that $\int_a^b [g''(x)]^2 dx$ is called the **roughness penalty**.

From the expression above, we can see that λ is the smoothing parameter that controls the bias-variance trade-off. Therefore, when $\lambda = 0$, we interpolate the data and that leads us to overfitting. When $\lambda = \infty$, then we return to linear least-squares regression. It turns out that the solution to the penalized residual sum of squares has to be a NCS. Indeed,

Theorem

$$\min_g RSS_\lambda(g) = \min_{\tilde{g}} RSS_\lambda(\tilde{g})$$

} where $\tilde{g}$ is a NCS with knots at the unique n data points.

Let $g(x)$ be the optimal solution. Since the loss part in $RSS_\lambda(g)$ only involves n data points, we can find define a natural cubic spline (NCS) fit that we can call $\tilde{g}(x)$ such that it matches $g(x)$ at the observations x_i , $i = 1, \dots, n$, i.e.

$$g(x_i) = \tilde{g}(x_i), i = 1, \dots, n$$

We can always find such $\tilde{g}$ since our space consists of n basis. Then, we can show that

$$\int g''^2 dx \geq \int \tilde{g}''^2 dx$$

meaning that we will *always* prefer the $\tilde{g}$, the NCS “representation” of g , since the *penalty* is smaller, and the *loss* doesn’t change.

4.3.2 Proof

To establish the Theorem, we essentially need to show that

$$\int g''^2 dx \geq \int \tilde{g}''^2 dx$$

Thus, we define $h(x) = g(x) - \tilde{g}(x)$ for which we know that $h(x_i) = 0$ for $i = 1, \dots, n$. Then

$$\int g''^2 dx = \int \tilde{g}''^2 dx + \int h''^2 dx + 2 \int \tilde{g}'' h'' dx$$

and without loss of generality assuming that the x_i s are ordered, we obtain:

$$\begin{aligned} \int \tilde{g}'' h'' dx &= \tilde{g}'' h' \Big|_a^b - \int_a^b h' \tilde{g}^{(3)} dx \\ &= - \sum_{i=1}^{n-1} \tilde{g}^{(3)}(x_j^+) \int_{x_j}^{x_{j+1}} h' dx \quad (\tilde{g}^{(3)} \text{ constant piecewise}) \\ &= - \sum_{i=1}^{n-1} \tilde{g}^{(3)}(x_j^+) (h(x_{j+1}) - h(x_j)) \end{aligned}$$

The second equation is because $\tilde{g}$ is a NCS and therefore we know that it has zero second derivative on the two boundaries a and b . The third equation is true, because $\tilde{g}$ is at most a 3rd order polynomial on any region and as a consequence has constant third derivatives, which we can pull out of the integration. The last equation holds because we said that $h(x) = 0$ on all the observation points x_i . Therefore, this shows that the roughness penalty of our NCS solution is no larger than the best solution g . If we also take into account that $\tilde{g}$ is also in the space $S[a, b]$, then g must be our NCS solution.

4.4 Fitting Smoothing Splines

From the preceding discussion we conclude that the minimizer admits a finite-dimensional representation

$$\hat{g}(x) = \sum_i \beta_i N_i(x)$$

where the N_i are a natural-cubic-spline basis with knots placed at the unique observed values of x . Substituting this expansion into the roughness penalty

yields

$$\begin{aligned}\int_a^b g''^2 dx &= \int \left(\sum_i \beta_i N_i''(x) \right)^2 dx \\ &= \sum_{i,j} \beta_i \beta_j \int N_i''(x) N_j''(x) dx \\ &= \beta^T \Omega \beta\end{aligned}$$

where the $n \times n$ penalty matrix has entries

$$\Omega_{ij} = \int N_i''(x) N_j''(x) dx$$

Consequently the penalized residual sum of squares becomes a function of the coefficient vector alone:

$$RSS_\lambda(\beta) = (\mathbf{y} - \mathbf{F}\beta)^T (\mathbf{y} - \mathbf{F}\beta) + \lambda \beta^T \Omega \beta$$

This is a ridge-type criterion whose minimizer is given in closed form by

$$\begin{aligned}\hat{\beta} &= \arg \min_{\beta} RSS_\lambda(\beta) \\ &= (\mathbf{F}^T \mathbf{F} + \lambda \Omega)^{-1} \mathbf{F}^T \mathbf{y}\end{aligned}$$

The smoothing spline version of the “hat” matrix is called the **smoother matrix**

$$\hat{\mathbf{y}} = \mathbf{F}(\mathbf{F}^T \mathbf{F} + \lambda \Omega)^{-1} \mathbf{F}^T \mathbf{y} = S_\lambda \mathbf{y}$$

Remarks

We have done the analysis of degrees of freedom for ridge type regression. The degrees of freedom of a smoothing spline are

$$df = \text{tr}(S_\lambda)$$

which ranges between 0 and n . Under a special choice of basis (Demmler and Reinsch, 1975), a basis with double orthogonality property, can lead to

$$\mathbf{F}^T \mathbf{F} = \mathbf{I}, \Omega = \text{diag}(d_i)$$

where d_i s are arranged in an increasing order and in addition $d_2 = d_1 = 0$. Using this basis we have

$$\hat{\beta} = (\mathbf{F}^T \mathbf{F} + \lambda \Omega)^{-1} \mathbf{F}^T \mathbf{y} = \left(\mathbf{I} + \lambda \text{diag}(d_i) \right)^{-1} \mathbf{F}^T \mathbf{y}$$

i.e.

$$\hat{\beta}_i = \frac{1}{1 + \lambda d_i} \hat{\beta}_i^{LS}$$

Choosing λ

The smoothing parameter is selected by the same criteria used for ridge regression.

- Leave-one-out CV:

$$\text{CV}(\lambda) = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{g}(x_i)}{1 - S_\lambda(i, i)} \right)^2.$$

- Generalized CV:

$$\text{GCV}(\lambda) = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{g}(x_i)}{1 - \frac{1}{n} \text{tr}(S_\lambda)} \right)^2.$$

4.5 The Birthrates Example

In this chapter we relaxed the linearity assumption of classical regression by introducing polynomial regression, regression splines, and smoothing splines. All three approaches remain linear in the parameters, so they can still be fit by (penalized) least squares, yet they are flexible enough to capture nonlinear relationships. We now illustrate smoothing splines—the method that requires the least manual tuning of knots—with a classic data set on U.S. birth rates.

The analysis of this data set is available in R and Python.

Remark:

In the following Python file, you will also find the code for all the R examples illustrated in this chapter: Examples in Python

Chapter 5

Introduction to Neural Networks for Regression

In the previous chapter we discussed **nonlinear regression** in the sense that it is a model that is **linear** in the parameters. We used *bases*, *polynomials*, *splines*, and *kernels* for example to approximate a nonlinear relationship between the response and the predictors. We called this function f before, let's further describe it as

$$f_{\beta}(x) = \beta^{\top} \phi(x)$$

for a feature map ϕ chosen by us. Obviously, if the feature map ϕ is wrong, then ridge, OLS or any other function will not be able to cover for the missing transformation.

In this chapter, **we focus on** ϕ . So, we keep *the loss*, *the linear last layer*, and **learns** ϕ . The resulting predictor is nonlinear in both β and x . The algorithm that makes the gradient of J with respect to β cheap **backpropagation**.

In machine learning courses, it is common to introduce neural networks when one talks about classification, e.g., image or text classification. We are going to view it in the **regression setting**.

In our case, the response $y \in \mathbb{R}$ **is continuous, and it is scored by the MSE**.

5.1 Learning by Fitting a Model

A method uses data E to fit a model f . We say it has **learned** if a performance measure P on a task T is better after seeing E than before:

data $E \longrightarrow$ algorithm $\longrightarrow$ model $f(\mathbf{x})$.

Linear regression already satisfies this definition. Assume that the **Task** T is to predict a house's price from size; the **Measure** P is the MSE, and the **experience** E are the past sales. Learning has occurred *if the test MSE becomes smaller* as more sales are observed.

A **neural net** is the same object, i.e. a predictor f fitted from E , with a different parameterization of f .

Consider a training set $\{(x_i, y_i)\}_{i=1}^n$ with $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. Nonlinear regression means that

$$f_\beta(x) = \beta^\top \phi(x)$$

with criterion

$$J(\beta) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (f_\beta(x_i) - y_i)^2,$$

and β is obtained by a linear method (OLS, ridge, a kernel solver). The map ϕ might be *monomials*, *splines*, a *radial basis*, or an *interaction*.

Two important things to keep in mind:

1. The **loss is a property of y** , not of ϕ . For a real-valued y we use a *squared error*.
2. Once $\phi(x)$ is computed, the last step is ordinary linear regression of y on those features.

Every network below is built so that these remain true. If $a = \phi_c(x)$ denotes the last hidden layer, the prediction is $Wa + b$:

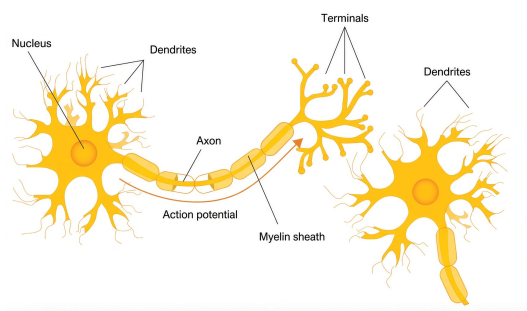
- Freeze c and you are back in the previous chapter.
- Train c jointly with (W, b) and you have a neural net.

The new challenges are statistical as much as algorithmic. The map $\beta \mapsto J(\beta)$ is no longer convex, so two random starts can stop at two different predictors. Adding hidden units (width) or layers (depth) makes the training MSE easier to drive down: the model is larger, and a wide net can always mimic a narrow one.

If you think of the houses example, the same extra capacity can go either way. It helps if the extra units capture signal the smaller net missed (a corner, an interaction). It hurts if they fit noise instead.

5.2 A Neuron: Two ways to think about it

There have always been two motivations for artificial neural nets: to understand biological computation, and to build predictors. In a statistical learning course, we focus on the second. Biology is just for the illustration of how neural nets work:

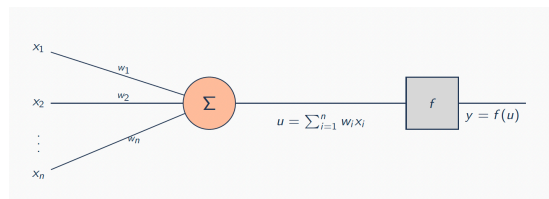


They are many simple units, densely connected, adapting in parallel.

The object we train is $a = \sigma(w^\top x + b)$, stacked, to predict a continuous y .

A Threshold Gate

McCulloch and Pitts (1943) replaced the biology with a **threshold unit**:



Inputs and output are bits,

$$y = \mathbf{1}_{\left\{\sum_{i=1}^n w_i x_i \geq \tau\right\}}, \quad x_i, y \in \{0, 1\}.$$

Networks of these units can compute any Boolean function. That is discrete universality. It does not give a real-valued y , and it does not give a way to fit w from data.

The Perceptron

Rosenblatt (1958) kept the gate and added a **learning rule**. Predict $\hat{y} = \mathbf{1}_{\{w^\top x + b \geq 0\}}$ on a labelled bit $y \in \{0, 1\}$. If the bit is right, do nothing. If it is wrong, move w parallel or anti-parallel to x :

$$\begin{aligned} w &\leftarrow w + \alpha (y - \hat{y}) x \\ b &\leftarrow b + \alpha (y - \hat{y}). \end{aligned}$$

The scalar $y - \hat{y}$ is 0, +1, or -1 . This model introduced the idea of error-driven local change of weights. However, it is not appropriate for a continuous response, since the output is still in $\{0, 1\}$.

The Modern Unit in the Regression Framework

Replace now the bits by *real-valued numbers* and the hard step by a *rectified linear unit* (ReLU):

$$\begin{aligned} z &= w^\top x + b, \\ a &= \sigma(z), \\ \sigma(z) &= \text{ReLU}(z) = \max\{z, 0\}. \end{aligned}$$

Three choices make this a **regression unit** rather than a *perceptron*:

1. The coordinates of x , w , b , and a are real.
2. The last layer of a network often has **no** σ : we want $\hat{y} = w^\top a + b \in \mathbb{R}$, free to be above or below any floor the hidden units have imposed.
3. We train by decreasing the MSE

$$J = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\hat{y}_i - y_i)^2.$$

The factor $1/2$ is used for scaling purposes (it will cancel a 2 later).

The McCulloch–Pitts step is a classifier and is not usefully differentiable. The identity $f(z) = z$ is differentiable, but a stack of identities is one linear map — last chapter’s model with $\phi(x) = x$. ReLU is zero on $(-\infty, 0]$ and the identity on

$[0, \infty)$: one corner, differentiable almost everywhere, cheap to stack. At $z = 0$ any subgradient in $[0, 1]$ is valid; usually software picks 0 or 1 by convention.

5.3 Illustration: Predicting House Prices

Our goal is to predict the selling price $y \in \mathbb{R}$ of a house, in thousands of dollars, from a given set of features: size, bedrooms, zip code, and a neighbourhood wealth index. A snapshot of the data is:

Size (ft ²)	Beds	ZIP	Wealth	Price (\$1000)
850	1	94305	0.4	420
1200	2	94305	0.6	610
2400	4	94301	1.2	980
3100	4	94022	2.1	1640
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Each row corresponds to an observation (x_i, y_i) , i.e. a house.

A linear predictor $f(x) = \beta^\top x$ that uses size has two problems: Nothing stops $\hat{y}$ from going negative for a small house, and a single slope cannot encode a floor such as “*below 1100 ft², extra square footage does not change the price*”.

Last chapter’s fix in such a problem was to put that floor into ϕ by hand: add the column $\text{ReLU}(\text{size} - 1100)$, for example, and run OLS on the expanded features. This chapter’s fix is to leave the columns raw and **let hidden units grow that floor**.

It is worth trying to do the following calculation on one toy example:

- Take $(x, y) = (2, 3)$, $w = 1$, $b = -1$.
- Then $z = wx + b = 1 \Rightarrow a = \text{ReLU}(z) = 1$, and $J = \frac{1}{2}(1 - 3)^2 = 2$.
- Since $z > 0$, $\text{ReLU}'(z) = 1$, so

$$\frac{\partial J}{\partial a} = -2, \quad \frac{\partial J}{\partial z} = -2, \quad \frac{\partial J}{\partial w} = -4, \quad \frac{\partial J}{\partial b} = -2.$$

A step of size $\alpha = 0.1$ yields $w \leftarrow 1.4$ and $b \leftarrow -0.8$. The next a moves toward 3. That is **backpropagation on one scalar unit**.

The simulated house prices can be downloaded here [houses.csv](#). A detailed analysis of the house prices data can be found here: [R](#) and [Python](#).

5.4 Same Loss, New f_β

The training process is the same as before:

1. You pick a loss $J(\beta)$ that is small when $\hat{y}$ is close to y .
2. You compute a gradient.
3. You take a step that decreases J .

What changed is only how $\hat{y} = f_\beta(x)$ is computed.

For a real-valued price the natural loss is still a squared error. For observation i ,

$$J_i(\beta) = \frac{1}{2}(f_\beta(x_i) - y_i)^2.$$

On the whole training set we average over all the data:

$$J(\beta) = \frac{1}{n} \sum_{i=1}^n J^{(i)}(\beta).$$

The factor $1/2$ is just a scaling to make the “2” from differentiation of the square disappear.

The factor $1/n$ is a convention. Dividing by n does not move the β that minimizes J . It does change, however, the size of ∇J , so a step size α that worked for the sum will be too small for the average.

The formula for J is the ordinary least-squares loss. If you freeze the hidden layers and treat the last layer as OLS on features $\phi(x)$, you are minimizing exactly this J . A neural net minimizes the same J ; it just uses a more complicated f_β .

To minimize J we still move β in the direction of $-\nabla J$. In other words, the solver is no longer “solves the normal equations”, but rather “follows $-\nabla J$ ”. Since we do not get a closed form, we have to follow the gradient steps.

Three common updates differ only in how many observations (houses in our example) enter the gradient:

- **Full Gradient Descent** uses every training point at every step:

$$\beta \leftarrow \beta - \alpha \nabla_\beta J(\beta).$$

The direction is exact (for this J), but each step costs a pass over all n houses.

- **Stochastic Gradient Descent** picks one index j at random and pretends $J^{(j)}$ is the whole objective:

$$\beta \leftarrow \beta - \alpha \nabla_{\beta} J^{(j)}(\beta).$$

Each step is cheap and noisy. Under standard conditions the noise averages out and β still heads toward a stationary point.

- **Mini-batch SGD** is the compromise used in practice. Draw B indices and average their gradients:

$$\beta \leftarrow \beta - \frac{\alpha}{B} \sum_{k=1}^B \nabla_{\beta} J^{(j_k)}(\beta).$$

$B = 32$ or $B = 64$ is typical.

Remark:

None of these procedures inherits the convexity of ridge. In the previous chapter, $J(\beta) = \|y - \Phi\beta\|^2 + \lambda\|\beta\|^2$ had one valley. Here f_{β} is a composition of nonlinear maps, so J can have many valleys. Two random starts can stop at two different predictors with similar training loss and different test loss. That is not a reason to avoid gradient methods. It is a reason to try more than one starts, and decide the “winner” on a test set rather than on training J alone.

5.5 Stacking Units is Learning ϕ

5.5.1 Hidden Units

Recall the house prices example, and take one house with 4 features, e.g.

$$x = (\text{size}, \text{\#beds}, \text{zip}, \text{wealth}) \in \mathbb{R}^4$$

In this scenario, *imagine* **three hidden quantities**

$$\begin{aligned} a_1 &= \text{ReLU}(\beta_1 x_1 + \beta_2 x_2 + \beta_3) && \text{“family size”,} \\ a_2 &= \text{ReLU}(\beta_4 x_3 + \beta_5) && \text{“walkable”,} \\ a_3 &= \text{ReLU}(\beta_6 x_3 + \beta_7 x_4 + \beta_8) && \text{“school quality”,} \end{aligned}$$

and a linear $\bar{f}_{\beta}(x) = \beta_9 a_1 + \beta_{10} a_2 + \beta_{11} a_3 + \beta_{12}$.

The labels “family size” and “school quality” are only nicknames to make the picture readable. Fitting the net does not force a_3 to mean school quality, and you should not read the hidden units that way.

What is forced is that the last-layer is still a linear model: once $a = (a_1, a_2, a_3)$ is in hand, $\hat{y}$ is ordinary linear regression of price on a . The new piece is that a is not a column in the “design matrix”. It is computed from the raw x , and the weights inside that computation are part of β .

5.5.2 Fully Connected Layers

In fully connected layers, we stop choosing which inputs feed the hidden unit. A fully connected layer does the opposite: every hidden unit is allowed to use every coordinate of $x \in \mathbb{R}^d$, and training decides which connections matter.

For $j = 1, \dots, m$,

$$z_j = (w_j^{[1]})^\top x + b_j^{[1]}, \quad a_j = \text{ReLU}(z_j), \quad \bar{f}_\beta(x) = (w^{[2]})^\top a + b^{[2]}.$$

Superscripts [1] and [2] label layers of parameters.

Stack the first-layer weights as rows of $W^{[1]} \in \mathbb{R}^{m \times d}$. Then

$$a = \text{ReLU}(W^{[1]}x + b^{[1]}), \quad \bar{f}_\beta(x) = W^{[2]}a + b^{[2]}.$$

That is a *one-hidden-layer net*. A **deeper net** iterates the same pattern: set $a^{[0]} := x$ and

$$a^{[k]} = \text{ReLU}(W^{[k]}a^{[k-1]} + b^{[k]}), \quad k = 1, \dots, r-1,$$

$$\bar{f}_\beta(x) = W^{[r]}a^{[r-1]} + b^{[r]}.$$

If $a^{[k]} \in \mathbb{R}^{m_k}$ then $W^{[k]} \in \mathbb{R}^{m_k \times m_{k-1}}$ and $b^{[k]} \in \mathbb{R}^{m_k}$, with $m_0 = d$. For scalar regression, $m_r = 1$. The parameter count, including biases, is

$$(d+1)m_1 + (m_1+1)m_2 + \dots + (m_{r-1}+1)m_r.$$

The name **multilayer perceptron** is historical, since these are not Rosenblatt perceptrons. They are the previous chapter’s linear model on top of a ϕ_c that is itself an iterated affine-plus- σ map.

If we write c for every parameter except the last layer, then $a^{[r-1]} = \phi_c(x)$ and

$$\bar{f}_\beta(x) = W^{[r]} \phi_\beta(x) + b^{[r]}.$$

- Freeze c : nonlinear regression with a fixed feature map.
- Train c : the feature map is part of the estimate.

The **statistical cost** of the second option is that ϕ_c can interpolate noise. The **statistical gain** is that ϕ_c can discover a corner or an interaction that we omitted.

5.6 Activations, Depth, and Approximation

ReLU is the hidden-unit nonlinearity we use in this chapter. Most other names you will hear are variations on the same idea: take the weighted sum $z = w^\top x + b$, then squash or bend it before passing it on.

Two older choices are the sigmoid $1/(1 + e^{-z})$ and $\tanh$. Both flatten out as $z \rightarrow \pm\infty$: the unit's output barely changes, and the derivative is almost 0. Gradients then die as they travel back through the layer. That is what “saturate” means. Those functions are still useful when you want an output in $(0, 1)$ or $(-1, 1)$, that is a probability, or a gate that turns a signal up or down, but they are a poor default in the middle of a regression net.

Leaky ReLU is ReLU with a small slope on the negative side instead of a hard zero, so a unit that lands below 0 can still send a little gradient. Swish (also called SiLU), GELU, and softplus are smooth versions of ReLU: same general shape, rounded at the corner so the derivative is defined everywhere.

A gated linear unit (GLU) is a different object. It is not a function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$. It takes a vector f , builds two linear maps of f , sends one through a sigmoid-like gate g , and multiplies the two pieces coordinatewise. Large language models use blocks of this form. You do not need it for a continuous response.

If σ is the Identity, Depth does nothing

If σ is the identity, then even with two layers

$$\bar{f}_\beta(x) = W^{[2]}W^{[1]}x = \widetilde{W}x.$$

Composition of linear maps is linear in which case Depth adds nothing and ϕ is the identity. So, we are back at OLS on raw x . Nonlinearity is what makes a hidden layer more than a change of basis that could have been absorbed into β .

Why one ReLU is not enough, and what “enough” means

One ReLU is one **corner**. A target with two corners/bends needs at least two hidden units. In one dimension, a one-hidden-layer ReLU net with m units is piecewise linear with at most m corners. Width adds pieces while depth composes pieces.

A standard universal-approximation result (Cybenko 1989; Hornik 1991) says that one hidden layer with a non-polynomial activation, and enough units, can approximate any continuous function on a compact set of $\mathbb{R}^d$ to arbitrary uniform accuracy.

A polynomial net (last chapter’s bases) builds curves from powers of x . A ReLU net builds curves from straight pieces joined at corners: on each region the map is linear, and the regions are cut out by the hyperplanes $w^\top x + b = 0$. So, ReLU networks have the same kind of density, with continuous piecewise-linear maps.

The theorem does **not** give a width, a rate, or a training algorithm. It does not say the SGD minimizer of J on n noisy y ’s will be close to the true regression function. It plays the same role here that Weierstrass played for polynomials: we already know the model class is rich enough to approximate reasonable functions. Whether a given n and a given width estimate something useful is a question that needs to be investigated empirically.

5.7 Backpropagation

This section is more technical than the rest of the course and is presented here for completeness. *There will be no quiz questions associated with it; it is just the explanation of how the method works.*

In the previous chapter, we already discussed that $f_\beta(x) = \beta^\top \phi(x)$ was linear in β , in which case ∇J was a matrix times a residual. Here, f_β is a composition,

$$x \rightarrow z^{[1]} = W^{[1]}x + b^{[1]} \rightarrow a^{[1]} = \sigma(z^{[1]}) \rightarrow \dots \rightarrow \hat{y},$$

and $J = \frac{1}{2}(\hat{y} - y)^2$. The chain rule still applies, but you take it **from the loss back to the inputs**.

On one observation the last residual is $\hat{y} - y$. That number is multiplied by the local slope of σ and by the previous layer’s weights, layer by layer. Each layer produces two things:

- the gradient of J with respect to *its* W, b (what you use to take a step) and
- the gradient with respect to its input (what the previous layer needs). That sweep is **backpropagation**.

You store the forward values $z^{[k]}, a^{[k]}$ because the local slopes depend on them.

The only complexity fact that matters: if scoring one observation needs N operations, getting every partial of J takes $O(N)$ operations as well, not one derivative per parameter. That is why treating ϕ_c as part of β is practical.

5.7.1 The Theorem Behind

A neural net is already a finite recipe of additions, multiplications, and maps such as ReLU: you evaluate it from the inputs toward $\hat{y}$, with no loops. It is also a **directed acyclic graph** since each node is a number, each arrow is one of those operations. **Backpropagation** is the statement that the same graph yields every partial of J at essentially the cost of one extra forward pass, not at d times the cost.

A **differentiable circuit of size N** is a directed acyclic graph of $+$, $-$, $\times$, $/$, and elementary maps (ReLU, exp, log, ...), each of which can be computed with its local derivative in constant time. The claim that makes this model class practical is that the gradient of such a circuit is essentially as cheap as the circuit.

Theorem (reverse-mode autodiff / cheap gradient)

Let $f : \mathbb{R}^\ell \rightarrow \mathbb{R}$ be computed by a differentiable circuit of size N , in which every elementary gate and the product of that gate's local derivative with a scalar can be evaluated in $O(1)$ time. Then ∇f can be computed by a circuit of size $O(N)$ in time $O(N)$.

When every gate is just $+$, $\times$, or multiplication by a constant, the same claim has a precise form: Baur and Strassen (1983) showed that if f can be computed with S such operations, then f and ∇f can be computed with at most $5S$ operations. Morgenstern (1985) gave a short proof by induction backward through the circuit. Griewank (1989) extended the idea beyond polynomials, to a general list of elementary functions. What follows is that argument written on a computation graph, i.e. a multilayer perceptron.

Proof

Label nodes $v_1, \dots, v_N$ in topological order. Each v_t is an input coordinate or φ_t applied to $O(1)$ earlier nodes (bounded fan-in). Write $\text{ch}(t)$ for the children of t . The output is the scalar $v_N = f(x)$.

The forward pass evaluates $t = 1, \dots, N$ and stores every v_t . Time and extra memory are $O(N)$.

Define the adjoint $\bar{v}_t := \partial f / \partial v_t$, and set $\bar{v}_N = 1$. The chain rule reads

$$\bar{v}_t = \sum_{s \in \text{ch}(t)} \bar{v}_s \cdot \frac{\partial \varphi_s}{\partial v_t}(v_{\text{pa}(s)}).$$

Each local factor depends only on stored parents of s and, by hypothesis, can be multiplied by $\bar{v}_s$ in constant time. Sweeping $t = N - 1, \dots, 1$ therefore fills every adjoint after its children's adjoints are known.

Each edge contributes one local derivative and one multiply-add, hence $O(N)$. The input adjoints are the coordinates of $\nabla f(x)$.

Correctness is induction down the reverse order. The base $\bar{v}_N = 1$ is tautological. If every child of t holds a correct adjoint, the displayed sum *is* the chain rule. Nodes off the path to the output stay at their initialized value 0. Attaching an adjoint node and $O(1)$ extra gates to each original node produces a circuit of size $O(N)$.

Why backpropagation?

Forward-mode autodiff pushes one directional derivative from the inputs to the output in $O(N)$ time. The full gradient then costs $O(\ell N)$. Reverse mode costs $O(N)$ because the output is a scalar. For maps $\mathbb{R}^\ell \rightarrow \mathbb{R}^m$ the costs swap. Fitting a network is $\ell \gg m = 1$: many parameters, one loss. Differentiating coordinate by coordinate is forward mode d times. Reverse mode recycles one forward tape across all coordinates.

If f is cheap, so is any scalar function of ∇f , including a one-step unrolled update $\ell(\beta - \eta \nabla \ell(\beta))$. If $g(x) = \langle \nabla f(x), v \rangle$, then $\nabla g(x) = \nabla^2 f(x) v$ costs $O(N + \ell)$: Hessian-vector products without forming the Hessian.

Modules, not Jacobians

If a scalar J depends on a tensor z , $\partial J / \partial z$ has the same shape as z . Differentiate scalars with respect to tensors. Do not materialize the Jacobian of a vector-valued map unless there is no other option.

For $u = g(z)$ and $J = f(u)$,

$$\frac{\partial J}{\partial z} = (Dg)^\top \frac{\partial J}{\partial u}.$$

The linear map $v \mapsto (Dg)^\top v$ is the **backward function** $B[g, z]$ of the module g at the stored input z . A network is a composition $J = M_k \circ \dots \circ M_1$. Forward: store each $u^{[i]} = M_i(u^{[i-1]})$. Backward: start from $\partial J / \partial u^{[k]} = 1$ and apply $B[M_i, \cdot]$ to move the adjoint onto the previous activation and onto the parameters inside M_i .

For $u = Wz + b$ with incoming adjoint $v = \partial J / \partial u$,

$$B[\text{MM}, z](v) = W^\top v, \quad B[\text{MM}, W](v) = vz^\top, \quad B[\text{MM}, b](v) = v.$$

The middle identity is the one people check once. Entrywise, $\partial u_k / \partial W_{ij}$ equals z_j if $k = i$ and equals 0 otherwise, so $(\partial J / \partial W)_{ij} = v_i z_j$, which is the outer product $vz^\top$. Shapes: v is $n \times 1$, $z^\top$ is $1 \times m$, $vz^\top$ is $n \times m$.

For an elementwise σ , $B[\sigma, z](v) = \sigma'(z) \odot v$ in linear time. For squared error, $\partial J / \partial z = z - y$ at the top of the backward pass (up to the conventional $1/2$).

The MLP in two Directions

Forward, storing every $z^{[k]}$ and $a^{[k]}$:

$$z^{[1]} = W^{[1]}x + b^{[1]}, \quad a^{[1]} = \sigma(z^{[1]}), \quad \dots, \quad z^{[r]} = W^{[r]}a^{[r-1]} + b^{[r]}, \quad J = \frac{1}{2}(z^{[r]} - y)^2$$

if the head is linear. Backward, starting from $\partial J / \partial z^{[r]} = z^{[r]} - y$ and writing $a^{[0]} = x$,

$$\begin{aligned} \frac{\partial J}{\partial W^{[k+1]}} &= \frac{\partial J}{\partial z^{[k+1]}} (a^{[k]})^\top, \\ \frac{\partial J}{\partial b^{[k+1]}} &= \frac{\partial J}{\partial z^{[k+1]}}, \\ \frac{\partial J}{\partial a^{[k]}} &= (W^{[k+1]})^\top \frac{\partial J}{\partial z^{[k+1]}}, \\ \frac{\partial J}{\partial z^{[k]}} &= \sigma'(z^{[k]}) \odot \frac{\partial J}{\partial a^{[k]}}. \end{aligned}$$

From memory, one layer: $z = Wa + b$, $a_{\text{out}} = \sigma(z)$; then $\delta \leftarrow (W^+)^\top \delta \odot \sigma'(z)$, $\nabla W \leftarrow \delta a^\top$, $\nabla b \leftarrow \delta$.

On a mini-batch, stack examples as columns of $X \in \mathbb{R}^{d \times B}$. Then $Z = WX + b$ with b broadcast, and $\partial J / \partial W = VX^\top$ if V holds the incoming adjoints as columns. Papers use that column convention. NumPy and PyTorch put batch first, so $X \in \mathbb{R}^{B \times d}$ and $Z = XW + b$. Transpose every matrix and flip every product; do not mix conventions.

5.8 Capacity, Nonconvexity, and Penalty

In the previous chapter $J(\beta)$ for OLS or ridge is convex. The $J(\beta)$ of even a shallow net is not! Hidden units can be permuted, signs can flip when σ is odd, and entire regions of parameter space implement essentially the same function. SGD from two seeds can stop at two different predictors with similar training

loss and different test loss. The practical response is the one already used for bandwidths and penalties: several starts, a holdout, and a simple regularizer.

Width and depth are capacity. A one-hidden-layer net with m units can form up to m corners in one dimension and can interpolate n points once m is large enough relative to n . Interpolation is not automatically bad, but it is a reason to look at test MSE as m grows, not only at training J .

The regularizer that sits closest to last chapter is **weight decay**. Adding $\lambda \sum_k \|W^{[k]}\|_F^2$ to J is ridge on the weights. In `nnet` this is the `decay` argument and it pulls hidden units toward zero.

This does not replace a test set!

5.9 When to specify ϕ instead of learning it!

A neural net is the wrong choice on a small tabular regression problem with a few smooth effects. If you already know that price is linear in size above 1100 ft² and linear in an interaction `beds × wealth`, type those columns and run OLS or ridge. You will get a standard error, an identifiable coefficient, and a model you can interpret. The net will try to rediscover the same ϕ_β , but it will not be clear how.

Nets are useful when ϕ is challenging: many inputs, unknown interactions, or a representation you intend to reuse. The next chapter adds a third option. A regression tree will cut if the data want that cut, without requiring a gradient through a corner. Trees and nets are two ways to let the data propose ϕ , while in the previous chapter we (as statisticians) proposed ϕ .

Chapter 6

Regression Trees

In this section, we are going to discuss **tree-based** models for **regression**.

Framework

- Response Variable: $Y \in \mathbb{R}$ – continuous
- Features: $X = (X_1, X_2, \dots, X_p)$

Regression trees are *non-linear* and *non-parametric* methods. They recursively partition the feature space into hyper-rectangular subsets and fit a simple prediction in each piece. There is no single global form such as “ Y is linear in X ”; complexity is controlled by tuning parameters (tree size, and later the ensemble size). For this chapter both the split criterion and the leaf prediction use **squared error**. Classification trees (Gini, deviance) will wait.

Below we can see a sketch of a regression tree for a **wines** data set in which we want to predict the *quality* of the wine (a numerical outcome) versus *wine characteristics* (a set of predictors):

We will restrict our focus to *binary* partitions of the tree. In the example above, we first split the space into **two** regions and model the response by the mean of Y in each region. *At each step*, we choose the variable and split-point to achieve the best fit. Then one or both regions are split into two more regions, and this process continues until *some* stopping rule is applied. In the example, the first split is whether **alcohol** is less than or greater than 10.85. The region with $\text{alcohol} < 10.85$ is then split on **volatile.acidity** at 0.2525, and the region to the right is split on **alcohol** again at 11.7417, and so on. When we stop, the predicted **quality** is the average of Y in that region.

Terminology

- **Leaf Node:** Each leaf node corresponds to a rectangular-shaped region in the *feature space*.
- **Internal Node:** These nodes are parent nodes with two children – left and right nodes.
- A **split** involves a variable j and a particular split value s .

A single tree is easy to interpret: a prediction is a short sequence of threshold rules. Splits depend only on the ordering of a predictor, so any monotone transformation of a predictor leaves the tree unchanged. The same splitting algorithm can be used when p is large, interactions appear when different variables are chosen along a path, and predictors that are never selected are effectively ignored.

The usual weakness is accuracy. One tree is often a high-variance fit; that is the reason for the ensemble methods later in this section.

6.1 Single-Tree Building

As we discussed, to build a single tree one recursively partitions the predictor space. Start by **initializing the root** with all of the training data. Then choose a **splitting rule** consisting of a predictor X_j and a constant c that splits the observations in the node:

$$\mathbf{1}_{\{X_j \leq c\}}$$

Then apply the same procedure to each child node:

We denote a node by (t) and its children by (t_L, t_R) . A split is a pair (variable j , cutpoint s), and the procedure ends at the leaves (terminal nodes). When we stop, we **predict** in each terminal node using the data in that node. Every leaf (a rectangular region $R_m \subset \mathcal{X}$) is assigned a constant, so a regression tree has the form

$$\hat{f}(X) = \sum_m c_m \mathbf{1}_{\{X \in R_m\}}.$$

The natural questions are:

1. **Where** to split the tree?
2. **When** to stop?
3. **How** to predict the response at each leaf node?

6.1.1 Prediction at Leaf Nodes

Each leaf corresponds to a region R_m that contains some training points. The **prediction** in that leaf is the *average of the response Y* for the observations in R_m , $m = 1, \dots, M$:

$$\hat{f}(X) = \sum_{m=1}^M c_m \mathbf{1}_{\{X \in R_m\}}.$$

The constants c_m are chosen by minimizing squared error in the leaf:

$$\min_{c_m} \sum_{i: x_i \in R_m} (y_i - c_m)^2.$$

The minimizer is the average of those y_i with $x_i \in R_m$.

6.1.2 Split Criterion

Finding a **global** partition of $\mathcal{X}$ with minimum residual sum of squares is computationally infeasible, so we use a greedy algorithm. This search is not guaranteed to find the piecewise-constant fit with globally smallest RSS; it only chooses the best single split at the current node. Start with all of the data at the root. A split variable j and split point s produce the half-planes

$$R_1(j, s) = \{X : X_j \leq s\} \quad \text{and} \quad R_2(j, s) = \{X : X_j > s\}.$$

We then choose the pair (j, s) that solves

$$\min_{j, s} \left[\min_{c_1} \sum_{x_i \in R_1(j, s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j, s)} (y_i - c_2)^2 \right].$$

For any fixed j and s , the inner minimization is solved by

$$\hat{c}_1 = \text{ave}(y_i : x_i \in R_1(j, s)) \quad \text{and} \quad \hat{c}_2 = \text{ave}(y_i : x_i \in R_2(j, s)).$$

For each splitting variable, the best cutpoint s can be found quickly by scanning the ordered unique values, so searching over all pairs (j, s) is feasible.

The same idea can be written as a *split criterion* $\Phi(j, s)$, here the **reduction in RSS**. Regression trees are grown *top-down* and *greedily*:

- Start at the *root* and consider
 - a. all variables $j = 1, \dots, p$ and
 - b. all candidate split values,
 - c. and pick the split with the best Φ .
- The data in the node are then sent to a left child and a right child.
- Repeat the procedure in each child.

Split Criterion $\Phi(j, s)$

Reduction in RSS if we split the samples at node t into t_L and t_R :

$$\Phi(j, s) = RSS(t) - [RSS(t_R) + RSS(t_L)]$$

where

$$RSS(t) = \sum_{x_i \in t} (y_i - c_t)^2, \quad \text{with} \quad c_t = \text{ave}\{y_i : x_i \in t\}.$$

Note that $\Phi(j, s) \geq 0$, with equality if and only if the two child means are equal.

Categorical Predictors

A categorical predictor with m levels can be used directly as a split variable. In principle there are $2^{m-1} - 1$ ways to assign the m labels to the left and right child. That search is feasible only for small m .

For regression with squared error there is a shortcut: order the m levels by their mean response, then treat the variable as if it were ordered. Only the $m - 1$ adjacent cuts need to be checked, and the best of those is the best binary partition of the labels.

Missing Predictor Values

Tree algorithms can handle missing predictors without discarding whole rows, which would otherwise shrink the training set quickly.

One approach is to evaluate the split using only the observations with X_j observed. After a split (j, s) is chosen, missing X_j can be filled in by an imputation rule.

A more specifically CART-style option is to build *surrogate* splits: other variables (and a majority rule) that predict the event $\{X_j \leq s\}$. An observation missing X_j is sent left or right by the best available surrogate, then the next, and so on.

6.1.3 Single-Tree Pruning

We have a leaf prediction and a split criterion. What remains is *when to stop*, which determines how large the tree is.

For a binary tree write $|T|$ for the number of **terminal nodes** (leaves). The number of splits is then $|T| - 1$. A large tree can overfit, but a small tree may miss structure that only appears after a weak first split. Balancing fit and size is the same “loss + penalty” idea as in penalized regression.

One possible rule is to split a node only if the decrease in RSS exceeds a certain threshold. That rule is short-sighted: a split that looks useless now may be followed by a valuable split below. The standard strategy is therefore to grow a large tree and then **prune**.

We start by growing a very large tree $T_{\max}$:

- until all terminal nodes are nearly pure, that is, the variability of Y in the node is small;
- or until the number of observations in each terminal node is below a threshold;
- or until the tree reaches a prescribed size.

The exact size of $T_{\max}$ is not critical as long as the tree is *large enough*.

Notation

Subtree $T' \preceq T$: any tree obtained from T by pruning, i.e. by collapsing some branches. Branch at node (t) : T_t

For any subtree $T \preceq T_{\max}$ define the **complexity cost**

$$R_\alpha(T) = R(T) + \alpha|T|,$$

where $R(T)$ is the RSS of T , $|T|$ is the number of leaves, and $\alpha \geq 0$ is the cost of a leaf. For each fixed α let

$$\begin{aligned} T(\alpha) &= \arg \min_{T \preceq T_{\max}} R_\alpha(T) \\ &= \arg \min_{T \preceq T_{\max}} [R(T) + \alpha|T|]. \end{aligned}$$

If several trees attain the minimum, take the smallest; that subtree $T^*(\alpha)$ is unique. We do **not** pick α by minimizing R_α on the training data. We compute the path $T^*(\alpha)$ and choose α by cross-validation.

For a pair of sister leaves (t_L, t_R) there is a critical value α^* such that

- for any $\alpha \geq \alpha^*$, we prefer to collapse them back to the parent t ;
- for any $\alpha \leq \alpha^*$, we prefer to keep the two leaves.

So α^* is *the highest price we are willing to pay to keep that split*. The same calculation extends to a whole **branch** T_t . Using only the samples that fall in t ,

- the cost of keeping the branch is

$$R_\alpha(T_t) = R(T_t) + \alpha|T_t|;$$

- the cost of cutting the branch (making t a leaf) is

$$R_\alpha(\{t\}) = R(\{t\}) + \alpha.$$

Equating the two costs gives

$$\alpha^* = \frac{R(\{t\}) - R(T_t)}{|T_t| - 1}.$$

If the working value of α exceeds this α^* , the branch is too expensive and t becomes a leaf.

Searching every subtree of $T_{\max}$ would be expensive, but it is unnecessary. *Weakest-link pruning* removes the least valuable branch at each step and produces a nested sequence of trees – the **cost-complexity path**. That path is what `cptable` and `plotcp` display later in `rpart`.

Weakest Link Pruning Algorithm

1. *Start* with $T_0 = T_{\max}$ and $\alpha_0 = 0$.
2. For any non-leaf node t ,

$$\alpha(t) = \text{the maximal price you would pay to keep } T_t.$$

3. $\alpha_1 = \min_t \alpha(t)$ The corresponding non-terminal node t_1 is called the **weakest link**.
4. *Cut* the branch at t_1 .
5. *Update* the maximal price for each non-leaf node (only the ancestors of t_1 change).
6. *Find* α_2 and cut the branch at the next weakest link.
7. *Keep* doing this until only the root remains.

The algorithm produces the nested path

$$T_{\max} = T_0 \succ T^*(\alpha_1) \succ T^*(\alpha_2) \succ \dots \succ \{\text{root node}\}$$

$$0 = \alpha_0 < \alpha_1 < \alpha_2 < \dots$$

All $\alpha \geq 0$ fall into $m + 1$ intervals

$$\begin{aligned} I_0 &= [0, \alpha_1) \\ I_1 &= [\alpha_1, \alpha_2) \\ &\vdots \\ I_m &= [\alpha_m, \infty) \end{aligned}$$

and every $\alpha \in I_i$ shares the same optimal subtree $T^*(\alpha_i)$.

Tuning of the hyperparameter α

Use K -fold cross-validation to choose α . Grow $T_{\max}$ on the full training data and compute the intervals $I_0, I_1, \dots, I_m$. Representative values in those intervals

are

$$\begin{aligned}\beta_0 &= 0 \\ \beta_1 &= \sqrt{\alpha_1 \alpha_2} \\ &\vdots \\ \beta_{m-1} &= \sqrt{\alpha_{m-1} \alpha_m} \\ \beta_m &= \infty,\end{aligned}$$

so each β_j stands for I_j .

For K -fold CV, partition the data into K folds and repeat $k = 1, \dots, K$. On the data with fold k held out, grow a large tree and extract the path

$$T_0 \succ T^*(\beta_1) \succ \dots \succ T^*(\beta_m) \succ \{\text{root node}\}.$$

Compute the prediction error of each of those trees on fold k . Average over folds to obtain a CV curve in α , and pick $\alpha_{\min}$ or α_{1se} .

6.2 Regression Forests

Regression forests extend a single tree by combining many trees into an **ensemble**. Two sources of randomness are used. First, each tree is grown on a bootstrap sample of the observations (bagging). Second, in a random forest, each split searches only a random subset of the p predictors. The extra randomness makes the trees less correlated. The ensemble prediction is the average of the individual trees, which typically lowers variance relative to one deep tree and improves accuracy and stability. The simple picture of a single tree is lost.

6.2.1 Weak and Strong Learners

Single trees have two familiar drawbacks: they are unstable – small changes in the data can produce very different trees – and they often lack accuracy. In that sense a single tree is a **weak learner**.

A standard response is to combine many weak learners into a **strong learner**. Ensemble methods of this type include bagging, random forests, and boosting.

Bagging and random forests grow trees under different *random perturbations* and *average* their predictions. Boosting is sequential: it *progressively* fits simple models and *adds* them together.

Conceptually, an **ensemble** of trees looks like this:

6.2.2 Bootstrap Aggregation (Bagging)

Bagging (bootstrap aggregation) is an ensemble method designed to reduce the variance of unstable predictors. It repeatedly *resamples* the training data *with*

replacement, fits a model on each bootstrap sample, and averages the predictions (regression) or takes a majority vote (classification).

Write the training data as $\mathbf{Z} = \{(x_i, y_i) : i = 1, \dots, n\}$. Draw **bootstrap samples** $\mathbf{Z}^{*b} = \{(x_i^{*b}, y_i^{*b}) : i = 1, \dots, n\}$ for $b = 1, \dots, B$,

$$\begin{aligned} \mathbf{Z}^{*1} &: (x_1^{*1}, y_1^{*1}), (x_2^{*1}, y_2^{*1}), \dots, (x_n^{*1}, y_n^{*1}) \\ \mathbf{Z}^{*2} &: (x_1^{*2}, y_1^{*2}), (x_2^{*2}, y_2^{*2}), \dots, (x_n^{*2}, y_n^{*2}) \\ &\vdots \\ \mathbf{Z}^{*B} &: (x_1^{*B}, y_1^{*B}), (x_2^{*B}, y_2^{*B}), \dots, (x_n^{*B}, y_n^{*B}). \end{aligned}$$

Fit $\hat{f}^{*b}$ on $\mathbf{Z}^{*b}$. The **bagging estimate** is

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x).$$

Averaging many trees can cut variance substantially. Two caveats: the readable structure of one tree is lost, and if the trees stay highly correlated the variance reduction is limited.

6.2.3 Random Forests

Random forests grew out of several 1990s ideas. Breiman's bagging showed that averaging unstable predictors such as trees can reduce variance. Ho's random subspace method (1998) showed that further *diversity* is available by training each tree on a random subset of features. Breiman combined the two in **random forests**: bagging plus a random subset of predictors *at every split*. The result is a large collection of de-correlated trees whose average is typically both more accurate and more stable than a single tree.

The algorithm has a few moving parts:

1. Bootstrap sampling (bagging): each tree is grown on a bootstrap sample of the training data.
2. Random feature selection: at each split, only a random subset of the p features is eligible. That extra randomness decorrelates the trees and improves the variance reduction from averaging.
3. Tree growth: forest trees are usually grown deep and not pruned, so that each tree can be highly flexible. Overfitting of any one tree is less of a concern because the average stabilizes the fit.
4. Prediction: for regression, average the B tree predictions.

5. Out-of-bag (OOB) error: each bootstrap sample leaves out about one-third of the rows. Those left-out observations give a built-in estimate of prediction error, close in spirit to cross-validation.

Sketch of the Algorithm

For $b = 1, \dots, B$

1. Draw a bootstrap sample of size n , $\mathbf{Z}^{*b}$, from the training data.
2. Grow a large tree T_b by repeating the following steps at each terminal node until a minimum node size is reached:
 1. Select m variables at random from the p variables.
 2. Pick the best variable and split-point among those m .
 3. Split the node into two children.

Output the forest $\{T_b\}_{b=1}^B$.

Predict at a new point x (regression)

$$\hat{f}^B(x) = \frac{1}{B} \sum_{b=1}^B T_b(x).$$

Restrictions

At every split, a subset of m features is drawn from the p available features, and the best split is chosen only among those m . A common rule of thumb is $m = \sqrt{p}$ for classification and $m = p/3$ for regression.

Restricting the candidate features makes it less likely that every tree leans on the same strong predictors, which lowers correlation among trees and helps the average.

6.2.4 Out-of-Bag (OOB) Samples

Out-of-bag samples are the observations that do not appear in $\mathbf{Z}^{*b}$ and so are not used to grow T_b . They are a natural test set for that tree. The OOB error aggregates, for each observation i , only the trees that did not train on i . Packages such as `randomForest` report predictions and error rates from this calculation. The estimate is typically close to K -fold cross-validation.

6.2.4.1 Variable Importance

Two common measures:

1. Reduction in RSS (impurity): each split records the drop in RSS and attributes that drop to the variable used. Sum those drops over all splits in a tree, then average over trees. This measures how much the variable is used to reduce training error. It tends to favor variables that offer many candidate cutpoints (numeric variables, or factors with many levels).
2. Permutation of predictors: for each tree T_b , compute the OOB prediction error (MSE for regression). Randomly permute predictor j in the OOB data and recompute the error. The increase is a measure of importance for j . Average the increase over trees; `randomForest` can also divide by the standard deviation of those increases (a scaled importance). Permutation importance can be misleading when predictors are highly correlated: the forest may split on one of several substitutes, and permuting a single j then understates the role of the group.

6.2.5 Boosting Trees

Boosting also builds an ensemble of trees, but the trees are grown **sequentially**. Each new tree is aimed at the errors of the current fit, rather than at a fresh bootstrap sample.

Write an additive model

$$F(x) = f_1(x) + f_2(x) + \dots + f_T(x).$$

Fitting all T terms at once is hard. Forward stagewise fitting adds one term at a time. For squared error, tree t is fit to the current residuals and then added with a learning rate λ :

$$F_t(x) = F_{t-1}(x) + \lambda \hat{h}_t(x).$$

Small λ and a large T , with T chosen by cross-validation, is the usual regularization.

Boosting often predicts better than bagging, and it can capture complex non-linear structure. It is more sensitive to noise and outliers than bagging, and the hyperparameters (`n.trees`, `shrinkage`, tree depth) need more care.

6.3 Tree-Based Regression Models in R

The Python version of this section can be found here: [Python Code for Regression Trees](#). It replicates (in spirit) the work that is done in this section. It also comments on differences between the two languages.

```
library(ggplot2)
library(rpart)
library(rpart.plot)
library(tree)
```

The `whitewines.csv` data set contains information on white variants of the Portuguese “Vinho Verde” wine. The columns are: (a) fixed acidity, (b) volatile acidity, (c) citric acid, (d) residual sugar, (e) chlorides, (f) free sulfur dioxide, (g) total sulfur dioxide, (h) density, (i) pH, (j) sulphates, (k) alcohol, (l) quality (score between 0 and 10). The data set can be downloaded here: [whitewines.csv](#)

```
whitewines = read.table("data/week6/whitewines.csv", sep = ";",
  header = TRUE)
head(whitewines)
```

```
##   fixed.acidity volatile.acidity citric.acid residual.sugar chlorides
## 1           7.0           0.27         0.36           20.7      0.045
## 2           6.3           0.30         0.34           1.6      0.049
## 3           8.1           0.28         0.40           6.9      0.050
## 4           7.2           0.23         0.32           8.5      0.058
## 5           7.2           0.23         0.32           8.5      0.058
## 6           8.1           0.28         0.40           6.9      0.050
##   free.sulfur.dioxide total.sulfur.dioxide density    pH sulphates alcohol
## 1                  45                  170 1.0010 3.00      0.45      8.8
## 2                  14                  132 0.9940 3.30      0.49      9.5
## 3                  30                   97 0.9951 3.26      0.44     10.1
## 4                  47                  186 0.9956 3.19      0.40      9.9
## 5                  47                  186 0.9956 3.19      0.40      9.9
## 6                  30                   97 0.9951 3.26      0.44     10.1
##   quality
## 1        6
## 2        6
## 3        6
## 4        6
## 5        6
## 6        6
```

```
dim(whitewines)
```

```
## [1] 4898   12
```

Two R packages fit **tree** models here: **tree** and **rpart**. We use **tree** only for **partition.tree**, which produced the first figures in the notes. The remaining code uses **rpart**.

A Simple Regression Tree

In R we use the **tree** package for this two-predictor illustration. The syntax for **tree** is similar to that for a linear model. For illustration, predict **quality** from **alcohol** and **volatile.acidity** only.

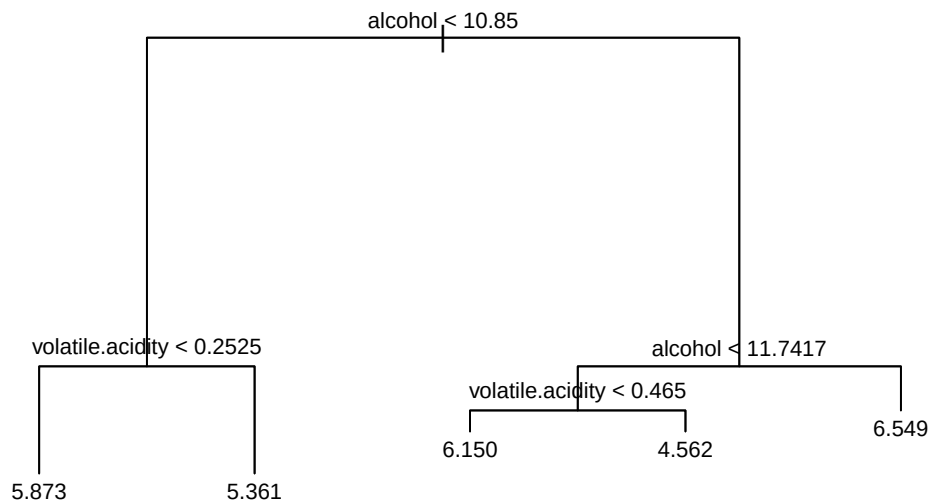
Fit the tree:

```
quality.tree = tree(quality ~ alcohol + volatile.acidity, data = whitewines)
quality.tree
```

```
## node), split, n, deviance, yval
##      * denotes terminal node
##
##  1) root 4898 3841.00 5.878
##    2) alcohol < 10.85 3085 1845.00 5.606
##      4) volatile.acidity < 0.2525 1475 868.00 5.873 *
##      5) volatile.acidity > 0.2525 1610 775.30 5.361 *
##    3) alcohol > 10.85 1813 1378.00 6.341
##      6) alcohol < 11.7417 881 670.00 6.121
##        12) volatile.acidity < 0.465 865 616.50 6.150 *
##        13) volatile.acidity > 0.465 16 13.94 4.562 *
##      7) alcohol > 11.7417 932 624.70 6.549 *
```

Plot the tree:

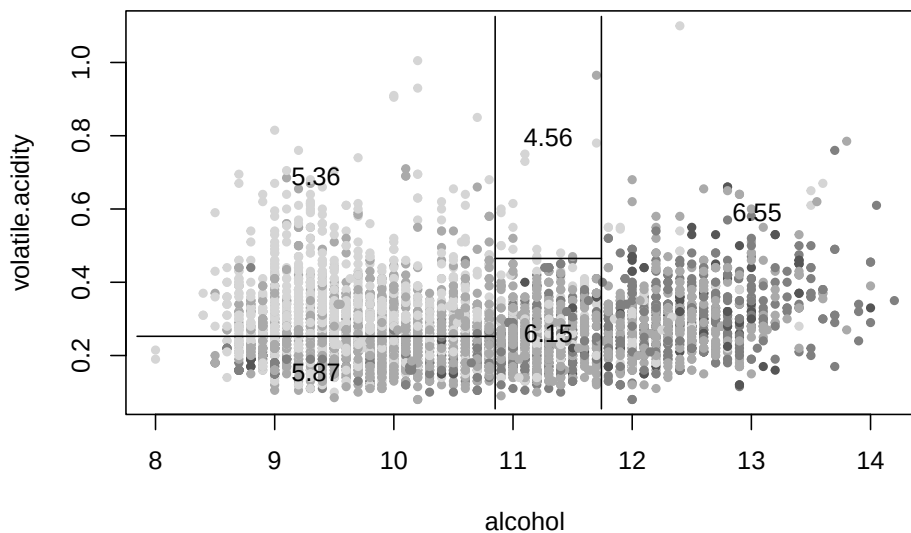
```
plot(quality.tree)
text(quality.tree, cex = 0.75)
```



The corresponding rectangular partition of the two-dimensional feature space is

```
quality.quantiles = cut(whitewines$quality, unique(quantile(whitewines$quality,
  0:5/5)), include.lowest = TRUE)
```

```
plot(whitewines$alcohol, whitewines$volatile.acidity, col = grey(5:1/6)[quality.quantiles],
  pch = 20, ylab = "volatile.acidity", xlab = "alcohol")
partition.tree(quality.tree, ordvars = c("alcohol", "volatile.acidity"),
  add = TRUE)
```

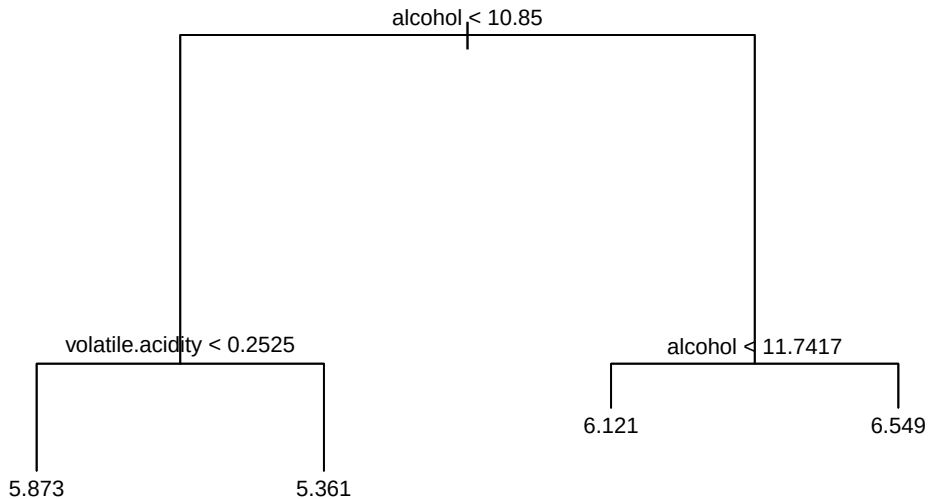


`prune.tree` shrinks the tree to a chosen number of leaves:

```
smaller.tree = prune.tree(quality.tree, best = 4)
smaller.tree
```

```
## node), split, n, deviance, yval
##      * denotes terminal node
##
## 1) root 4898 3841.0 5.878
##    2) alcohol < 10.85 3085 1845.0 5.606
##      4) volatile.acidity < 0.2525 1475 868.0 5.873 *
##      5) volatile.acidity > 0.2525 1610 775.3 5.361 *
##    3) alcohol > 10.85 1813 1378.0 6.341
##      6) alcohol < 11.7417 881 670.0 6.121 *
##      7) alcohol > 11.7417 932 624.7 6.549 *
```

```
plot(smaller.tree)
text(smaller.tree, cex = 0.75)
```



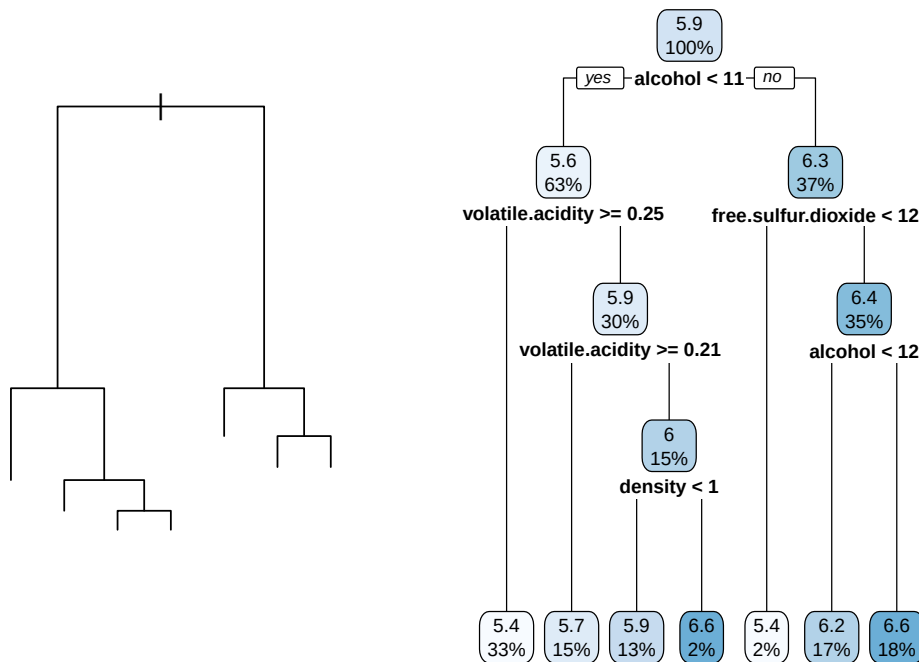
Detach `tree` before moving to `rpart` so that the two `plot` methods do not clash.

```
detach("package:tree")
```

Fitting and Pruning a Regression Tree

In R we use `rpart`. The syntax for `rpart` is again the usual formula interface. The default plot is plain; `rpart.plot` is easier to read.

```
set.seed(598)
wine.tree1 = rpart(quality ~ ., data = whitewines)
par(mfrow = c(1, 2))
plot(wine.tree1)
rpart.plot(wine.tree1)
```



Unlike the `tree` example, this fit uses every predictor.

Reading the Tree

Read the `rpart.plot` from the root down. The root prints the mean of `quality` over the whole sample and 100% of the rows. The first split is on `alcohol`. The left child holds the lower-alcohol rows and a lower mean; the right child holds the rest. Leaves sit at the bottom; their shading tracks the size of the prediction. Exact means and percentages will match the printed tree for this data set and this `rpart` version.

6.3.1 Cross-validation and the CP Table

Cross-validation is built into `rpart`. Use `printcp` to inspect the pruning table.

CP is the **complexity parameter**, a scaled version of α – the *price* of a leaf.

```
printcp(wine.tree1)
```

```
##
## Regression tree:
## rpart(formula = quality ~ ., data = whitewines)
##
## Variables actually used in tree construction:
```

```
## [1] alcohol          density          free.sulfur.dioxide
## [4] volatile.acidity
##
## Root node error: 3841/4898 = 0.7842
##
## n= 4898
##
##          CP nsplit rel error  xerror    xstd
## 1 0.161007      0  1.00000 1.00030 0.021275
## 2 0.052469      1  0.83899 0.84260 0.020094
## 3 0.027342      2  0.78652 0.79453 0.019601
## 4 0.017711      3  0.75918 0.76429 0.018478
## 5 0.010355      4  0.74147 0.75134 0.018065
## 6 0.010000      6  0.72076 0.74656 0.017995
```

In the output above:

- **nsplit** is the number of splits. The number of leaves is **nsplit** + 1.
- **rel error** is the training RSS of that subtree, divided by the RSS of the root.
- **xerror** is the same quantity estimated by cross-validation (also relative to the root). **xstd** is the standard error of **xerror**.

cp is a **scaled** version of the penalty α from cost-complexity pruning. If T_0 denotes the root and $R(T)$ is RSS, **rpart** uses

$$R_{\text{cp}}(T) = \frac{R(T)}{R(T_0)} + \text{cp} |T|.$$

That is the same criterion as $R_\alpha(T) = R(T) + \alpha|T|$ with

$$\text{cp} = \frac{\alpha}{R(T_0)}.$$

(The constant that distinguishes a penalty on leaves from a penalty on splits does not change which subtree is optimal for a given **cp**.)

In the **cptable**, when **nsplit** increases by one from row i to row $i + 1$,

$$\text{rel error}_i - \text{rel error}_{i+1} = \text{CP}_i.$$

That difference is the drop in *relative* RSS from the extra split. It is the largest `cp` for which that split is still worth keeping on the training data. Cross-validation (`xerror`) is what we use to decide whether the split is worth keeping for prediction.

Pruning

`prune` returns a subtree for a chosen `cp`:

```
prune(wine.tree1, cp = 0.3)
```

```
## n= 4898
##
## node), split, n, deviance, yval
##      * denotes terminal node
##
## 1) root 4898 3840.99 5.877909 *
```

```
prune(wine.tree1, cp = 0.2)
```

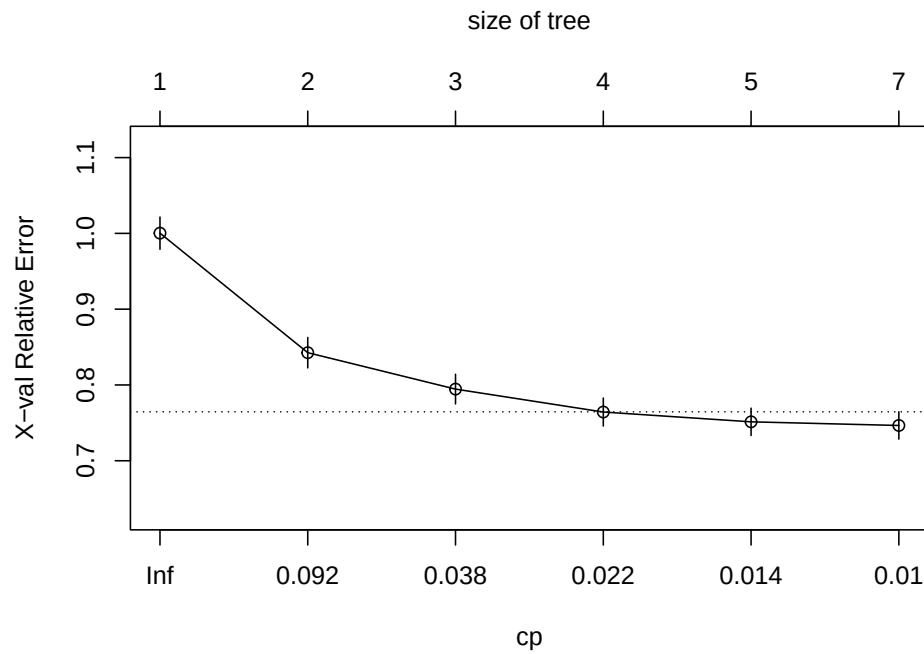
```
## n= 4898
##
## node), split, n, deviance, yval
##      * denotes terminal node
##
## 1) root 4898 3840.99 5.877909 *
```

```
prune(wine.tree1, cp = 0.156)
```

```
## n= 4898
##
## node), split, n, deviance, yval
##      * denotes terminal node
##
## 1) root 4898 3840.990 5.877909
##   2) alcohol< 10.85 3085 1844.906 5.605511 *
##   3) alcohol>=10.85 1813 1377.659 6.341423 *
```

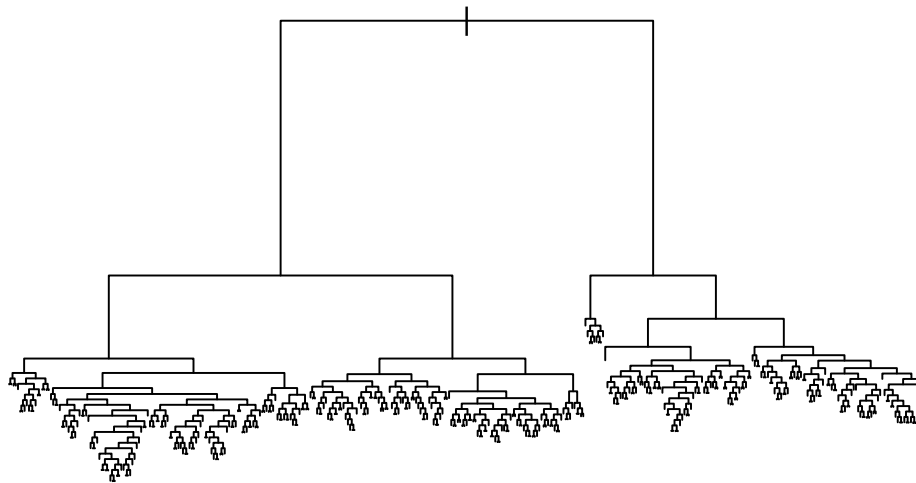
The CV path:

```
plotcp(wine.tree1)
```

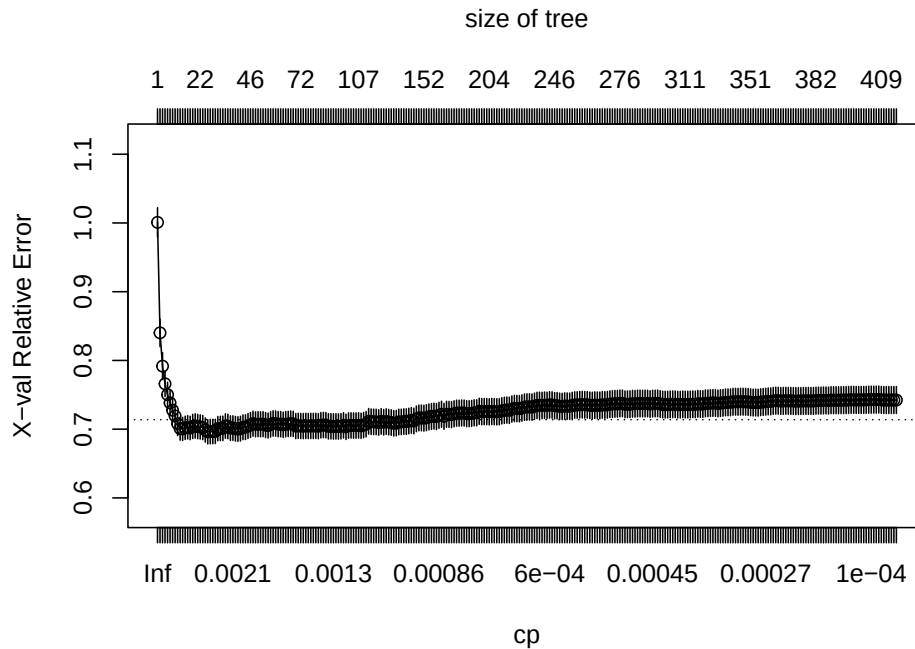


If the default tree looks too small to have reached the CV minimum, grow a larger tree by setting `cp = 0`:

```
wine.tree2 = rpart(quality ~ ., data = whitewines, control = list(cp = 0,
  xval = 10))
plot(wine.tree2)
```



```
plotcp(wine.tree2)
```



Optimal CP Value

Choose *cp* by the *minimum cross-validation error* or by the *one-standard-error (1-SE) rule*.

```
# Index of CP with lowest xerror
optimal.cp = which.min(wine.tree2$cptable[, "xerror"])

# The optimal CP value (column 'xerror')
wine.tree2$cptable[optimal.cp, 4]

## [1] 0.6961474

# upper bound for equivalent optimal xerror
wine.tree2$cptable[optimal.cp, 4] + wine.tree2$cptable[optimal.cp,
5]

## [1] 0.713746

# smallest tree whose error is within one SE of the
# minimum
tmp.id = which(wine.tree2$cptable[, 4] <= wine.tree2$cptable[optimal.cp,
4] + wine.tree2$cptable[optimal.cp, 5])
tmp.id = min(tmp.id)
```

```

# CP.1se: midpoint of the cp values on either side of that
# row
CP.1se = (wine.tree2$cptable[tmp.id - 1, 1] + wine.tree2$cptable[tmp.id,
  1])/2

# Prune tree with CP.1se
wine.tree3 = prune(wine.tree2, cp = CP.1se)

```

The identity between column 1 (CP) and the adjacent drops in column 3 (rel error) is the same one stated after `printcp`. The commented line below checks it numerically.

```

# cbind(wine.tree2$cptable[, 1],
# c(-diff(wine.tree2$cptable[, 3]), 0))

```

After the tree is fit, `predict()` (the `predict.rpart` method) produces fitted values on new or existing rows.

Random Forests in R

Load `randomForest` and hold out a test set.

```

library(randomForest)

## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##      margin
n = nrow(whitewines)
ntest = round(n * 0.3)
set.seed(598)
test.id = sample(1:n, ntest)

```

Model Fitting

Predict quality from every other column.

```
rf.wines = randomForest(quality ~ ., data = whitewines[-test.id,
], importance = T, ntree = 400)
```

`mtry` is the number of variables eligible at each split.

```
rf.wines$mtry
```

```
## [1] 3
```

Here `mtry` is 3, which matches the usual regression default $\lfloor p/3 \rfloor$.

Test-set mean squared error:

```
yhat.test = predict(rf.wines, whitewines[test.id, ])

sum((whitewines$quality[test.id] - yhat.test)^2)/length(test.id)
```

```
## [1] 0.4129956
```

There are two training-side errors: the error of `predict()` on the training rows, and the out-of-bag error stored in the forest.

```
yhat.train = predict(rf.wines, whitewines[-test.id, ])
sum((whitewines$quality[-test.id] - yhat.train)^2)/(n - ntest)
```

```
## [1] 0.07462265
```

The in-sample training error is much smaller than the test error.

```
sum((whitewines$quality[-test.id] - rf.wines$predicted)^2)/(n -
ntest)
```

```
## [1] 0.3717869
```

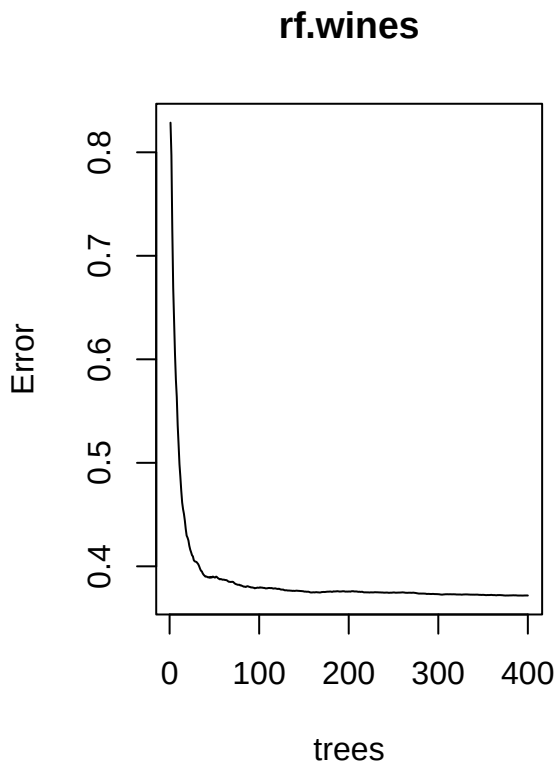
The OOB error is closer to the test error and is a more reliable estimate of generalization performance. How often each observation was left out of a bootstrap sample (first 10 rows):

```
rf.wines$oob.times[1:10]
```

```
## [1] 134 139 136 156 153 146 127 151 167 149
```

Performance vs. Number of Trees

```
par(mfrow = c(1, 2))
plot(rf.wines)
```



The plot is OOB MSE against the number of trees. Error usually drops quickly and then levels off. Extra trees rarely make the forest worse; they may simply be unnecessary.

Variable Importance

Because we set `importance = TRUE`, the forest stores an importance table.

```
rf.wines$importance
```

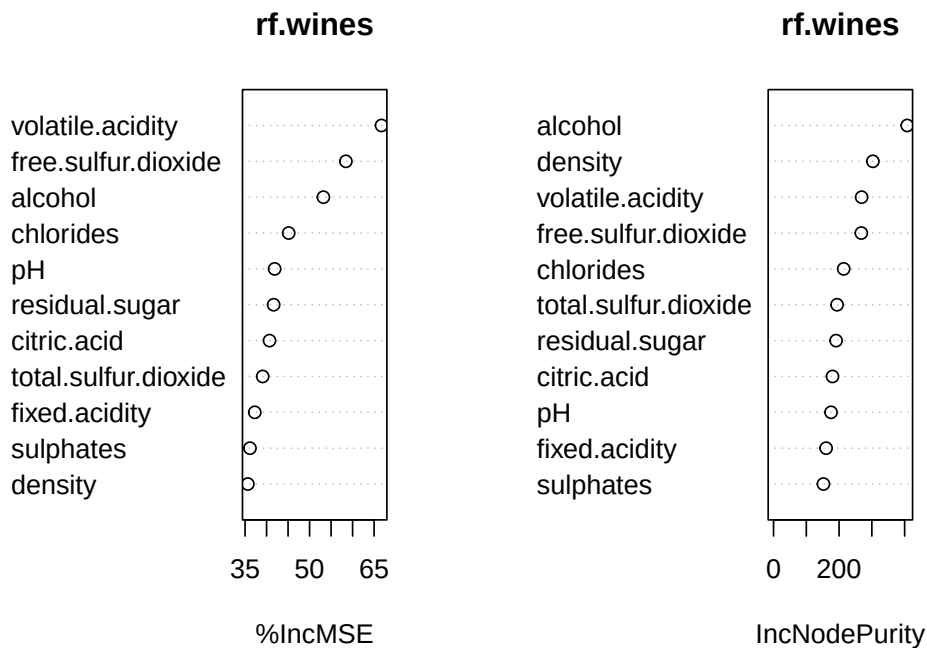
##	%IncMSE	IncNodePurity
## fixed.acidity	0.06065622	160.4879
## volatile.acidity	0.17216691	268.8119
## citric.acid	0.08534572	180.0529
## residual.sugar	0.14440143	190.9511
## chlorides	0.11077918	214.0642
## free.sulfur.dioxide	0.13763805	267.8467
## total.sulfur.dioxide	0.09552685	193.8788
## density	0.24357653	303.1417
## pH	0.08025148	175.5309
## sulphates	0.04998015	152.2842
## alcohol	0.35362770	407.9546

%IncMSE is permutation importance. IncNodePurity is the accumulated RSS gain. Scaled and unscaled versions are available, with standard errors for the permutation measure.

```
importance(rf.wines, scale = F)
```

##		%IncMSE	IncNodePurity
##	fixed.acidity	0.06065622	160.4879
##	volatile.acidity	0.17216691	268.8119
##	citric.acid	0.08534572	180.0529
##	residual.sugar	0.14440143	190.9511
##	chlorides	0.11077918	214.0642
##	free.sulfur.dioxide	0.13763805	267.8467
##	total.sulfur.dioxide	0.09552685	193.8788
##	density	0.24357653	303.1417
##	pH	0.08025148	175.5309
##	sulphates	0.04998015	152.2842
##	alcohol	0.35362770	407.9546

```
par(mfrow = c(1, 2))
varImpPlot(rf.wines, type = 1)
varImpPlot(rf.wines, type = 2)
```



Gradient Boosting Machines in R

A GBM is a sum of trees built **sequentially**. For squared error (`distribution = "gaussian"`), tree t is fit to the current residuals and then added with a learning rate λ :

$$F_t(x) = F_{t-1}(x) + \lambda \hat{h}_t(x).$$

The arguments that matter for `gbm` are:

- `distribution`: loss. Use `"gaussian"` for squared error.
- `n.trees`: number of trees T (default 100). Too small underfits; too large can overfit if λ is not small.
- `shrinkage`: learning rate λ (default 0.1). Smaller λ usually needs a larger `n.trees`.
- `interaction.depth`: size of each tree (default 1, a stump). Depth d allows interactions up to order d .
- `bag.fraction`: fraction of the training rows used to fit each tree (default 0.5). This is stochastic gradient boosting; 1 uses all rows every time.
- `cv.folds`: folds for choosing `n.trees` (default 0, no CV error).

```
library(gbm)
```

```
## Loaded gbm 2.3.1
```

```
## This version of gbm is no longer under development. Consider transitioning to gbm3,
```

The first specification uses no shrinkage and no subsampling, so it is close to unregularized stagewise fitting. That is a poor default in practice; we use it only as a baseline.

```
gbm.wines = gbm(quality ~ ., data = whitewines[-test.id, ], distribution = "gaussian",
  n.trees = 100, shrinkage = 1, interaction.depth = 3, bag.fraction = 1,
  cv.folds = 5)
```

```
gbm.wines
```

```
## gbm(formula = quality ~ ., distribution = "gaussian", data = whitewines[-test.id,
##      ], n.trees = 100, interaction.depth = 3, shrinkage = 1, bag.fraction = 1,
##      cv.folds = 5)
```

```
## A gradient boosted model with gaussian loss function.
```

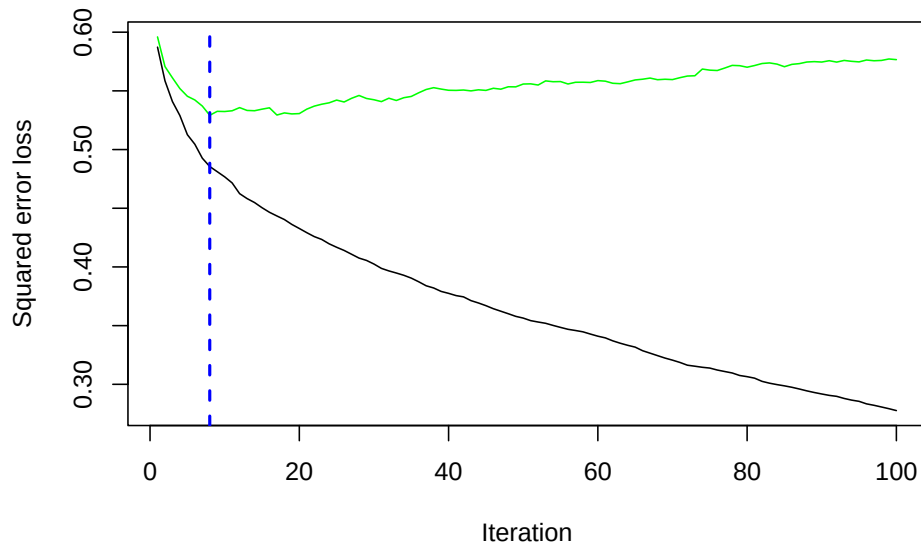
```
## 100 iterations were performed.
```

```
## The best cross-validation iteration was 8.
```

```
## There were 11 predictors of which 10 had non-zero influence.
```

Plot the CV error against the number of trees:

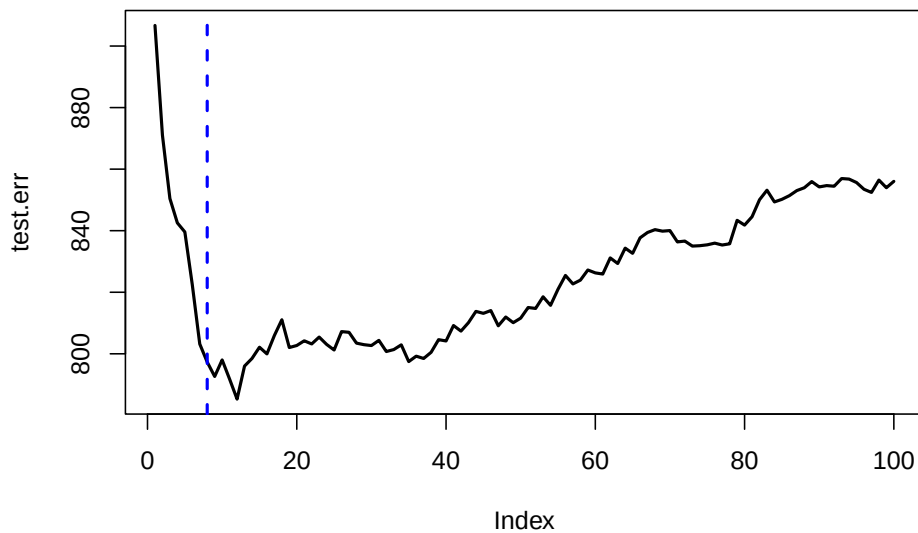
```
opt.size = gbm.perf(gbm.wines)
```



`gbm.perf` picks a number of trees from the CV curve. With $\lambda = 1$, that optimum is often early: later trees add little or start to overfit.

Test MSE as a function of the number of trees:

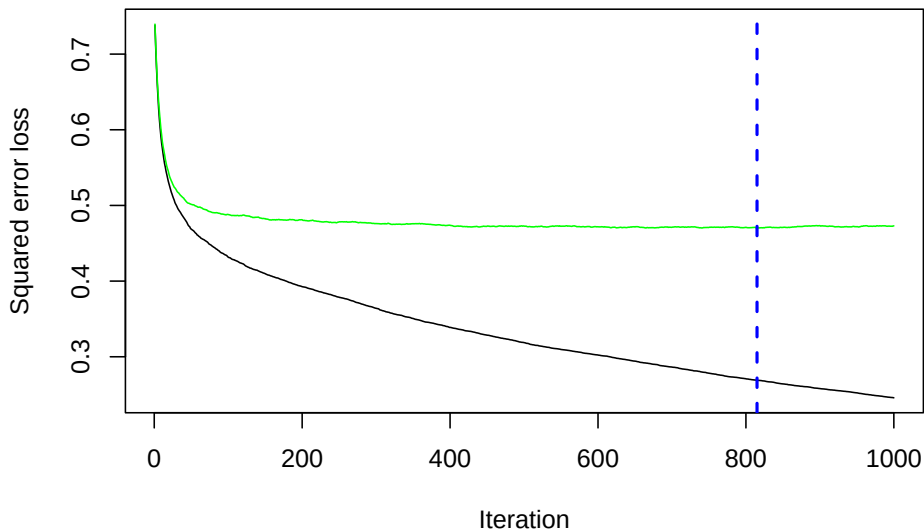
```
size = 1:gbm.wines$n.trees
test.err = rep(0, length(size))
for (i in 1:length(size)) {
  y.pred = predict(gbm.wines, whitewines[test.id, ], n.trees = size[i])
  test.err[i] = sum((whitewines$quality[test.id] - y.pred)^2)
}
plot(test.err, type = "n")
lines(size, test.err, lwd = 2)
abline(v = opt.size, lwd = 2, lty = 2, col = "blue")
```



A more standard specification: small learning rate, more trees, and `bag.fraction = 0.5`.

```
gbm.wines.2 = gbm(quality ~ ., data = whitewines[-test.id, ],
  distribution = "gaussian", n.trees = 1000, shrinkage = 0.1,
  interaction.depth = 3, bag.fraction = 0.5, cv.folds = 5)

opt.size.2 = gbm.perf(gbm.wines.2)
```

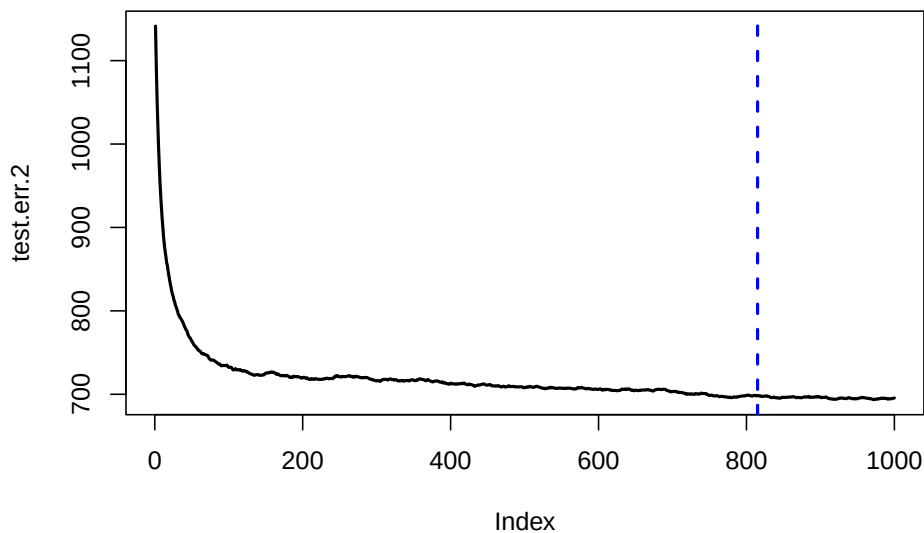


```
size.2 = 1:gbm.wines.2$n.trees
test.err.2 = rep(0, length(size.2))
for (i in 1:length(size.2)) {
```

```

y.pred.2 = predict(gbm.wines.2, whitewines[test.id, ], n.trees = size.2[i])
test.err.2[i] = sum((whitewines$quality[test.id] - y.pred.2)^2)
}
plot(test.err.2, type = "n")
lines(size.2, test.err.2, lwd = 2)
abline(v = opt.size.2, lwd = 2, lty = 2, col = "blue")

```



Variable importance from the baseline fit (relative influence and a permutation measure):

```

par(mfrow = c(1, 2))
summary(gbm.wines, cBars = 10, method = relative.influence, las = 2)

```

```

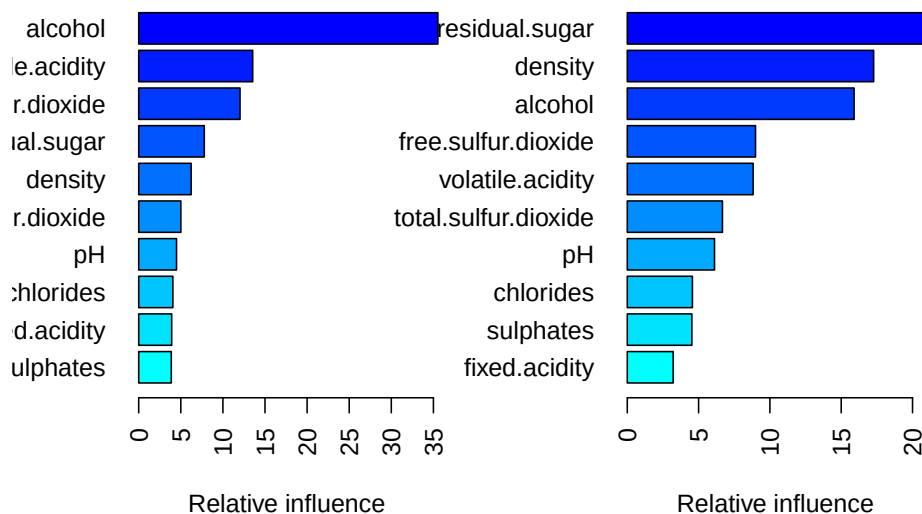
##              var    rel.inf
## alcohol              alcohol 35.531879
## volatile.acidity    volatile.acidity 13.540395
## free.sulfur.dioxide free.sulfur.dioxide 12.034661
## residual.sugar      residual.sugar  7.791508
## density              density  6.230189
## total.sulfur.dioxide total.sulfur.dioxide 5.006459
## pH                    pH  4.494230
## chlorides             chlorides  4.058416
## fixed.acidity         fixed.acidity  3.918535
## sulphates             sulphates  3.871500
## citric.acid           citric.acid  3.522228

```

```

summary(gbm.wines, cBars = 10, method = permutation.test.gbm,
        las = 2)

```



##	var	rel.inf
## 1	residual.sugar	20.976784
## 2	density	17.275031
## 3	alcohol	15.905093
## 4	free.sulfur.dioxide	8.993392
## 5	volatile.acidity	8.822918
## 6	total.sulfur.dioxide	6.674502
## 7	pH	6.117880
## 8	chlorides	4.568066
## 9	sulphates	4.529224
## 10	fixed.acidity	3.218505
## 11	citric.acid	2.918603

6.4 Conclusion

A regression tree approximates a mean function by a piecewise-constant fit on hyper-rectangles. The rectangles are built by greedy binary splits that minimize squared error, and each leaf is the average of Y in that cell. One tree is easy to read and does not require a global linear form, but it is a high-variance predictor: a small change in the data can change the split sequence. The usual remedy is to grow a large tree and prune it with the cost-complexity criterion $R_\alpha(T) = R(T) + \alpha|T|$, choosing α by cross-validation (minimum CV error or the 1-SE rule). Ensembles trade the single-tree picture for accuracy. Bagging averages trees grown on bootstrap samples; a random forest does the same and, at each split, searches only m of the p predictors so that the trees are less correlated. Boosting grows trees sequentially, at each step fitting the current residuals and adding the new tree with a learning rate λ . In short: use one

pruned tree when you need a short list of rules; use a forest or a boosted ensemble when the goal is prediction.

Chapter 7

Introduction to Classification

Classification is a supervised learning task whose goal is to *predict a categorical outcome* from a set of input features. Unlike regression, where the response is continuous, the labels here are discrete—for example, “spam” versus “not spam,” or “diseased” versus “healthy.” We learn a rule that maps features to a *class*, typically by estimating class probabilities or by constructing a decision boundary. Methods range from relatively simple models such as logistic regression and discriminant analysis to more flexible ones such as trees, support vector machines, neural networks, and ensembles. As in the rest of statistical learning, the central tension is between flexibility and overfitting: we want a rule that predicts well on *new* observations, not only on the training sample.

Classification Framework

We start with **training data** $\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^p$ is a p -dimensional feature vector. For now we focus on the **binary** case, with response $y_i \in \{0, 1\}$. The particular coding of the two classes is not essential; $\{-1, +1\}$ is equally common.

The goal is to find a **classifier** $f : \mathbb{R}^p \rightarrow \{0, 1\}$, that is, a map from feature space to the label set. To score a prediction $f(x)$ when the true label is y , we use the **0-1 loss**, also called the **misclassification error**:

$$L(f(x), y) = \begin{cases} 0, & \text{if } y = f(x), \\ 1, & \text{otherwise.} \end{cases}$$

The training (empirical) risk of f is then

$$\widehat{R}_n[f] = \frac{1}{n} \sum_{i=1}^n L(f(x_i), y_i).$$

Minimizing $\widehat{R}_n[f]$ over *all* functions f is not a useful recipe by itself: the minimizer can interpolate the training labels and still fail on future data. What we actually want is a rule with small *expected* error on a new pair (X, Y) .

The Optimal Classifier

If we knew the data-generating distribution P , or if we had infinitely many samples, the empirical risk would converge to the expected risk,

$$\widehat{R}_n[f] \longrightarrow R[f] := E_{X,Y} L(f(X), Y).$$

Optimal classifier

The optimal classifier minimizes expected 0-1 risk:

$$f^* = \arg \min_f R[f], \quad R[f] = E_{X,Y} L(f(X), Y).$$

Suppose we may use *any* function f . What is f^* ?

Write $p(x)$ for the marginal density of X , $p(y | x)$ for the conditional distribution of Y given $X = x$, and $p(x, y) = p(x)p(y | x)$ for the joint distribution. Then

$$\begin{aligned} R[f] &= E_{X,Y} L(f(X), Y) \\ &= \int_x \sum_{y \in \{0,1\}} L(f(x), y) p(y | x) p(x) dx \\ &= \int_x \left[\sum_{y \in \{0,1\}} L(f(x), y) p(y | x) \right] p(x) dx. \end{aligned}$$

Because $p(x) \geq 0$, it is enough to minimize the inner sum *separately at each* x :

$$f^*(x) = \arg \min_{a \in \{0,1\}} \sum_{y \in \{0,1\}} L(a, y) p(y | x).$$

Let $\eta(x) := P(Y = 1 | X = x)$. Then $Y | X = x$ is Bernoulli, and the inner risk is just the conditional probability of a mistake:

$$\sum_y L(f(x), y) p(y | x) = \begin{cases} 1 - \eta(x), & \text{if } f(x) = 1, \\ \eta(x), & \text{if } f(x) = 0. \end{cases}$$

Hence

$$f^*(x) = \begin{cases} 1, & \text{if } \eta(x) \geq 1/2, \\ 0, & \text{if } \eta(x) < 1/2. \end{cases}$$

(The assignment at the tie $\eta(x) = 1/2$ is arbitrary.) This f^* is the **Bayes classifier** (or **Bayes rule**). Its risk $R[f^*]$ is the **Bayes risk**, also called the **Bayes error**. No other classifier can beat it in expected 0-1 loss; in practice we never know $\eta(x)$, so every method is an attempt to approximate f^* .

How about multi-class classification?

The same argument extends immediately to K classes, $y \in \{1, \dots, K\}$. The Bayes rule is

$$f^*(x) = \arg \max_{k \in \{1, \dots, K\}} P(Y = k \mid X = x).$$

A classifier partitions $\mathbb{R}^p$ into labeled regions. We often describe the **decision boundary** (the interface between regions) rather than f itself. A *linear classifier* is one whose decision boundaries are hyperplanes.

7.1 Discriminant Analysis

Discriminant analysis builds a classifier by modeling how the *features* are distributed *within each class*, then applying Bayes' theorem to recover $P(Y = k \mid X = x)$. The three standard variants differ mainly in how they model $X \mid Y = k$, and therefore in the shape of the resulting boundary:

1. Quadratic discriminant analysis (QDA)
2. Linear discriminant analysis (LDA), and its connection to Fisher discriminant analysis (FDA)
3. Naive Bayes

All three start from the factorization $P(x, y) = P(x \mid y) P(y)$. We estimate

- $f_k(x) = P(X = x \mid Y = k)$, the class-conditional density of the features, and
- $\pi_k = P(Y = k)$, the class prior,

and then flip the conditioning.

Bayes' theorem for classification

$$P(Y = k \mid X = x) = \frac{P(X = x \mid Y = k) P(Y = k)}{P(X = x)} = \frac{\pi_k f_k(x)}{P(X = x)} \propto \pi_k f_k(x).$$

- $f_k(x)$: class-conditional density of $X \mid Y = k$
- $\pi_k = P(Y = k)$: prior probability of class k

The Bayes classifier therefore has the equivalent form

$$f(x) = \arg \max_k \pi_k f_k(x) = \arg \max_k [\log \pi_k + \log f_k(x)].$$

To implement it we only need estimates of π_k and f_k .

7.1.1 Quadratic Discriminant Analysis

Assume that, given $Y = k$, the feature vector is multivariate normal, $\mathbf{x} \sim N_p(\mu_k, \Sigma_k)$, with class-specific mean μ_k and class-specific covariance Σ_k . The

density is

$$f_k(\mathbf{x}) = (2\pi)^{-p/2} |\Sigma_k|^{-1/2} \exp\left\{-\frac{1}{2}(\mathbf{x} - \mu_k)^\top \Sigma_k^{-1}(\mathbf{x} - \mu_k)\right\}.$$

The quadratic form in the exponent is

$$(\mathbf{x} - \mu_k)^\top \Sigma_k^{-1}(\mathbf{x} - \mu_k) = \sum_{j=1}^p \sum_{\ell=1}^p \theta_{k,j\ell} (x_j - \mu_{k,j})(x_\ell - \mu_{k,\ell}),$$

where $\theta_{k,j\ell}$ are the entries of Σ_k^{-1} . Dropping terms that do not depend on k ,

$$P(Y = k \mid \mathbf{x}) \propto \pi_k f_k(\mathbf{x}) \propto e^{-d_k(\mathbf{x})/2},$$

where

$$d_k(\mathbf{x}) = (\mathbf{x} - \mu_k)^\top \Sigma_k^{-1}(\mathbf{x} - \mu_k) + \log |\Sigma_k| - 2 \log \pi_k.$$

The first term is the **Mahalanobis distance** from $\mathbf{x}$ to μ_k . We assign $\mathbf{x}$ to the class with the smallest $d_k(\mathbf{x})$. Because d_k is a quadratic function of $\mathbf{x}$, the decision boundaries are quadratic, which is why the method is called **quadratic discriminant analysis (QDA)**.

In practice we replace the unknown parameters by sample estimates:

- Prior and mean:

$$\hat{\pi}_k = \frac{n_k}{n}, \quad \hat{\mu}_k = \frac{1}{n_k} \sum_{i:y_i=k} \mathbf{x}_i, \quad k = 1, \dots, K.$$

- Class covariance:

$$\hat{\Sigma}_k = \frac{1}{n_k - 1} \sum_{i:y_i=k} (\mathbf{x}_i - \hat{\mu}_k)(\mathbf{x}_i - \hat{\mu}_k)^\top.$$

Remark. If $\hat{\Sigma}_k$ is singular (common when p is large relative to n_k), replace $\hat{\Sigma}_k^{-1}$ by $(\hat{\Sigma}_k + \eta I_p)^{-1}$ for a small $\eta > 0$, or use a generalized inverse. This is a simple form of regularization.

7.1.2 Linear Discriminant Analysis

Now assume a *common* covariance, $\Sigma_k = \Sigma$ for all k . Then

$$d_k(\mathbf{x}) = (\mathbf{x} - \mu_k)^\top \Sigma^{-1}(\mathbf{x} - \mu_k) + \log |\Sigma| - 2 \log \pi_k.$$

Expanding the quadratic form,

$$(\mathbf{x} - \mu_k)^\top \Sigma^{-1}(\mathbf{x} - \mu_k) = \mathbf{x}^\top \Sigma^{-1} \mathbf{x} - 2\mathbf{x}^\top \Sigma^{-1} \mu_k + \mu_k^\top \Sigma^{-1} \mu_k.$$

The term $\mathbf{x}^\top \Sigma^{-1} \mathbf{x}$ does not depend on k , nor does $\log |\Sigma|$. What remains is a **linear function of $\mathbf{x}$** , so the decision boundaries are hyperplanes. This is **linear discriminant analysis (LDA)**.

The shared covariance is estimated by the **pooled** sample covariance

$$\hat{\Sigma} = \frac{1}{n-K} \sum_{k=1}^K \sum_{i: y_i=k} (\mathbf{x}_i - \hat{\mu}_k)(\mathbf{x}_i - \hat{\mu}_k)^\top.$$

Remark. If $\hat{\Sigma}$ is singular, use $(\hat{\Sigma} + \eta I_p)^{-1}$ or a generalized inverse. For example, if the SVD is $\hat{\Sigma} = U \text{diag}(d_1, \dots, d_r, 0, \dots, 0) U^\top$, invert only the positive singular values:

$$\hat{\Sigma}^+ = U \text{diag}(d_1^{-1}, \dots, d_r^{-1}, 0, \dots, 0) U^\top.$$

7.1.3 Reduced-Rank LDA

As a further simplification, take $\Sigma = I_p$. The LDA discriminant reduces to

$$d_k(\mathbf{x}) = \|\mathbf{x} - \mu_k\|^2 - 2 \log \pi_k.$$

Only the squared Euclidean distance from $\mathbf{x}$ to the class center μ_k depends on the features.

This distance can be computed in a subspace of dimension at most $K-1$. In that sense, LDA is a dimension-reduction method: it automatically collapses p coordinates to $K-1$ coordinates without changing the classification rule (under $\Sigma = I_p$).

Without loss of generality, translate the data so that the grand mean $\bar{\mu}$ is at the origin. The K class means then lie in a subspace $\mathcal{A}$ of dimension at most $K-1$. Decompose any $\mathbf{x} \in \mathbb{R}^p$ as

$$\mathbf{x} = \mathbf{x}_1 + \mathbf{x}_2, \quad \mathbf{x}_1 \in \mathcal{A}, \quad \mathbf{x}_2 \in \mathcal{A}^\perp.$$

In matrix form, if $B = (\mu_1, \dots, \mu_{K-1})$ and $H = B(B^\top B)^{-1} B^\top$ is the orthogonal projector onto $\mathcal{A}$, then $\mathbf{x}_1 = H\mathbf{x}$ and $\mathbf{x}_2 = \mathbf{x} - \mathbf{x}_1$. Orthogonality gives

$$\|\mathbf{x}\|^2 = \|\mathbf{x}_1\|^2 + \|\mathbf{x}_2\|^2,$$

and, because each μ_k already lies in $\mathcal{A}$,

$$\|\mathbf{x} - \mu_k\|^2 = \|\mathbf{x}_1 - \mu_k\|^2 + \|\mathbf{x}_2\|^2.$$

The residual length $\|\mathbf{x}_2\|^2$ does not depend on k , so

$$d_k(\mathbf{x}) = \|\mathbf{x}_1 - \mu_k\|^2 - 2 \log \pi_k + \text{constant}.$$

Classification using the original $\mathbf{x}$ is identical to classification using only its projection $\mathbf{x}_1$ onto the class-mean subspace.

When $\Sigma \neq I_p$, first **whiten** the features. If $\Sigma = UD^2U^\top$ is an SVD, set

$$\Sigma^{-1/2} := UD^{-1}U^\top, \quad \tilde{\mathbf{x}} = \Sigma^{-1/2}\mathbf{x}.$$

Then $\text{Cov}(\tilde{\mathbf{x}}) = I_p$:

$$\text{Cov}(\tilde{\mathbf{x}}) = \Sigma^{-1/2} \Sigma (\Sigma^{-1/2})^\top = UD^{-1}U^\top UD^2U^\top UD^{-1}U^\top = I_p.$$

(The one-dimensional analogue is dividing a scalar by its standard deviation.) After whitening, apply the $\Sigma = I_p$ argument to $\tilde{\mathbf{x}}$ and the whitened means $\tilde{\mu}_k = \Sigma^{-1/2}\mu_k$.

Remark. $\Sigma^{-1/2}$ is symmetric and satisfies $\Sigma^{-1/2}\Sigma^{-1/2} = \Sigma^{-1}$.

Summary of reduced-rank LDA

Training. From $\{(x_i, y_i)\}_{i=1}^n$, compute $(\hat{\pi}_k, \hat{\mu}_k)_{k=1}^K$ and $\hat{\Sigma}$.

Prediction at a test point $\mathbf{x}^*$:

1. Compute a square-root inverse $A = \hat{\Sigma}^{-1/2}$ from the SVD $\hat{\Sigma} = UD^2U^\top$, $A = UD^{-1}U^\top$.
2. Form the whitened, centered class means and the projector onto their span:

$$B = [A(\hat{\mu}_1 - \tilde{\mu}), \dots, A(\hat{\mu}_{K-1} - \tilde{\mu})],$$

$$H = B(B^\top B)^{-1}B^\top.$$

3. Project the whitened test point: $\tilde{\mathbf{x}}^* = H A \mathbf{x}^*$. Let $\tilde{\mu}_k = H A \hat{\mu}_k$ be the corresponding projected means.
4. For $k = 1, \dots, K$, compute

$$d_k(\mathbf{x}^*) = \|\tilde{\mathbf{x}}^* - \tilde{\mu}_k\|^2 - 2 \log \hat{\pi}_k.$$

5. Predict $\arg \min_k d_k(\mathbf{x}^*)$.

When $p \geq K$, LDA therefore lives in a $(K-1)$ -dimensional subspace (a single line, in the binary case). The same subspace appears in **Fisher discriminant analysis (FDA)**, derived from a different criterion.

Risk of overfitting

Each of the $K-1$ directions is a linear combination of the original p coordinates, with weights estimated from the training sample. If p is large, those weights can lock onto noise and produce excellent *training* separation that does not reproduce on test data.

7.2 Fisher Discriminant Analysis (FDA)

Supervised dimension reduction

FDA looks for directions $a \in \mathbb{R}^p$ such that the projected data $a^\top x$ are well separated by class: class means should be far apart relative to the spread *within* each class. Equivalently, we maximize the ratio

$$\frac{\text{between-class variation}}{\text{within-class variation}} = \frac{a^\top B a}{a^\top W a}.$$

Here W and B are the within-class and between-class covariance matrices

$$W = \frac{1}{n-K} \sum_{k=1}^K \sum_{i: y_i=k} (\mathbf{x}_i - \hat{\mu}_k)(\mathbf{x}_i - \hat{\mu}_k)^\top,$$

$$B = \frac{1}{K-1} \sum_{k=1}^K n_k (\hat{\mu}_k - \bar{\mu})(\hat{\mu}_k - \bar{\mu})^\top,$$

where $\bar{\mu} = n^{-1} \sum_i \mathbf{x}_i$ is the grand mean. The matrix W is exactly the pooled covariance $\hat{\Sigma}$ from LDA. If we write each point as

$$\mathbf{x}_i = (\mathbf{x}_i - \hat{\mu}_{y_i}) + \hat{\mu}_{y_i},$$

then W is the covariance of the residuals $\mathbf{x}_i - \hat{\mu}_{y_i}$, and B is (up to scaling) the covariance of the fitted class means $\hat{\mu}_{y_i}$. Overall constants in B and W do not change the maximizer of the ratio.

Generalized Eigenvalue Problem

$$\max_a \frac{a^\top B a}{a^\top W a} \iff \max_a a^\top B a \quad \text{subject to} \quad a^\top W a = 1.$$

Assume $W = U D^2 U^\top$ and set $b = W^{1/2} a$ with $W^{1/2} = U D U^\top$, so $W^{-1/2} = U D^{-1} U^\top$ and $a = W^{-1/2} b$. Then

$$a^\top B a = b^\top (W^{-1/2} B W^{-1/2}) b,$$

subject to $\|b\|^2 = 1$. This is an ordinary eigenvalue problem. Solve it sequentially:

- b_1 = leading eigenvector of $W^{-1/2} B W^{-1/2}$, then $a_1 = W^{-1/2} b_1$;
- b_2 = second eigenvector, then $a_2 = W^{-1/2} b_2$;
- and so on.

The b_j are orthonormal, but the a_j are orthogonal only *with respect to* W :

$$\begin{aligned} b_j^\top b_j &= 1, & a_j^\top W a_j &= 1, \\ b_j^\top b_\ell &= 0, & a_j^\top W a_\ell &= 0 \quad (j \neq \ell). \end{aligned}$$

At most $K-1$ directions exist, because $\text{rank}(B) \leq K-1$. Those directions span the same subspace obtained from reduced-rank LDA.

LDA vs. FDA

LDA is a *classifier*: it produces a decision rule. FDA is a *supervised dimension-reduction* method: it returns directions, not labels. FDA does not start from a Gaussian model, but the subspace it finds coincides with that of LDA when the class-conditional covariances are equal—the same working assumption that makes LDA’s boundary linear. Unlike PCA, which ignores Y , FDA uses the labels. The same idea can be used as a pre-processing step for regression, but in high dimensions the estimated directions overfit unless we regularize.

Risk of overfitting

A caricature: $n = 6$ points in high dimension, three from class 1 and three from class 0. One can always find a direction that sends the first class to (near) 1 and the second class to (near) 0. The separation is an artifact of p being comparable to n ; even pure noise features can be combined to separate the training labels.

Two simple remedies:

- **Shrinkage / ℓ_1 penalties.** Regularize the estimated means or the inverse covariances (or the FDA criterion itself) so that the estimated directions cannot use every coordinate.
- **Independence.** Pretend that the p coordinates are independent within each class, so that Σ_k is diagonal. That is **naive Bayes**, and it is often a surprisingly strong high-dimensional baseline.

7.3 Naive Bayes Classifiers

The multi-class Bayes rule is still

$$\arg \max_k P(Y = k \mid X = \mathbf{x}) = \arg \max_k \pi_k f_k(\mathbf{x}).$$

Naive Bayes imposes the product factorization

$$f_k(\mathbf{x}) = \prod_{j=1}^p f_{kj}(x_j),$$

i.e., the coordinates of $\mathbf{x}$ are treated as independent given the class. Independence is almost never literally true; it is a *regularizer* for high-dimensional problems.

Each univariate density f_{kj} is estimated separately within class k : histograms or multinomials for discrete features; kernel density estimates (nonparametric naive Bayes) or univariate Gaussians (parametric / Gaussian naive Bayes) for numeric features.

Summary of parametric (Gaussian) naive Bayes

Training. From $\{(x_i, y_i)\}_{i=1}^n$, compute $\hat{\pi}_k$ and, for each class k and coordinate j , the marginal mean and variance $(\hat{\mu}_{kj}, \hat{\sigma}_{kj}^2)$.

Prediction at $\mathbf{x}^* = (x_1^*, \dots, x_p^*)$:

$$d_k(\mathbf{x}^*) = -2 \log \hat{\pi}_k + \sum_{j=1}^p \left[\log \hat{\sigma}_{kj}^2 + \frac{(x_j^* - \hat{\mu}_{kj})^2}{\hat{\sigma}_{kj}^2} \right].$$

Predict $\arg \min_k d_k(\mathbf{x}^*)$. (Additive constants that do not depend on k have been dropped.)

Summary

Discriminant analysis estimates the *class-conditional* factorization

$$P(X = x, Y = k) = P(X = x | Y = k) P(Y = k)$$

and then recovers $P(Y = k | X = x)$ by Bayes' theorem. The idea is conceptually clean and works well when p is modest and the Gaussian (or product) model is not badly wrong. In high dimensions, unrestricted QDA/LDA overfit; pooling, shrinking, reducing rank, or assuming coordinate-wise independence are the usual ways to buy stability.

7.4 The Digits Recognition Example

Discriminant analysis in R

The Python version of this section can be found here: [Discriminant Analysis in Python](#).

To make the ideas concrete we use the handwritten ZIP-code digits from *The Elements of Statistical Learning*. The data originally in `ElemStatLearn` are now available in `tensorBSS`. Each observation is a 16×16 grayscale image, stored as 256 pixels, together with a label in $\{0, 1, \dots, 9\}$. The task is to classify the digit. We will try LDA, QDA, FDA, and naive Bayes.

The package also provides a proper test set `zip.test` ($n = 2007$). Below we instead draw a small training/test split from `zip.train` so that the demonstration stays fast; you should feel free to repeat the analysis on the official `zip.test`.

```
library(tensorBSS) ## ESL ZIP-code digits
```

```
## Loading required package: JADE
```

```
## Loading required package: fICA
```

```
library(MASS) ## lda(), qda()
library(e1071) ## naiveBayes()
```

```
##
## Attaching package: 'e1071'
```

```
## The following object is masked from 'package:ggplot2':
##
##      element
data(zip.train)
```

Column 1 is the digit label; columns 2 through 257 are the pixels. A few training images:

```
par(mfrow = c(2, 3), mai = c(0.15, 0.15, 0.15, 0.15))
for (i in 1:6) {
  digit.img <- matrix(zip.train[i, -1], nrow = 16, byrow = TRUE)
  digit.img <- t(digit.img)[, 16:1]
  image(1:16, 1:16, digit.img, axes = FALSE, col = gray.colors(256,
    rev = TRUE))
}
```



```
set.seed(598)
n <- nrow(zip.train)

train.index <- sample(1:n, 800)
remaining.index <- setdiff(1:n, train.index)
test.index <- sample(remaining.index, 200)

train.zip <- zip.train[train.index, ]
test.zip <- zip.train[test.index, ]

X.train <- train.zip[, 2:257]
Y.train <- train.zip[, 1]
X.test <- test.zip[, 2:257]
Y.test <- test.zip[, 1]
```

Linear Discriminant Analysis (LDA)

```
digits.lda <- lda(X.train, Y.train)

Y.predicted <- predict(digits.lda, X.test)$class
acc.lda <- mean(Y.predicted == Y.test)
acc.lda

## [1] 0.845

# table(Y.test, Y.predicted) # uncomment for the confusion
# matrix
```

On this particular split the test accuracy is 84.5%. The number will move if you change the seed or the split sizes; the official `zip.test` is a harder benchmark.

Quadratic Discriminant Analysis (QDA)

A direct call to `qda()` fails here. With $p = 256$ pixels and only a few hundred observations per digit, each class covariance is rank-deficient, and `MASS::qda` does not regularize automatically.

```
## digits.qda <- qda(X.train, Y.train) # rank-deficient
## Sigma_k
```

Fisher Discriminant Analysis (FDA)

The matrix `lda()$scaling` contains the Fisher directions (at most $K - 1 = 9$ of them for ten digits).

```
FDA.digits.directions <- digits.lda$scaling
dim(FDA.digits.directions)
```

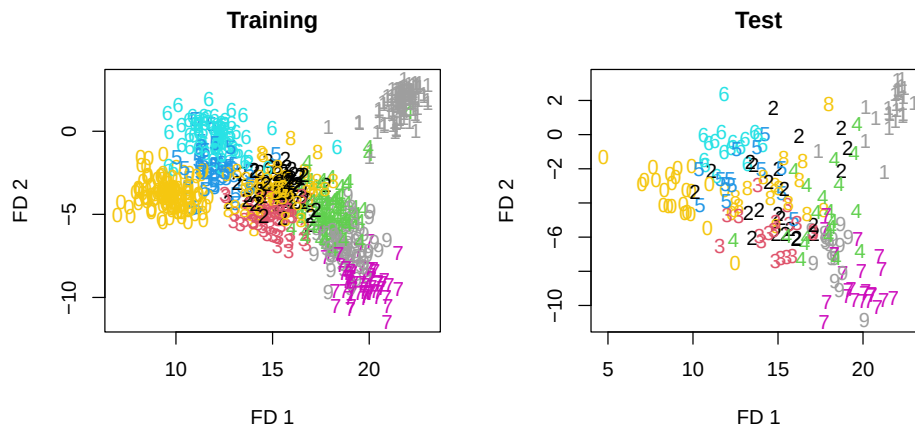
```
## [1] 256 9
```

Project training and test data onto the first two Fisher directions:

```
F.train <- X.train %*% FDA.digits.directions
F.test <- X.test %*% FDA.digits.directions

par(mfrow = c(1, 2))
plot(F.train[, 1], F.train[, 2], type = "n", xlab = "FD 1", ylab = "FD 2",
     main = "Training")
text(F.train[, 1], F.train[, 2], Y.train, col = Y.train + 7)

plot(F.test[, 1], F.test[, 2], type = "n", xlab = "FD 1", ylab = "FD 2",
     main = "Test")
text(F.test[, 1], F.test[, 2], Y.test, col = Y.test + 7)
```



The training cloud (left) looks neatly separated. The test cloud (right) does not reproduce that separation. That gap is overfitting: the nine directions were tuned to the 800 training images.

Feel free to plot other pairs of columns of `F.train` / `F.test`; different planes tell different stories.

Computational remarks

We can recover the same directions by forming W and B explicitly. A no-intercept dummy regression of the features on the class labels yields residuals $\mathbf{x}_i - \hat{\mu}_{y_i}$ and fitted values $\hat{\mu}_{y_i}$:

```
tmp.lm <- lm(X.train ~ as.factor(Y.train) - 1)
W <- cov(tmp.lm$residuals)
B <- cov(tmp.lm$fitted)

eigenW <- eigen(W, symmetric = TRUE)
## Guard tiny/negative eigenvalues so W^{-1/2} is defined
ev <- pmax(eigenW$values, 1e-08)
A <- eigenW$vectors %*% diag(1/sqrt(ev)) %*% t(eigenW$vectors)

FDA.eigen <- eigen(A %*% B %*% t(A), symmetric = TRUE)
FDA.eigen.dir <- A %*% FDA.eigen$vectors[, 1:9]

## Signs of eigenvectors are arbitrary, so correlations may
## be ±1
for (j in 1:9) {
  print(cor(FDA.eigen.dir[, j], FDA.digits.directions[, j]))
}

## [1] 1
## [1] 1
## [1] 1
```

```
## [1] 1
## [1] -1
## [1] 1
## [1] 1
## [1] 1
## [1] 1
```

FDA vs. PCA

PCA ignores the labels. The leading principal axes need not be good separating directions.

```
Sigma <- cov(X.train)
dig.eigen <- eigen(Sigma, symmetric = TRUE)
PCA.dir <- dig.eigen$vectors[, 1:100]
dim(PCA.dir)
```

```
## [1] 256 100
```

PCA directions are orthonormal in the usual Euclidean inner product:

```
round(t(PCA.dir[, 1:10]) %*% PCA.dir[, 1:10], digits = 2)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,]    1    0    0    0    0    0    0    0    0    0
## [2,]    0    1    0    0    0    0    0    0    0    0
## [3,]    0    0    1    0    0    0    0    0    0    0
## [4,]    0    0    0    1    0    0    0    0    0    0
## [5,]    0    0    0    0    1    0    0    0    0    0
## [6,]    0    0    0    0    0    1    0    0    0    0
## [7,]    0    0    0    0    0    0    1    0    0    0
## [8,]    0    0    0    0    0    0    0    1    0    0
## [9,]    0    0    0    0    0    0    0    0    1    0
## [10,]   0    0    0    0    0    0    0    0    0    1
```

FDA directions are not:

```
round(t(FDA.eigen.dir) %*% FDA.eigen.dir, digits = 2)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## [1,] 112.81 -36.31 -1.72 -74.75 21.61 -23.03 28.10 84.45 25.98
## [2,] -36.31 76.33 10.78 36.52 -16.64 0.05 -63.48 -22.84 -3.74
## [3,] -1.72 10.78 75.85 5.00 4.98 2.03 -18.44 -8.86 1.40
## [4,] -74.75 36.52 5.00 221.53 -15.22 20.63 -0.99 -78.55 -47.91
## [5,] 21.61 -16.64 4.98 -15.22 113.25 -24.92 -8.64 -0.74 -12.94
## [6,] -23.03 0.05 2.03 20.63 -24.92 85.31 36.17 -22.76 -2.01
## [7,] 28.10 -63.48 -18.44 -0.99 -8.64 36.17 217.71 6.06 -2.48
## [8,] 84.45 -22.84 -8.86 -78.55 -0.74 -22.76 6.06 165.63 6.32
## [9,] 25.98 -3.74 1.40 -47.91 -12.94 -2.01 -2.48 6.32 101.53
```

They are orthogonal with respect to W :

```
round(t(FDA.eigen.dir) %*% W %*% FDA.eigen.dir, digits = 2)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## [1,]    1    0    0    0    0    0    0    0    0
## [2,]    0    1    0    0    0    0    0    0    0
## [3,]    0    0    1    0    0    0    0    0    0
## [4,]    0    0    0    1    0    0    0    0    0
## [5,]    0    0    0    0    1    0    0    0    0
## [6,]    0    0    0    0    0    1    0    0    0
## [7,]    0    0    0    0    0    0    1    0    0
## [8,]    0    0    0    0    0    0    0    1    0
## [9,]    0    0    0    0    0    0    0    0    1
```

A practical compromise is to reduce dimension with PCA first and then run LDA/FDA in the principal subspace. That often trades a little training separation for a more stable test result.

```
newX.train <- X.train %*% PCA.dir
newX.test  <- X.test  %*% PCA.dir
new.digits.lda <- lda(newX.train, Y.train)
Y.test.pred <- predict(new.digits.lda, newX.test)$class
table(Y.test, Y.test.pred)
```

```
##      Y.test.pred
## Y.test  0  1  2  3  4  5  6  7  8  9
##      0 25  0  0  0  0  0  0  0  0
##      1  0 25  0  0  0  0  0  0  0
##      2  0  0 23  0  0  0  0  0  1
##      3  0  0  0 16  0  0  0  1  1
##      4  0  2  0  0 18  0  0  0  0
##      5  0  0  0  4  0  9  0  0  0
##      6  0  0  0  0  0  0 17  0  0
##      7  0  0  0  0  0  0  0 20  0
##      8  0  1  1  0  2  0  0  0 12  0
##      9  0  0  0  0  0  0  0  2  0 15
```

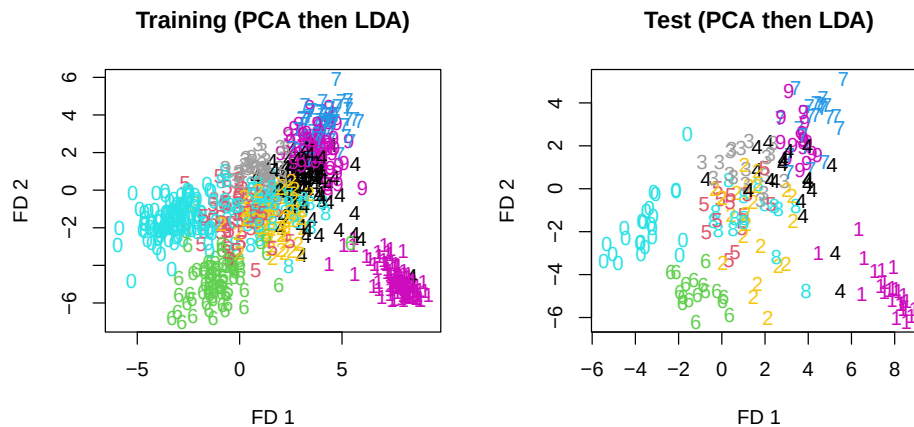
```
mean(Y.test != Y.test.pred) # test error
```

```
## [1] 0.1
```

```
newF <- newX.train %*% new.digits.lda$scaling
newFtest <- newX.test %*% new.digits.lda$scaling

par(mfrow = c(1, 2))
plot(newF[, 1], newF[, 2], type = "n", xlab = "FD 1", ylab = "FD 2",
     main = "Training (PCA then LDA)")
text(newF[, 1], newF[, 2], Y.train, col = Y.train + 5)
```

```
plot(newFtest[, 1], newFtest[, 2], type = "n", xlab = "FD 1",
     ylab = "FD 2", main = "Test (PCA then LDA)")
text(newFtest[, 1], newFtest[, 2], Y.test, col = Y.test + 5)
```



Training separation is less dramatic after the $256 \rightarrow 100$ compression, but the training and test pictures are more consistent—the usual signature that we have given up some overfit.

Naive Bayes Prediction

As a high-dimensional baseline we fit Gaussian naive Bayes to a two-class slice (digits 1 versus 7), using `e1071::naiveBayes`. Restricting to two classes makes the confusion matrix easy to read; the same function will run on all ten digits.

```
zip17 <- train.zip[train.zip[, 1] %in% c(1, 7), ]
zip17.test <- test.zip[test.zip[, 1] %in% c(1, 7), ]
Y.17 <- as.factor(zip17[, 1])
X.17 <- zip17[, -1]
```

```
NBfit <- naiveBayes(X.17, Y.17)
y_pred <- predict(NBfit, zip17.test[, -1])
acc.nb <- mean(y_pred == zip17.test[, 1])
acc.nb
```

```
## [1] 0.9791667
```

```
table(Predicted = y_pred, Actual = zip17.test[, 1])
```

```
##           Actual
## Predicted  1   7
##           1 25  1
##           7  0 22
```

Chapter 8

Logistic Regression

In the previous section, we discussed how discriminant analysis models the joint distribution of the predictors and the class variable. In this section, we focus on logistic regression, which takes a different approach: it **directly** models the conditional probability of class membership as a function of the predictors. For *binary outcomes*, this is achieved through the *logistic function*, which maps a linear combination of predictors to values between 0 and 1.

Logistic regression has the advantage of making fewer assumptions about the distribution of the predictors, provides coefficients that can be interpreted in terms of odds ratios, and can be extended to multiple classes through multinomial formulations. Because of its simplicity, interpretability, and solid probabilistic foundation, logistic regression is one of the most widely used methods for classification.

8.1 Fitting a Logistic Regression Model

Logistic regression starts by directly modeling the *conditional* probability

$$\eta(x) = P(Y = 1 \mid X = x),$$

where $\eta(x)$ is a function bounded in $[0, 1]$.

Specifically, consider a **link** function g that maps $\eta(x)$ into $(-\infty, \infty)$, i.e.

$$g(\eta(x)) = \mathbf{x}^T \beta.$$

Based on this formulation, the response Y conditional on X follows a Bernoulli distribution:

$$P(Y = y_i \mid X = x_i) = \eta(x_i)^{y_i} (1 - \eta(x_i))^{1-y_i},$$

where

$$\eta(x) = P(Y = 1 \mid X = x) \quad \text{and} \quad 1 - \eta(x) = P(Y = 0 \mid X = x).$$

For logistic regression, we use the **logit link**, defined by

$$\log \frac{\eta(x)}{1 - \eta(x)} = x^T \beta \quad \Leftrightarrow \quad \eta(x) = \frac{\exp(x^T \beta)}{1 + \exp(x^T \beta)}.$$

The quantity $\log \frac{\eta(x)}{1 - \eta(x)}$ is called the **log-odds**, and we model it as a **linear** function of x .

8.1.1 Maximum Likelihood Approach

At each data point we observe (x_i, y_i) , and we want to use these pairs to estimate $\eta(x)$, or equivalently β . Instead of minimizing a squared-error loss, as we did for a continuous response, we follow a maximum likelihood approach.

Maximum likelihood estimation is a method that provides the *most likely value* of a parameter, given the data we observed. In a parametric framework, the likelihood is a function of the model parameters and the data, and the maximum likelihood estimator is obtained by maximizing the likelihood with respect to the parameters.

Recall that, conditionally on X , the response Y follows a Bernoulli distribution with probability $\eta(x)$. Therefore, the likelihood function is

$$\begin{aligned} \mathcal{L}(\beta \mid \mathbf{y}, \mathbf{x}) &= \prod_{i=1}^n p(y_i \mid x_i, \beta) \\ &= \prod_{i=1}^n \eta(x_i)^{y_i} (1 - \eta(x_i))^{1-y_i}. \end{aligned}$$

To fit the logistic regression model, we maximize the likelihood with respect to β :

$$\max_{\beta} \mathcal{L}(\beta \mid \mathbf{y}, \mathbf{x}).$$

This is equivalent to maximizing the log of the likelihood, the *log-likelihood function*:

$$\begin{aligned} \ell(\beta \mid \mathbf{y}, \mathbf{x}) &= \sum_{i=1}^n \log \left\{ \eta(x_i)^{y_i} (1 - \eta(x_i))^{1-y_i} \right\} \\ &= \sum_{i=1}^n \left\{ y_i \log \frac{\eta(x_i)}{1 - \eta(x_i)} + \log(1 - \eta(x_i)) \right\}. \end{aligned}$$

Recall that we defined

$$\log \frac{\eta(x)}{1 - \eta(x)} = x^T \beta.$$

Combining the expressions above, after a few lines of algebra, the log-likelihood becomes

$$\ell(\beta \mid \mathbf{y}, \mathbf{x}) = \sum_{i=1}^n \left\{ y_i x_i^T \beta - \log(1 + \exp(x_i^T \beta)) \right\},$$

and the maximum likelihood estimator (MLE) $\hat{\beta}$ is obtained by *maximizing*

$$\max_{\beta} \ell(\beta \mid \mathbf{y}, \mathbf{x}) = \max_{\beta} \sum_{i=1}^n \left\{ y_i x_i^T \beta - \log(1 + \exp(x_i^T \beta)) \right\}.$$

Note that β is a vector, so we compute the gradient $\frac{\partial \ell(\beta \mid \mathbf{y}, \mathbf{x})}{\partial \beta}$, set it equal to 0, solve for β , and check the Hessian $\frac{\partial^2 \ell(\beta \mid \mathbf{y}, \mathbf{x})}{\partial \beta \partial \beta^T}$ to confirm that the critical point is a maximum.

Solving for the MLE

Differentiating with respect to β and using the chain rule, the gradient is

$$\begin{aligned} \frac{\partial \ell(\beta \mid \mathbf{y}, \mathbf{x})}{\partial \beta} &= \sum_{i=1}^n \frac{\partial}{\partial \beta} (y_i x_i^T \beta) - \frac{\partial}{\partial \beta} \left(\log(1 + \exp(x_i^T \beta)) \right) \\ &= \sum_{i=1}^n y_i x_i^T - \sum_{i=1}^n \frac{1}{1 + \exp(x_i^T \beta)} \cdot \frac{\partial}{\partial \beta} (1 + \exp(x_i^T \beta)) \\ &= \sum_{i=1}^n y_i x_i^T - \sum_{i=1}^n \frac{1}{1 + \exp(x_i^T \beta)} \cdot \exp(x_i^T \beta) x_i^T \\ &= \sum_{i=1}^n y_i x_i^T - \sum_{i=1}^n \frac{\exp(x_i^T \beta) x_i^T}{1 + \exp(x_i^T \beta)}. \end{aligned}$$

To obtain the Hessian, we differentiate again with respect to β , again using the chain rule:

$$\begin{aligned}
\frac{\partial^2 \ell(\beta \mid \mathbf{y}, \mathbf{x})}{\partial \beta \partial \beta^T} &= \sum_{i=1}^n \frac{\partial}{\partial \beta} y_i x_i^T - \frac{\partial}{\partial \beta} \left(\frac{\exp(x_i^T \beta) x_i^T}{1 + \exp(x_i^T \beta)} \right) \\
&= - \sum_{i=1}^n \frac{\partial}{\partial \beta} \left(\frac{\exp(x_i^T \beta) x_i^T}{1 + \exp(x_i^T \beta)} \right) \\
&= - \sum_{i=1}^n \frac{\partial}{\partial (\exp(x_i^T \beta))} \left(\frac{\exp(x_i^T \beta) x_i^T}{1 + \exp(x_i^T \beta)} \right) \frac{\partial (\exp(x_i^T \beta))}{\partial \beta} \\
&= - \sum_{i=1}^n x_i \frac{\exp(x_i^T \beta)}{1 + \exp(x_i^T \beta)} \frac{1}{1 + \exp(x_i^T \beta)} x_i^T \\
&= - \sum_{i=1}^n x_i \eta(x_i) (1 - \eta(x_i)) x_i^T.
\end{aligned}$$

Finally, we have

- **Gradient vector**

$$\frac{\partial \ell(\beta \mid \mathbf{y}, \mathbf{x})}{\partial \beta} = \sum_{i=1}^n y_i x_i^T - \sum_{i=1}^n \frac{\exp(x_i^T \beta) x_i^T}{1 + \exp(x_i^T \beta)} = \mathbf{X}^T (\mathbf{y} - \mathbf{p}),$$

where

$$p_i = \eta(x_i) = P(Y = 1 \mid X = x_i).$$

- **Hessian matrix**

$$\frac{\partial^2 \ell(\beta \mid \mathbf{y}, \mathbf{x})}{\partial \beta \partial \beta^T} = - \sum_{i=1}^n x_i \eta(x_i) (1 - \eta(x_i)) x_i^T = -\mathbf{X}^T \mathbf{W} \mathbf{X}.$$

The Hessian is *negative semi-definite*, so the log-likelihood is *concave*, and therefore *any local maximum is a global maximum*.

There is no closed-form expression for the MLE in logistic regression, but it can be obtained with an iterative Newton–Raphson procedure.

Newton–Raphson for the Logistic MLE

$$\begin{aligned}
f(x) &\approx f(x_0) + f'(x_0)(x - x_0) \\
\Rightarrow x &= x_0 - \frac{f(x_0)}{f'(x_0)}
\end{aligned}$$

In our case,

$$\begin{aligned}\beta &= \beta_0 + (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T (\mathbf{y} - \mathbf{p}) \\ &= (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} (\mathbf{X} \beta_0 + \mathbf{W}^{-1} (\mathbf{y} - \mathbf{p})).\end{aligned}$$

We can also obtain the logistic regression MLE via an *iteratively reweighted least squares* (IRLS) algorithm:

1. Start with some initial values β^0 .
2. Calculate the corresponding $p_i^0 = \eta^0(x_i)$ for $i = 1, \dots, n$.
3. Define $\mathbf{W} = \text{diag}(p_i^0(1 - p_i^0))_{i=1}^n$.
4. Calculate

$$\mathbf{z} = \mathbf{X} \beta^0 + \mathbf{W}^{-1} (\mathbf{y} - \mathbf{p}^0).$$

5. Update $\beta^0 \leftarrow \beta^1$ with

$$\beta^1 = (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \mathbf{z}.$$

6. Iterate the above steps until convergence.

Interpreting Parameters

What is the effect of β ? A one-unit increase in X_j increases the log-odds of $Y = 1$ by β_j (holding the other predictors fixed).

8.2 Logistic Regression with Separable Data

Consider the following scenario with two classes, each containing two points. Assume that the intercept is zero for simplicity.

In the log-likelihood, the contribution of the **orange points** ($y = 1$) involves

$$\frac{e^{\beta_1 x_1 + \beta_2 x_2}}{1 + e^{\beta_1 x_1 + \beta_2 x_2}} = \frac{1}{1 + e^{-(\beta_1 x_1 + \beta_2 x_2)}},$$

while the contribution of the **blue points** ($y = 0$) involves

$$\frac{1}{1 + e^{\beta_1 x_1 + \beta_2 x_2}}.$$

The only difference between the two expressions is a minus sign in the exponent.

In this setting, as the β s grow in a separating direction, the fitted probabilities approach 0 or 1 and the likelihood keeps increasing, so the algorithm does not converge. The decision boundary itself stays the same, so the fitted classifier can still be used for prediction.

Remark: A natural question is whether regularization helps. An L_2 (ridge) penalty with $\lambda > 0$, and typically an L_1 (lasso) penalty as well, keeps the coefficients finite. The unpenalized MLE is the quantity that diverges: when the classes are perfectly separable, the likelihood increases as $\|\beta\| \rightarrow \infty$ along a separating direction, and (β_0, β) is no longer a well-defined interior maximizer.

8.3 Retrospective Sampling

So far we have assumed that the data are i.i.d. draws from a population of interest. **But** in some settings, such as studying a rare disease, sampling the population as it is would require a very large sample—thousands or millions of observations—which is highly inefficient.

Instead, we draw a balanced sample from patients with the disease and from healthy controls; say 50 individuals in each group. When we later evaluate the model on a test sample from the original population, the class balance will not match the 50–50 split used in training.

To address this in the model, define a random variable Z indicating whether an individual was sampled. The sampling scheme leads us to work with

$$P(Y = 1 \mid Z = 1, X = x)$$

rather than $P(Y = 1 \mid X = x)$. In the logistic regression framework, write

$$r_0 = P(Z = 1 \mid Y = 0), \quad r_1 = P(Z = 1 \mid Y = 1).$$

Then

$$\begin{aligned} P(Y = 1 \mid Z = 1, X = x) &= \frac{P(Z = 1 \mid Y = 1, x) P(Y = 1 \mid x)}{P(Z = 1, Y = 0 \mid x) + P(Z = 1, Y = 1 \mid x)} \\ &= \frac{r_1 \exp(\alpha + x^T \beta)}{r_0 + r_1 \exp(\alpha + x^T \beta)} \\ &= \frac{\exp(\alpha^* + x^T \beta)}{1 + \exp(\alpha^* + x^T \beta)}, \end{aligned}$$

where $\alpha^* = \alpha + \log \frac{r_1}{r_0}$. Although the data were sampled retrospectively, the logistic model still applies, with the same slope coefficients β and a different intercept.

8.4 Logistic Regression for the Heart Data Set in R

The Python version of the example can be found here: [Python Code for Logistic](#).

In this example we work with the South African heart data, available [here](#).

```
SAheart = read.csv("data/week8/SAheart.csv", header = TRUE)
```

The `SAheart` data set comes from a study of heart-disease risk factors among South African men of mixed ancestry. It contains 462 observations on lifestyle, medical history, and physiological measurements—age, alcohol intake, tobacco use, blood pressure, cholesterol, and family history of heart disease, among others.

Our goal is to estimate the probability that an individual has coronary heart disease (CHD).

The variables are:

- `sbp` - systolic blood pressure
- `tobacco` - cumulative tobacco (kg)
- `ldl` - low-density lipoprotein cholesterol
- `adiposity` - a numeric measure of relative body fat
- `famhist` - family history of heart disease (`Present`, `Absent`)
- `typea` - Type A behavior
- `obesity`
- `alcohol` - current alcohol consumption
- `age` - age at onset
- `chd` - response: coronary heart disease (1 = yes, 0 = no)

```
heart.logistic = glm(chd ~ ., data = SAheart, family = binomial)
summary(heart.logistic)
```

```
##
## Call:
## glm(formula = chd ~ ., family = binomial, data = SAheart)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -5.9207616   1.3265724  -4.463 8.07e-06 ***
## row.names     -0.0008844   0.0008950  -0.988 0.323042
## sbp           0.0076602   0.0058574   1.308 0.190942
```

```
## tobacco      0.0777962  0.0266602   2.918 0.003522 **
## ldl          0.1701708  0.0597998   2.846 0.004432 **
## adiposity    0.0209609  0.0294496   0.712 0.476617
## famhistPresent 0.9385467  0.2287202   4.103 4.07e-05 ***
## typea       0.0376529  0.0124706   3.019 0.002533 **
## obesity     -0.0661926  0.0443180  -1.494 0.135285
## alcohol      0.0004222  0.0045053   0.094 0.925346
## age         0.0441808  0.0121784   3.628 0.000286 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 596.11  on 461  degrees of freedom
## Residual deviance: 471.16  on 451  degrees of freedom
## AIC: 493.16
##
## Number of Fisher Scoring iterations: 5
```

We obtain fitted values from the model and summarize the classifications at the 0.5 threshold:

```
yhat = (heart.logistic$fitted.values > 0.5)
table(yhat, SAheart$chd)
```

```
##
## yhat      0    1
## FALSE 255   79
## TRUE   47   81
```

8.4.1 Goodness of Fit

In logistic regression we do not have a residual sum of squares (RSS), so we need other metrics to assess fit. Deviance measures, which are also based on the likelihood, are the standard choice. When all of the x_i are unique, the residual deviance is

$$\text{Deviance} = -2 \left(\sum_{i:y_i=1} \log \hat{p}_i + \sum_{i:y_i=0} \log(1 - \hat{p}_i) \right),$$

where $\hat{p}_i$ is the fitted probability $P(Y = 1 \mid X = x_i)$.

At the bottom of the `summary` output in R we see two deviances:

- **Null deviance:** the deviance of a logistic regression with only an intercept. Here the estimated probability $P(Y = 1 \mid X = x_i)$ is the sample mean of the observed y_i values and does not depend on x_i .

- **Residual deviance:** the deviance of the fitted model under consideration, using the model's fitted probabilities $\hat{p}_i$.

We can also compute them directly:

```
p.hat = heart.logistic$fitted.values

-2 * (sum(log(p.hat[SAheart$chd == 1])) + sum(log(1 - p.hat[SAheart$chd ==
0])))

## [1] 471.1591

p0.hat = rep(mean(SAheart$chd), length(SAheart$chd))

-2 * (sum(log(p0.hat[SAheart$chd == 1])) + sum(log(1 - p0.hat[SAheart$chd ==
0])))

## [1] 596.1084
```

Solving this problem using numerical optimization, without glm

```
## Log-likelihood (not the negative log-likelihood)
logistic.loglik <- function(b, x, y) {
  bm = as.matrix(b)
  xb = x %*% bm
  # this returns the log-likelihood
  return(sum(y * xb) - sum(log(1 + exp(xb))))
}

## Gradient
logistic.gradient <- function(b, x, y) {
  bm = as.matrix(b)
  expxb = exp(x %*% bm)
  return(t(x) %*% (y - expxb/(1 + expxb)))
}

## We can now use the `optim` function to solve the
## problem:

SAheart$famhist = ifelse(SAheart$famhist == "Present", 1, 0)

x = as.matrix(cbind(intercept = 1, SAheart[, 2:10]))
y = as.matrix(SAheart[, 11])
b = rep(0, ncol(x))

# The objective above is the log-likelihood, so we maximize
# (equivalently, optim minimizes -ell when fnscale = -1):
```

```

optim(b, fn = logistic.loglik, gr = logistic.gradient, method = "BFGS",
      x = x, y = y, control = list(fnscale = -1))

## $par
## [1] -6.150733305  0.006504017  0.079376464  0.173923988  0.018586578
## [6]  0.925372019  0.039595096 -0.062909867  0.000121675  0.045225500
##
## $value
## [1] -236.07
##
## $counts
## function gradient
##      74      16
##
## $convergence
## [1] 0
##
## $message
## NULL

```

The result matches the `glm()` solution. If a general-purpose solver is not available, we can use Newton–Raphson instead; that requires a function for the Hessian.

8.4.2 Variable Selection

Variable selection can also be needed in a logistic regression setting. We next ask which of the methods and criteria from linear regression carry over.

AIC/BIC

We can still use AIC and BIC because the logistic model has a likelihood. Unlike linear regression, however, we cannot use the leaps-and-bounds algorithm to enumerate all submodels, because the logistic likelihood is not a residual sum of squares. Instead we use a greedy search: stepwise, backward, or forward selection.

As an example:

```

step.logistic = step(heart.logistic, scope = list(upper = ~.,
lower = ~1))

## Start:  AIC=493.16
## chd ~ row.names + sbp + tobacco + ldl + adiposity + famhist +
##      typea + obesity + alcohol + age
##

```

```

##           Df Deviance    AIC
## - alcohol    1   471.17 491.17
## - adiposity   1   471.67 491.67
## - row.names   1   472.14 492.14
## - sbp         1   472.88 492.88
## <none>        1   471.16 493.16
## - obesity     1   473.47 493.47
## - ldl         1   479.65 499.65
## - tobacco     1   480.27 500.27
## - typea       1   480.75 500.75
## - age         1   484.76 504.76
## - famhist     1   488.29 508.29
##
## Step:  AIC=491.17
## chd ~ row.names + sbp + tobacco + ldl + adiposity + famhist +
##       typea + obesity + age
##
##           Df Deviance    AIC
## - adiposity   1   471.69 489.69
## - row.names   1   472.14 490.14
## - sbp         1   472.94 490.94
## <none>        1   471.17 491.17
## - obesity     1   473.49 491.49
## + alcohol     1   471.16 493.16
## - ldl         1   479.68 497.68
## - tobacco     1   480.73 498.73
## - typea       1   480.80 498.80
## - age         1   484.82 502.82
## - famhist     1   488.43 506.43
##
## Step:  AIC=489.69
## chd ~ row.names + sbp + tobacco + ldl + famhist + typea + obesity +
##       age
##
##           Df Deviance    AIC
## - row.names   1   472.55 488.55
## - sbp         1   473.55 489.55
## <none>        1   471.69 489.69
## - obesity     1   473.86 489.86
## + adiposity   1   471.17 491.17
## + alcohol     1   471.67 491.67
## - typea       1   481.05 497.05
## - tobacco     1   481.35 497.35
## - ldl         1   481.68 497.68
## - famhist     1   488.89 504.89
## - age         1   493.97 509.97

```

```
##
## Step:  AIC=488.55
## chd ~ sbp + tobacco + ldl + famhist + typea + obesity + age
##
##           Df Deviance    AIC
## - sbp      1   473.98 487.98
## <none>      1   472.55 488.55
## - obesity  1   474.65 488.65
## + row.names 1   471.69 489.69
## + adiposity 1   472.14 490.14
## + alcohol   1   472.55 490.55
## - tobacco   1   482.54 496.54
## - ldl        1   482.95 496.95
## - typea     1   483.19 497.19
## - famhist   1   489.38 503.38
## - age       1   495.48 509.48
##
## Step:  AIC=487.98
## chd ~ tobacco + ldl + famhist + typea + obesity + age
##
##           Df Deviance    AIC
## - obesity  1   475.69 487.69
## <none>      1   473.98 487.98
## + sbp      1   472.55 488.55
## + adiposity 1   473.47 489.47
## + row.names 1   473.55 489.55
## + alcohol   1   473.94 489.94
## - tobacco   1   484.18 496.18
## - typea     1   484.30 496.30
## - ldl        1   484.53 496.53
## - famhist   1   490.58 502.58
## - age       1   502.11 514.11
##
## Step:  AIC=487.69
## chd ~ tobacco + ldl + famhist + typea + age
##
##           Df Deviance    AIC
## <none>      1   475.69 487.69
## + obesity   1   473.98 487.98
## + sbp       1   474.65 488.65
## + row.names 1   475.26 489.26
## + adiposity 1   475.44 489.44
## + alcohol   1   475.65 489.65
## - ldl        1   484.71 494.71
## - typea     1   485.44 495.44
## - tobacco   1   486.03 496.03
```

```
## - famhist      1    492.09 502.09
## - age          1    502.38 512.38
```

8.4.3 Penalized Logistic Regression

As in ordinary regression, we can penalize the logistic log-likelihood to handle collinearity or high-dimensional settings. In R this is again available through the `glmnet` package.

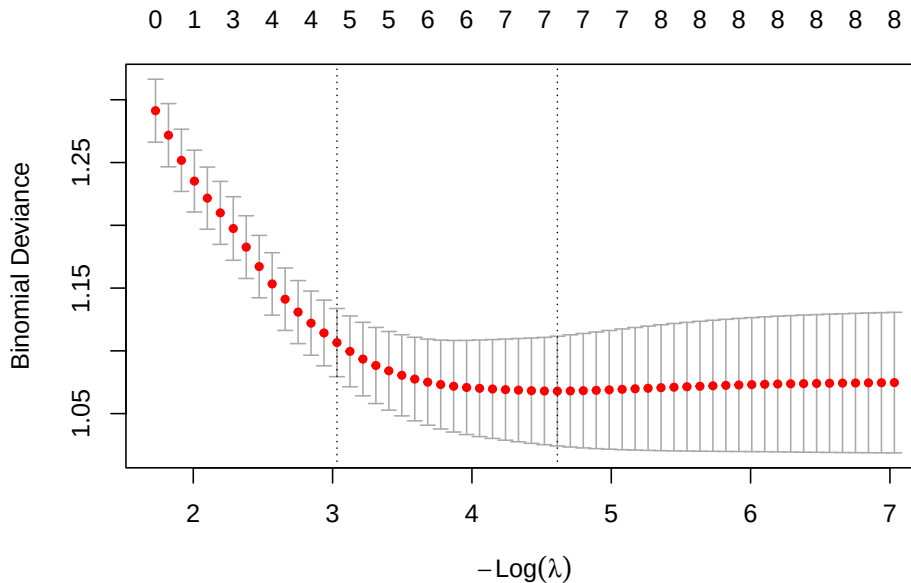
```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 5.0
```

```
lasso.fit = cv.glmnet(x = data.matrix(SAheart[, 2:10]), y = SAheart[,
  11], nfolds = 10, family = "binomial")
```

```
plot(lasso.fit)
```



```
coef(lasso.fit, s = "lambda.min")
```

```
## 10 x 1 sparse Matrix of class "dgCMatrix"
##           lambda.min
## (Intercept) -5.73717333
## sbp          0.00416794
## tobacco      0.07056290
## ldl          0.14789200
## adiposity    .
```

```
## famhist      0.81077289
## typea       0.02967640
## obesity     -0.01618876
## alcohol      .
## age         0.04396417
```

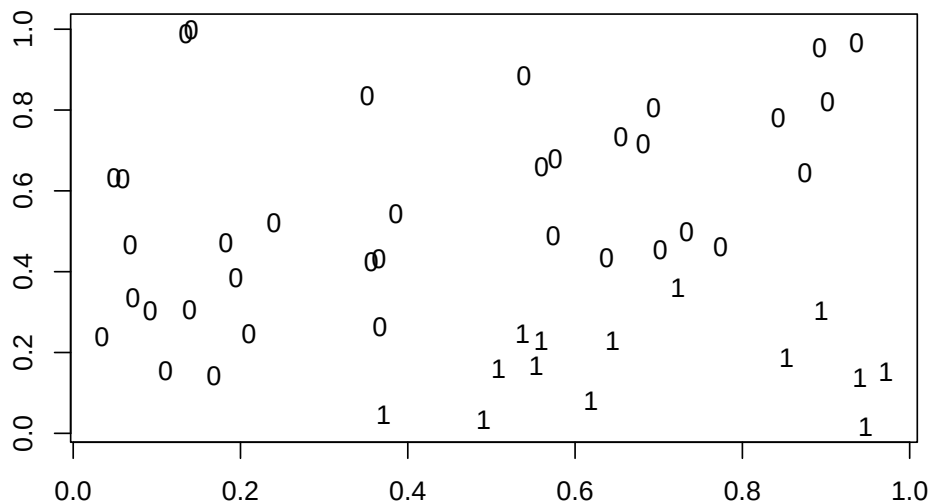
8.4.4 Illustration of Divergence of the MLE

Consider a very simple case in which

$$y = \begin{cases} 1, & \text{if } x_1 - 2x_2 > 0, \\ 0, & \text{otherwise.} \end{cases}$$

Simulate such data:

```
x = matrix(runif(100), 50, 2)
y = ifelse(x[, 1] - 2 * x[, 2] > 0, 1, 0)
plot(x[, 1], x[, 2], type = "n", xlab = "", ylab = "")
text(x[, 1], x[, 2], y)
```



```
logistic.fit = glm(y ~ x, family = binomial)
```

```
## Warning: glm.fit: algorithm did not converge
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

The warning is exactly the phenomenon discussed above: the MLE of the coefficients tends to $\pm\infty$ and the algorithm does not converge.

Even so, the separating line is well defined, so the fitted model can still be used for prediction.

```
round(logistic.fit$fitted.values, 0)
```

```
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
##  0  0  0  1  1  0  1  0  1  0  1  1  0  0  0  0  0  0  1  0  0  0  1  0  0  0
## 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
##  0  0  0  0  0  1  0  1  1  1  0  1  0  0  0  0  0  0  0  0  0  0  1  0  0
```


Chapter 9

Support Vector Machines

Support Vector Machines (SVMs) are a powerful approach to *supervised learning*, especially for classification. Rather than searching for *any* boundary that separates the classes, SVMs seek the boundary that maximizes the **margin**, that is the distance between the separating hyperplane and the nearest training points from each class. This focus on the margin is a form of geometric regularization. In the hard-margin linear case it amounts to keeping $\|\beta\|$ small, which controls a complexity measure of the classifier (its VC dimension, or a Rademacher bound). That is a *reason* to expect better generalization to *new* data; it is not a guarantee on any given sample. The same construction highlights the *support vectors*: the subset of training points that actually define the decision boundary.

The flexibility of SVMs extends far beyond linear settings. Through *kernel functions*, the data can be treated as if they had been mapped into a higher-dimensional feature space, so that a linear rule in that space becomes a nonlinear rule in the original x . The mapping itself need not be computed. In this way, SVMs combine geometric intuition, optimization, and statistical regularization into a single framework.

9.1 Linear SVM in the Separable Case: *Separation Margin*

We begin with the hard-margin linear classifier, the version associated with Vapnik and Chervonenkis in the 1960s and later developed into the modern SVM (Cortes and Vapnik, 1995). Assume we observe **training data** $\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^p$ are the *features* and $y_i \in \{-1, 1\}$ is the class label coded as -1 or $+1$.

The goal is to estimate a real-valued function $f(x)$ and the classification rule

$C(x) = \text{sign}\{f(x)\}$. The Bayes-optimal classifier (equal misclassification costs) is

$$C^*(x) = \text{sign}\left\{P(Y = 1 \mid X = x) - P(Y = -1 \mid X = x)\right\}.$$

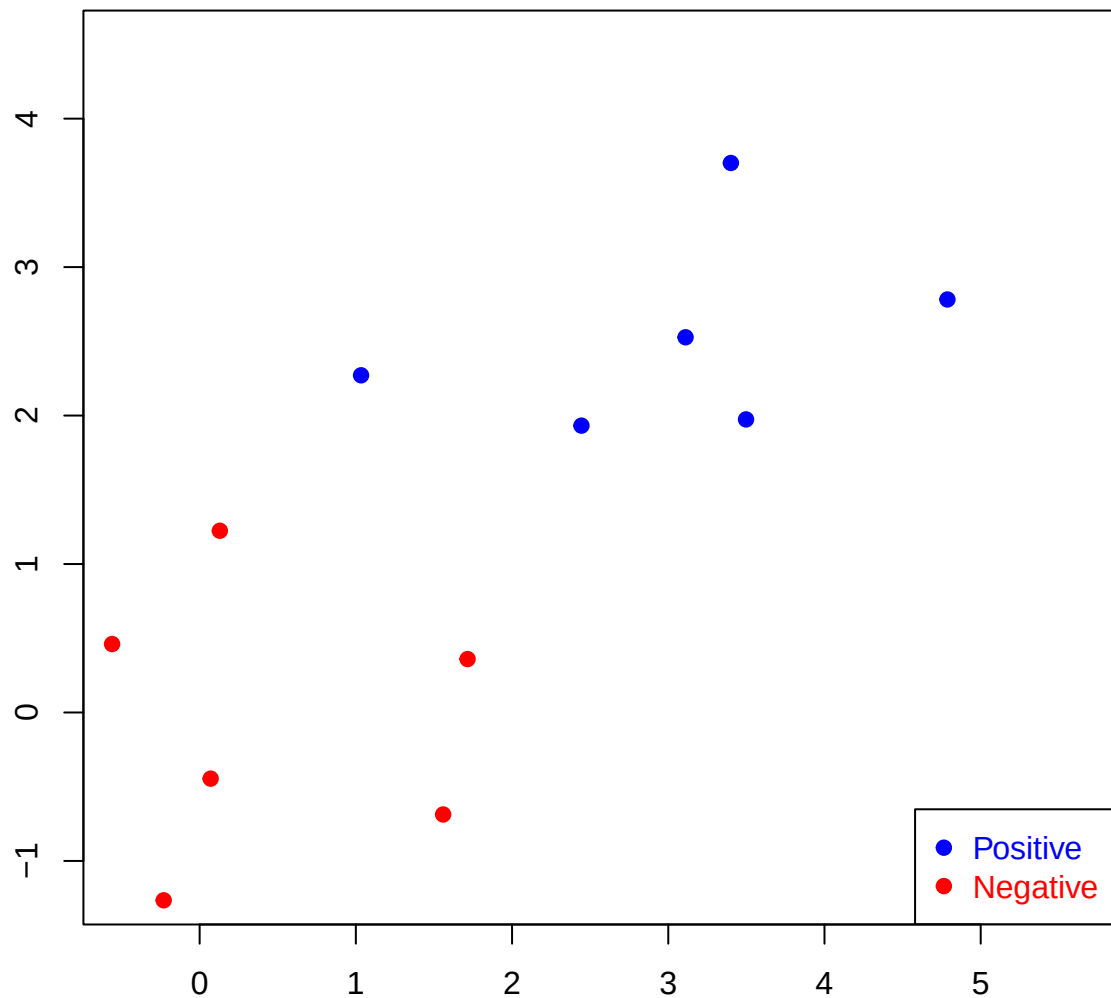
In contrast to logistic regression or DA, SVMs do **not** model $P(Y \mid X)$ or the class-conditional densities $P(X \mid Y)$. They **directly model the decision boundary**. The fitted score $f(x)$ is (up to scale) a signed distance to that boundary, not a class probability. If probabilities are needed later, a common device is Platt scaling: a logistic regression of y on $f(x)$, typically on a hold-out sample. That step is not part of the SVM itself.

9.1.1 Maximum-Margin Classifier

With labels $y_i \in \{-1, 1\}$, the classification rule based on $f(x)$ is

$$\begin{aligned}\hat{y} &= +1 && \text{if } f(x) > 0, \\ \hat{y} &= -1 && \text{if } f(x) < 0.\end{aligned}$$

The classification is correct if $y_i f(x_i) > 0$. The quantity $y_i f(x_i)$ is the **functional margin**: if it is *positive*, the point is on the correct side; if it is *negative*, the point is on the wrong side. Larger positive values mean a more confident correct classification.



If the classes are **linearly separable**, we look for a linear function $f(x) = x^T \beta + \beta_0$ that separates the two groups. Many such hyperplanes exist:

Among all these lines, which one is best?

Although many lines can perfectly separate the two classes, SVMs define the **best** line as the line that maximizes the **separation margin**, that is the width between the two dotted lines:

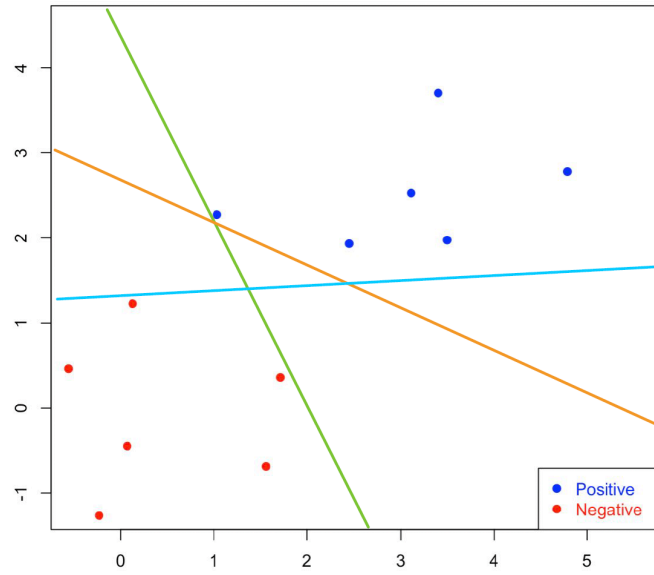


Figure 9.1: Separating lines: which one is best?

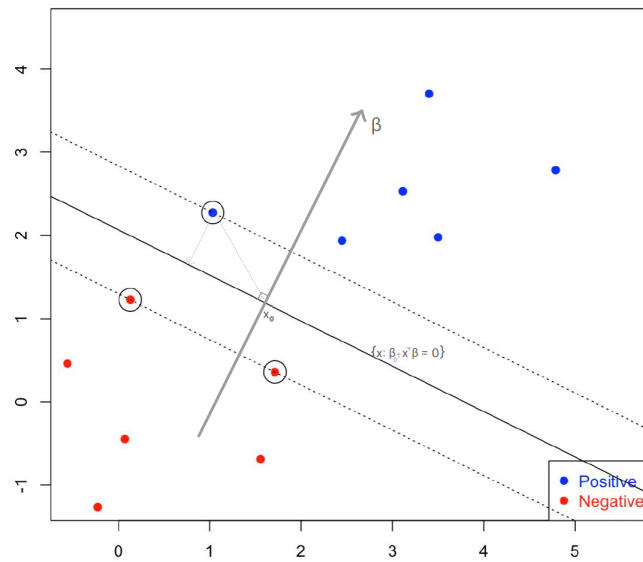


Figure 9.2: SVM: maximum-margin separating line

Mathematical Formulation

Write the linear decision function as

$$f(x) = \beta_0 + x^T \beta.$$

The **separating hyperplane** is the level set

$$L = \{x : \beta_0 + x^T \beta = 0\}.$$

In the separable case, every observation with $y_i = +1$ satisfies $f(x_i) > 0$, and every observation with $y_i = -1$ satisfies $f(x_i) < 0$.

The **signed distance** of a point x to the hyperplane is

$$\left\langle \frac{\beta}{\|\beta\|}, x - x_0 \right\rangle,$$

where $x_0 \in L$.

Remark. When we define a distance we usually take an unsigned version, $|x - x_0|$ or a square, which is always nonnegative. In the SVM derivation we *keep the sign*, because the sign records which side of the hyperplane the point occupies. A positive signed distance means x lies on the side toward which β points; a negative signed distance means x lies on the opposite side.

To compute the signed distance, pick any $x_0 \in L$, so

$$f(x_0) = 0 \quad \Leftrightarrow \quad x_0^T \beta = -\beta_0.$$

Project $x - x_0$ onto the unit normal $\beta/\|\beta\|$:

$$\begin{aligned} \left\langle \frac{\beta}{\|\beta\|}, x - x_0 \right\rangle &= (x - x_0)^T \frac{\beta}{\|\beta\|} \\ &= \frac{1}{\|\beta\|} (x^T \beta + \beta_0) = \frac{1}{\|\beta\|} f(x). \end{aligned}$$

Thus $f(x)$ is proportional to the signed distance from x to L . When $\|\beta\| = 1$, $f(x)$ is that signed distance.

The **goal** of the SVM is to separate the two classes and maximize the distance to the closest points from either class (Vapnik, 1996), that is, to construct the **maximum margin**:

$$\begin{aligned} &\max_{\beta, \beta_0, \|\beta\|=1} M \\ \text{subject to} \quad &y_i(x_i^T \beta + \beta_0) \geq M, \quad i = 1, \dots, n. \end{aligned}$$

Every training point is then at least a signed distance M from the decision boundary. Equivalently, if $y_i = +1$ we require $f(x_i) \geq M$, and if $y_i = -1$ we

require $f(x_i) \leq -M$. Maximizing the minimum distance is exactly maximizing the margin.

The constraint $\|\beta\| = 1$ is inconvenient because the geometric margin also depends on the scale of β . Dropping the unit-norm restriction, the same requirement can be written

$$\frac{1}{\|\beta\|} y_i (x_i^T \beta + \beta_0) \geq M.$$

The scale of β is now free, so we may set $\|\beta\| = 1/M$ and rewrite the problem as

$$\begin{aligned} \min_{\beta, \beta_0} \quad & \frac{1}{2} \|\beta\|^2 \\ \text{subject to} \quad & y_i (x_i^T \beta + \beta_0) \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

This constraint says that every point is at least $1/\|\beta\|$ away from the separating plane and lies on the correct side of the corresponding margin line:

$$\begin{cases} x_i^T \beta + \beta_0 \geq 1, & \text{if } y_i = +1, \\ x_i^T \beta + \beta_0 \leq -1, & \text{if } y_i = -1. \end{cases}$$

The full margin width between the two dotted lines is $2/\|\beta\|$.

If x_i lies **on one of the dotted lines**, that is, if $y_i(x_i^T \beta + \beta_0) - 1 = 0$, the i th constraint is *active*. If $y_i(x_i^T \beta + \beta_0) - 1 > 0$, it is *inactive*. At the optimum there must be at least two active points, one from each class; otherwise the margin could be widened.

This is the primal form of the hard-margin SVM:

Primal form (hard margin)

$$\begin{aligned} \text{minimize}_{\beta, \beta_0} \quad & \frac{1}{2} \|\beta\|^2 \\ \text{subject to} \quad & y_i (x_i^T \beta + \beta_0) \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

The max-margin problem is a convex quadratic program with linear inequality constraints, so it has no local-optima issues *as a mathematical program*. Feature scaling, a barely separable sample, and solver tolerances can still give a poor numerical fit. This primal is feasible only in the perfectly separable case.

9.1.2 The KKT Conditions: From the Primal to the Dual

Statistically, the SVM is regularized hinge loss: $\sum_i (1 - y_i f(x_i))_+ + \lambda \|\beta\|^2$, with $\lambda = 1/C$. We derive the dual because that program is equivalent to the hinge problem and because the dual depends on the data only through inner products

$x_i^T x_j$. The quadratic program over α is the algorithm; the loss-plus-penalty criterion is the definition. We will return to that view after the kernels.

The Karush–Kuhn–Tucker (KKT) conditions are necessary conditions for optimality in constrained problems (and they are sufficient here because the program is convex). We review the idea briefly, then apply it to the SVM.

With no constraints, a minimum of a differentiable function f requires $\nabla f(x) = 0$: there is no direction in which f can still decrease.

Equality-Constrained Optimization

Now minimize $f(x)$ subject to $h(x) = 0$. The feasible set is $\mathcal{S} = \{x : h(x) = 0\}$. At a constrained minimum the gradient of f need not vanish, but any descent direction of f must be forbidden by the constraint: it must leave $\mathcal{S}$. That is expressed by

$$\nabla f(x) + \lambda \nabla h(x) = 0, \quad \lambda \in \mathbb{R}.$$

Equivalently, the Lagrangian

$$\mathcal{L}(x, \alpha) = f(x) + \alpha h(x)$$

is stationary in x . Here α is a Lagrange multiplier. Geometrically, $\nabla h(x)$ is orthogonal to the constraint surface. At a constrained local minimum, ∇f must be orthogonal to that surface as well, so ∇f and ∇h are parallel:

$$\nabla f(x^*) = -\alpha \nabla h(x^*).$$

Inequality-Constrained Optimization

Consider

$$\text{minimize}_x g(x) \quad \text{subject to} \quad h_i(x) \leq 0, \quad i = 1, \dots, n.$$

The Lagrangian is

$$\mathcal{L}(x, \alpha) = g(x) + \sum_{i=1}^n \alpha_i h_i(x), \quad \alpha_i \geq 0.$$

Two nested problems are associated with $\mathcal{L}$:

- The **primal** problem,

$$\min_x \max_{\alpha \geq 0} \mathcal{L}(x, \alpha).$$

If x violates a constraint, $h_i(x) > 0$, we can send $\alpha_i \rightarrow \infty$ and make $\mathcal{L}$ arbitrarily large. Hence the inner maximization is finite only for feasible x .

- The **dual** problem,

$$\max_{\alpha \geq 0} \min_x \mathcal{L}(x, \alpha).$$

In general

$$\underbrace{\max_{\alpha \geq 0} \min_x \mathcal{L}(x, \alpha)}_{\text{dual}} \leq \underbrace{\min_x \max_{\alpha \geq 0} \mathcal{L}(x, \alpha)}_{\text{primal}}.$$

Equality holds (strong duality) when g and the h_i are convex and a mild feasibility condition is satisfied. Both conditions hold for the SVM.

Rewrite the hard-margin SVM as an inequality-constrained program:

$$\begin{aligned} & \min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 \\ & \text{subject to} \quad -\{y_i(x_i^T \beta + \beta_0) - 1\} \leq 0, \quad i = 1, \dots, n. \end{aligned}$$

The Lagrangian is

$$\mathcal{L}(\beta, \beta_0, \alpha) = \frac{1}{2} \|\beta\|^2 - \sum_{i=1}^n \alpha_i \{y_i(x_i^T \beta + \beta_0) - 1\}, \quad \alpha_i \geq 0.$$

We solve the dual: first minimize $\mathcal{L}$ in (β, β_0) , then maximize in α .

9.1.3 Solving the Dual Problem

Stationarity in β and β_0 gives

$$\begin{aligned} \nabla_{\beta} \mathcal{L} = 0 & \Leftrightarrow \beta = \sum_{i=1}^n \alpha_i y_i x_i, \\ \nabla_{\beta_0} \mathcal{L} = 0 & \Leftrightarrow \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

Substitute these relations back into $\mathcal{L}$ to obtain the dual objective

$$\mathcal{L}(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y_i y_j \alpha_i \alpha_j x_i^T x_j.$$

The dual problem is therefore

$$\begin{aligned} & \max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n y_i y_j \alpha_i \alpha_j x_i^T x_j \\ & \text{subject to} \quad \alpha_i \geq 0, \quad i = 1, \dots, n, \quad \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

This is again a convex quadratic program. Complementary slackness implies that if a point is *inactive* (strictly on the correct side of its margin line), then $\alpha_i = 0$. The points with $\alpha_i > 0$ are the **support vectors**. Only they enter the expansion of $\hat{\beta}$.

Linear SVM algorithm (hard-margin dual)

1. Solve the dual for $\hat{\alpha}$.
2. Recover

$$\hat{\beta} = \sum_{i=1}^n \hat{\alpha}_i y_i x_i.$$

3. Recover $\hat{\beta}_0$ from any support vector, using $y_i(x_i^T \hat{\beta} + \hat{\beta}_0) = 1$. In the hard-margin case we can also take the midpoint of the two closest support vectors from opposite classes:

$$\hat{\beta}_0 = -\frac{1}{2} \left(\max_{i: y_i = -1} x_i^T \hat{\beta} + \min_{i: y_i = +1} x_i^T \hat{\beta} \right).$$

This midpoint formula is for the hard-margin problem only. Once slack is allowed, the closest points need not lie on the margin. In `e1071` the intercept used in the plots is `-rho` from the solver.

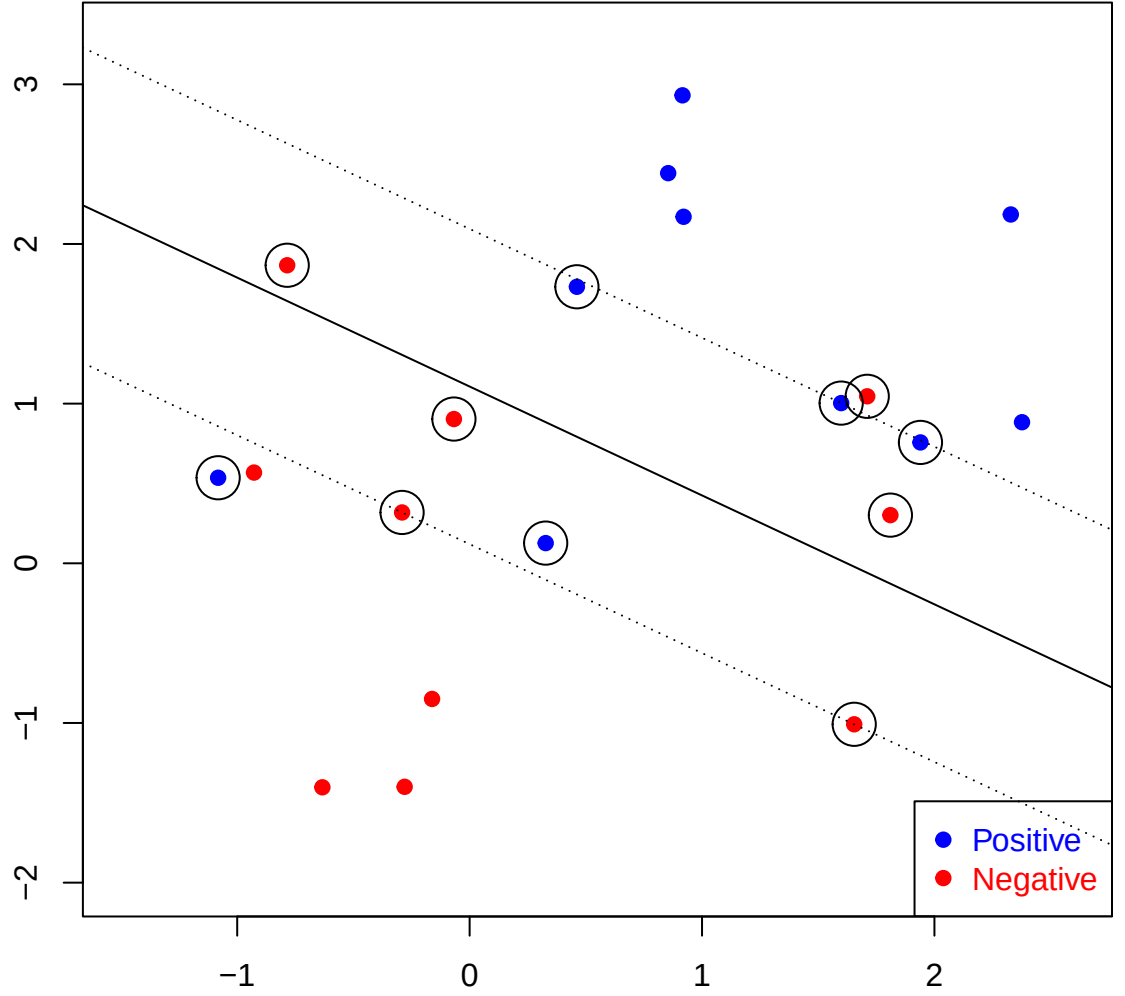
4. For a new x , predict $\text{sign}(x^T \hat{\beta} + \hat{\beta}_0)$.

Remarks

Linear separability means that many hyperplanes classify the training data perfectly. The SVM selects the one with the largest margin. The primal constraints enforce correct classification with a functional margin of at least one. In the dual, the Lagrange multipliers α_i are nonzero only for points that sit on the margin. Those points are the support vectors: if they were removed, the fitted hyperplane would change. All other training points can be moved (a little) or even deleted without affecting $\hat{f}$.

9.2 Linear SVM in the Non-Separable Case: *Slack Variables*

If the classes are not linearly separable, a zero training error is impossible and the hard-margin problem is infeasible.



We therefore introduce **slack variables** $\{\xi_i\}_{i=1}^n$ and relax the margin constraints to

$$y_i(x_i^T \beta + \beta_0) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The value of ξ_i has a direct geometric reading:

- $\xi_i = 0$: the point is on the correct side and outside or on the margin.
- $0 < \xi_i \leq 1$: the point is on the correct side but inside the margin.
- $\xi_i > 1$: the point is on the wrong side of the separating hyperplane.

The soft-margin problem is

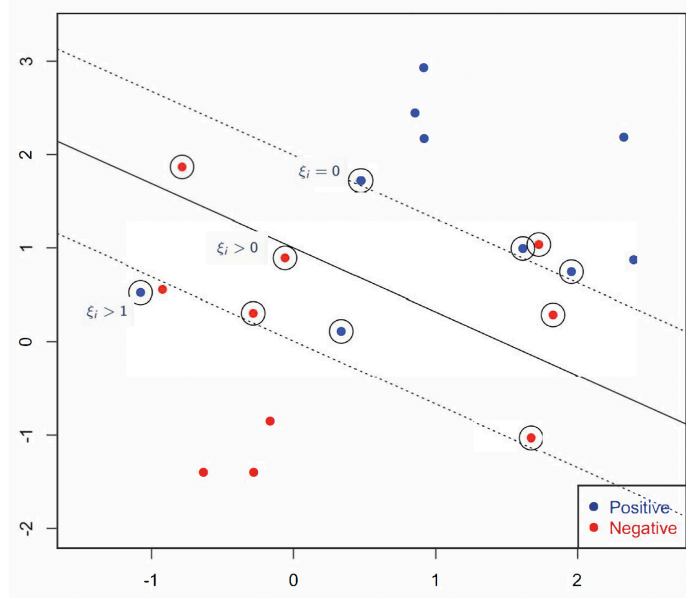


Figure 9.3: Slack variables in the linearly non-separable case

$$\text{minimize}_{\beta, \beta_0, \xi} \quad \frac{1}{2} \|\beta\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{subject to} \quad y_i(x_i^T \beta + \beta_0) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n.$$

The objective trades a large margin $2/\|\beta\|$ against the total slack $\sum_i \xi_i$. The tuning parameter $C > 0$ is the *cost* of slack. A large C penalizes violations heavily and recovers the hard-margin problem as $C \rightarrow \infty$. A small C tolerates more violations in exchange for a wider margin. In practice C is chosen by cross-validation.

In these notes the parameter is C . In `e1071::svm()` the same quantity is called `cost`.

Solving the SVM with Slack Variables

The Lagrangian of the primal is

$$\mathcal{L}(\beta, \beta_0, \xi, \alpha, \gamma) = \frac{1}{2} \|\beta\|^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i \{y_i(x_i^T \beta + \beta_0) - (1 - \xi_i)\} - \sum_{i=1}^n \gamma_i \xi_i,$$

with $\alpha_i \geq 0$ and $\gamma_i \geq 0$. Stationarity yields

$$\nabla_{\beta} \mathcal{L} = 0 \quad \Leftrightarrow \quad \beta = \sum_{i=1}^n \alpha_i y_i x_i,$$

$$\nabla_{\beta_0} \mathcal{L} = 0 \quad \Leftrightarrow \quad \sum_{i=1}^n \alpha_i y_i = 0,$$

$$\nabla_{\xi_i} \mathcal{L} = 0 \quad \Leftrightarrow \quad C - \alpha_i - \gamma_i = 0.$$

The last identity, together with $\gamma_i \geq 0$, forces $0 \leq \alpha_i \leq C$. Substituting back produces the **soft-margin dual**:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n y_i y_j \alpha_i \alpha_j \langle x_i, x_j \rangle \\ \text{subject to} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n, \quad \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

The inner product $\langle x_i, x_j \rangle$ is just $x_i^T x_j$. As before,

$$\hat{\beta} = \sum_{i=1}^n \hat{\alpha}_i y_i x_i.$$

The KKT complementary slackness conditions are

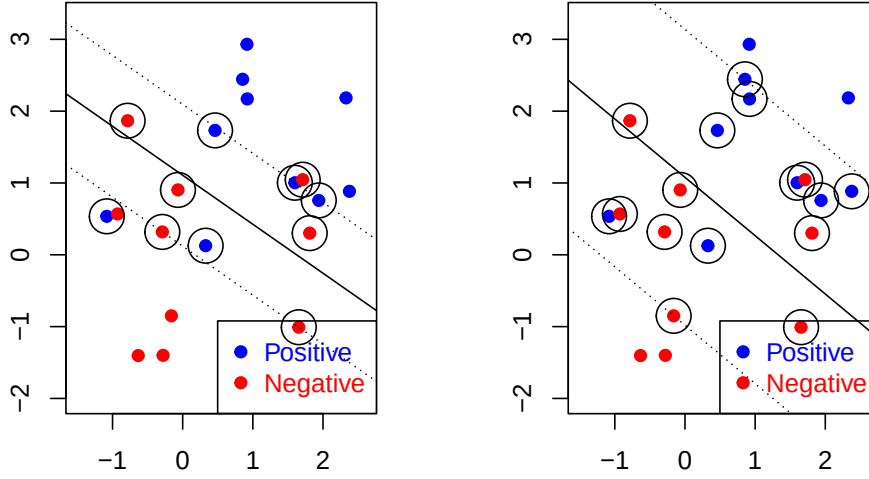
$$\begin{aligned} \alpha_i \{y_i(x_i^T \beta + \beta_0) - (1 - \xi_i)\} &= 0, \\ \gamma_i \xi_i &= 0, \\ y_i(x_i^T \beta + \beta_0) - (1 - \xi_i) &\geq 0. \end{aligned}$$

They partition the training points into three groups:

- $\alpha_i = 0$ (hence $\xi_i = 0$): points strictly beyond the margin. They do not affect $\hat{\beta}$.
- $0 < \alpha_i < C$ (hence $\xi_i = 0$): points *on* the margin. These are the free support vectors and are the ones used to recover $\hat{\beta}_0$.
- $\alpha_i = C$: points inside the margin or on the wrong side, with $\xi_i = 1 - y_i(x_i^T \beta + \beta_0) \geq 0$.

To compute $\hat{\beta}_0$, use any *free* support vector ($0 < \hat{\alpha}_i < C$), for which $y_i(x_i^T \hat{\beta} + \hat{\beta}_0) = 1$. Averaging over all such points is more stable numerically. Do not use the hard-margin midpoint formula here: with slack, the closest points from each class may be inside the margin or misclassified.

The two panels below use the same data with $C = 1000$ (left) and $C = 0.1$ (right). The plotted intercept is the solver intercept `-svm.fit$rho`, not the midpoint formula.



A large C puts more weight on avoiding slack than on widening the margin. A small C does the reverse and is influenced more by points far from the boundary. In estimation language: large C is closer to empirical 0-1 risk; small C is more regularization. The same soft-margin program can be used on separable data as well: we may deliberately allow a few points to cross a margin line if that substantially increases the margin. The parameter C controls that trade-off.

9.3 Nonlinear SVM: *The Kernel Trick*

In a linear SVM the decision boundaries are hyperplanes in the original input space. In many problems that is not flexible enough.

Setting default kernel parameters

To obtain nonlinear boundaries we enlarge the feature space with a map

$$\Phi : \mathcal{X} \rightarrow \mathcal{F}, \quad \Phi(x) = (\phi_1(x), \phi_2(x), \dots),$$

where $\mathcal{F}$ may be finite- or infinite-dimensional, and then run a *linear* SVM in $\mathcal{F}$. A linear function of $\Phi(x)$ is typically a nonlinear function of the original x . The decision function becomes

$$f(x) = \langle \Phi(x), \beta \rangle = \Phi(x)^T \beta.$$

Kernel trick

Do not compute $\Phi(x)$ explicitly. The dual SVM depends on the data only through inner products, and

$$K(x, z) = \langle \Phi(x), \Phi(z) \rangle$$

can often be evaluated directly in the original space.

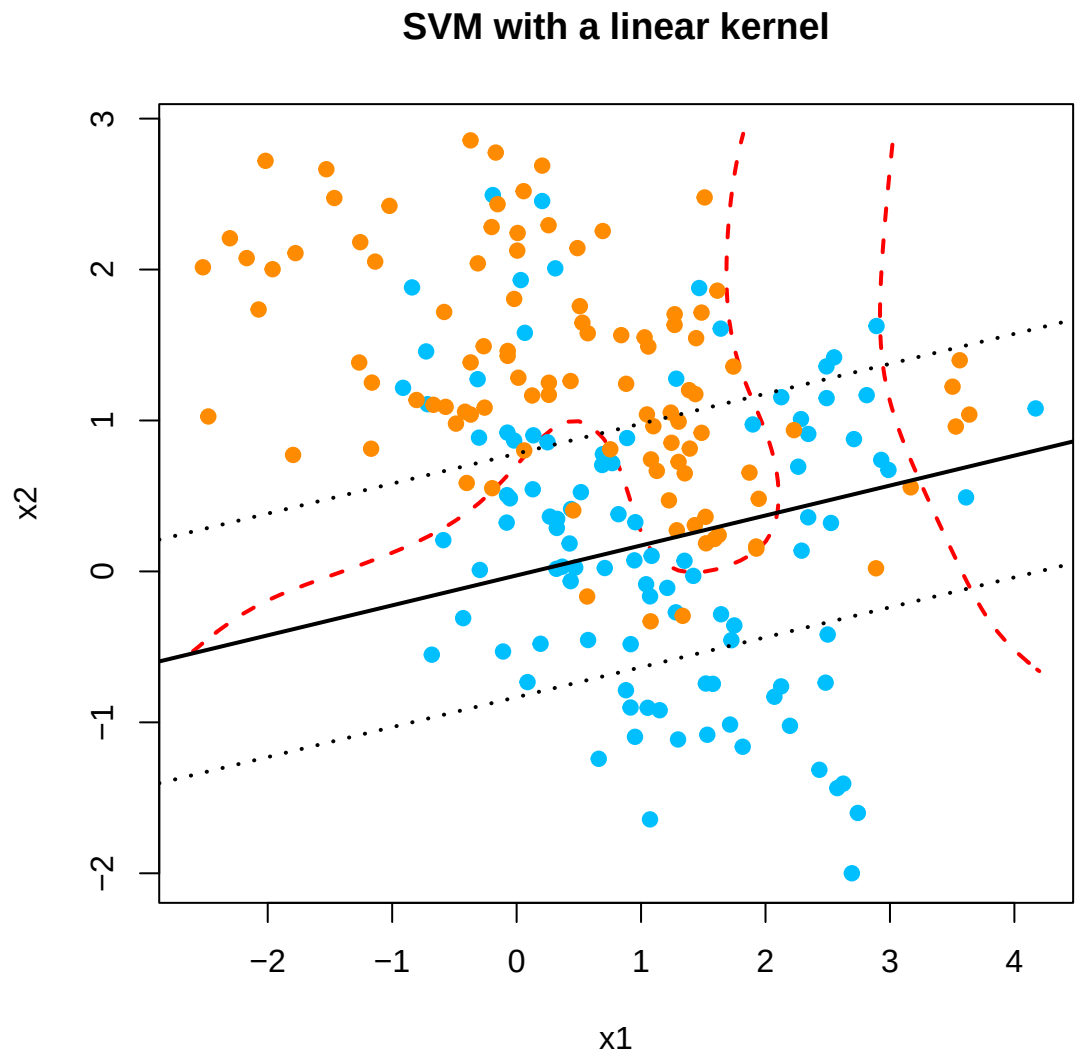


Figure 9.4: A linear SVM on a nonlinear problem (ESL mixture data)

Naive Approach

If Φ were known, we could form $\Phi(x_i)$ for each i and solve

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n y_i y_j \alpha_i \alpha_j \underbrace{\langle \Phi(x_i), \Phi(x_j) \rangle}_{=K(x_i, x_j)} \\ \text{subject to} \quad & 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

The kernel trick lets us evaluate $K(x_i, x_j)$ without ever forming $\Phi(x_i)$.

A simple example: the homogeneous quadratic kernel $K(x, z) = (x^T z)^2$ with $x, z \in \mathbb{R}^p$. Let $\Phi(x)$ collect all monomials $x_k x_\ell$ for $1 \leq k, \ell \leq p$. Then

$$\begin{aligned} K(x, z) &= \left(\sum_{k=1}^p x_k z_k \right) \left(\sum_{\ell=1}^p x_\ell z_\ell \right) \\ &= \sum_{k=1}^p \sum_{\ell=1}^p (x_k x_\ell) (z_k z_\ell) \\ &= \langle \Phi(x), \Phi(z) \rangle. \end{aligned}$$

The kernel trick is useful for two reasons.

- The dual depends on the training data only through the pairwise values $K(x_i, x_j)$. We never have to form $\Phi(x_i)$.
- The fitted decision function can be written in terms of those same kernel evaluations,

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i y_i K(x, x_i)$$

(or an equivalent expansion, depending on how α is coded). In other words, $\hat{f}$ lives in the span of $\{K(\cdot, x_i)\}_{i=1}^n$.

Remark. A sufficient condition is that the kernel matrix $(K(x_i, x_j))$ be positive semidefinite for every finite sample. Then K is an inner product $\langle \Phi(x), \Phi(z) \rangle$ for some feature map Φ (Mercer's theorem). We will use that fact and will not develop the RKHS formalism further here.

Computational Advantage {-}

- Evaluating $K(x, z) = (x^T z)^2$ costs $\mathcal{O}(p)$ operations.
- Forming $\Phi(x)$ and $\Phi(z)$ and then taking an inner product costs $\mathcal{O}(p^2)$.

That is why we derived the dual: the primal is written in terms of $\beta \in \mathcal{F}$, which we cannot even store when $\mathcal{F}$ is infinite-dimensional, whereas the dual is written entirely in terms of $K(x_i, x_j)$.

Common kernels

- Degree- d polynomial:

$$K(x, z) = (1 + x^T z)^d$$

- Radial basis (Gaussian):

$$K(x, z) = \exp(-\|x - z\|^2/c)$$

- Sigmoid (listed for completeness; do not use it by default):

$$K(x, z) = \tanh(\gamma x^T z + r)$$

Remark. Not every function K is a valid kernel. If the kernel matrix fails to be positive semidefinite, the dual need not be convex and there may be no feature map Φ . The sigmoid kernel is a historical choice from early neural-network comparisons. It is not always positive semidefinite; prefer a polynomial or radial kernel unless you have a specific reason not to.

Remark on Convexity

Is the dual still convex after we replace $x_i^T x_j$ by $K(x_i, x_j)$? The quadratic term is

$$\sum_{i,j=1}^n y_i y_j \alpha_i \alpha_j K(x_i, x_j) = \alpha^T \text{diag}(y) K \text{diag}(y) \alpha.$$

This is a convex function of α whenever K is positive semidefinite, which is the same condition that lets us treat $K(x, z)$ as an inner product $\langle \Phi(x), \Phi(z) \rangle$.

9.4 SVM as a Penalization Method

The soft-margin constraint $y_i f(x_i) \geq 1 - \xi_i$ with $\xi_i \geq 0$ is equivalent to $\xi_i \geq (1 - y_i f(x_i))_+$. Substituting the smallest feasible slacks into the primal objective gives

$$\text{minimize}_f \sum_{i=1}^n (1 - y_i f(x_i))_+ + \lambda \|\beta\|^2,$$

where $\lambda = 1/C$ and $f(x) = x^T \beta + \beta_0$. This is a **loss + penalty** problem. The loss is the **hinge loss**

$$L(y, f(x)) = (1 - y f(x))_+ = \max(0, 1 - y f(x)).$$

Hinge loss is not differentiable at $y f(x) = 1$, so we cannot use ordinary gradient descent without modification (a subgradient does exist). Other convex surrogate losses used for classification include:

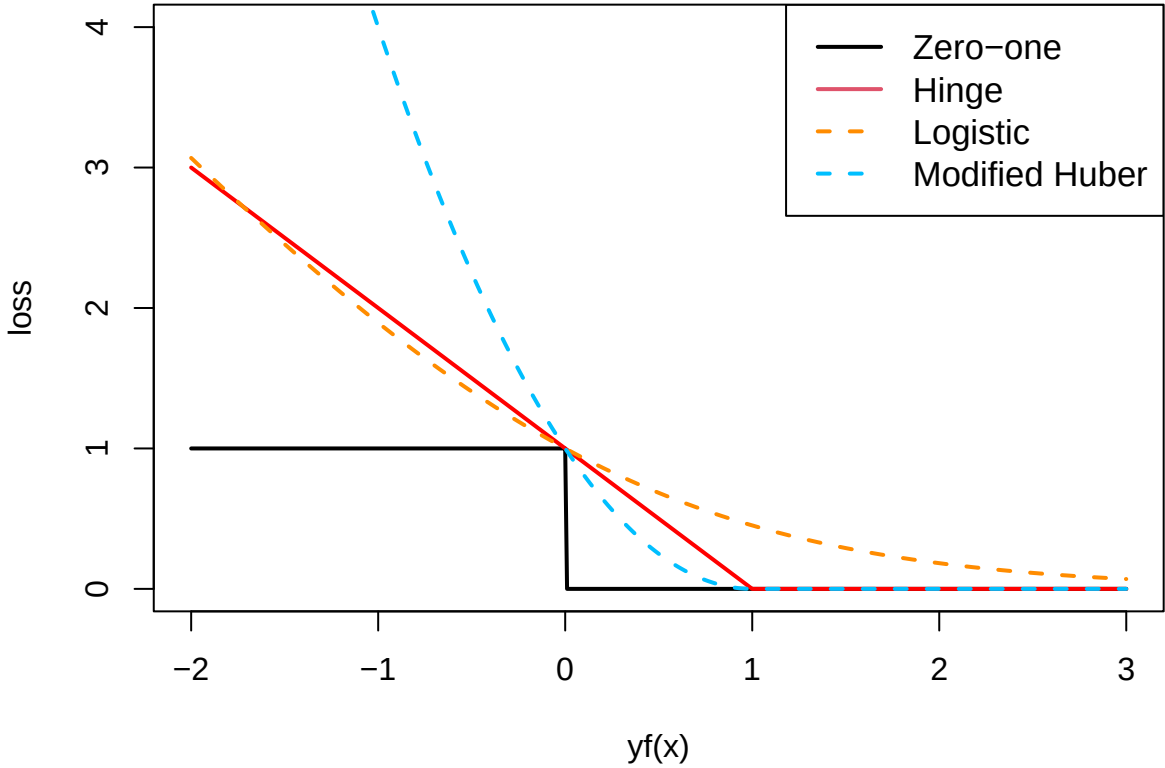
- Logistic loss:

$$L(y, f(x)) = \log(1 + e^{-y f(x)})$$

- Modified Huber loss:

$$L(y, f(x)) = \begin{cases} (1 - yf(x))^2, & yf(x) \geq -1, \\ -4yf(x), & \text{otherwise.} \end{cases}$$

Classification loss functions



Zero-one loss is the target, but it is discontinuous and nonconvex. Hinge, logistic, and modified Huber are convex surrogates and are much easier to optimize. Logistic and modified Huber are differentiable, so gradient methods apply directly and can be attractive when n is large.

The contrast with the previous chapter is useful here. Logistic loss is (up to constants) the negative log-likelihood of a Bernoulli model, so we can map $\hat{f}$ to $\hat{P}(Y = 1 \mid X = x)$ by the logistic transform. Hinge loss only cares that $yf(x)$ be at least 1; once a point is beyond the margin it costs nothing further. That is why an SVM is a classifier and not, by itself, a probability model.

A nonlinear SVM with hinge loss solves

$$\min_{f \in \mathcal{H}_K} \sum_{i=1}^n (1 - y_i f(x_i))_+ + \lambda \|f\|_{\mathcal{H}_K}^2,$$

where $\mathcal{H}_K$ is the RKHS of the kernel K . By the representer theorem the solution has the form

$$\hat{f}(x) = \alpha_0 + \sum_{i=1}^n \alpha_i K(x, x_i).$$

The problem reduces to an optimization over $\alpha \in \mathbb{R}^n$,

$$\sum_{i=1}^n L(y_i, K_i^T \alpha) + \lambda \alpha^T K \alpha,$$

where K is the $n \times n$ kernel matrix and K_i is its i th column. If L is differentiable this is an unconstrained smooth problem.

In this view a kernel SVM first builds n new predictors, each measuring similarity to one training point, and then fits a linear model in those predictors with an ℓ_2 -type penalty $\alpha^T K \alpha$.

Two restrictions of the classical SVM are worth questioning once we adopt the loss-plus-penalty view:

- The penalty on α is a ridge (RKHS-norm) penalty. A lasso-type penalty on α would produce a sparser expansion, using fewer kernel centers.
- The kernel is required to be positive semidefinite. Many useful similarity functions are not.

Relaxing either condition takes us outside the textbook definition of an SVM, but the resulting *kernel machines* are often useful supervised learning methods.

When an SVM is the wrong tool

An SVM is a strong binary classifier. It is not a default replacement for logistic regression or DA.

- It does not produce $\widehat{P}(Y = 1 | X)$ without an extra calibration step (Platt scaling and relatives). If the scientific target is a probability, start with logistic regression.
- Multiclass problems are handled by one-versus-one or one-versus-rest reductions. That works, but it is bolted on; there is no single native multiclass hinge in the same way that multinomial logistic regression is native.
- When n is large or p is huge and *sparsity in the original features* is the point, ℓ_1 -penalized logistic regression is often the simpler statistical and computational story. The SVM sparsity is in the α_i (support vectors), not in the coordinates of x .
- A radial kernel with a poorly chosen width will look excellent on the training plot and fail on a new draw. Scale the features, and treat C and the kernel width as tuning parameters chosen by prediction error, not by how pretty the decision regions are.

Use an SVM when you want a flexible decision boundary and you are willing to treat $f(x)$ as a score rather than a probability. Use logistic regression or DA when the probability model, or the linear structure in x , is the object of interest.

9.5 SVM Examples in R

This section is an illustration of the methods discussed in this week's lectures. The code shows how the objects from the notes, e.g., β , C , support vectors, kernels appear in a fitted SVM.

The Python version of the example can be found here: [Python Code for SVM](#).

The calculations use `e1071::svm()`. Its argument `cost` is the C of the notes. We keep `scale = FALSE` when we want coefficients and plots in the original units of X . In applications one would usually scale the features.

Three comparisons:

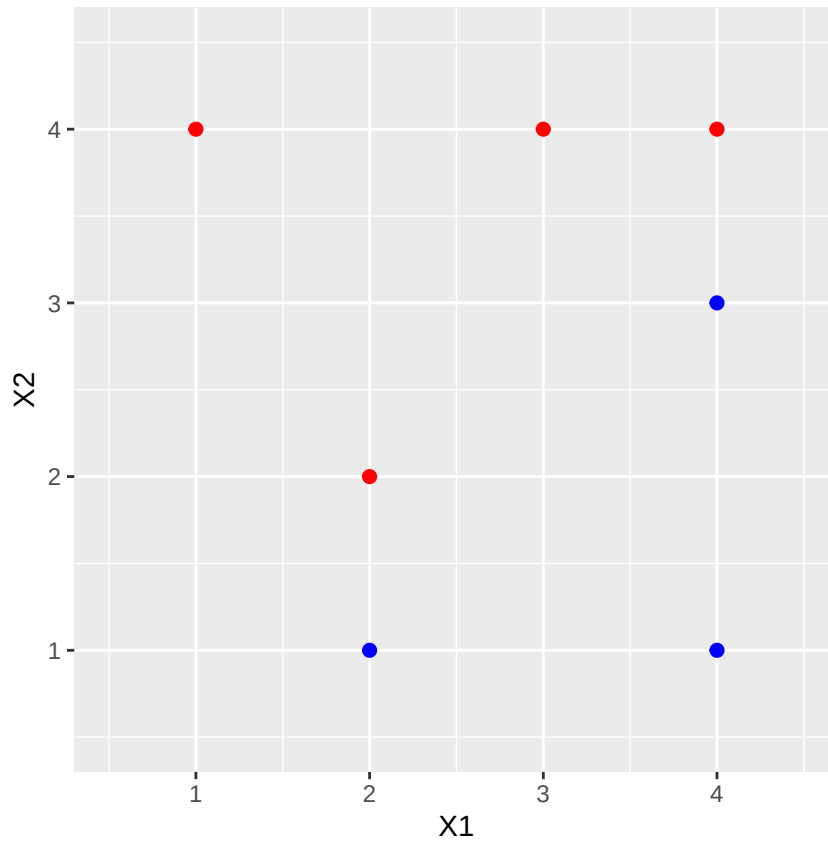
- a hard margin versus a soft margin (what C does);
- a linear SVM versus logistic regression on the same points (who influences the fit);
- a linear kernel versus a radial kernel, with the Bayes boundary as a reference.

Linear Separable Case: Maximal Margin Classifier

We start with a small maximal-margin example (adapted from an ISLR exercise).

X1	X2	Y
3	4	red
2	2	red
4	4	red
1	4	red
2	1	blue
4	3	blue
4	1	blue

A scatterplot of the data:



The linear separating hyperplane has the form

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0.$$

We compute it from a hard-margin SVM fit in `e1071` by taking a very large cost:

```
library(e1071)
# large C approximates a hard margin
svm.toy <- svm(Y ~ ., data = toy.data, kernel = "linear", cost = 1e+05,
  scale = FALSE)
```

The coefficients of the separating line are

```
b <- t(svm.toy$coefs) %*% svm.toy$SV
c <- -svm.toy$rho
b
```

```
##           X1           X2
## [1,] -1.999846 1.999693
```

```
c
```

```
## [1] 1.00041
```

so the optimal separating hyperplane is

$$1 - 2X_1 + 2X_2 = 0.$$

```
slope <- -b[1]/b[2]
b0 <- -c/b[2]
slope
```

```
## [1] 1.000077
```

```
b0
```

```
## [1] -0.5002816
```

The decision boundary is the line

$$X_2 = X_1 - 0.5,$$

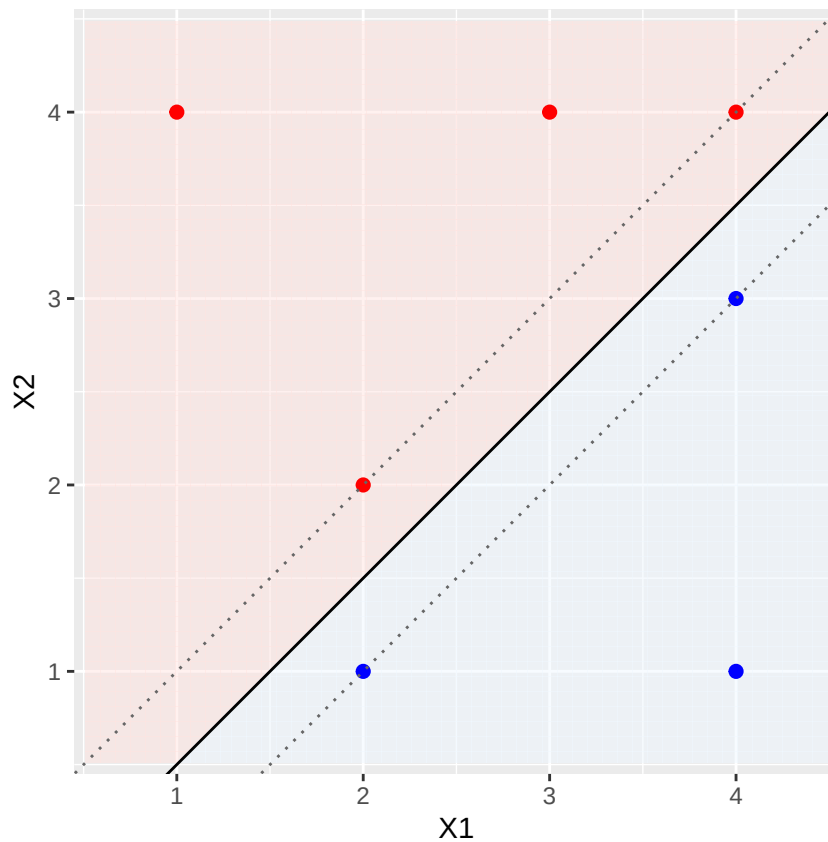
and the maximal-margin classifier can be written

$$f(X) = X_2 - X_1 + 0.5.$$

A new point x^* is assigned to the red class if $f(x^*) > 0$ and to the blue class otherwise. The margin width is $2/\|\beta\|$:

```
margin <- 2/sqrt(sum(b^2))
margin
```

```
## [1] 0.7071882
```



The solid line is the optimal separating hyperplane; the dotted lines are the margins. The fit used `scale = FALSE` and a large `cost` so that this is, for practical purposes, the hard-margin solution in the original coordinates. In `e1071`, `svm.toy$index` identifies the support vectors and `svm.toy$rho` is the intercept used in the plot (not the hard-margin midpoint formula).

```
svm.toy$index
```

```
## [1] 2 3 6
```

```
svm.toy$nSV
```

```
## [1] 2 1
```

Those indices are the support vectors: the active points on the two dotted lines. For a hard-margin fit, $y_i f(x_i) \geq 1$ at every training point, with equality only on this subset. The printed margin $2/\|\beta\|$ is the width between the dotted lines.

Non-Separable Data and the Role of C

Add a red point at $(X_1, X_2) = (3, 1)$. The data are no longer linearly separable.

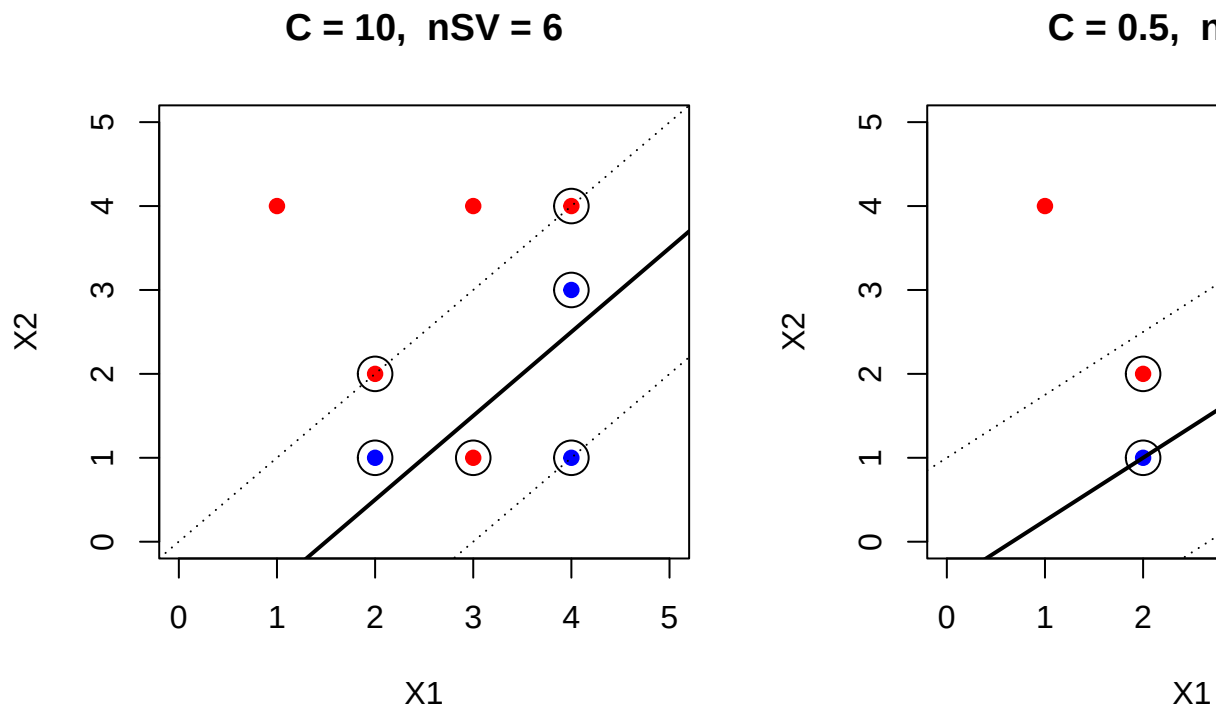
```
toy.data.2 <- data.frame(X1 = c(3, 2, 4, 1, 2, 4, 4, 3), X2 = c(4,
  2, 4, 4, 1, 3, 1, 1), Y = as.factor(c("red", "red", "red",
  "red", "blue", "blue", "blue", "red")))
```

A finite cost is now required. Large C tries to avoid slack (narrower margin, fewer violations). Small C allows more violations and a wider margin. The plotted intercept is again $-\text{fit}\$rho$.

```
plot_linear_svm <- function(fit, data, main) {
  b <- as.numeric(t(fit$coefs) %*% fit$SV)
  b0 <- -fit$rho
  x <- data[, c("X1", "X2")]
  plot(x, col = ifelse(data$Y == "blue", "blue", "red"), xlim = c(0,
    5), ylim = c(0, 5), pch = 19, xlab = "X1", ylab = "X2",
    main = main)
  abline(a = -b0/b[2], b = -b[1]/b[2], lwd = 2)
  points(x[fit$index, ], col = "black", cex = 2.4)
  abline(a = (-b0 - 1)/b[2], b = -b[1]/b[2], lty = 3)
  abline(a = (-b0 + 1)/b[2], b = -b[1]/b[2], lty = 3)
}

svm.hiC <- svm(Y ~ ., data = toy.data.2, kernel = "linear", cost = 10,
  scale = FALSE)
svm.loC <- svm(Y ~ ., data = toy.data.2, kernel = "linear", cost = 0.5,
  scale = FALSE)

par(mfrow = c(1, 2))
plot_linear_svm(svm.hiC, toy.data.2, main = paste0("C = 10, nSV = ",
  sum(svm.hiC$nSV)))
plot_linear_svm(svm.loC, toy.data.2, main = paste0("C = 0.5, nSV = ",
  sum(svm.loC$nSV)))
```



The extra red point is expensive when $C = 10$: the boundary moves toward it and the margin shrinks. When $C = 0.5$ that point is tolerated as slack, more points become support vectors, and the margin widens. This is an estimation statement, not just a geometric one. Large C is closer to empirical 0-1 risk (fit the training labels, even the noisy ones). Small C is more regularization (a smaller $\|\beta\|$, a wider margin, more slack). The left panel weaves toward the extra point; the right panel is willing to ignore it.

Linear SVM versus Logistic Regression

On the same eight points, fit a linear logistic regression. Both methods give a line; they do not use the data in the same way.

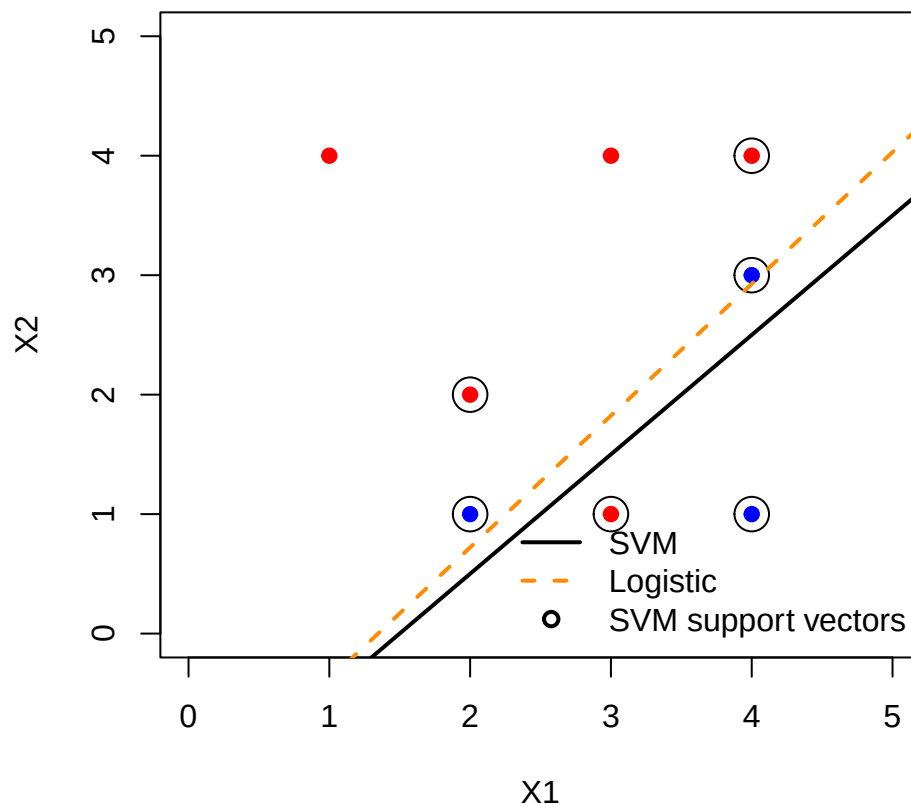
```
toy.data.2$y01 <- as.numeric(toy.data.2$Y == "red")
glm.toy <- glm(y01 ~ X1 + X2, data = toy.data.2, family = binomial)

b.svm <- as.numeric(t(svm.hiC$coefs) %*% svm.hiC$SV)
b0.svm <- -svm.hiC$rho
b.glm <- coef(glm.toy)

plot(toy.data.2$X1, toy.data.2$X2, col = ifelse(toy.data.2$Y ==
  "blue", "blue", "red"), pch = 19, xlim = c(0, 5), ylim = c(0,
  5), xlab = "X1", ylab = "X2", main = "Linear SVM (C = 10) vs logistic")
points(toy.data.2[svm.hiC$index, c("X1", "X2")], cex = 2.4)
```

```
abline(a = -b0.svm/b.svm[2], b = -b.svm[1]/b.svm[2], lwd = 2)
abline(a = -b.glm[1]/b.glm[3], b = -b.glm[2]/b.glm[3], col = "darkorange",
      lwd = 2, lty = 2)
legend("bottomright", c("SVM", "Logistic", "SVM support vectors"),
      col = c("black", "darkorange", "black"), lty = c(1, 2, NA),
      pch = c(NA, NA, 1), lwd = 2, bty = "n")
```

Linear SVM (C = 10) vs logistic



Logistic regression is a likelihood for $P(Y = 1 | X)$. Every point contributes to the score equations. The SVM with hinge loss ignores points that already lie beyond the margin; only the support vectors (circled) determine the black line. The two boundaries need not agree, even when both are linear. Logistic regression still produces $\widehat{P}(Y = 1 | X)$; the SVM does not.

Nonlinear SVM: Linear Kernel, Radial Kernel, Bayes Rule

The mixture data from ESL are not linearly separable in a way that matches the Bayes boundary (the red dashed $P(Y = 1 | X) = 1/2$ contour). A linear

SVM can only draw a line. A radial kernel can bend.

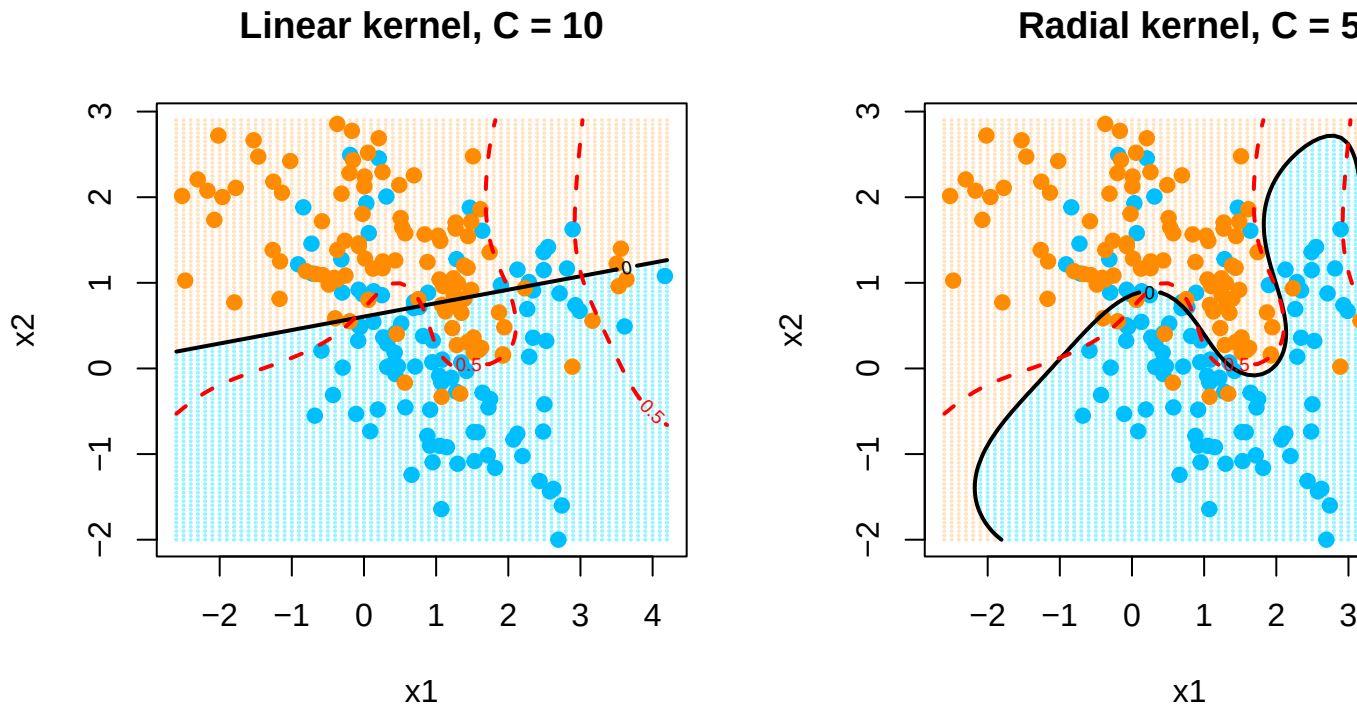
```
library(mda)
data(ESL.mixture)

px1 <- ESL.mixture$px1
px2 <- ESL.mixture$px2
mix <- data.frame(X1 = ESL.mixture$x[, 1], X2 = ESL.mixture$x[,
  2], y = factor(ESL.mixture$y))
prob <- ESL.mixture$prob
xgrid <- expand.grid(X1 = px1, X2 = px2)

svm.lin <- svm(y ~ ., data = mix, kernel = "linear", cost = 10,
  scale = FALSE)
svm.rad <- svm(y ~ ., data = mix, kernel = "radial", cost = 5,
  scale = FALSE)

plot_mixture <- function(fit, main) {
  dec <- predict(fit, xgrid, decision.values = TRUE)
  z <- attributes(dec)$decision
  yhat <- predict(fit, xgrid)
  plot(xgrid, col = ifelse(yhat == levels(mix$y)[2], "bisque",
    "cadetblue1"), pch = 20, cex = 0.2, xlab = "x1", ylab = "x2",
    main = main)
  points(mix$X1, mix$X2, col = ifelse(mix$y == 1, "darkorange",
    "deepskyblue"), pch = 19)
  contour(px1, px2, matrix(z, length(px1), length(px2)), levels = 0,
    add = TRUE, lwd = 2)
  contour(px1, px2, matrix(prob, length(px1), length(px2)),
    levels = 0.5, add = TRUE, col = "red", lty = 2, lwd = 2)
}

par(mfrow = c(1, 2))
plot_mixture(svm.lin, "Linear kernel, C = 10")
plot_mixture(svm.rad, "Radial kernel, C = 5")
```



The red dashed curve is the Bayes boundary. The linear kernel cannot track it. The radial kernel can, but only for a reasonable C (and kernel width). That is the point of this figure: the kernel changes the *class of boundaries*, C changes how hard we fit within that class.

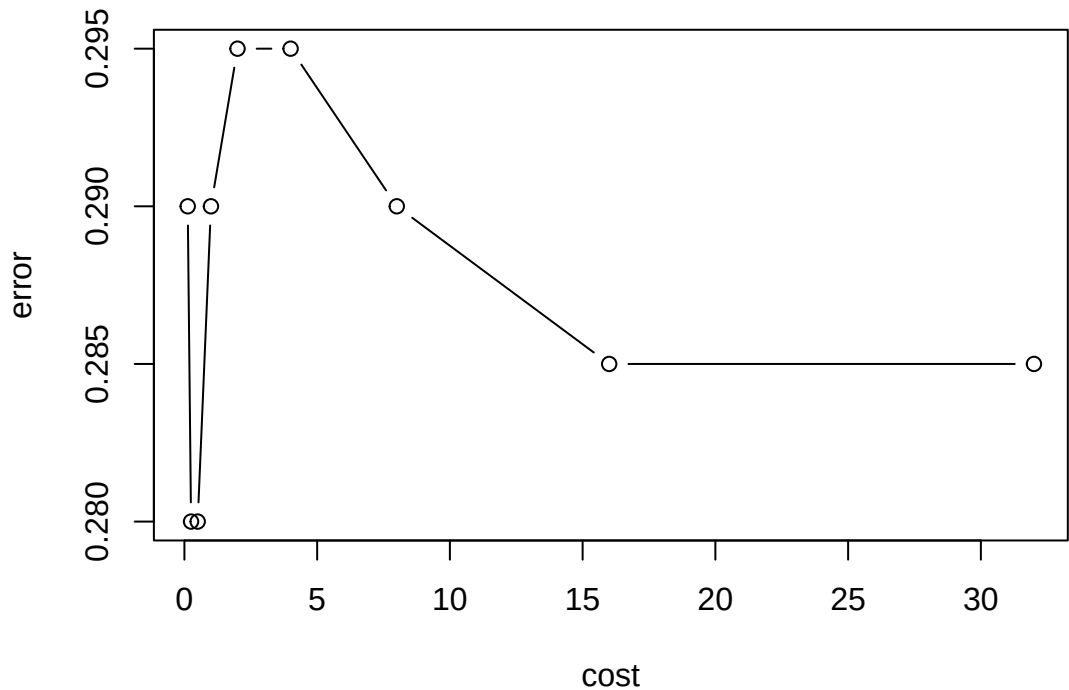
Tuning C by Cross-Validation

C is a regularization parameter. We choose it by cross-validation, not by looking at the training plot. `e1071::tune()` does a grid search and returns the CV error. The radial kernel also has a width parameter `gamma` (large `gamma` is a narrow kernel).

```
set.seed(1)
tune.lin <- tune(svm, y ~ ., data = mix, kernel = "linear", scale = FALSE,
  ranges = list(cost = 2^seq(-3, 5, by = 1)))
tune.lin$best.parameters

## cost
## 2 0.25
plot(tune.lin)
```

Performance of 'svm'



```
set.seed(1)
tune.rad <- tune(svm, y ~ ., data = mix, kernel = "radial", scale = FALSE,
  ranges = list(cost = c(0.5, 1, 5, 10), gamma = c(0.5, 1,
    2)))
tune.rad$best.parameters

##   cost gamma
##    7     5    1
tune.rad$best.performance

## [1] 0.165
```

Read the linear plot as you would a ridge CV curve: very small C underfits (wide margin, too much slack), very large C starts to overfit the training labels. The “best” C is the one with the smallest estimated prediction error, not the prettiest training margin.

If you already use `caret` in the course, the same idea can be written as `train(..., method = "svmRadial")`, where `C` is C and `sigma` is the radial width. What matters is the CV curve, not the wrapper.

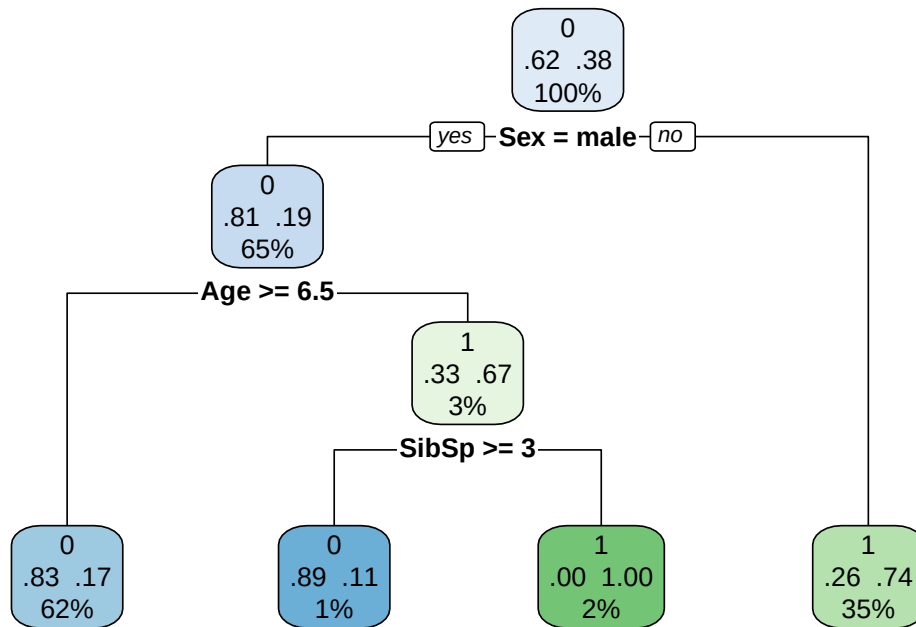
Chapter 10

Classification Trees & Ensemble Methods

In this section we revisit **trees** in a different context. In contrast to regression trees, the response is now a **categorical** variable taking values in a discrete set, $Y \in \{1, 2, \dots, K\}$. Given features $X = (X_1, X_2, \dots, X_p)$, the goal is to assign a new observation to one of the K classes.

Tree-based methods, as discussed earlier, are *nonparametric* procedures that recursively partition the feature space into hyper-rectangular regions and make a constant prediction in each region. Historically there are two main lines of tree algorithms: *Classification and Regression Trees* (CART; Breiman, Friedman, Olshen, and Stone, 1984) and the *ID3/C4.5* family (Quinlan, 1986, 1993).

As an illustration we use the *Titanic survival* data, a staple example for classification. The version in the chunk below was downloaded from Kaggle [here](#). We use *passenger class*, *sex*, *age*, *number of siblings/spouses aboard*, and *number of parents/children aboard* to predict whether a passenger survived. Pruning works in the same way as for regression trees.



The terminal nodes show the idea of a classification tree: the algorithm recursively splits the feature space until each region is dominated by *one class*, and that majority class is the prediction for the region.

Once the tree is grown, the prediction in a terminal node $\mathcal{T}$ is the majority class,

$$\hat{y}(x) = \arg \max_k \hat{p}_k(\mathcal{T}(x)),$$

where $\mathcal{T}(x)$ is the leaf that contains x and $\hat{p}_k$ is the sample frequency of class k in that leaf. If a probability is needed later—for example to compare the tree with logistic regression—use $\hat{p}_k(\mathcal{T}(x))$ itself, not only the hard label.

The official Kaggle test file has no survival labels, so a fair comparison uses a holdout slice of the *labeled* training data. The table below reports misclassification for a majority-class baseline, the pruned tree, and a logistic regression on the same features.

```

set.seed(10)

# Complete cases so the tree and the logistic model see the
# same rows
titanic.cc = titanic.train[complete.cases(titanic.train[, c("Survived",
  "Pclass", "Sex", "Age", "SibSp", "Parch"))], ]
n.t = nrow(titanic.cc)
hold = sample(seq_len(n.t), size = floor(0.3 * n.t))
tr = titanic.cc[-hold, ]
te = titanic.cc[hold, ]

```

```

maj = as.numeric(names(which.max(table(tr$Survived))))
base.err = mean(te$Survived != maj)

tree.hold = rpart(as.factor(Survived) ~ Pclass + Sex + Age +
  SibSp + Parch, data = tr)
tree.hold = prune(tree.hold, cp = 0.02)
tree.err.tr = mean(predict(tree.hold, tr, type = "class") !=
  factor(tr$Survived))
tree.err.te = mean(predict(tree.hold, te, type = "class") !=
  factor(te$Survived))

glm.hold = glm(Survived ~ Pclass + Sex + Age + SibSp + Parch,
  data = tr, family = binomial())
glm.hat = as.integer(predict(glm.hold, te, type = "response") >
  0.5)
glm.err.te = mean(glm.hat != te$Survived)

data.frame(method = c("Majority class (holdout)", "Pruned tree (train)",
  "Pruned tree (holdout)", "Logistic regression (holdout)"),
  misclassification = round(c(base.err, tree.err.tr, tree.err.te,
    glm.err.te), 3))

##                method misclassification
## 1    Majority class (holdout)           0.453
## 2      Pruned tree (train)             0.158
## 3      Pruned tree (holdout)           0.201
## 4 Logistic regression (holdout)        0.192

```

A single tree is already better than the base rate. Whether it beats the logistic model depends on the split and on how nonlinear the signal is; the point of the table is to put the tree on the same scoreboard as the methods from the previous chapters, not to declare a winner.

The classification methods we have already discussed either transform the feature space (*SVMs*) or rely on a distance (*kNN*, kernel methods). A tree attacks the same problem by cutting the feature space directly into rectangles, using a **binary splitting rule** of the form $\mathbf{1}_{\{x \leq c\}}$.

10.1 Splitting Rules in Classification Trees

To see how the splitting rule works, start with a toy example. Draw $X_1, X_2 \sim \text{Uniform}[-1, 1]$. Generate the label so that about 90% of the Class 1 (blue) points fall *inside* the circle $x_1^2 + x_2^2 < 0.6$, while about 90% of the Class 2 (orange) points fall *outside* it.

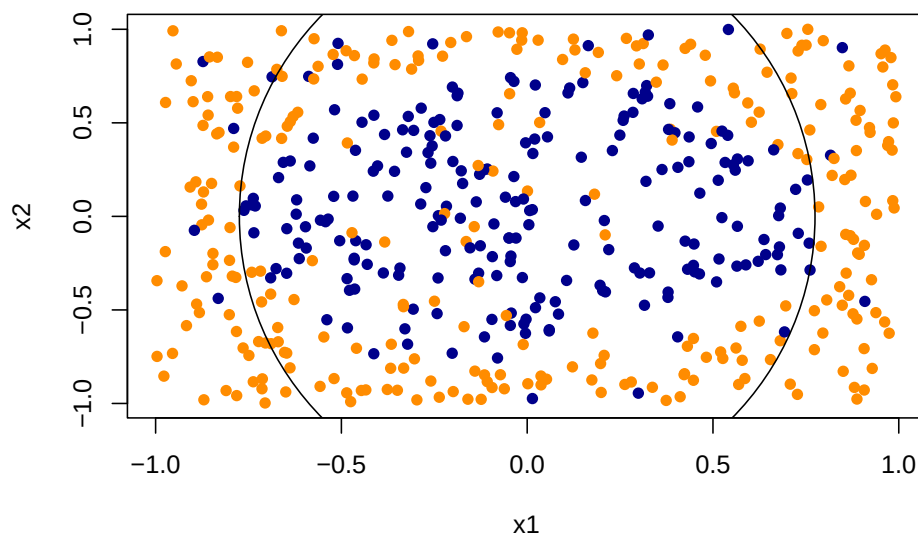
```
## Simulation

set.seed(1)

x1 = runif(500, -1, 1)
x2 = runif(500, -1, 1)

y = rbinom(500, size = 1, prob = ifelse(x1^2 + x2^2 < 0.6, 0.9,
0.1))

plot(x1, x2, col = ifelse(y == 1, "darkblue", "darkorange"),
     pch = 16)
symbols(0, 0, circles = sqrt(0.6), add = TRUE, inches = FALSE)
```

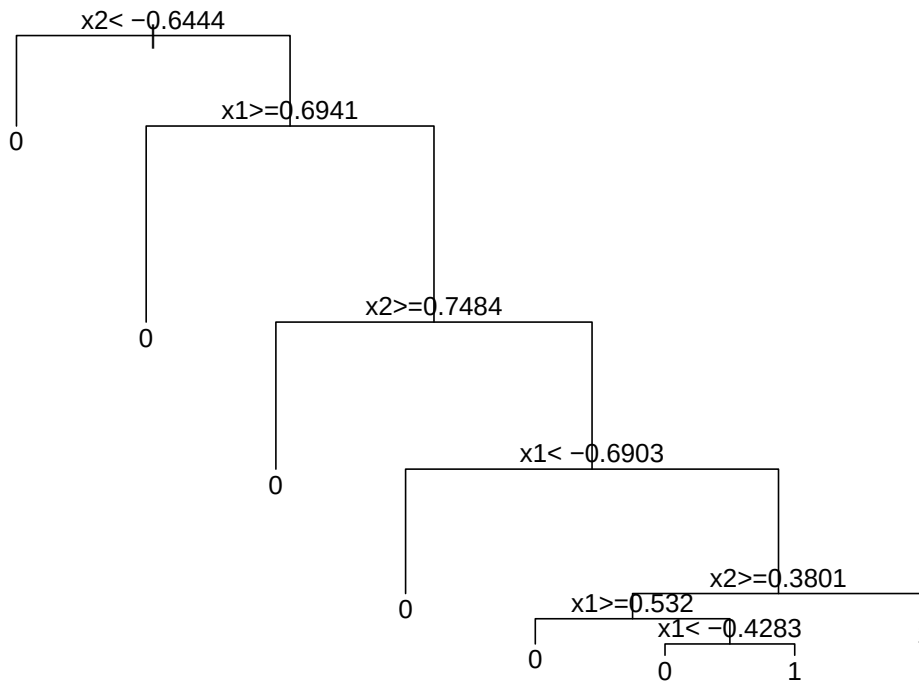


A classification tree recursively splits this plane so that each resulting region is dominated by one class.

```
library(rpart)

rpart.fit = rpart(as.factor(y) ~ x1 + x2, data = data.frame(x1,
x2, y))

par(mar = rep(0.5, 4))
plot(rpart.fit)
text(rpart.fit)
```



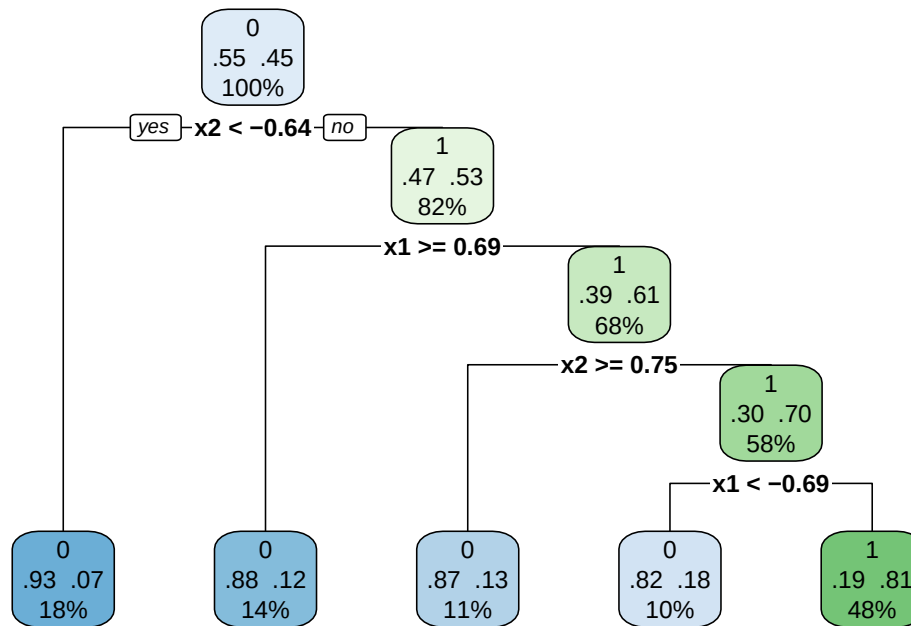
The complexity table records the sequence of splits and the associated cost-complexity parameters:

```
rpart.fit$cptable
```

```
##          CP nsplit rel error   xerror   xstd
## 1 0.17040359      0 1.0000000 1.0000000 0.04984280
## 2 0.14798206      3 0.4843049 0.6816143 0.04612343
## 3 0.01121076      4 0.3363229 0.4125561 0.03885386
## 4 0.01000000      7 0.3004484 0.3721973 0.03730931
```

As before, we can prune the tree and plot it with `rpart.plot`:

```
pruned.tree = prune(rpart.fit, cp = 0.041)
rpart.plot(pruned.tree, type = 2, extra = 104, fallen.leaves = TRUE)
```



The Bayes boundary in this example is a circle, but every split is perpendicular to a coordinate axis. The tree can only approximate that curve by a staircase of rectangles. That is approximation bias from the axis-aligned partition, not a coding failure. Bagging and random forests average many such staircases; they do not remove the axis-aligned constraint.

At a given node the algorithm compares all candidate splits on all variables. For each variable it finds the **best cutoff**, then it compares those best cutoffs across variables and keeps the single split that most improves the criterion. It then repeats on each child.

For a continuous predictor the split has the form $\mathbf{1}_{\{X^{(j)} \leq c\}}$: node $\mathcal{T}$ is partitioned into two children. We evaluate the **impurity** of the parent and of the two children (there are K classes) and choose the split with the largest reduction in impurity.

That raises the question: which criterion should we use to compare cutoffs? Three standard choices are:

- Gini impurity (CART)
- Shannon entropy (C4.5)
- Misclassification error

10.1.1 Impurity Measures

To choose the variable and the cutoff we quantify the **gain** in an impurity measure. If $i(t)$ denotes the impurity at node t , the gain of a split (j, s) is

$$\Phi(j, s) = i(t) - (p_L \cdot i(t_L) + p_R \cdot i(t_R)),$$

where

$$i(t) = I(\hat{p}_t(1), \dots, \hat{p}_t(K)),$$

$I(\cdot)$ is an impurity function, and $\hat{p}_t(k)$ is the sample frequency of class k at node t . We pick the split (j, s) with the *largest reduction in impurity*.

A proposed split $\mathbf{1}_{\{X^{(j)} \leq c\}}$ partitions the observations in the node into two disjoint children, $\mathcal{T}_L$ and $\mathcal{T}_R$. Writing $N_{\mathcal{T}}$, $N_{\mathcal{T}_L}$, and $N_{\mathcal{T}_R}$ for the corresponding sample sizes, the same gain is

$$\Phi(j, s) = I(\mathcal{T}) - \left(\underbrace{\frac{N_{\mathcal{T}_L}}{N_{\mathcal{T}}}}_{\hat{p}_L} I(\mathcal{T}_L) + \underbrace{\frac{N_{\mathcal{T}_R}}{N_{\mathcal{T}}}}_{\hat{p}_R} I(\mathcal{T}_R) \right).$$

The algorithm searches over variables j and cut points c and keeps the split with the best score.

We now define an impurity measure formally.

Definition: Impurity measure

An impurity measure is a function $I(p_1, \dots, p_K)$ defined on $p_k \geq 0$, $\sum_k p_k = 1$, with the following properties:

- (a) the maximum occurs at the uniform distribution $(1/K, \dots, 1/K)$;
- (b) the minimum occurs at a vertex $p_k = 1$ (a perfectly pure node);
- (c) I is symmetric in $(p_1, \dots, p_K)$: permuting the class probabilities does not change I .

The impurity measures used most often in classification are the following.

- **Gini index (CART)**

$$\text{Gini}(\mathcal{T}) = \sum_{k=1}^K \hat{p}_k(1 - \hat{p}_k) = 1 - \sum_{k=1}^K \hat{p}_k^2,$$

where

$$\hat{p}_k = \frac{\sum_i \mathbf{1}_{\{y_i=k\}} \mathbf{1}_{\{x_i \in \mathcal{T}\}}}{\sum_i \mathbf{1}_{\{x_i \in \mathcal{T}\}}}$$

is the within-node frequency of class k .

- **Shannon entropy (ID3/C4.5)**

$$\text{Entropy}(\mathcal{T}) = - \sum_{k=1}^K \hat{p}_k \log \hat{p}_k.$$

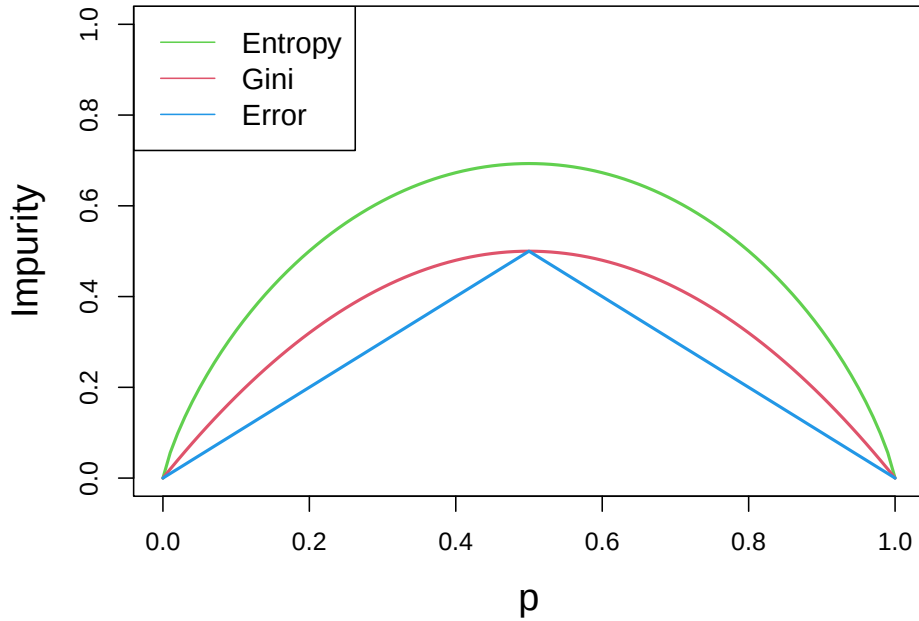
- **Misclassification rate**

$$\text{Error}(\mathcal{T}) = 1 - \max_{k=1, \dots, K} \hat{p}_k.$$

When there are only two classes ($K = 2$) and p denotes the frequency of class 1, these reduce to

- Gini index: $2p(1-p)$
- Shannon entropy: $p \log(1/p) + (1-p) \log(1/(1-p))$
- misclassification error: $\min(p, 1-p)$

Plotting the three binary impurity curves:



A smaller value of the impurity measure means that the node is more “pure,” so there is less uncertainty about the class prediction. The point of a split is to produce two children that contain less variation—that is, children that are as pure as possible. We therefore compute the criterion before and after each candidate split and keep the cutoff with the largest reduction.

Take the Gini index as an example. For an internal node $\mathcal{T}$,

$$\Phi_{\text{Gini}}(j, c) = \text{Gini}(\mathcal{T}) - \left(\frac{N_{\mathcal{T}_L}}{N_{\mathcal{T}}} \text{Gini}(\mathcal{T}_L) + \frac{N_{\mathcal{T}_R}}{N_{\mathcal{T}}} \text{Gini}(\mathcal{T}_R) \right).$$

The two children produced by a split of variable j at cutoff c are

$$\begin{aligned}\mathcal{T}_L &= \{x : x \in \mathcal{T}, x_j \leq c\}, \\ \mathcal{T}_R &= \{x : x \in \mathcal{T}, x_j > c\}.\end{aligned}$$

The first term is the uncertainty in the parent; the second is the size-weighted uncertainty of the two children. A larger score is a better split, and we take

$$\arg \max_{j,c} \Phi_{\text{Gini}}(j, c).$$

A Binary Classification Problem: Toy Example

Consider the following split of a node of size 10:

	Class 1	Class 2	$\hat{p}_1$	$\hat{p}_2$	Gini	Entropy	Misclassification
$\mathcal{T}$	7	3	7/10	3/10	0.420	0.611	0.3
$\mathcal{T}_{\text{left}}$	3	0	3/3	0	0	0	0
$\mathcal{T}_{\text{right}}$	4	3	4/7	3/7	0.490	0.683	3/7

The three impurity gains are

$$\begin{aligned}\Phi_{\text{Gini}} &= 0.420 - \left(\frac{3}{10} \cdot 0 + \frac{7}{10} \cdot 0.490\right) = 0.077, \\ \Phi_{\text{Entropy}} &= 0.611 - \left(\frac{3}{10} \cdot 0 + \frac{7}{10} \cdot 0.683\right) = 0.133, \\ \Phi_{\text{Error}} &= \frac{3}{10} - \left(\frac{3}{10} \cdot 0 + \frac{7}{10} \cdot \frac{3}{7}\right) = 0.\end{aligned}$$

Gini and Shannon entropy respond to changes in the node probabilities and tend to produce purer children. Misclassification error is a natural criterion for *evaluating* a finished tree, but it is often not sensitive enough for *growing* one. Entropy and Gini are strictly concave, so the split gain Φ is nonnegative and is strictly positive unless the two children have the same class frequencies. That is one reason they are the default criteria for growing trees.

10.1.2 Splitting Rules: Categorical Predictors

If a predictor $X^{(j)}$ is categorical and takes values in $\{1, \dots, M\}$, we search over a subset $\mathcal{A} \subset \{1, \dots, M\}$ and use the split

$$\mathbf{1}_{\{X^{(j)} \in \mathcal{A}\}}.$$

There are at most $2^{M-1} - 1$ distinct partitions into two nonempty groups, so the search becomes expensive when M is large. Heuristics are then used; for example, one can try several random assignments of categories to a child.

For *binary* classification there is a useful shortcut (a classical CART result): order the M levels by the proportion of class 1, treat that ordered list as if it were a continuous variable, and search only the $M - 1$ adjacent cuts. That reduces the problem from exponential in M to linear in M .

10.2 Random Forests

In the regression notes we already saw the drawback of a single tree and the benefit of aggregating unstable, relatively inaccurate “weak learners” into a stronger predictor.

Bagging, *boosting*, and *random forests* all combine many trees. Bagging and random forests grow individual trees with random perturbations and then aggregate them. Boosting instead grows models sequentially, each with a small contribution, and adds them to the current fit.

10.2.1 Bagging Predictors for Classification

Draw bootstrap samples with replacement (so some observations appear more than once). Fit a CART tree to each bootstrap sample (the trees themselves may be tuned by cross-validation) and combine the B predictions by majority vote:

$$\hat{f}_{\text{bag}}(x) = \text{majority vote}\{\hat{f}_b(x)\}_{b=1}^B.$$

Averaging in this way can reduce the variance of a single tree substantially. In the construction below, CART could be replaced by another unstable learner. The toy example compares one tree with a bagged ensemble:

```
# Simulate data to use for the example
set.seed(2)
n = 1000
x1 = runif(n, -1, 1)
x2 = runif(n, -1, 1)
y = rbinom(n, size = 1, prob = ifelse((x1 + x2 > -0.5) & (x1 +
  x2 < 0.5), 0.8, 0.2))
xgrid = expand.grid(x1 = seq(-1, 1, 0.01), x2 = seq(-1, 1, 0.01))

par(mfrow = c(1, 2))

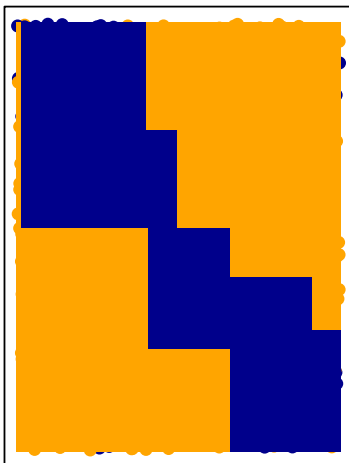
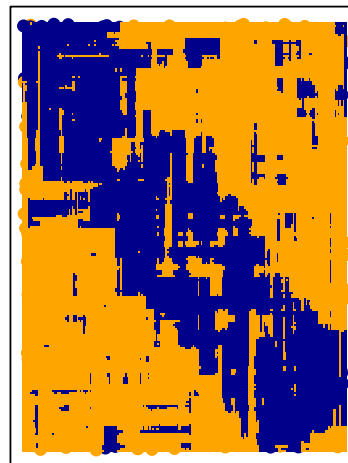
# Fit CART using rpart
library(rpart)
rpart.fit = rpart(as.factor(y) ~ x1 + x2, data = data.frame(x1,
  x2, y))

# A different tree from one bootstrap sample: rpart.fit =
# rpart(as.factor(y) ~ x1 + x2, data = data.frame(x1, x2,
```

```
# y[sample(1:n, n, replace = TRUE), ]

pred = matrix(predict(rpart.fit, xgrid, type = "class") == 1,
               201, 201)
contour(seq(-1, 1, 0.01), seq(-1, 1, 0.01), pred, levels = 0.5,
        labels = "", axes = FALSE)
points(x1, x2, col = ifelse(y == 1, "darkblue", "orange"), pch = 16,
       yaxt = "n", xaxt = "n")
points(xgrid, pch = ".", cex = 1.2, col = ifelse(pred, "darkblue",
        "orange"))
box()
title("CART")

# Fit bagging
library(ipred)
bag.fit = bagging(as.factor(y) ~ x1 + x2, data = data.frame(x1,
        x2, y), nbagg = 200)
pred = matrix(predict(bag.fit, xgrid) == 1, 201, 201)
contour(seq(-1, 1, 0.01), seq(-1, 1, 0.01), pred, levels = 0.5,
        labels = "", axes = FALSE)
points(x1, x2, col = ifelse(y == 1, "darkblue", "orange"), pch = 16,
       yaxt = "n", xaxt = "n")
points(xgrid, pch = ".", cex = 1.2, col = ifelse(pred, "darkblue",
        "orange"))
box()
title("Bagging")
```

CART**Bagging**

Bagging can cut the variance of unstable learners such as trees and often im-

proves prediction. The price is interpretability: the simple rectangular structure of a single tree is lost. A second limitation is that bagged trees tend to be highly correlated, which reduces the benefit of averaging.

10.2.2 Random Forests

Random forests keep the bootstrap (or subsample) step and add extra randomness through tuning parameters. To fit a forest one must choose at least the following.

- The number of trees. A large number stabilizes the ensemble; once the estimate has settled, extra trees mainly cost computation.
- How many observations to use for each tree. With sampling *with* replacement one often uses a sample of size n .
- The number of randomly selected predictors available at each split (`mtry`). This is the main tuning parameter: larger values are more adaptive when the signal is sparse; smaller values help when predictors are highly correlated. For classification the usual default is $\text{mtry} = \lfloor \sqrt{p} \rfloor$.

Splitting stops when a node has no more than `nodesize` observations. That role is a little like the neighborhood size k in k NN, although the effect interacts strongly with the other tuning parameters.

OOB error and the number of trees. Growing more trees does not overfit in the usual sense: after the ensemble has settled, extra trees mostly cost time. Each observation is left out of about one-third of the bootstrap samples; the votes on those left-out cases are the *out-of-bag* (OOB) predictions. The OOB error rate is a built-in estimate of test error, which is the main reason a forest can be used without a separate validation set.

```
# Generate some data with larger p
set.seed(542)
n = 1000
p = 10
X = matrix(runif(n * p, -1, 1), n, p)
x1 = X[, 1]
x2 = X[, 2]
y = rbinom(n, size = 1, prob = ifelse((x1 + x2 > -0.5) & (x1 +
  x2 < 0.5), 0.8, 0.2))
xgrid = expand.grid(x1 = seq(-1, 1, 0.01), x2 = seq(-1, 1, 0.01))

# Fit a random forest with a selected tuning
library(randomForest)

rf.fit = randomForest(X, as.factor(y), ntree = 1000, mtry = 7,
  nodesize = 10, sampsize = 800)

# Last row of err.rate is the OOB error after all trees
```

```
rf.fit$err.rate[nrow(rf.fit$err.rate), "OOB"]
```

```
##    OOB
## 0.316
```

Instead of manufacturing a test-set labeling by hand, compare the forest with the Bayes rule on a fresh sample.

```
# Testing data
Xtest = matrix(runif(n * p, -1, 1), n, p)

# Bayes rule for this simulation
BayesRule = ifelse((Xtest[, 1] + Xtest[, 2] > -0.5) & (Xtest[,
  1] + Xtest[, 2] < 0.5), 1, 0)

mean((predict(rf.fit, Xtest) == "1") == BayesRule)
```

```
## [1] 0.826
```

That number is the fraction of test points on which the forest *agrees with the Bayes rule*, not the ordinary misclassification rate.

Variable importance

A forest (or a boosted tree model) no longer offers a single split path, but it does give a ranking of predictors. Two common scores are:

- the total decrease in impurity (Gini) attributed to splits on that variable;
- a permutation score: scramble one column in the OOB sample and record how much the OOB error rises.

```
importance(rf.fit)
```

```
##      MeanDecreaseGini
## 1          57.67159
## 2          48.22224
## 3          25.00175
## 4          23.08845
## 5          24.86713
## 6          24.75000
## 7          25.19696
## 8          22.78443
## 9          25.59233
## 10         27.78592
```

Large importance is not a causal claim, and correlated predictors share credit. The `xgboost` chunk at the end of the chapter prints a gain-based ranking of the same kind.

10.3 Boosting

Consider a sequence of learners added together,

$$F_T(x) = \sum_{t=1}^T f_t(x).$$

How should each $f_t(x)$ be trained? At iteration t , given $f_1, \dots, f_{t-1}$, we look for a new function $h(x)$ that reduces the loss:

$$\min_h \sum_{i=1}^n L\left(y_i, \sum_{k=1}^{t-1} f_k(x_i) + h(x_i)\right).$$

In boosting we typically do not add the whole h . We take only a fraction of it and update the current model by $\alpha_t h(x)$, then move to the next iteration.

Boosting is an additive model, but it is not the same as a generalized additive model, where each component is a function of a single variable. Here each $f_t(x)$ can use several features, and we may add a large number of such terms. It is also different from a random forest, which is additive as well but grows its trees independently, so the trees cannot borrow information from one another.

10.3.1 AdaBoost

AdaBoost (Freund and Schapire, 1997) is the special case of the framework above that uses exponential loss for classification. Labels are coded as $y_i \in \{-1, 1\}$.

AdaBoost algorithm

1. Initialize weights $w_i^{(1)} = 1/n$, $i = 1, 2, \dots, n$.
2. For $t = 1, \dots, T$, repeat (a)–(d):
 - (a) Fit a classifier $f_t(x) \in \{-1, 1\}$ to the training data using weights $w_i^{(t)}$.
 - (b) Compute the weighted training error

$$\varepsilon_t = \sum_i w_i^{(t)} \mathbf{1}_{\{y_i \neq f_t(x_i)\}}.$$

- (c) Set

$$\alpha_t = \frac{1}{2} \log \frac{1 - \varepsilon_t}{\varepsilon_t}.$$

- (d) Update the weights

$$w_i^{(t+1)} = \frac{w_i^{(t)}}{Z_t} \exp\{-\alpha_t y_i f_t(x_i)\},$$

where Z_t renormalizes the weights to a probability distribution:

$$Z_t = \sum_{i=1}^n w_i^{(t)} \exp\{-\alpha_t y_i f_t(x_i)\}.$$

3. Output

$$F_T(x) = \sum_{t=1}^T \alpha_t f_t(x).$$

The classification rule is $\text{sign}(F_T(x))$.

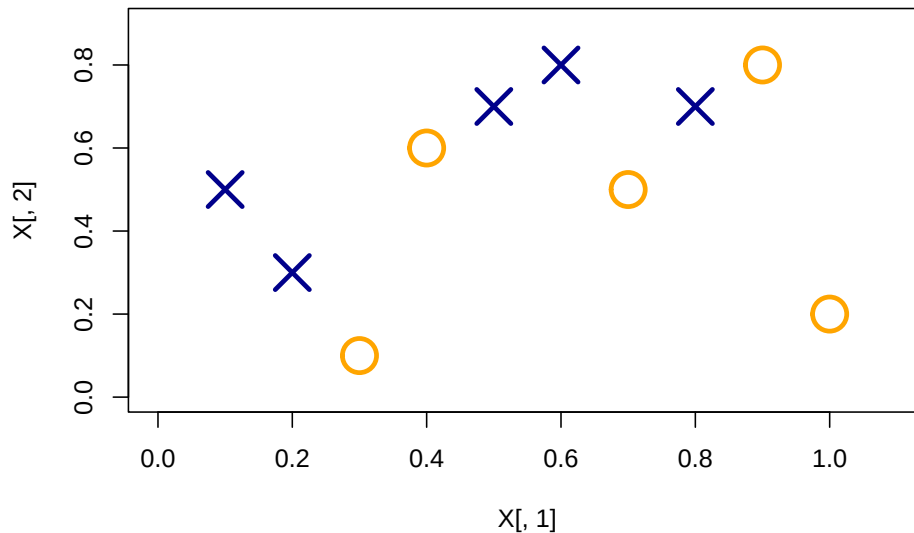
Scope of this algorithm. The version above needs labels in $\{-1, 1\}$ and a weak learner whose weighted error is below $1/2$. It is a two-class method. Multi-class boosting uses a different algorithm (for example SAMME, or a multinomial gradient-boosted tree). Do not apply this α_t update unchanged to a factor with $K > 2$ levels.

In the next example each $f_t(x)$ is a tree with a single split. The final classifier is the weighted sum of those stumps.

```
x1 = seq(0.1, 1, 0.1)
x2 = c(0.5, 0.3, 0.1, 0.6, 0.7, 0.8, 0.5, 0.7, 0.8, 0.2)

y = c(1, 1, -1, -1, 1, 1, -1, 1, -1, -1)
X = cbind(x1 = x1, x2 = x2)
xgrid = expand.grid(x1 = seq(0, 1.1, 0.01), x2 = seq(0, 0.9,
0.01))

plot(X[, 1], X[, 2], col = ifelse(y > 0, "darkblue", "orange"),
     pch = ifelse(y > 0, 4, 1), xlim = c(0, 1.1), lwd = 3, ylim = c(0,
0.9), cex = 3)
```



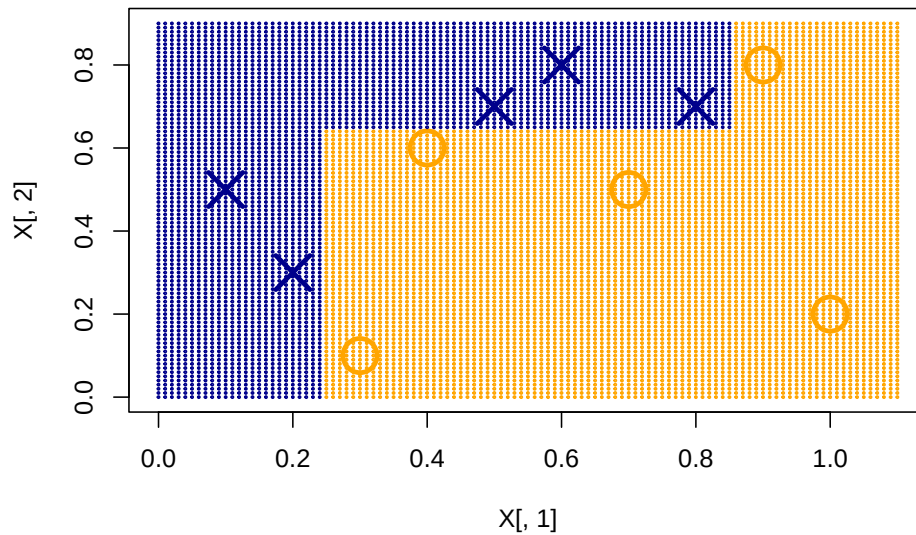
```
# Fit gbm with 3 trees
library(gbm)

gbm.fit = gbm(y ~ ., data.frame(x1, x2, y = as.numeric(y == 1)),
  distribution = "adaboost", interaction.depth = 1, n.minobsinnode = 1,
  n.trees = 3, shrinkage = 1, bag.fraction = 1)

pretty.gbm.tree(gbm.fit, i.tree = 1)
```

```
## SplitVar SplitCodePred LeftNode RightNode MissingNode ErrorReduction Weight
## 0 0 0.25 1 2 3 2.5 10
## 1 -1 1.00 -1 -1 -1 0.0 2
## 2 -1 -0.25 -1 -1 -1 0.0 8
## 3 -1 0.00 -1 -1 -1 0.0 10
## Prediction
## 0 0.00
## 1 1.00
## 2 -0.25
## 3 0.00
```

```
plot(X[, 1], X[, 2], col = ifelse(y > 0, "darkblue", "orange"),
  pch = ifelse(y > 0, 4, 1), xlim = c(0, 1.1), lwd = 3, ylim = c(0,
  0.9), cex = 3)
pred = predict(gbm.fit, xgrid)
points(xgrid, col = ifelse(pred > 0, "darkblue", "orange"), cex = 0.2)
```

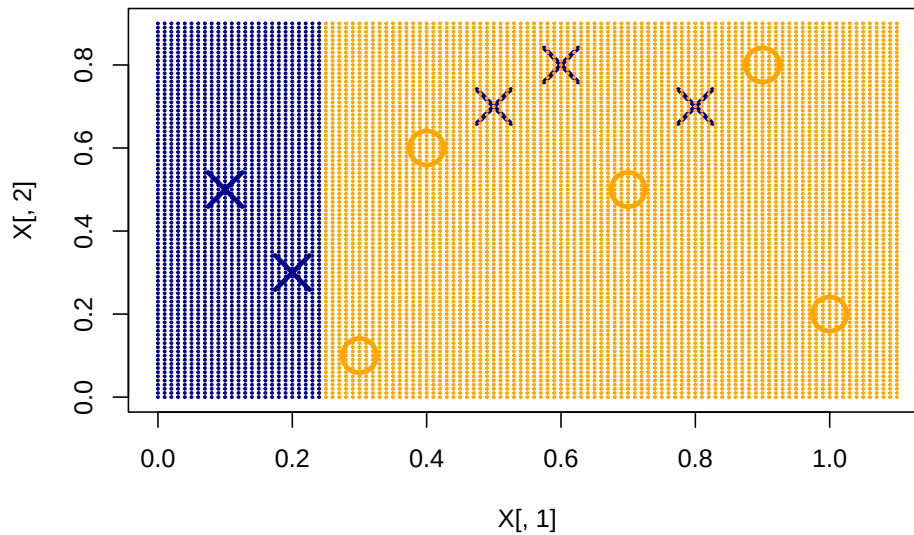


```
w1 = rep(1/10, 10)
f1 <- function(x) ifelse(x[, 1] < 0.25, 1, -1)
e1 = sum(w1 * (f1(X) != y))
a1 = 0.5 * log((1 - e1)/e1)

w2 = w1 * exp(-a1 * y * f1(X))
w2 = w2/sum(w2)

# First tree
plot(X[, 1], X[, 2], col = ifelse(y > 0, "darkblue", "orange"),
     pch = ifelse(y > 0, 4, 1), xlim = c(0, 1.1), lwd = 3, ylim = c(0,
     0.9), cex = 3)

pred = f1(xgrid)
points(xgrid, col = ifelse(pred > 0, "darkblue", "orange"), cex = 0.2)
```



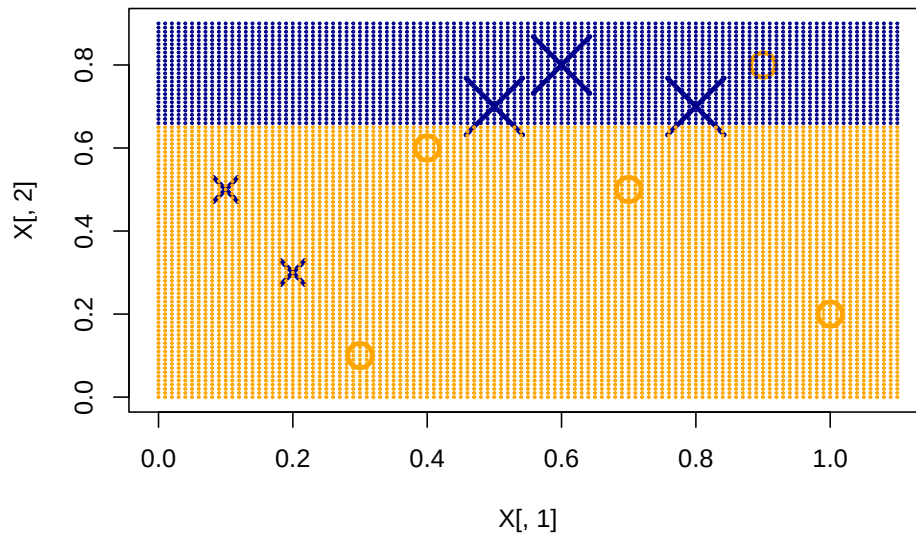
```
# Weights after the first tree
plot(X[, 1], X[, 2], col = ifelse(y > 0, "darkblue", "orange"),
     pch = ifelse(y > 0, 4, 1), xlim = c(0, 1.1), lwd = 3, ylim = c(0,
     0.9), cex = 30 * w2)

f2 <- function(x) ifelse(x[, 2] > 0.65, 1, -1)
e2 = sum(w2 * (f2(X) != y))
a2 = 0.5 * log((1 - e2)/e2)

w3 = w2 * exp(-a2 * y * f2(X))
w3 = w3/sum(w3)

# Second tree
plot(X[, 1], X[, 2], col = ifelse(y > 0, "darkblue", "orange"),
     pch = ifelse(y > 0, 4, 1), xlim = c(0, 1.1), lwd = 3, ylim = c(0,
     0.9), cex = 30 * w2)

pred = f2(xgrid)
points(xgrid, col = ifelse(pred > 0, "darkblue", "orange"), cex = 0.2)
```

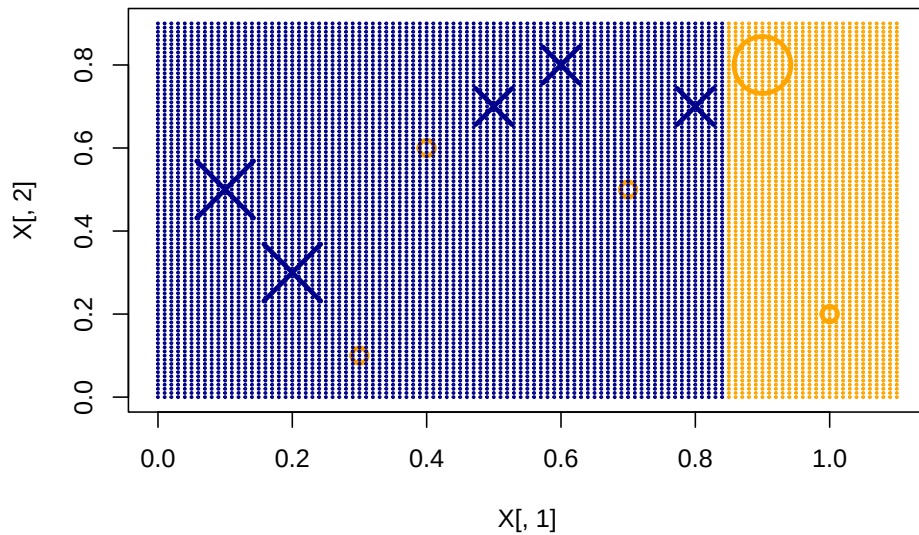


```
# Weights after the second tree
plot(X[, 1], X[, 2], col = ifelse(y > 0, "darkblue", "orange"),
     pch = ifelse(y > 0, 4, 1), xlim = c(0, 1.1), lwd = 3, ylim = c(0,
     0.9), cex = 30 * w3)

f3 <- function(x) ifelse(x[, 1] < 0.85, 1, -1)
e3 = sum(w3 * (f3(X) != y))
a3 = 0.5 * log((1 - e3)/e3)

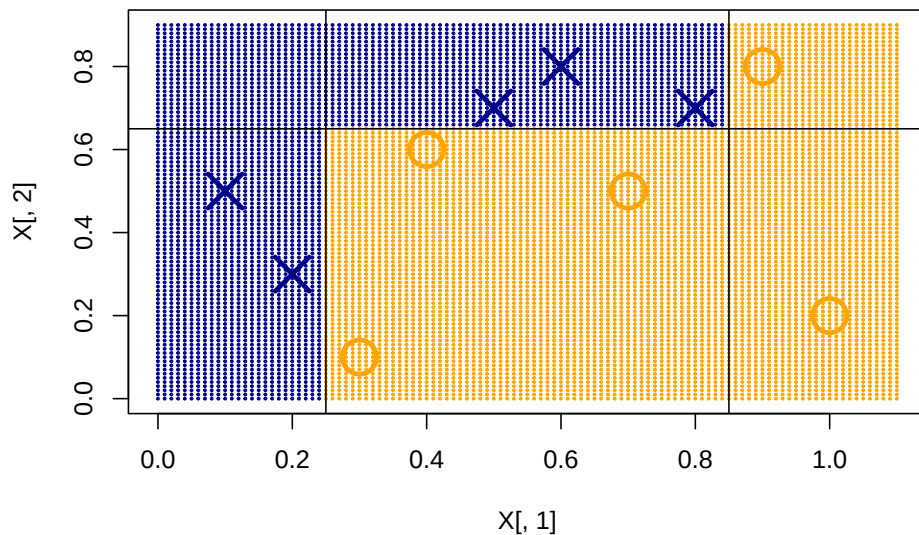
# Third tree
plot(X[, 1], X[, 2], col = ifelse(y > 0, "darkblue", "orange"),
     pch = ifelse(y > 0, 4, 1), xlim = c(0, 1.1), lwd = 3, ylim = c(0,
     0.9), cex = 30 * w3)

pred = f3(xgrid)
points(xgrid, col = ifelse(pred > 0, "darkblue", "orange"), cex = 0.2)
```



```
# Final decision rule
plot(X[, 1], X[, 2], col = ifelse(y > 0, "darkblue", "orange"),
     pch = ifelse(y > 0, 4, 1), xlim = c(0, 1.1), lwd = 3, ylim = c(0,
0.9), cex = 3)

pred = a1 * f1(xgrid) + a2 * f2(xgrid) + a3 * f3(xgrid)
points(xgrid, col = ifelse(pred > 0, "darkblue", "orange"), cex = 0.2)
abline(v = 0.25) # f1
abline(h = 0.65) # f2
abline(v = 0.85) # f3
```



At the first iteration the stump splits X_1 at 0.25. The misclassified positive

points on the right have their weights increased. At the second iteration the stump splits X_2 at 0.65, which updates the weights again and produces a new α_t . At the third iteration the stump splits X_1 at 0.85. The stacked model is

$$F_3(x) = 0.4236 f_1(x) + 0.6496 f_2(x) + 0.9229 f_3(x).$$

The intuition is simple. All observations start with equal weight. After we fit f_t , the misclassified points are upweighted for the next round. We add $\alpha_t f_t(x)$ to the current model and continue with the new weights.

A very small training error is possible, but that does not prevent overfitting. The next example shows the pattern.

```
# One-dimensional classification example
```

```
n = 1000
```

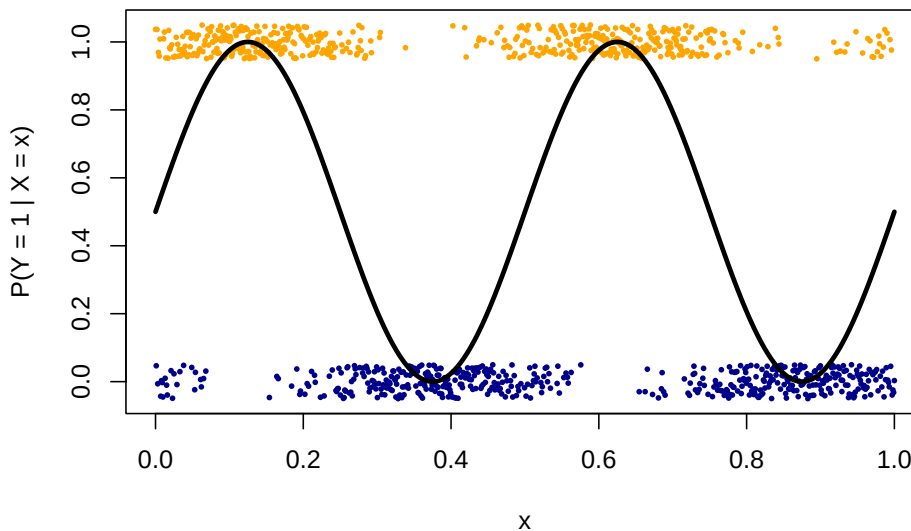
```
set.seed(1)
```

```
x = cbind(seq(0, 1, length.out = n), runif(n))
```

```
py = (sin(4 * pi * x[, 1]) + 1)/2
```

```
y = rbinom(n, 1, py)
```

```
plot(x[, 1], y + runif(n, -0.05, 0.05), pch = 16, ylim = c(-0.05, 1.05),
      cex = 0.5, col = ifelse(y == 1, "orange", "darkblue"),
      xlab = "x", ylab = "P(Y = 1 | X = x)")
points(x[, 1], py, type = "l", lwd = 3)
```



```
# AdaBoost with a large shrinkage factor, to make the  
# stages visible
```

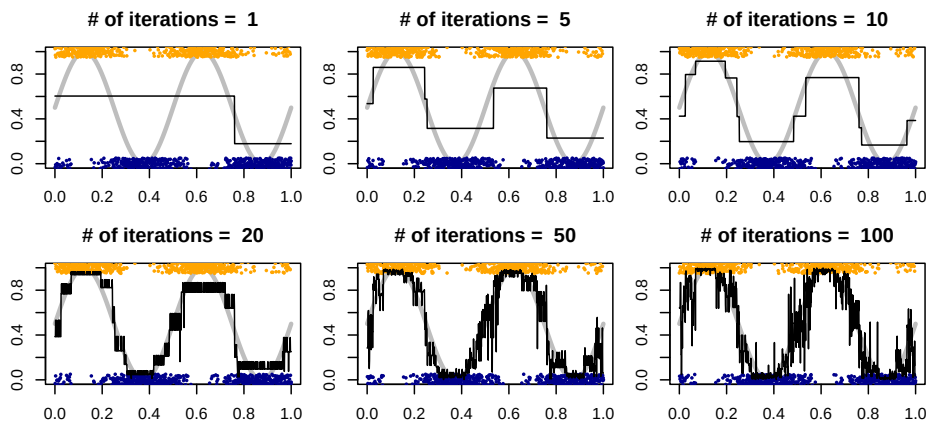
```
gbm.fit = gbm(y ~ ., data.frame(x, y), distribution = "adaboost",  
              n.minobsinnode = 2, n.trees = 200, shrinkage = 1, bag.fraction = 0.8,  
              cv.folds = 10)
```

```

par(mfrow = c(3, 3))
# Plot the decision function F(x), not sign(F(x))
size = c(1, 5, 10, 20, 50, 100)

for (i in 1:6) {
  par(mar = c(2, 2, 3, 1))
  plot(x[, 1], py, type = "l", lwd = 3, ylab = "P(Y = 1 | X = x)",
       col = "gray")
  points(x[, 1], y + runif(n, -0.05, 0.05), pch = 16, cex = 0.5,
        col = ifelse(y == 1, "orange", "darkblue"))
  Fx = predict(gbm.fit, n.trees = size[i])
  lines(x[, 1], 1/(1 + exp(-2 * Fx)), lwd = 1)
  title(paste("# of iterations = ", size[i]))
}

```

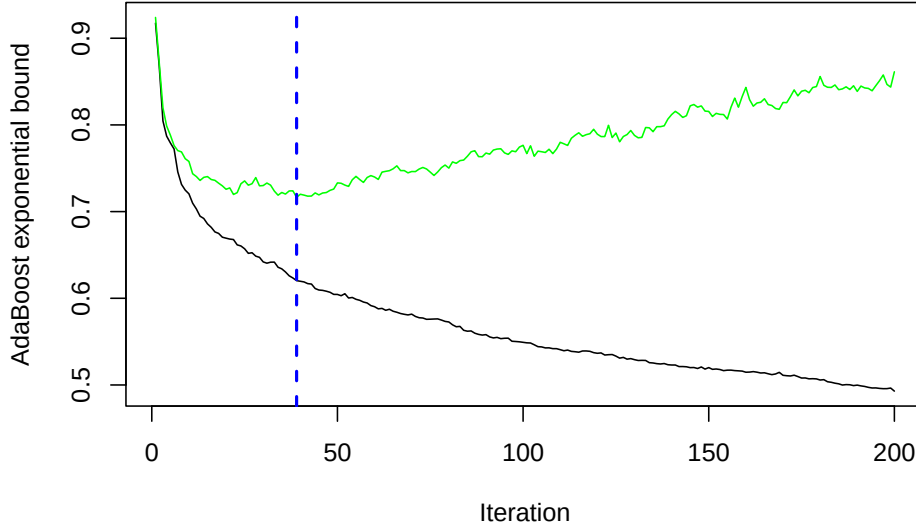


With few iterations the fit is too smooth (underfitting); with many iterations it tracks noise (overfitting). The number of trees therefore has to be chosen. One can use the out-of-bag estimate of the exponential loss, or ordinary cross-validation.

```

# Best number of trees from cross-validation (use method =
# 'OOB' if no CV was run)
gbm.perf(gbm.fit, method = "cv")

```



[1] 39

10.3.2 Training Error Bound

A useful property of AdaBoost is the following: if every iteration produces a classifier that is better than random guessing, the *training* error tends to zero. To see why, unwind the weights after the last update,

$$w_i^{(T+1)} = \frac{1}{Z_T} w_i^{(T)} \exp\{-\alpha_T y_i f_T(x_i)\}.$$

The same relation holds at the previous step,

$$w_i^{(T)} = \frac{1}{Z_{T-1}} w_i^{(T-1)} \exp\{-\alpha_{T-1} y_i f_{T-1}(x_i)\},$$

and so on back to the first iteration. Therefore

$$\begin{aligned} w_i^{(T+1)} &= \frac{1}{Z_1 \cdots Z_T} w_i^{(1)} \prod_{t=1}^T \exp\{-\alpha_t y_i f_t(x_i)\} \\ &= \frac{1}{Z_1 \cdots Z_T} \frac{1}{n} \exp\left\{-y_i \sum_{t=1}^T \alpha_t f_t(x_i)\right\} \\ &= \frac{1}{Z_1 \cdots Z_T} \frac{1}{n} \exp\{-y_i F_T(x_i)\}. \end{aligned}$$

The weights sum to 1, so

$$1 = \sum_{i=1}^n w_i^{(T+1)} = \frac{1}{Z_1 \cdots Z_T} \frac{1}{n} \sum_{i=1}^n \exp\{-y_i F_T(x_i)\},$$

or

$$Z_1 \cdots Z_T = \frac{1}{n} \sum_{i=1}^n \exp\{-y_i F_T(x_i)\}.$$

The right-hand side is the exponential loss. Two elementary comparisons are:

- if y and $F(x)$ have the same sign, then $\exp(-yF(x)) \leq 1$;
- if they have opposite signs, then $\exp(-yF(x)) > 1$.

In particular, exponential loss dominates 0–1 loss:

$$\begin{aligned} Z_1 \cdots Z_T &= \frac{1}{n} \sum_{i=1}^n \exp\{-y_i F_T(x_i)\} \\ &\geq \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\{y_i \neq \text{sign}(F_T(x_i))\}}. \end{aligned}$$

Each factor Z_t can be written more explicitly. Because $f_t(x_i) \in \{-1, 1\}$,

$$\begin{aligned} Z_t &= \sum_{i=1}^n w_i^{(t)} \exp\{-\alpha_t y_i f_t(x_i)\} \\ &= e^{-\alpha_t} \sum_{y_i = f_t(x_i)} w_i^{(t)} + e^{\alpha_t} \sum_{y_i \neq f_t(x_i)} w_i^{(t)} \\ &= (1 - \varepsilon_t) e^{-\alpha_t} + \varepsilon_t e^{\alpha_t}, \end{aligned}$$

where

$$\varepsilon_t = \sum_i w_i^{(t)} \mathbf{1}_{\{y_i \neq f_t(x_i)\}}$$

is the weighted misclassification rate at step t . The value of α_t used by AdaBoost is exactly the choice that minimizes this Z_t . Differentiating in α_t gives

$$-(1 - \varepsilon_t) e^{-\alpha_t} + \varepsilon_t e^{\alpha_t} = 0,$$

hence

$$\alpha_t = \frac{1}{2} \log \frac{1 - \varepsilon_t}{\varepsilon_t}.$$

Substituting that value yields

$$Z_t = 2\sqrt{\varepsilon_t(1 - \varepsilon_t)}.$$

The product $\varepsilon_t(1 - \varepsilon_t)$ is at most $1/4$, with equality only when $\varepsilon_t = 1/2$. Thus $Z_t < 1$ whenever the weak learner beats random guessing, and the product $Z_1 \cdots Z_T$ is driven toward 0.

The Training Error

Write $\gamma_t = 1/2 - \varepsilon_t$ for the edge over random guessing. Then

$$Z_t = 2\sqrt{\varepsilon_t(1 - \varepsilon_t)} = \sqrt{1 - 4\gamma_t^2} \leq \exp(-2\gamma_t^2),$$

using $1 - u \leq e^{-u}$. Therefore the AdaBoost training error satisfies

$$\begin{aligned} \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\{y_i \neq \text{sign}(F_T(x_i))\}} &\leq \frac{1}{n} \sum_{i=1}^n \exp\{-y_i F_T(x_i)\} \\ &= Z_1 \cdots Z_T \\ &\leq \exp\left\{-2 \sum_{t=1}^T \gamma_t^2\right\}. \end{aligned}$$

Whenever each f_t is better than a coin flip, the right-hand side tends to 0 as T grows.

Remarks

- Does AdaBoost output a classifier F_T with small *test* error? **Not automatically.** T still has to be tuned; the algorithm can overfit.
- Does the 0–1 training error of F_T decrease at every T ? **Not necessarily.** What decreases at each step is a particular *upper bound* on that error. Over a long run the training error is driven toward zero, but not monotonically.
- Can we use a classifier that is *worse* than random guessing? **Yes.** That classifier is equivalent to its reverse with $\alpha_t < 0$.

In practice the weak learner is a classification tree (often a stump or a shallow tree). One can also read off an estimated class probability from the exponential loss. The population criterion $E[\exp(-yF(x))]$ equals

$$e^{-F(x)} P(Y = 1 \mid x) + e^{F(x)} P(Y = -1 \mid x).$$

Setting the derivative in $F(x)$ to zero gives

$$-e^{-F(x)} P(Y = 1 \mid x) + e^{F(x)} P(Y = -1 \mid x) = 0,$$

so the minimizer is

$$F(x) = \frac{1}{2} \log \frac{P(Y = 1 \mid x)}{P(Y = -1 \mid x)}, \quad P(Y = 1 \mid x) = \frac{e^{2F(x)}}{1 + e^{2F(x)}}.$$

10.3.3 Gradient Boosting: Forward Stagewise Additive Models

A more general additive expansion is

$$F_T(x) = \sum_{t=1}^T \alpha_t f(x; \theta_t),$$

fitted by minimizing

$$\min_{\{\alpha_t, \theta_t\}_{t=1}^T} \sum_{i=1}^n L(y_i, F_T(x_i)).$$

One chooses a loss L that matches the problem and a base learner $f(x; \theta)$ (a linear term, a tree, ...). Minimizing jointly over all $\{\alpha_t, \theta_t\}$ is hard, so the fit is built stage by stage.

Algorithm

1. Set $F_0(x) = 0$.
2. For $t = 1, \dots, T$:
 - (a) Choose $\{\alpha_t, \theta_t\}$ to minimize

$$\min_{\alpha, \theta} \sum_{i=1}^n L(y_i, F_{t-1}(x_i) + \alpha f(x_i; \theta)).$$

- (b) Update $F_t(x) = F_{t-1}(x) + \alpha_t f(x; \theta_t)$.

AdaBoost is this procedure with exponential loss. It does not insist on an optimal $f(x; \theta)$ at each step: any learner better than random is enough. Given that learner, the step size α_t is optimized.

Another forward-stagewise example is **forward-stagewise linear regression**. At each step one uses a single-variable update

$$f(x, j) = \text{sign}(\text{Cor}(X_j, r)) X_j,$$

where $r_i = y_i - F_{t-1}(x_i)$ is the current residual and j indexes the predictor most correlated with r . A very small step α_t (for example 10^{-5}) is then taken in that direction. As t varies, F_t traces a path that is nearly the lasso path.

Equivalently, r_i is the negative gradient of squared-error loss:

$$r_{it} = - \left[\frac{\partial}{\partial F(x_i)} (y_i - F(x_i))^2 \right]_{F(x_i) = F_{t-1}(x_i)}.$$

One fits the weak learner f_t to these residuals and updates F_t . The same idea applies to a general loss L :

- At iteration t , compute the *pseudo-residuals* (negative gradient)

$$g_{it} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x_i) = F_{t-1}(x_i)}.$$

- Fit $f_t(x; \theta_t)$ to $\{g_{it}\}$.

- Choose a step length

$$\alpha_t = \arg \min_{\alpha} \sum_{i=1}^n L(y_i, F_{t-1}(x_i) + \alpha f(x_i; \theta_t)).$$

- Update $F_t(x) = F_{t-1}(x) + \alpha_t f(x; \theta_t)$.

Changing the outcome type then amounts to changing the loss and the implied pseudo-residuals. When the base learner is CART, one can go further and choose a separate step size in each terminal node.

10.3.4 xgboost

AdaBoost and `gbm` already show how a boosted classifier is trained and how T is chosen. What remains is the implementation students are most likely to meet in practice.

`xgboost` (Chen and Guestrin, 2016) is a fast implementation of gradient boosting with explicit regularization (`lambda`, `alpha`, `max_depth`, column and row subsampling) and built-in early stopping. The chunk below uses squared-error loss so the API is uncluttered. A classification fit is the same call with `objective` set to `"binary:logistic"` or `"multi:softprob"`; the iteration logic does not change.

```
set.seed(2025)
n <- 2000
p <- 15
X <- matrix(rnorm(n * p), n, p)
f_true <- function(x) 2 * sin(x[, 1]) + 0.5 * x[, 2]^2 - 1.5 * (x[, 3] > 0)
y <- f_true(X) + rnorm(n, sd = 1)

idx <- sample.int(n, floor(0.7 * n))
dtrain <- xgboost::xgb.DMatrix(X[idx, ], label = y[idx])
dvalid <- xgboost::xgb.DMatrix(X[-idx, ], label = y[-idx])

params <- list(
  objective = "reg:squarederror",
  eval_metric = "rmse",
  eta = 0.05,
  max_depth = 4,
  subsample = 0.8,
  colsample_bytree = 0.8,
  min_child_weight = 5,
  lambda = 1.0,
  alpha = 0
)
```

```

watch <- list(train = dtrain, valid = dvalid)
fit <- xgboost::xgb.train(
  params = params,
  data = dtrain,
  nrounds = 3000,
  watchlist = watch,
  early_stopping_rounds = 50,
  verbose = 0
)

## Warning in throw_err_or_depr_msg("Parameter '", match_old, "' has been renamed
## to '", : Parameter 'watchlist' has been renamed to 'evals'. This warning will
## become an error in a future version.

fit$best_iteration

## NULL

fit$best_score

## NULL

imp <- xgboost::xgb.importance(model = fit, feature_names = paste0("x", 1:p))
head(imp, 8)

##      Feature      Gain      Cover  Frequency
##      <char>      <num>      <num>      <num>
## 1:      x1 0.52082853 0.17986857 0.13098649
## 2:      x2 0.17765382 0.20785955 0.16332378
## 3:      x3 0.16081670 0.10099492 0.08350389
## 4:     x11 0.01634937 0.04621493 0.06221858
## 5:     x14 0.01426370 0.04683192 0.06221858
## 6:      x4 0.01410662 0.05919056 0.06099059
## 7:     x10 0.01354525 0.03752183 0.04952927
## 8:      x6 0.01240869 0.04412002 0.05771592

pred <- predict(fit, dvalid)
rmse <- sqrt(mean((pred - y[-idx])^2))
rmse

## [1] 1.085705

```

10.4 A Controlled Comparison

The Python version of this example and of all the other examples in this chapter can be found here: [Python Code for Classification Trees And Ensemble](#) .

The figures above use different data sets, so they cannot answer a simple question: on *one* problem, how do a single tree, bagging, a random forest, and boosting compare? We reuse the circular Bayes rule from the beginning of the chapter, with one training draw and one independent test draw.

```
library(rpart)
library(ipred)
library(randomForest)
library(gbm)

set.seed(10)
n = 1000

sim_circle = function(n) {
  x1 = runif(n, -1, 1)
  x2 = runif(n, -1, 1)
  p = ifelse(x1^2 + x2^2 < 0.6, 0.9, 0.1)
  y = rbinom(n, 1, p)
  data.frame(x1 = x1, x2 = x2, y = y)
}

train = sim_circle(n)
test = sim_circle(n)
train$yf = factor(train$y)
test$yf = factor(test$y)

bayes_rule = function(x1, x2) as.integer(x1^2 + x2^2 < 0.6)

# 1. Pruned CART
cart.fit = rpart(yf ~ x1 + x2, data = train)
cp.best = cart.fit$cptable[which.min(cart.fit$cptable[, "xerror"]),
  "CP"]
cart.pr = prune(cart.fit, cp = cp.best)

# 2. Bagging
bag.fit = bagging(yf ~ x1 + x2, data = train, nbagg = 200)

# 3. Random forest (classification default mtry =
# floor(sqrt(p)) = 1)
rf.fit = randomForest(yf ~ x1 + x2, data = train, ntree = 500,
  mtry = 1)

# 4. Gradient boosting; choose T by CV
gbm.fit = gbm(y ~ x1 + x2, data = train, distribution = "bernoulli",
  n.trees = 400, shrinkage = 0.05, interaction.depth = 2, bag.fraction = 0.8,
  cv.folds = 5)
```

```

best.iter = gbm.perf(gbm.fit, method = "cv", plot.it = FALSE)

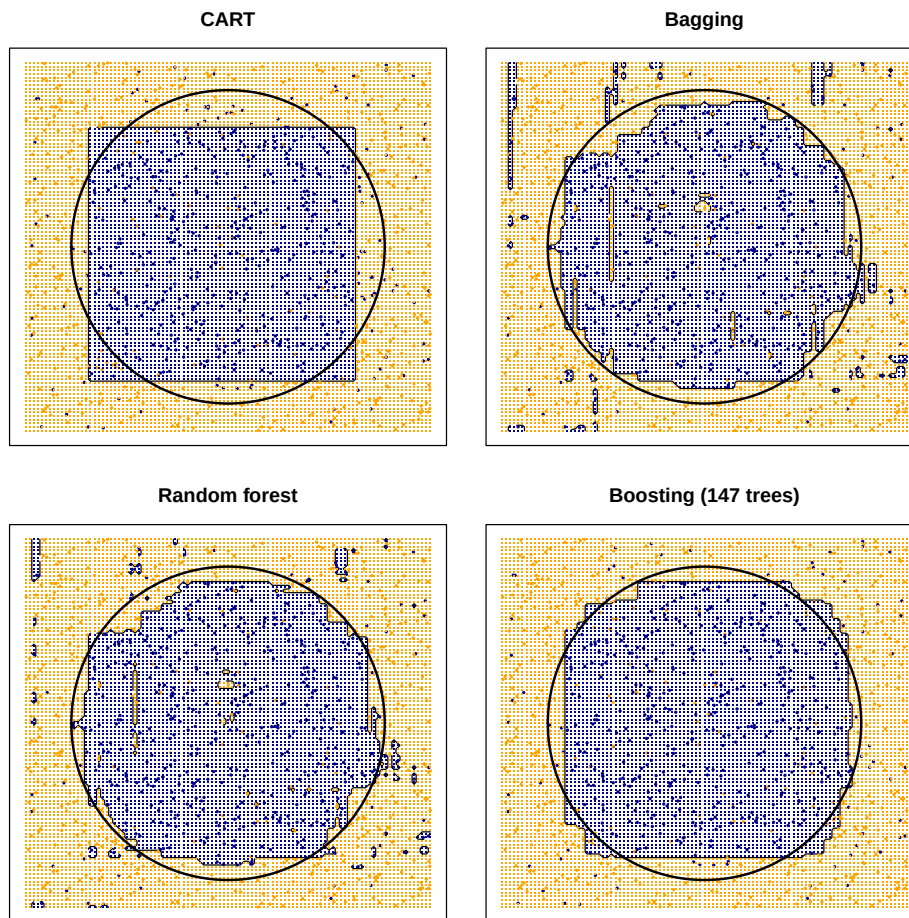
xgrid = expand.grid(x1 = seq(-1, 1, 0.02), x2 = seq(-1, 1, 0.02))

pred_grid = list(CART = as.integer(predict(cart.pr, xgrid, type = "class")) -
  1, Bagging = as.integer(predict(bag.fit, xgrid)) - 1, Forest = as.integer(predict(
  xgrid)) - 1, Boosting = as.integer(predict(gbm.fit, xgrid,
  n.trees = best.iter, type = "response") > 0.5))

plot_panel = function(title, pred) {
  z = matrix(pred, nrow = length(unique(xgrid$x1)))
  contour(seq(-1, 1, 0.02), seq(-1, 1, 0.02), z, levels = 0.5,
    labels = "", axes = FALSE)
  points(train$x1, train$x2, col = ifelse(train$y == 1, "darkblue",
    "orange"), pch = 16, cex = 0.45)
  points(xgrid, pch = ".", cex = 1.1, col = ifelse(pred ==
    1, "darkblue", "orange"))
  symbols(0, 0, circles = sqrt(0.6), add = TRUE, inches = FALSE,
    lwd = 2)
  box()
  title(title)
}

par(mfrow = c(2, 2), mar = c(1, 1, 3, 1))
plot_panel("CART", pred_grid$CART)
plot_panel("Bagging", pred_grid$Bagging)
plot_panel("Random forest", pred_grid$Forest)
plot_panel(paste0("Boosting (", best.iter, " trees)"), pred_grid$Boosting)

```



The black circle is the Bayes boundary. The single tree builds a coarse staircase. Bagging, the forest, and boosting keep the axis-aligned constraint but average (or add) enough rectangles to track the circle more closely.

```
pred_test = list(Majority = rep(as.integer(names(which.max(table(train$y))))),
  nrow(test)), CART = as.integer(predict(cart.pr, test, type = "class")) -
  1, Bagging = as.integer(predict(bag.fit, test)) - 1, Forest = as.integer(predict(rf.fit,
  test)) - 1, Boosting = as.integer(predict(gbm.fit, test,
  n.trees = best.iter, type = "response") > 0.5))
truth = test$y
bayes = bayes_rule(test$x1, test$x2)

cmp = data.frame(method = names(pred_test), test_error = sapply(pred_test,
  function(ph) mean(ph != truth)), agree_with_Bayes = sapply(pred_test,
  function(ph) mean(ph == bayes)))
cmp$test_error = round(cmp$test_error, 3)
cmp$agree_with_Bayes = round(cmp$agree_with_Bayes, 3)
```

```
cmp
```

```
##           method test_error agree_with_Bayes
## Majority Majority      0.491         0.515
## CART      CART       0.173         0.915
## Bagging   Bagging     0.154         0.934
## Forest    Forest      0.141         0.949
## Boosting  Boosting     0.141         0.947
```

Two remarks follow from the table and the figure, and they are the point of putting all four methods on the same draw.

- Test error and agreement with the Bayes rule need not rank methods in the same order. A procedure can match the circle and still miss labels near the boundary, where the class probability is 0.9 on one side and 0.1 on the other—so even the Bayes rule has error 0.1.
- The forest used the default `mtry = 1` and a large `ntree`; extra trees after the OOB curve has flattened cost time rather than decisions. Boosting chose T from the CV trace (`best.iter` in the plot title). That is the practical difference advertised earlier: a forest is mostly insensitive to using “too many” trees, and a boosted model is not.

Which method, and when?

The last four classification chapters are different answers to the same prediction problem:

- **Logistic regression** when you want coefficients, standard errors, and calibrated probabilities from a linear predictor on the logit scale.
- **Discriminant analysis** when a Gaussian (or similar) class-conditional model is plausible and you want a rule that uses the pooled or class-specific covariance.
- **SVM** when the margin story matters and you are willing to work with a kernel rather than with probabilities.
- **A single tree** when the decision rule itself has to be shown—a split on sex and class, for example.
- **A random forest** as a strong default among tree methods: little tuning beyond `mtry`, and OOB error as a built-in check.
- **Boosting** (`gbm`, `xgboost`) when you will tune the number of iterations and care about the last bits of accuracy. It is more sensitive to T and to shrinkage than a forest is.

Chapter 11

Recommender Systems

A **recommender system** (also called a recommendation system or recommendation platform) is a subclass of information filtering systems that seeks to predict the rating or preference a user would give to an item.

Recommender systems are everywhere: in movie/series streaming platforms (e.g., Netflix), e-commerce websites (e.g., Amazon), social network platforms (e.g., Facebook, LinkedIn), or dating apps. These systems typically leverage historical data to automatically recommend items or products that a user is likely to be interested in.

So, let's consider how those recommender systems are constructed. A useful starting point is the *non-personalized* baseline that many platforms still display: top-5 lists. A few years ago, the recommendations you could find were about top-5 destinations for the summer, top-5 Black Friday deals, and so on. Those lists are based on expert knowledge or aggregate statistics, and they lack personalization. This is exactly where recommender systems come in.

In this section, we explore fundamental recommendation methods: content-based methods, collaborative filtering (item-based or user-based), and latent factor models. If we classify the different methods, we notice that they are based on mainly two types of information: (i) characteristics or information about the items, and/or (ii) user-item interactions. This approach leads to

- **Content-based methods:** focusing on attributes of a specific item.
- **Collaborative Filtering** (item-based / user-based) methods: based on the analysis of user-item interaction data.
- **Baseline predictors:** global, user, and item effects as a first model for the rating matrix.
- **Latent Factor Models:** also based on the analysis of user-item interaction data.

Hybrid methods combine content-based ideas with collaborative filtering, and modern production systems often use deep learning. We mention those only briefly at the end; the rest of this chapter develops the three families above.

Several of these ideas were sharpened in industry, notably around the **Netflix Prize** (2006–2009), a public competition that asked teams to predict movie ratings and scored them by RMSE. Matrix factorization became a standard latent-factor method in large part because of that competition.

11.1 Content-Based Methods

The content-based method is the more intuitive of the two classical approaches. Assume that we are an online bookstore and we want a catalogue in which every item (book) can be represented accurately. We represent each book as a multidimensional feature vector. The features could include the author, the publisher, the genre, and the year—anything that we believe describes the type of book—and the resulting vector is the *item profile*. We then visualize every item as a distinct point within a multidimensional feature space (the orange dots in the figure).

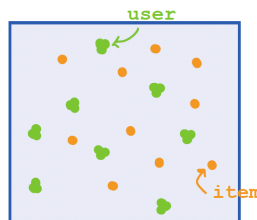


Figure 11.1: User and Item feature space

We keep a bank of item profiles, and we also construct a *user profile* in the same feature space (the green dots in the figure). Building the user profile is harder. Typically this is done either *directly*, by asking users for feedback, or *indirectly*, by monitoring activity on the website (clicks, viewing times, and analysis of items the user has already interacted with). These profiles usually give more weight to recent interactions than to older ones. The recommendation is then based on the items that are **closer** to the user in this multidimensional feature space. In other words, *closeness equates to preference*, which gives us a way to make tailored recommendations.

Advantages. The main advantage of this approach is that it does not require any other user’s data to make a recommendation. It is therefore possible to start recommending from day one, without waiting for an expensive collection of user-item interaction data. The method also does not discriminate on the basis of item popularity: a rare book and a bestseller are treated the same way if they sit at the same distance from the user. Recommendations are tied to

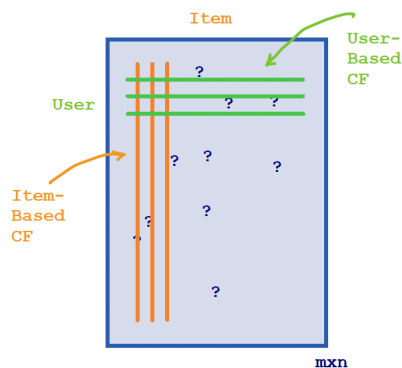
explicit features, which makes the system relatively transparent. It is easy to say why an item was recommended (matching author, genre, and so on) and how it aligns with the user's interests.

Limitations. The same mechanism produces *limited discovery*. Because every recommendation is based on items similar to those the user has already interacted with, the system tends to keep suggesting more of the same and rarely surfaces something unexpected. New users with empty profiles are also hard to place in the feature space until some feedback has been collected.

11.2 Collaborative Filtering

The collaborative filtering method relies on an interaction matrix between users and items, known as **the rating matrix**. This is an $m \times n$ grid, where rows represent users and columns represent items. Within this matrix, some entries are known via *explicit* or *implicit* feedback. Explicit feedback might be direct ratings, such as a user's rating of a movie on Netflix on a scale from one to five. Implicit feedback comes from behavior such as clicking on an item or spending time viewing a particular movie, which provides only indirect clues about preference.

Mathematically, the problem we need to solve is *matrix completion*: we fill in the blanks by predicting preferences for items a user has not yet interacted with. Throughout, we write $r_{aj} = ?$ when user a has no observed rating for item j .



There are two types of collaborative filtering:

- **User-based CF:** “*Similar users will like similar items.*” For example, if User A liked an item and User B has a similar profile to User A, it is likely that User B will also like that item. The challenge is defining what makes users similar.
- **Item-based CF:** “*Users will like items similar to those they have already liked.*” For example, if User A liked Item X, and Item Y is similar to Item

X, then User A is likely to like Item Y as well.

User-based Collaborative Filtering

The challenge in user-based collaborative filtering is to quantify resemblance between users (or, later, items) even when most ratings are missing. This means that we need a notion of **similarity**. Some of the most commonly used similarity metrics include:

- **Jaccard Similarity**: compares users based on the items they have both interacted with, or compares items based on shared users. It is ideal for binary data. For explicit ratings one first binarizes (for example, “rated” versus “not rated”, or “liked” versus “not liked”):

$$\frac{|A \cap B|}{|A \cup B|}, \quad \text{where } A, B \text{ are two sets.}$$

- **Cosine Similarity**: measures the cosine of the angle between two vectors. Missing values are ignored, or equivalently treated as zeros:

$$\frac{u^\top v}{\|u\| \cdot \|v\|}, \quad \text{where } u, v \text{ are two vectors.}$$

- **Centered Cosine Similarity (Pearson Correlation)**: A variation of cosine similarity that first subtracts a user’s (or item’s) mean rating:

$$\frac{(u - \bar{u})^\top (v - \bar{v})}{\|u - \bar{u}\| \cdot \|v - \bar{v}\|}, \quad \text{where } u, v \text{ are two vectors.}$$

Cosine and Pearson coefficients fall in the range $[-1, 1]$, whereas a similarity used as a weight is often expected to lie in $[0, 1]$. Negative similarities are awkward in a weighted average, so in practice one either keeps only positive similarities or maps the coefficient to $[0, 1]$ via $(1 + \cos)/2$. In what follows we treat the similarity s_{au} as a weight and assume that convention has already been chosen.

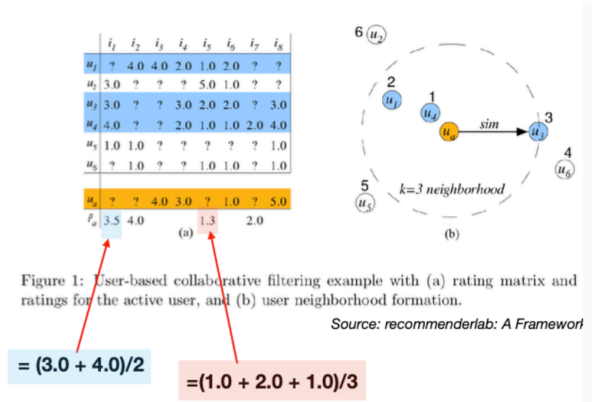
The measures are computed from the *shared*, non-missing elements between two entities. For users u and v , the calculation uses only items that both users have rated. Consequently the effective length of the vector for u can change depending on whether we compare u with v_1 or u with v_2 . A common pathology is that two users who have rated only one item in common receive similarity 1.

Two conventions for centering are in use, and they are not the same:

- **Center on all of a user’s ratings, then take cosine on co-rated items.** If user u has rated 20 movies and 5 of those are also rated by user v , the mean $\bar{u}$ is computed from all 20 ratings, while the cosine uses only the 5 shared ratings. This is the usual implementation of Pearson correlation in recommender systems.

- **Restrict everything, including the means, to co-rated items.** In the same example, both the centering and the cosine use only the 5 shared ratings.

The displayed formula above corresponds to the second convention if $\bar{u}$ and $\bar{v}$ are computed on the co-rated support only. Finally, the denominator can become zero when all non-missing entries of a (centered) vector are constant, so those pairs must be skipped or regularized.



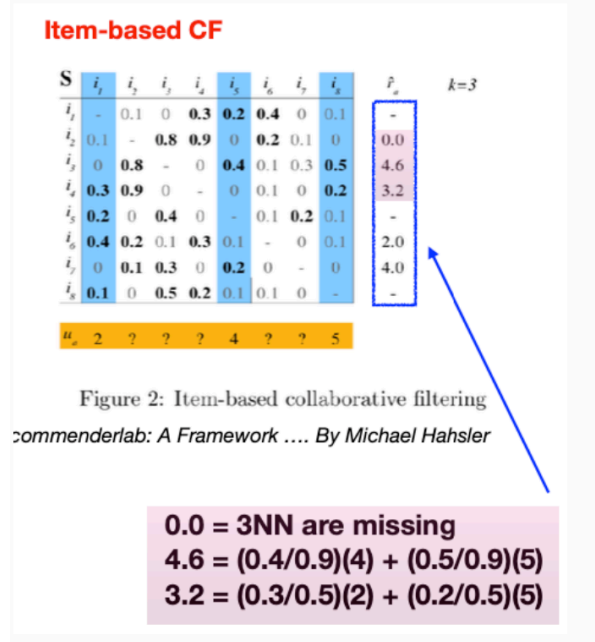
In the illustration above we used simple averages. We can extend this to **weighted averages**, using weights based on proximity to the user of interest. Let $N(a)$ be the set of k nearest neighbors of user a . The predicted rating of user a for item i is

$$\hat{r}_{ai} = \frac{\sum_{u \in N(a): r_{ui} \neq ?} s_{au} r_{ui}}{\sum_{u \in N(a): r_{ui} \neq ?} s_{au}}.$$

The prediction is undefined if no neighbor of a has rated item i . In practice, to reduce that problem, we select the nearest neighbors *among users who have rated the item in question*. The neighborhood of a may therefore change with i . A mean-centered variant replaces r_{ui} by $r_{ui} - \bar{r}_u$ and then adds $\bar{r}_a$ back after the weighted average.

Item-Based Collaborative Filtering

In item-based collaborative filtering, we consider similarities between items instead of users. Below, a similarity matrix for items is constructed.



The top three entries in each row are highlighted in bold, since we consider only 3-nearest neighbors. For a test user, represented in orange, who has provided ratings for items 1, 5, and 8, these respective columns are highlighted in blue. These are the items the test user has rated, and their ratings will serve as the foundation for our predictions.

For the test user in question, the aim is to predict the ratings for the unreviewed items using the known ratings for items 1, 5, and 8.

Let $S(i)$ be the set of k nearest items to item i (in the figure, $k = 3$). The **predicted rating** $\hat{r}_{ai}$ for item i from user a is a weighted average of a 's ratings on those neighbors that a has actually rated, with weights given by item-item similarities s_{ij} :

$$\hat{r}_{ai} = \frac{\sum_{j \in S(i): r_{aj} \neq ?} s_{ij} r_{aj}}{\sum_{j \in S(i): r_{aj} \neq ?} s_{ij}}.$$

The prediction is missing if the set $\{j \in S(i) : r_{aj} \neq ?\}$ is empty, i.e., when none of the similar items have been rated by the user. As in the user-based case, we can choose the neighbors of i dynamically among items that user a has rated, so that the neighborhood used for $\hat{r}_{ai}$ can depend on a .

Pros and Cons of Collaborative Filtering

Collaborative filtering does not rely on pre-defined item features: user interactions *become* the features, so the system can recommend items outside the user's usual content profile. It cannot start from day one, because it needs a build-up of user-item interaction data. New items with no interaction history cannot be recommended; this is the **cold start** problem (the same issue appears for brand-new users). There is also a risk of popularity bias, in which items that many people have already rated are recommended to everyone and personalization is weakened.

Item-Based vs User-Based Collaborative Filtering

The choice between user-based and item-based collaborative filtering often depends on the specific context and platform.

For instance, e-commerce sites and streaming services such as Amazon and Netflix have often found item-based CF more effective. Item similarities tend to be more stable over time than user similarities, and “users who liked this also liked that” is a more reliable relation than the less predictable notion of similar people.

Neither algorithm is cheap if one identifies nearest neighbors by a brute-force scan of a large catalog. Approximation algorithms, clustering, and efficient data structures are used to keep the cost manageable.

Item-based CF also has a *computational* advantage in production. Both methods must find nearest neighbors, but item-based CF can push most of the work into an offline training phase: one builds and sorts an item-to-item similarity matrix once, then serves predictions from it. User-based CF has no comparable static training phase; similarities between users are computed at prediction time, which is expensive in real time.

The tradeoff is memory. The precomputed item-item matrix must be stored and accessed efficiently when recommendations are generated. If the catalog is huge one typically keeps only the top- k similarities per item.

11.3 Baseline Predictors

Before we go to latent factors, it is useful to look at a much simpler model. Some users rate everything highly, some items are universally liked, and there is an overall average level of ratings. So a natural first prediction for a missing entry is not a neighbor or a latent factor, but a *baseline*.

Start with the global average of all observed ratings, μ . Then allow each user their own offset a_i and each item its own offset b_j :

$$\hat{R}_{ij} = \mu + a_i + b_j.$$

This is just a regression on dummy variables for users and items, fit only on the observed entries. We can think of a ladder of baselines:

- predict everything by the global mean μ ,
- then by a user mean $\mu + a_i$,
- then by an item mean $\mu + b_j$,
- then by $\mu + a_i + b_j$.

Already this removes a lot of the obvious structure in the rating matrix. The residual

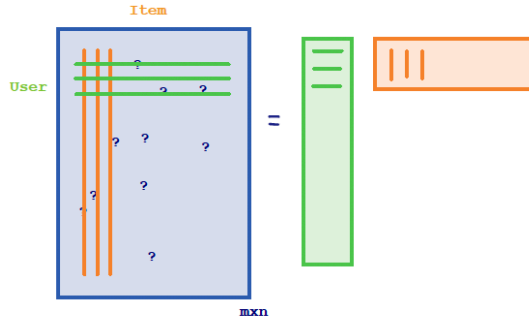
$$\tilde{R}_{ij} = R_{ij} - \mu - a_i - b_j$$

is what neighborhood methods or latent factors should be asked to explain. In that sense, collaborative filtering and matrix factorization are models for the leftover, not for the raw stars.

We usually regularize a_i and b_j as well, especially for users or items with very few ratings — the same shrinkage idea as in penalized regression. A user who has rated only one movie should not get a huge a_i .

11.4 Latent Factor Model

The latent factor model is another way to work with the rating matrix. The idea is inspired by a truncated singular value decomposition (SVD): we approximate the $m \times n$ rating matrix by a product of two thinner factor matrices that embed users and items in a shared d -dimensional space.



The decomposition is

$$R_{m \times n} \approx U_{m \times d} V_{d \times n}^T,$$

where V is $n \times d$. The d columns of U are user factors and the d columns of V (equivalently, the d rows of V^T) are item factors. A user's predicted preference for an item is the inner product of the corresponding factor vectors. In a full

SVD one would also have a diagonal matrix of singular values; here those scales are absorbed into U and V .

Classical SVD cannot be applied directly, because R has missing entries. Instead we minimize a quadratic loss *only over observed ratings*. If every entry were observed, that minimizer would recover (a truncation of) the ordinary SVD. To avoid overfitting we add penalty terms on the factors. The resulting objective is

$$\sum_{R_{ij} \neq ?} (R_{ij} - u_i^\top v_j)^2 + \lambda_1 \text{Pen}(U) + \lambda_2 \text{Pen}(V),$$

where u_i , the i -th row of U , is the latent vector for user i , and v_j , the j -th row of V , is the latent vector for item j . Typical penalties are squared Frobenius norms, $\text{Pen}(U) = \|U\|_F^2$ and $\text{Pen}(V) = \|V\|_F^2$.

This loss is convex in U for fixed V and convex in V for fixed U , but it is **not jointly convex** in (U, V) . In practice one therefore uses alternating least squares (ALS), cycling between the two blocks, or stochastic gradient descent over observed ratings. After the factors have been fit, a missing entry is predicted by the inner product $u_i^\top v_j$.

Taking this model further, we put the baseline from the previous section back in. The latent product is then asked only to explain what is left after $\mu + a_i + b_j$:

$$R_{ij} = \underbrace{\mu}_{\text{Overall Average}} + \underbrace{a_i}_{\text{User Effect}} + \underbrace{b_j}_{\text{Item Effect}} + u_i^\top v_j.$$

Substituting that into the loss, and regularizing a and b as well, gives

$$\sum_{R_{ij} \neq ?} (R_{ij} - \mu - a_i - b_j - u_i^\top v_j)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2 + \|a\|_2^2 + \|b\|_2^2).$$

The same ALS or SGD procedures apply. In practice the baseline alone already does a lot; the factors are there to capture the leftover user–item interactions.

11.5 Evaluating Recommender Systems

Evaluation is often treated as a prediction task, but the way we *split* the data matters as much as the metric.

The unit of splitting is a *rating*, not a row of a table. If we randomly shuffle users the way we did in classification, we can leak a user’s later ratings into the training set and then “predict” their earlier ones. A standard protocol is:

- hold out some observed entries, at random, or better, the most recent ratings of each user,

- fit the model on the remaining observed entries only,
- score only the held-out cells.

RMSE (and MAE) make sense when the system is really asked to predict a number of stars. RMSE was the official score of the Netflix Prize, and it is still the default number people report. It has drawbacks. In many cases we care more about higher ratings (such as 4 or 5) that signal strong interest, and about what appears at the *top of a list*. Ranking metrics such as Precision@ K , Recall@ K , or NDCG@ K are usually closer to how recommendations are actually shown.

One more warning: when we compute Precision@ K we have to decide what counts as a “relevant” item among the ones the user never saw. Held-out high ratings are the usual proxy. That proxy is imperfect, which is one reason live experiments still matter once a system is in production.

11.6 Challenges & Strategies

1. **Computational Efficiency:** To find similar items or users efficiently, a pre-clustering step is often used. By grouping users into clusters beforehand, the search for the most similar user can be restricted to the same cluster, which reduces computational overhead.
2. **Combining Multiple Systems:** In practice, recommender systems often combine several approaches. For instance, Amazon may showcase the week’s top deals, popular products within specific categories, and items you have previously shown interest in. This hybrid strategy accommodates different user preferences and different data constraints (including cold start).
3. **Contextual Information:** Modern recommendation systems also take contextual factors into account—a user’s location, time of access, and device (e.g., phone, computer, or tablet). The browsing context often changes what a good recommendation is.
4. **Evaluating Recommender Systems:** See the previous section. The short version: split by held-out *ratings* (ideally the most recent ones), report RMSE if you are predicting stars, and report a ranking metric such as Precision@ K if you are producing a list.
5. **Balance between accuracy and diversity:** Accuracy matters, but so does diversity. If every recommendation is a near-duplicate of something the user already likes, the list is precise and still unhelpful. Serendipity—helping users discover unexpected items they end up enjoying—is part of what makes a system feel useful.
6. **Incorporating User Feedback:** It is important to distinguish a user’s lack of interest in an item from mere unawareness of it. Tracking clicks (and skips) on recommended items provides feedback that ratings alone

do not.

7. **Challenges in Recommendation Systems:** Scalability and data sparsity remain the main engineering constraints. The cold-start problem—how to recommend for a new user, or how to promote a newly added item—has no single universal solution; it is usually handled by mixing content-based signals, popularity baselines, and the collaborative or latent-factor models of this chapter.

Chapter 12

Clustering Analysis

So far, all the methods we discussed were presented in a supervised learning framework. However, in our discussion of clustering analysis we will focus on **unsupervised learning problems**.

In the sequel, in our framework we no longer have a response variable Y , but we only have access to the features $\{x_i\}_{i=1}^n$, and our goal is to learn patterns in $\mathbf{X}$.

Some examples of unsupervised learning include estimation of the density or the covariance or the network structure using a set of features, which can become very challenging in high-dimensional settings. Clustering analysis is also primarily used in an unsupervised learning framework.

Clustering analysis is the identification of multiple regions of the feature space that contain modes of density, or in other words, regions of the feature space with some *common similarities*. In some cases, we are also interested in dimension reduction, in which case we want to identify low-dimensional manifolds within the feature space $\mathcal{X}$ that represent high data density.

Clustering

When we cluster, we group the data set into subsets so that those within each subset are more closely related (*similar*) to each other than those objects assigned to other subsets. Each subset is called a **cluster**. We have two main types of clustering: *Flat Clustering* and *Hierarchical Clustering*. Flat Clustering divides the data into k clusters, while hierarchical clustering arranges the clusters into a natural hierarchy.

Clustering results are dependent on the **measure of similarity** (or distance) between the “points” to be clustered. A distance metric or a distance function is a function that defines the dissimilarity of two elements (points, sets, etc.)

Mathematically, a distance function is non-negative; it is zero when two points are the same; it is symmetric and it satisfies the triangle inequality. The most popular examples of distances we will use are:

- The **Euclidean Distance** (appropriate for continuous variables):

$$d(u, v) = \|u - v\|_2 = \sqrt{\sum_{j=1}^p (u_j - v_j)^2}$$

The *squared* Euclidean distance $\|u - v\|_2^2$ is not a metric (it fails the triangle inequality), but it is the quantity minimized by K -means. The two distances yield the same nearest-center assignments.

- The **Hamming Distance** (suited for categorical variables):

$$d(u, v) = \sum_{j=1}^p \mathbf{1}_{u_j \neq v_j}$$

In general, there is no universally best approach, so distance measures should be defined based on the application.

The Python code for this chapter is available [here](#) Python Code for Clustering . It reproduces the main R examples (kmeans, hclust, and the spectral clustering demo).

12.1 Introduction to Clustering

Consider a set of n data points. The **goal** is to form $K \ll n$ clusters, indexed by $k \in \{1, \dots, K\}$. This implies that we need to search for a function $C : \{1, \dots, n\} \rightarrow \{1, \dots, K\}$ that minimizes the within cluster distance, i.e.

$$W(C) = \frac{1}{2} \sum_{k=1}^K \sum_{C(i)=C(i')=k} d(x_i, x_{i'})$$

where $C(\cdot)$ is a cluster index function that assigns the i th observation to cluster $C(i)$.

This is equivalent to maximizing the between cluster distance:

$$B(C) = \frac{1}{2} \sum_{k=1}^K \sum_{C(i)=k} \sum_{C(i') \neq k} d_{ii'}$$

Note that the **total distance** can be broken down into

$$\begin{aligned}
 T &= \frac{1}{2} \sum_{i=1}^n \sum_{i'=1}^n d_{ii'} \\
 &= \frac{1}{2} \sum_{k=1}^K \sum_{C(i)=k} \left\{ \sum_{C(i')=k} d_{ii'} + \sum_{C(i') \neq k} d_{ii'} \right\} \\
 &= W(C) + B(C)
 \end{aligned}$$

Since, the total distance is fixed for a given set of data, we have that

$$\text{Minimize } W(C) \Leftrightarrow \text{Maximize } B(C)$$

Given a specific distance measure $d(\cdot, \cdot)$, several algorithms can be used to find the clusters:

- i. Combinatorial Algorithms
- ii. k -means Clustering
- iii. Hierarchical Clustering

Combinatorial Algorithms

For small n and K , we could minimize W by using brute-force search. However, the number of “tries” needed to complete the search is

$$S(n, K) = \frac{1}{K!} \sum_{k=0}^K (-1)^{K-k} \binom{K}{k} k^n$$

(the Stirling number of the second kind; for $n > 0$ the $k = 0$ term vanishes). This counts the number of ways to partition n observations into K nonempty unlabeled clusters.

It is clear that this is not feasible for large n and K , since the number of distinct assignments can be extremely large. So, we need to develop more efficient algorithms that may not be optimal but they could be a reasonably good sub-optimal partition.

12.2 *K*-means Clustering

Consider an enlarged optimization problem

$$\min_{C, \{m_k\}_{k=1}^K} \sum_{k=1}^K \sum_{C(i)=k} \|x_i - m_k\|^2$$

We solve this with respect to both: *the cluster function* $C(\cdot)$ and the *cluster centers* m_k , $k = 1, \dots, K$. But, this problem is NP-hard for ≥ 2 dimensions and a combinatorial algorithm is too expensive.

We could instead consider an algorithm that alternatively updates the two components:

- C the cluster assignments
- $\{m_k\}_{k=1}^K$ the cluster means

So, we can achieve this by first fixing C , and then finding the best $\{m_k\}_{k=1}^K$, and second by fixing $\{m_k\}_{k=1}^K$, and then finding the best C . Specifically, in K -means Clustering, we need to alternate between the following two steps until convergence:

1. Fix C , i.e. the cluster label of each subject is known. Then, for any set $\{i : C(i) = k\}$, we only need to find the mean

$$m_k = \arg \min_m \sum_{C(i)=k} \|x_i - m\|^2$$

which boils down to simply finding the mean within cluster k , i.e.

$$m_k = \frac{\sum_{C(i)=k} x_i}{\sum_i \mathbf{1}_{\{C(i)=k\}}}$$

2. Fix the cluster means $\{m_k\}_{k=1}^K$. Then, to find the new cluster assignments, we simply recalculate the distance from an observation to each of the cluster means:

$$C(i) = \arg \min_{k \in \{1, \dots, K\}} d(x_i, m_k)$$

Hence each point will be assigned to the closest cluster mean.

The K -means Clustering Algorithm can be summarized as follows:

K -means Clustering Algorithm

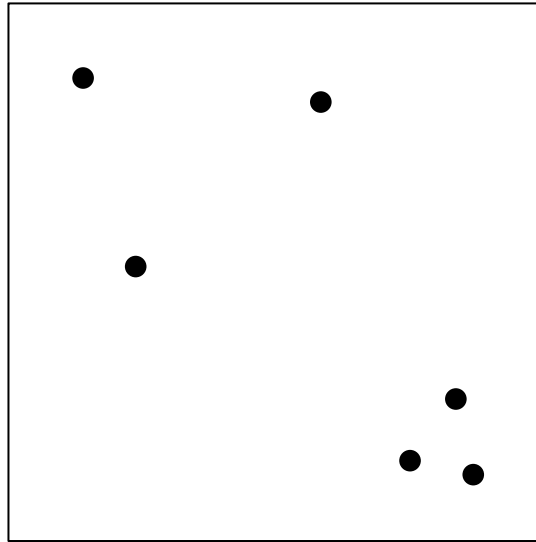
1. Randomly split the data set into K different subsets. Assign each subset a cluster label. Then iterate between (2) and (3):
2. Given the cluster assignment C , calculate the cluster mean vectors $m_1, \dots, m_K$.
3. Given the current set of means $\{m_1, \dots, m_K\}$, assign each observation to the closest current cluster mean.

4. Stop the algorithm when C does not change.

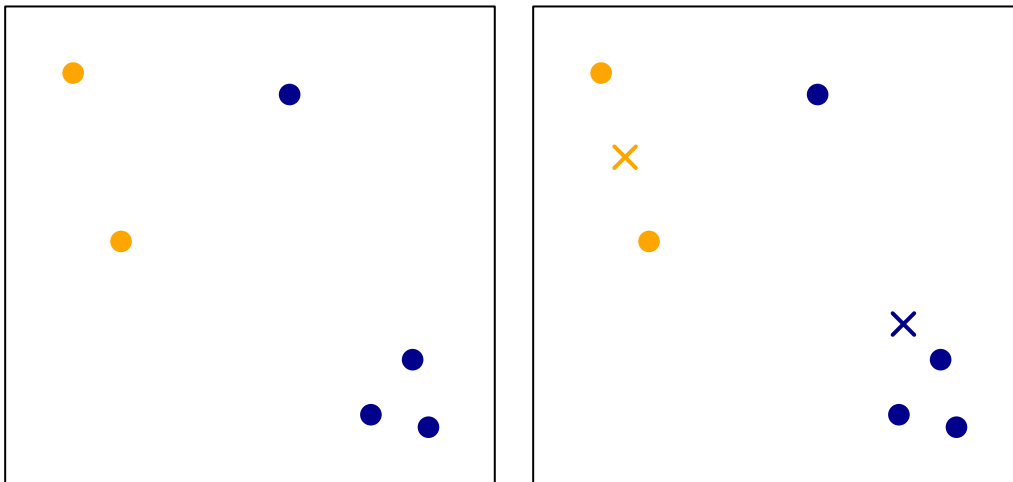
Below is an illustration of the way the algorithm works in a toy example

Illustration of *K*-Means Clustering

Suppose we have a set of *six* observations.

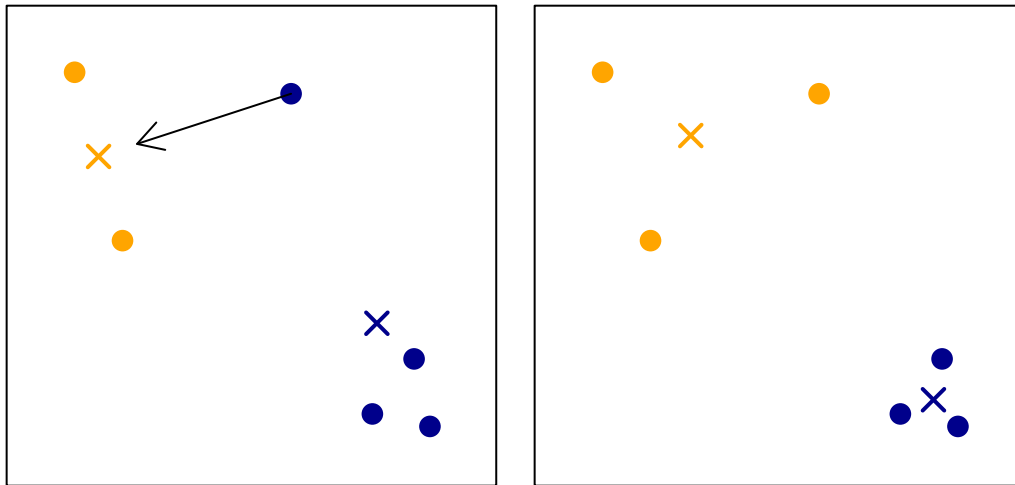


We first *randomly assign them into two clusters* (initiate a random C function). Based on this cluster assignment, we can calculate the corresponding cluster mean m_k 's:



Then we will assign each observation to the closest cluster mean. In this exam-

ple, only the blue point on the top will be moved to a new cluster. Then the cluster means can then be recalculated.



When there is nothing to move anymore, the algorithm stops. Keep in mind that we started with a random cluster assignment, and this objective function is not convex which implies that we may obtain different results if we start with different values. The solution is to try different starting points and use the best final results. This can be tuned using the `nstart` parameter in the `kmeans()` function.

```
# some random data
set.seed(1)
mat = matrix(rnorm(1000), 50, 20)

# if we use only one starting point
kmeans(mat, centers = 3, nstart = 1)$tot.withinss
```

```
## [1] 885.8913
```

```
# if we use multiple starting points and pick the best one
kmeans(mat, centers = 3, nstart = 100)$tot.withinss
```

```
## [1] 883.8241
```

Note that the clusters are typically **initialized randomly**. However, this algorithm does not guarantee a global minimum, although it still leads to a local minimizer.

Two modeling assumptions are easy to miss and matter in practice. First, *K*-means with Euclidean (or squared Euclidean) distance treats every coordinate as comparable, so features should be put on a common scale before clustering; otherwise a variable measured in thousands will dominate one measured in

fractions. Second, minimizing $\sum \|x_i - m_k\|^2$ favors compact, roughly spherical clusters of similar size. Elongated or unequal-variance groups will be split or merged for reasons that have nothing to do with “natural” labels.

The same squared-error criterion will reappear in the next chapter: K -means is a hard-assignment, equal-covariance version of EM for a Gaussian mixture. That is the hinge between geometric clustering and model-based clustering.

An alternative approach is the K -**medoids** method. We need to replace the **first step** (updating the centers) by searching for *the one* observation that minimizes the distance to all others in the cluster, i.e.

$$i_k^* = \arg \min_{i: C(i)=k} \sum_{C(i')=k} d(x_i, x_{i'})$$

Then we use $x_{i_k^*}$ as the “center” of cluster k . The assignment step is unchanged: each point is assigned to the closest medoid.

12.2.1 Categorical Variables in Clustering

By design, the k -means method cannot handle categorical variables as is unless we change them to numerical, e.g., using one-hot encoding, or ordinal encoding. Alternatively, we may also use distance metrics that are specifically designed for categorical data, e.g., Hamming distance, or the Jaccard distance.

We can also choose clustering algorithms that support categorical variables, such as K -Modes or K -prototypes.

12.2.2 Tuning K -Means: How to Choose K

When using K -Means, we need to identify an appropriate value for the number of clusters, k . Some of the available methods, include:

- Gap Statistics
- Silhouette Statistics
- Prediction Strength

When identifying the appropriate number of clusters, we need to consider a *measure of compactness/tightness* of clusters. A measure that quantifies the **compactness** or **tightness** of clusters can be computed based on

$$SS(K) = \sum_{k=1}^K \sum_{C(i)=k} \|x_i - m_k\|^2$$

A smaller SS indicates tighter clusters. As the number of clusters (K) increases, the SS inherently decreases for the same data set which means that relying only on SS can be misleading when selecting the optimal K .

Gap Statistic

The **gap statistic** (Tibshirani, Walther and Hastie, 2001) compares the clustering of actual data against a *random clustering from a reference distribution*.

$$\begin{aligned} G(K) &= E_0 \left(\log SS^*(K) \right) - \log SS_{obs}(K) \\ &\approx \frac{1}{B} \sum_{b=1}^B \log SS_b^*(K) - \log SS_{obs}(K) \end{aligned}$$

This reference set is derived from a distribution that has *no intrinsic clustering*, meaning an ideal number of clusters would be one. As K grows, even though the SS shrinks, the difference (or gap) may not always decrease. A high gap statistic suggests that the SS for the observed data at a particular K is notably smaller than its reference counterpart, indicating good clustering.

There are two proposed methods for generating data from the reference distribution:

- **Uniform Sampling:** The reference data is uniformly sampled over the range of the observed data. This method may not be effective if the observed data has distinct shapes.
- **Principal Component Based Sampling:** This method samples over the range of the principal components of the observed data, ensuring better alignment with the data structure.

So, to determine the **optimal** K with the **gap statistic**, we plot the gap statistic values for different values of K . The optimal K is determined either by identifying the highest gap statistic or, in a sequential approach, by selecting the first K where its gap statistic exceeds that of $K + 1$.

Since the gap statistic is based on random sampling, there is inherent variability: the one-standard-error principle is used to account for this uncertainty. We compare the gap statistic at K to the lower bound of the gap statistic for $K + 1$ (subtracting one standard error). If the former is greater, we consider that K as optimal:

$$K_{opt} = \arg \min_K \{ K : G(K) \geq G(K + 1) - s_{K+1} \}$$

where

$$s_K = \text{sd}_K \sqrt{1 + 1/B}$$

and sd_K is the standard deviation of $\log SS_b^*(K)$ over the B reference samples.

Silhouette Statistic

The **silhouette statistic** of the i th observation measures how well this observation fits in its own cluster versus how well it fits in its next closest cluster.

In the original definition (Rousseeuw, 1987), $a(i)$ is the average distance from i to the other points in its own cluster, and $b(i)$ is the smallest average distance from i to the points of any other cluster. A common adaptation for K -means uses distances to centroids instead:

$$a(i) = \|x_i - m_k\|, \quad b(i) = \|x_i - m_\ell\|$$

where $i \in C_k$ and C_ℓ is the next-closest cluster to x_i . Then, its silhouette is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

For well-clustered observations where $a(i)$ is significantly smaller than $b(i)$, $s(i)$ approaches 1. If the i th observation lies on the borderline of two clusters, both $a(i)$ and $b(i)$ will be similar, rendering $s(i)$ close to 0.

In the K -means scenario with centroid distances, $b(i)$ should be greater than $a(i)$; otherwise, the observation would belong to a different cluster. With the pairwise-average definition, or with other clustering algorithms, $b(i)$ can be smaller than $a(i)$, potentially resulting in a negative silhouette statistic. Even then, the value of $s(i)$ remains above -1 . Extremely negative silhouette statistics indicate poor clustering for the i th observation.

Silhouette Coefficient

The **Silhouette Coefficient (SC)**, also referred to as the Silhouette Width, is defined as the average silhouette statistic over all samples.

A rule of thumb taken from Kaufman and Rousseeuw (1990); see also Struyf, Hubert and Rousseeuw (1997), “*Clustering in an Object-Oriented Environment*”:

- SC > 70%: A strong structure has been found.
- SC > 50%: A reasonable structure has been found.
- SC > 26%: The structure is weak and could be artificial; try additional methods.
- SC < 26%: No substantial structure has been found.

We can also plot the silhouette coefficient against varying K values. The optimal K can either be the one yielding the largest silhouette coefficient or a K surpassing a specific threshold.

Prediction Strength

The **Prediction Strength** method is another strategy for choosing the optimal number of clusters K . We start by splitting the data set into two subsets: A (training set) and B (test set).

1. Assume there are m observations in test set B . Cluster these m samples into K clusters using your chosen clustering algorithm. Label these clusters as $C_1, \dots, C_K$. Each cluster contains m_j observations. The sum of the sizes of these clusters $m_1, \dots, m_K$ is equal to m . This step determines the true clustering structure for the test data.
2. Then, cluster the training data A into K clusters. Use the resulting clustering rule to allocate the m observations from test set B into these K clusters. This essentially means predicting the cluster membership for the test set based on the cluster centers determined from the training data. Denote these predicted clusters as $C'_1, \dots, C'_K$.

For each cluster obtained in *Step 1* (the clusters found on the test set B), examine every pair of observations within that cluster. Check whether those pairs remain in the same cluster under the assignment from *Step 2*. Calculate the average co-membership accuracy for each test cluster, indicating how often pairs of observations remain together.

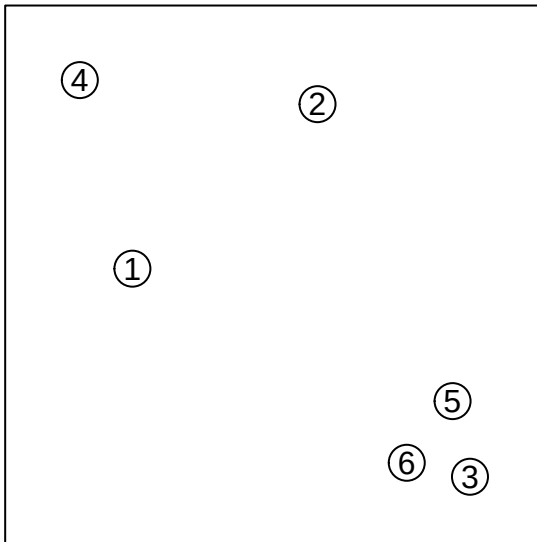
The “prediction strength” (Tibshirani and Walther 2005) for K clusters is the worst (smallest) accuracy among these clusters. Choosing the worst accuracy avoids potential biases. If one were to take the average accuracy, the method might be skewed by predominantly classifying many observations into one cluster, artificially inflating the average. A common rule is to choose the largest K whose prediction strength stays above a threshold such as 0.8 or 0.9.

12.3 Hierarchical Clustering

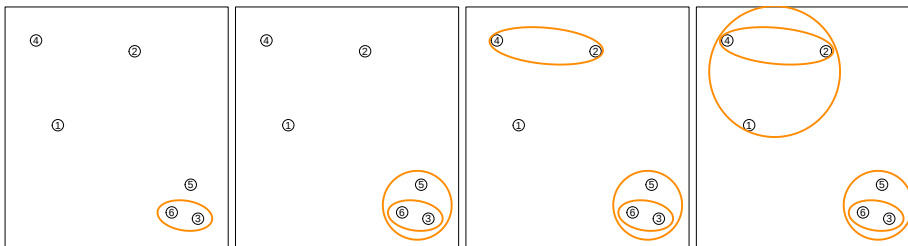
Choosing the number of clusters K may be difficult. In a **Hierarchical Representation** each cluster contains a single observation at the lowest level, and there is only one cluster containing all observations at the highest level. We then use a dendrogram to display the clustering result.

Hierarchical Clustering Example

Suppose we have a set of six observations:



The goal is to progressively group them together until there is only one group.



During this entire process, only the pair-wise distances are used to determine which observations are grouped. Hence, the R function `hclust()` will take a distance matrix and perform the clustering. A distance matrix is an $n \times n$ matrix. As a default, we can use the Euclidean distance.

```
# the Euclidean distance can be computed using dist()
as.matrix(dist(x))
```

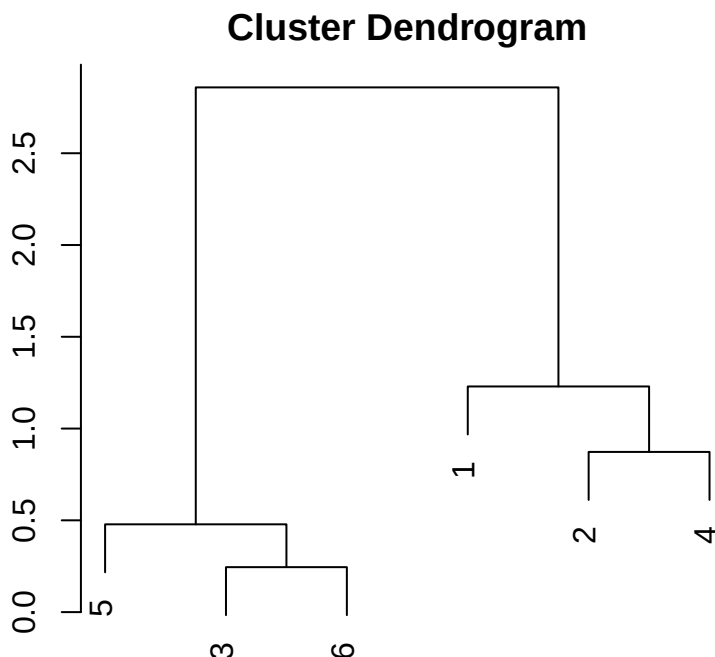
```
##           1           2           3           4           5           6
## 1 0.000000  1.2294164  1.7864196  1.1971565  1.4246185  1.5698349
## 2 1.229416  0.0000000  2.3996575  0.8727261  1.9243764  2.2708670
## 3 1.786420  2.3996575  0.0000000  2.8586738  0.4782442  0.2448835
## 4 1.197156  0.8727261  2.8586738  0.0000000  2.4219048  2.6741260
## 5 1.424618  1.9243764  0.4782442  2.4219048  0.0000000  0.4204479
## 6 1.569835  2.2708670  0.2448835  2.6741260  0.4204479  0.0000000
```

We then use this distance matrix in the hierarchical clustering algorithm.

```
# the Euclidean distance can be computed using dist()
as.matrix(dist(x))
```

```
##           1           2           3           4           5           6
## 1 0.000000 1.2294164 1.7864196 1.1971565 1.4246185 1.5698349
## 2 1.229416 0.0000000 2.3996575 0.8727261 1.9243764 2.2708670
## 3 1.786420 2.3996575 0.0000000 2.8586738 0.4782442 0.2448835
## 4 1.197156 0.8727261 2.8586738 0.0000000 2.4219048 2.6741260
## 5 1.424618 1.9243764 0.4782442 2.4219048 0.0000000 0.4204479
## 6 1.569835 2.2708670 0.2448835 2.6741260 0.4204479 0.0000000
```

```
# pass the distance matrix to hclust() we use a complete
# link function
hcf1t <- hclust(dist(x), method = "complete")
par(mar = c(2, 2, 2, 1))
plot(hcf1t)
```



Hence, the merging process is exactly the same as we demonstrated previously. The height of each split represents how separated the two subsets are. Selecting the number of clusters is still a tricky problem. Usually, we pick a cutoff where the height of the next split is short. Hence, the above example fits well with two clusters.

An **agglomerative algorithm** is a bottom-up approach:

1. Begin with every observation representing a singleton cluster.
2. At each step, merge two closest clusters into one cluster and reduce the number of clusters by one.

3. Stop when all observations are in the same cluster.

To choose which two clusters to merge, we need (i) a distance measure between any two observations $d(x_i, x_j)$, and (ii) a distance measure between any two sets of observations $d(G, H)$.

A Distance $d(G, H)$ between two clusters G and H defined as:

- **Complete Linkage:** the furthest pair

$$d(G, H) = \max_{i \in G, i' \in H} d_{ii'}$$

- **Single Linkage:** the closest pair

$$d(G, H) = \min_{i \in G, i' \in H} d_{ii'}$$

- **Average Linkage:** average dissimilarity

$$d(G, H) = \frac{1}{n_G n_H} \sum_{i \in G} \sum_{i' \in H} d_{ii'}$$

- **Ward Linkage:** merge the pair of clusters whose combination increases the within-cluster sum of squares the least. This linkage is the closest hierarchical analogue of the K -means objective.

Different choices may result in different hierarchical structures.

To perform a hierarchical clustering, a matrix of all the pair-wise distances is sufficient, and we do not need to know the values of the original observations. This is an $n \times n$ matrix: the (i, i') element represents the distance between x_i and $x_{i'}$. This matrix is also called a **dissimilarity matrix**. This is a symmetric matrix with diagonal elements that are zero.

12.4 Self-Organizing Maps

Self-organizing Map (SOM) is another popular clustering method. The main difference between SOM and K -means is that the cluster means of a SOM have a geometric relationship preserving the geometry of the underlying data. An SOM starts with **Input Data:** $\{x_i\}_{i=1}^n$, with each $x_i \in \mathbb{R}^p$. We then randomly initialize the **Cluster Means**, i.e. creating a grid of centers $w_{ij} \in \mathbb{R}^p$, $i = 1, \dots, q_1$, $j = 1, \dots, q_2$. These are similar to the centers in a K -means algorithm, but they also preserve some geometric relationship on the grid. Then, we pick a **Learning Rate:** $\alpha \in [0, 1]$. This controls how fast the w_{ij} 's are updated, and it will decrease progressively. We also need to choose the **Radius:** r . This controls how many w_{ij} 's will be updated at each iteration, and it will also decrease progressively.

SOM Algorithm

For $k = 1, \dots, n$, we will stream in one new observation x_k and perform the following update:

1. For all w_{ij} , calculate the distance between each w_{ij} and x_k . \ Let $d_{ij} = \|x_k - w_{ij}\|$ (use Euclidean distance).
2. Select the closest w_{ij} , denoted as w_* .
3. Update each w_{ij} based on the formula

$$w_{ij} \leftarrow w_{ij} + \alpha h(w_*, w_{ij}, r) (x_k - w_{ij})$$

where h is a neighborhood function on the *grid* (typically depending on the grid distance between units and the current radius r). Units close to the winner on the map are pulled toward x_k ; units outside the neighborhood are left unchanged.

4. Decrease the value of α and r .

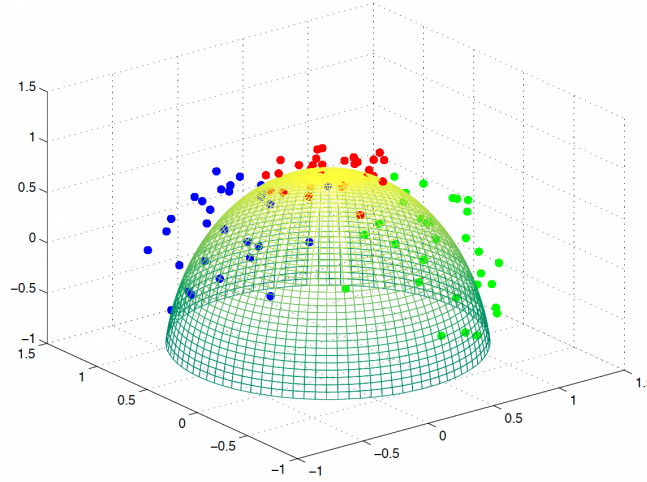


Figure 12.1: Self-Organizing Map (Ref. ESL textbook)

12.5 Spectral Clustering

In some applications, we do not have the value of each data point, instead, we have the similarities between data points. Note that for similarity measures, larger means more similar, while for distance measures, larger is further away. A way to represent the data is the **Similarity Graph** $G = (V, E)$ (an undirected graph) in which V is a set of vertices $\{x_1, x_2, \dots, x_n\}$, and E is the set of edges $\{(i, j)\}_{ij}$.

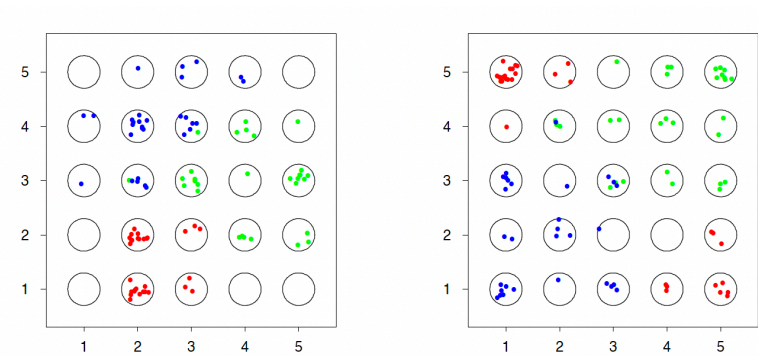


Figure 12.2: Self-Organizing Map (Ref. ESL textbook)

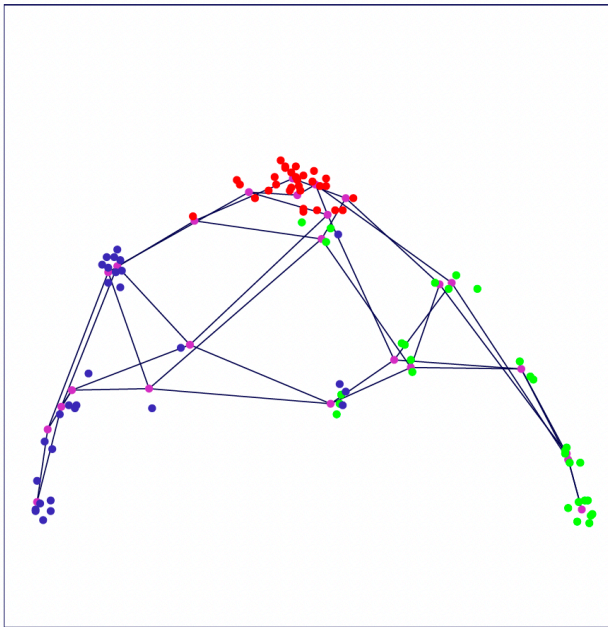


Figure 12.3: Self-Organizing Map (Ref. ESL textbook)

For our case, this graph is weighted by an **adjacency matrix**

$$\mathbf{W}_{n \times n} = \{w_{ij}\}_{ij}$$

We can define $\mathbf{W}$ in many different ways. Each w_{ij} is the *similarity* between vertices i and j which means that $\mathbf{W}$ is symmetric $w_{ij} = w_{ji}$.

We also define the **degree matrix** $\mathbf{D}$ as a *diagonal* matrix

$$\text{diag}(d_1, \dots, d_n)$$

where d_i is the degree of vertex i :

$$d_i = \sum_{j=1}^n w_{ij}$$

Graph Laplacian

There are many different ways to define a graph Laplacian matrix:

- Unnormalized graph Laplacian:

$$\mathbf{L} = \mathbf{D} - \mathbf{W}$$

- Normalized graph Laplacians:

$$\begin{aligned}\mathbf{L}_{sym} &= \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{W} \mathbf{D}^{-1/2} \\ \mathbf{L}_{rw} &= \mathbf{D}^{-1} \mathbf{L} = \mathbf{I} - \mathbf{D}^{-1} \mathbf{W}\end{aligned}$$

Each of them has some unique properties: For the unnormalized graph Laplacian, we have, for any f

$$\begin{aligned}f' \mathbf{L} f &= f' \mathbf{D} f - f' \mathbf{W} f = \sum_{i=1}^n d_i f_i^2 - \sum_{i,j} f_i f_j w_{ij} \\ &= \frac{1}{2} \left\{ \sum_i d_i f_i^2 - 2 \sum_{i,j} f_i f_j w_{ij} + \sum_j d_j f_j^2 \right\} \\ &= \frac{1}{2} \sum_{i,j} w_{ij} (f_i - f_j)^2\end{aligned}$$

L here is positive semi-definite. The smallest eigen-value is 0, with eigen-vector 1. The number of 0 eigen-values depends on the number of connected components.

For the normalized graph Laplacian, we have

$$f' \mathbf{L}_{sym} f = \frac{1}{2} \sum_{i,j} w_{ij} \left(\frac{f_i}{\sqrt{d_i}} - \frac{f_j}{\sqrt{d_j}} \right)^2$$

The eigenvalue λ and eigenvector $\mathbf{v}$ of $\mathbf{L}_{rw}$ can be solved from

$$\mathbf{L}\mathbf{v} = \lambda \mathbf{D}\mathbf{v}$$

The smallest eigenvalue of both $\mathbf{L}_{sym}$ and $\mathbf{L}_{rw}$ are 0, with eigenvectors $\mathbf{1}$ and $\mathbf{D}^{1/2}\mathbf{1}$ respectively.

The spectral clustering is very simple:

Clustering Algorithm

1. Construct a weighted adjacency matrix $\mathbf{W}$, using e.g.,

$$w_{ij} = \exp\left(-\frac{1}{2\sigma^2} \|x_i - x_j\|^2\right)$$

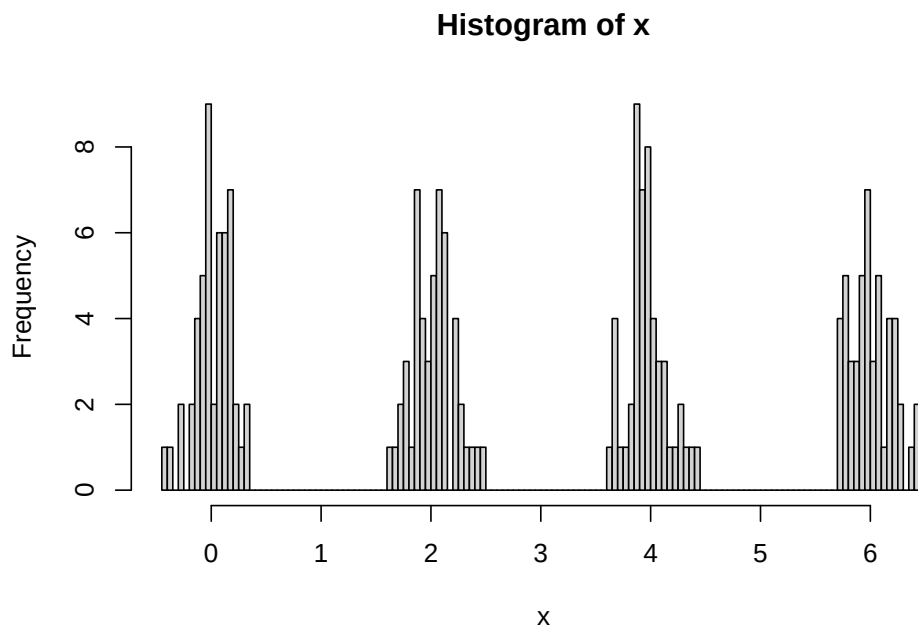
2. Compute the Laplacian $\mathbf{L}$ (or normalized versions $\mathbf{L}_{sym}$ and $\mathbf{L}_{rw}$)
3. Compute the smallest k eigenvectors, denote as $\mathbf{V}_{n \times k}$. (For $\mathbf{L}_{sym}$, a common extra step is to row-normalize $\mathbf{V}$ before clustering.)
4. Treat $\mathbf{V}_{n \times k}$ as the matrix of the observed data, and perform k -means clustering.
5. Output the k cluster labels

Below is an example from Von Luxburg, Ulrike. “A tutorial on spectral clustering.” *Statistics and computing* 17, no. 4 (2007): 395-416.

Spectral Clustering Example

Spectral clustering essentially consists of two steps. First, we construct the graph Laplacian $\mathbf{L}$ (or normalized version), then we perform eigen-decomposition of the matrix. Let's show an example, replicated from Von Luxburg (2007).

```
set.seed(1)
n = 50
x = c(rnorm(n, 0, 0.2), rnorm(n, 2, 0.2), rnorm(n, 4, 0.2), rnorm(n,
  6, 0.2))
hist(x, breaks = 100)
```



We use the adjacency matrix defined as

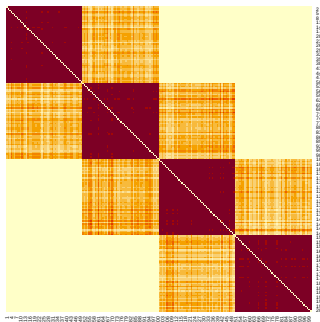
$$w_{ij} = \exp \left\{ \frac{-\|x_i - x_j\|^2}{2\sigma^2} \right\},$$

and calculate the Laplacian

$$\mathbf{L} = \mathbf{D} - \mathbf{W}.$$

We can then use the eigen decomposition to recover the underlying features.

```
# construct the adjacency matrix
W = as.matrix(exp(-dist(as.matrix(x))^2/4))
heatmap(W, Rowv = NA, Colv = NA, symm = TRUE, revC = TRUE)
```

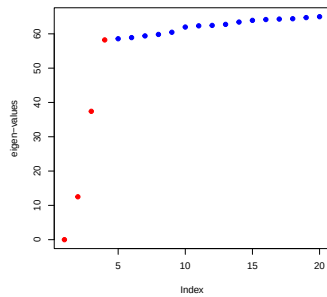


```
# compute the degree of each vertex
d = colSums(W)
```

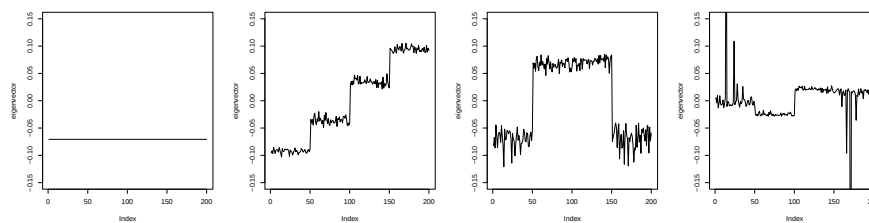
```
# the laplacian matrix
L = diag(d) - W

# eigen decomposition
f = eigen(L, symmetric = TRUE)

# plot the eigen values we need the smallest ones
plot(rev(f$values)[1:20], pch = 19, ylab = "eigen-values", col = c(rep("red",
4), rep("blue", 196)))
```



```
# plot the last four eigen vectors
par(mfrow = c(1, 4))
plot(f$vectors[, 200], type = "l", ylab = "eigenvector", ylim = c(-0.15,
0.15))
plot(f$vectors[, 199], type = "l", ylab = "eigenvector", ylim = c(-0.15,
0.15))
plot(f$vectors[, 198], type = "l", ylab = "eigenvector", ylim = c(-0.15,
0.15))
plot(f$vectors[, 197], type = "l", ylab = "eigenvector", ylim = c(-0.15,
0.15))
```



On the other hand, if we use an adjacency matrix that contains several non-connected blocks, then we would observe four zero eigen values. For example, using the KNN adjacency index, we may obtain four separated blocks.

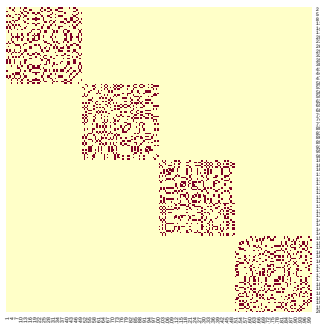
```
library(FNN)
```

```
##
## Attaching package: 'FNN'
```

```
## The following objects are masked from 'package:class':
##
##      knn, knn.cv
nn = get.knn(x, k = 10)
W = matrix(0, 200, 200)
for (i in 1:200) W[i, nn$nn.index[i, ]] = 1

# W is not necessarily symmetric
W = pmax(W, t(W))

heatmap(W, Rowv = NA, Colv = NA, symm = TRUE, revC = TRUE)
```



We use a normalized graph Laplacian, defined as

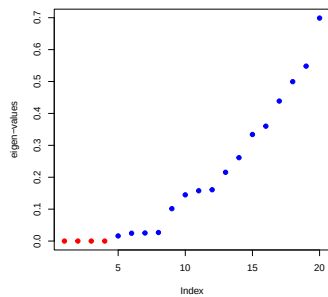
$$\mathbf{L}_{\text{sym}} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{W} \mathbf{D}^{-1/2}$$

```
# compute the degree of each vertex
d = colSums(W)

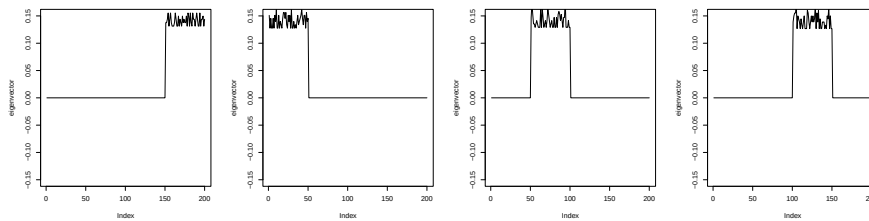
# the laplacian matrix
L = diag(200) - diag(1/sqrt(d)) %*% W %*% diag(1/sqrt(d))

# eigen decomposition
f = eigen(L, symmetric = TRUE)

# plot the eigen values we need the smallest ones
plot(rev(f$values)[1:20], pch = 19, ylab = "eigen-values", col = c(rep("red",
4), rep("blue", 196)))
```

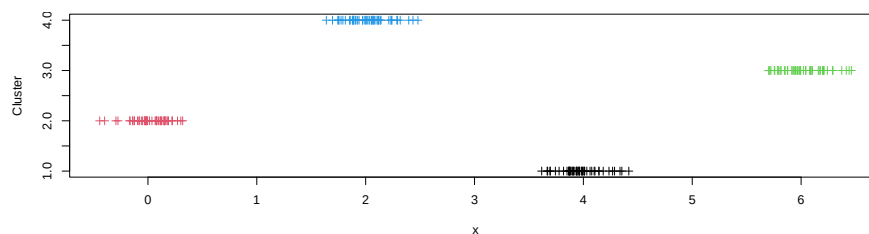


```
# plot the last four eigen vectors
par(mfrow = c(1, 4))
plot(f$vectors[, 200], type = "l", ylab = "eigenvector", ylim = c(-0.15,
0.15))
plot(f$vectors[, 199], type = "l", ylab = "eigenvector", ylim = c(-0.15,
0.15))
plot(f$vectors[, 198], type = "l", ylab = "eigenvector", ylim = c(-0.15,
0.15))
plot(f$vectors[, 197], type = "l", ylab = "eigenvector", ylim = c(-0.15,
0.15))
```



We can then perform k -means clustering using the top eigen vectors as the features.

```
cl = kmeans(f$vectors[, 197:200], centers = 4, nstart = 100)
plot(x, cl$cluster, ylab = "Cluster", col = cl$cluster, pch = 3)
```



Chapter 13

Model-Based Clustering: Latent Structure Models

This chapter is **model-based clustering**. Last week we partitioned $\mathbf{X}$ with a distance and a criterion such as $W(C)$. Here we treat the cluster labels as missing data, put a probability model on $\mathbf{X}$, and estimate that model. The main example is a Gaussian mixture; a hidden Markov model is the same idea when the labels form a Markov chain in time.

Model-based clustering works by fitting a mixture model to the data, with each cluster represented by one component of that mixture.

In this framework, clustering becomes equivalent to estimating a mixture model that captures the underlying distribution of the variable X . Choosing the correct number of clusters, K , can be guided by model selection criteria such as AIC or BIC.

A major difficulty in model-based clustering—and in mixture modeling more broadly—is estimating the model parameters. The EM (Expectation–Maximization) algorithm is the standard tool used to obtain maximum likelihood estimates of these parameters.

Although our focus here is on clustering, mixture models belong to the wider family of hierarchical models, and their applications reach well beyond clustering alone.

13.1 Gaussian Mixtures

Consider two *populations* with means μ_1 and μ_2 and the same variance, say $\sigma^2 = 1$, and let X be the **observed outcome** (from one of the two populations),

and $Z \in \{0, 1\}$ be a **hidden variable** (not observable) that indicates the population label, with $P(Z = 1) = \pi$.

Using only the observed data, $\{x_i\}_{i=1}^n$, we want to estimate the two population means and the mixing probability $\theta = (\mu_1, \mu_2, \pi)$. The density of X is a *mixture of two Gaussian* random variables, i.e.

$$p_X(x) = (1 - \pi) \phi_{\mu_1}(x) + \pi \phi_{\mu_2}(x)$$

where ϕ_μ is the density function of $\mathcal{N}(\mu, 1)$.

The log-likelihood function based on n observed training data computes as

$$\ell(\mathbf{x}|\theta) = \sum_{i=1}^n \log \left((1 - \pi) \phi_{\mu_1}(x_i) + \pi \phi_{\mu_2}(x_i) \right)$$

We can solve this by *gradient descent*, but it is often slow. Instead, we can treat the labels from which distribution each sample comes from (call them Z) as **missing** and then we can use the EM algorithm.

13.2 EM Algorithm

Instead of directly optimizing the log-likelihood $\ell(\mathbf{x}|\theta)$, we introduce the **latent variable** Z , and write the joint *log-likelihood* as

$$\begin{aligned} \ell(\mathbf{x}, \mathbf{z}|\theta) &= \sum_{i=1}^n \left((1 - z_i) \log \phi_{\mu_1}(x_i) + z_i \log \phi_{\mu_2}(x_i) \right) \\ &\quad + \sum_{i=1}^n \left((1 - z_i) \log(1 - \pi) + z_i \log \pi \right) \end{aligned}$$

EM algorithm: We optimize this likelihood function by iteratively updating the unknowns: $\mathbf{z}$ and θ .

- *E-step (Expectation)*: we treat both $\mathbf{x}$ and (μ_1, μ_2, π) as known, and calculate the conditional probability of each z_i .
- *M-step (Maximization)*: we treat $\mathbf{x}$ and $\mathbf{z}$ as known, and solve the parameters by maximizing the likelihood.

EM Algorithm: E-Step

If both $\mathbf{x}$ and $\theta = (\mu_1, \mu_2, \pi)$ are known, then the conditional probability of each z_i can be calculated as:

$$\begin{aligned} P(Z_i = 1 | \theta^{(k)}, \mathbf{x}) &= \frac{P(Z_i = 1, x_i | \theta^{(k)})}{p(x_i | \theta^{(k)})} \\ &= \frac{\pi^{(k)} \phi_{\mu_2^{(k)}}(x_i)}{(1 - \pi^{(k)}) \phi_{\mu_1^{(k)}}(x_i) + \pi^{(k)} \phi_{\mu_2^{(k)}}(x_i)} \end{aligned}$$

This is simple since we just need to calculate the density functions of each x_i under the current parameter $\theta^{(k)}$ for each possible label ($z_i = 0$ or 1).

EM Algorithm: M-Step

We already have $P(\mathbf{Z} = \mathbf{z} | \mathbf{x}, \theta^{(k)})$, we can replace all the z_i values (since they are unknown anyways) in the likelihood function $\ell(\mathbf{x}, \mathbf{z} | \theta)$ by their expectations (from the E-step):

$$\hat{p}_i = P(Z_i = 1 | \mathbf{x}, \theta^{(k)})$$

After this, what remains in the likelihood only involves $\mathbf{x}$ and θ , so we can solve (the M-step) for the MLE of θ based on this new likelihood function.

It turns out that these estimators are just *weighted means*:

$$\hat{\mu}_1 = \frac{\sum_{i=1}^n (1 - \hat{p}_i) x_i}{\sum_{i=1}^n (1 - \hat{p}_i)}, \quad \hat{\mu}_2 = \frac{\sum_{i=1}^n \hat{p}_i x_i}{\sum_{i=1}^n \hat{p}_i}, \quad \hat{\pi} = \frac{1}{n} \sum_{i=1}^n \hat{p}_i$$

We will then *iterate the E- and M- steps until convergence*.

Connection with k -Means

Suppose in the E-step, we do not use the “*soft*” label (the probability), instead, we use a “*hard*” label:

$$\mathbf{1}_{\{P(Z_i=1|\mathbf{x},\theta^{(k)})>0.5\}}$$

This is essentially comparing the two densities and seeing which one is larger. Notice that, with equal variance, the log density of a Gaussian differs from a constant only by the squared Euclidean norm, so we are just choosing the “closer” cluster mean (of the previous iteration). The M-step is just re-calculating the cluster means based on the new assignments. In that sense, K -means is a hard-assignment, equal-covariance version of EM for a Gaussian mixture. If the component variances are allowed to differ, the hard assignment is no longer “nearest mean.”

The same EM argument extends immediately to K components in p dimensions,

$$p(x) = \sum_{k=1}^K \pi_k \phi(x; \mu_k, \Sigma_k),$$

with $\sum_k \pi_k = 1$. The E-step produces responsibilities $\hat{p}_{ik} = P(Z_i = k | x_i, \theta^{(k)})$, and the M-step updates each π_k , μ_k , and Σ_k as weighted means and weighted covariances. Constraining $\Sigma_k = \sigma^2 I$ recovers spherical clusters; allowing a common Σ or free Σ_k lets clusters be elliptical. The observed-data likelihood is then available, so the number of components can be chosen by AIC or BIC, which is the model-based counterpart of the gap and silhouette statistics from the previous chapter. In practice one often initializes EM from a K -means run.

Responsibilities, labels, and choosing K

The numbers $\hat{p}_{ik}$ are the reason to prefer a mixture model over K -means when the model is plausible. They are soft cluster memberships: $\hat{p}_{ik}$ is the estimated probability that observation i belongs to component k , given the current fit. One can report a hard label $\arg \max_k \hat{p}_{ik}$, but one can also keep the vector $(\hat{p}_{i1}, \dots, \hat{p}_{iK})$ as a measure of uncertainty. Points with two comparable $\hat{p}_{ik}$ sit on the boundary between clusters; K -means has no analogue of that statement.

Two caveats come with the likelihood. First, $\ell(\theta | \mathbf{x})$ is unchanged if we permute the component labels, so the maximizer is not unique; this is **label switching**. Second, the observed-data likelihood is not concave, so EM converges to a local maximum. Different K -means starts (or several random starts) can yield different fits; one keeps the run with the largest $\ell(\theta | \mathbf{x})$.

Once a fit is in hand, the same observed-data log-likelihood is what we use to choose K . With ℓ_K the maximized log-likelihood of a K -component mixture and d_K the number of free parameters,

$$\begin{aligned} \text{AIC}(K) &= -2\ell_K + 2d_K, \\ \text{BIC}(K) &= -2\ell_K + d_K \log n. \end{aligned}$$

For a p -dimensional Gaussian mixture with free mixing weights, means, and covariances,

$$d_K = (K - 1) + Kp + K \cdot \frac{p(p + 1)}{2}.$$

Constraints on Σ_k (spherical, diagonal, or shared across k) reduce d_K . We select the K that minimizes AIC or BIC. BIC penalizes complexity more heavily and is the more common default for mixtures. This is a different answer to “how many clusters?” than gap or silhouette: here K indexes a list of probability models, and we score those models on the observed data.

13.3 EM Algorithm: General Framework

Suppose that we want to maximize the log-likelihood $\log p(\mathbf{x}|\theta)$. Under some scenarios, this likelihood can be difficult to derive. For example, X are generated from a mixture of several models, however, we do not know which is the underlying true model for each observation. Or, X contains missing values, where we have $\mathbf{X} = (\mathbf{X}_{obs}, \mathbf{X}_{mis})$. In such cases, it would be easier if we introduced a latent variable Z , such that the joint likelihood of $p(\mathbf{x}, \mathbf{z}|\theta)$ is much easier to derive.

For example, if Z represents the hidden label, then

$$p(\mathbf{x}, \mathbf{z}|\theta) = p(\mathbf{z}|\theta) p(\mathbf{x}|\mathbf{z}, \theta)$$

where both probabilities are easier to write out given the underlying model. In general, for a discrete case of Z , we need to maximize the log-likelihood

$$\log p(\mathbf{x}|\theta) = \log \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}|\theta)$$

For a continuous case, we maximize the log-likelihood

$$\log p(\mathbf{x}|\theta) = \log \int_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}|\theta) d\mathbf{z}$$

This can still be difficult to solve since there is a summation in the log function. The **Expectation - Maximization (EM) algorithm** is designed to solve this problem (Dempster, Laird, and Rubin, 1977).

General EM Algorithm

1. **E-Step:** Under the current value of θ , denoted as $\theta^{(k)}$, find $p(\mathbf{z}|\mathbf{x}, \theta^{(k)})$, the distribution of the unobserved variables given the data and $\theta^{(k)}$. Then calculate the conditional expectation:

$$\begin{aligned} g(\theta) &= \mathbf{E}_{\mathbf{Z}|\mathbf{x}, \theta^{(k)}} \log p(\mathbf{x}, \mathbf{Z}|\theta) \\ &= \begin{cases} \sum_{\mathbf{z}} P(\mathbf{Z} = \mathbf{z}|\mathbf{x}, \theta^{(k)}) \log p(\mathbf{x}, \mathbf{z}|\theta) \\ \int_{\mathbf{z}} P(\mathbf{Z} = \mathbf{z}|\mathbf{x}, \theta^{(k)}) \log p(\mathbf{x}, \mathbf{z}|\theta) d\mathbf{z} \end{cases} \end{aligned}$$

2. **M-Step:** Re-estimate the parameter θ to maximize $g(\theta)$:

$$\theta^{(k+1)} = \arg \max_{\theta} g(\theta)$$

To summarize, three components are involved in an EM algorithm: 1. The outcome variable $\mathbf{X}$, with observed values $\mathbf{x}$. 2. The distribution parameters θ . 3. The latent variables $\mathbf{Z}$.

The EM algorithm bounces back and forth between two processes:

- E-step: Given the current parameters and the observed data, estimate the (conditional mean of) latent variables.
- M-step: Given the observed data and the latent variables, estimate the parameters.

13.4 The MM Algorithm

The **MM algorithm** (Minorize-Maximize, or Majorize-Minimize) is a general framework for optimization. The EM algorithm can be treated as a special case of MM. We will show that both MM and EM have the ascend (or descend) property.

General Philosophy

Suppose we want to maximize the function $f(\theta)$. We perform the optimization by *recursively updating* the θ value.

- At each iteration k :
 - Let $\theta^{(k)}$ represent the current parameter value.
 - We first find a function $g(\theta|\theta^{(k)})$ that *minorizes* $f(\theta)$ (lies below f and touches it at the current point):
 - For all θ , $g(\theta|\theta^{(k)}) \leq f(\theta)$ and $g(\theta^{(k)}|\theta^{(k)}) = f(\theta^{(k)})$.
 - Update θ with

$$\theta^{(k+1)} = \arg \max_{\theta} g(\theta|\theta^{(k)})$$

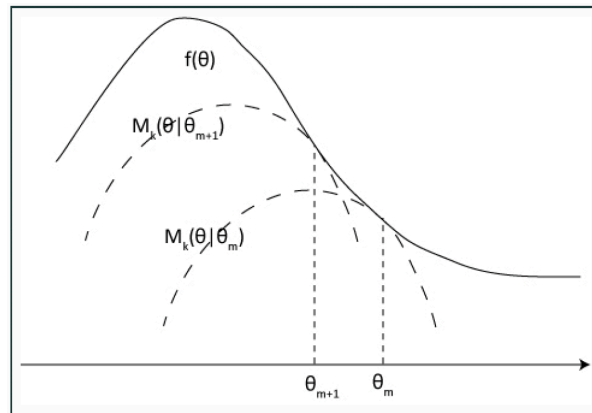


Figure 13.1: MM Algorithm (Ref. Wikipedia)

13.4.1 Ascend Property

The MM algorithm is guaranteed to ascend:

$$\begin{aligned} f(\theta^{(k+1)}) &= g(\theta^{(k+1)}|\theta^{(k)}) + \left(f(\theta^{(k+1)}) - g(\theta^{(k+1)}|\theta^{(k)}) \right) \\ &\geq g(\theta^{(k+1)}|\theta^{(k)}) \\ &\geq g(\theta^{(k)}|\theta^{(k)}) = f(\theta^{(k)}) \end{aligned}$$

If we instead minimize f using a majorizing surrogate $g \geq f$, the same argument shows that the algorithm is guaranteed to descend.

Ascend Property: EM Algorithm

The EM algorithm is a special case of MM. We can show that

$$\ell(\theta^{(k+1)}|\mathbf{x}) = \log p(\mathbf{x}|\theta^{(k+1)}) \geq \log p(\mathbf{x}|\theta^{(k)}) = \ell(\theta^{(k)}|\mathbf{x})$$

The proof relies on Jensen's inequality:

$$\begin{aligned} E(f(X)) &\geq f(E(X)), \text{ if } f \text{ is convex} \\ E(f(X)) &\leq f(E(X)), \text{ if } f \text{ is concave} \end{aligned}$$

Since $\mathbf{x}$ is observed and $\mathbf{z}$ is the missing part

$$\ell(\theta|\mathbf{x}) = \log p(\mathbf{x}|\theta) = \log \frac{p(\mathbf{x}, \mathbf{z}|\theta)}{p(\mathbf{z}|\mathbf{x}, \theta)}$$

The difference of two likelihood functions under $\theta^{(k)}$ and $\theta^{(k+1)}$

$$\begin{aligned} \ell(\theta^{(k+1)}|\mathbf{x}) - \ell(\theta^{(k)}|\mathbf{x}) &= \log p(\mathbf{x}, \mathbf{z}|\theta^{(k+1)}) - \log p(\mathbf{x}, \mathbf{z}|\theta^{(k)}) \\ &\quad - \left(\log p(\mathbf{z}|\mathbf{x}, \theta^{(k+1)}) - \log p(\mathbf{z}|\mathbf{x}, \theta^{(k)}) \right) \end{aligned}$$

This identity holds for every $\mathbf{z}$. We take the expectation of both sides with respect to $\mathbf{z}$ given $\mathbf{x}$ and $\theta^{(k)}$. The left-hand side does not depend on $\mathbf{z}$, so it is unchanged.

The right-hand side becomes

$$\begin{aligned} &\underbrace{E\left(\log p(\mathbf{x}, \mathbf{z}|\theta^{(k+1)}) - \log p(\mathbf{x}, \mathbf{z}|\theta^{(k)}) \mid \mathbf{x}, \theta^{(k)}\right)}_{g(\theta^{(k+1)}|\mathbf{x}, \theta^{(k)}) - g(\theta^{(k)}|\mathbf{x}, \theta^{(k)})} \\ &\quad - E\left(\log p(\mathbf{z}|\mathbf{x}, \theta^{(k+1)}) - \log p(\mathbf{z}|\mathbf{x}, \theta^{(k)}) \mid \mathbf{x}, \theta^{(k)}\right). \end{aligned}$$

The first term is nonnegative because $\theta^{(k+1)}$ maximizes $g(\cdot|\mathbf{x}, \theta^{(k)})$ in the M-step.

The second term is

$$E \left(\log \frac{p(\mathbf{z}|\mathbf{x}, \theta^{(k+1)})}{p(\mathbf{z}|\mathbf{x}, \theta^{(k)})} \middle| \mathbf{x}, \theta^{(k)} \right).$$

Since log is concave, Jensen's inequality gives

$$\begin{aligned} & E \left(\log \frac{p(\mathbf{z}|\mathbf{x}, \theta^{(k+1)})}{p(\mathbf{z}|\mathbf{x}, \theta^{(k)})} \middle| \mathbf{x}, \theta^{(k)} \right) \\ & \leq \log E \left(\frac{p(\mathbf{z}|\mathbf{x}, \theta^{(k+1)})}{p(\mathbf{z}|\mathbf{x}, \theta^{(k)})} \middle| \mathbf{x}, \theta^{(k)} \right) \\ & = \log \int \frac{p(\mathbf{z}|\mathbf{x}, \theta^{(k+1)})}{p(\mathbf{z}|\mathbf{x}, \theta^{(k)})} p(\mathbf{z}|\mathbf{x}, \theta^{(k)}) d\mathbf{z} \\ & = \log(1) = 0. \end{aligned}$$

(The same quantity is $-D_{\text{KL}}(p(\mathbf{z}|\mathbf{x}, \theta^{(k)}) \| p(\mathbf{z}|\mathbf{x}, \theta^{(k+1)})) \leq 0$.)

Hence the difference of observed-data log-likelihoods is a nonnegative term minus a nonpositive term, and we have the ascend property

$$\ell(\theta^{(k+1)}|\mathbf{x}) - \ell(\theta^{(k)}|\mathbf{x}) \geq 0.$$

13.5 Hidden Markov Models

A hidden Markov model is model-based clustering in time: the observations X_t are still generated from component distributions, but the latent labels Z_t are no longer independent. They form a Markov chain, so nearby time points tend to share a cluster.

We will start by discussing an example taken from Durbin et al. (1999). A *dishonest casino* uses two dice, one of them is **fair** and the other one is **biased**:

Face/Prob	"1"	"2"	"3"	"4"	"5"	"6"
Fair Die	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$
Loaded Die	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{2}$

The observer does not know which die is actually taken (the state is *hidden*), but the sequence of throws (observations) can be used to infer which die (state) was used.

Hidden Markov Model (HMM)

A **Hidden Markov Model (HMM)** for $(\mathbf{Z}, \mathbf{X}) = (Z_1, \dots, Z_n, X_1, \dots, X_n)$ consists of

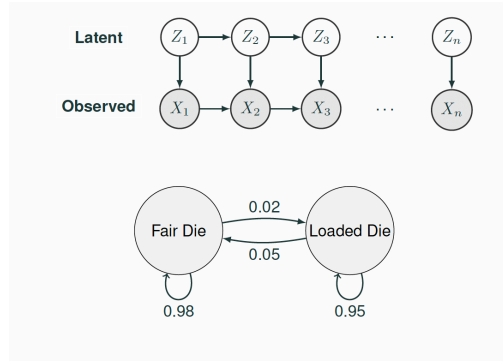
- X_i s are observed (face of a die)

- Z_t s are hidden (fair or biased)

where we assume that both Z and X are *discrete* random variables, taking m_z and m_x possible values, respectively.

So the HMM is parametrized by $\theta = (w, A, B)$ where * $w_{m_z \times 1}$: Distribution of Z_1 , an initial state * $A_{m_z \times m_z}$: the transition probability matrix from Z_t to Z_{t+1} * $B_{m_z \times m_x}$: the probability matrix (the emission distribution) for observing X_t under each hidden state Z_t .

The behavior of a HMM is fully determined by the three probabilities w , A , and B , and implicitly by m_z and m_x



For the *Dishonest Casino example*, we have

- $m_z = 2$ and $m_x = 6$.
- $A = \begin{bmatrix} 0.98 & 0.02 \\ 0.05 & 0.95 \end{bmatrix}$
- $B = \begin{bmatrix} \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} \\ \frac{1}{10} & \frac{1}{10} & \frac{1}{10} & \frac{1}{10} & \frac{1}{10} & \frac{1}{2} \end{bmatrix}$
- $w = (\frac{1}{2}, \frac{1}{2})$ equal probabilities if no strong prior.

We can calculate the probabilities of the observed data based on any given parameter value.

How should we model the data and detect the underlying states (which die was used)?

The underlying states $\{Z_t; t = 1, \dots, n\}$ are a **Markov chain**, that satisfies the following assumptions:

- Memoryless Assumption

$$p(Z_t | Z_{t-1}, \dots, Z_1) = p(Z_t | Z_{t-1})$$

- Stationarity Assumption

$$p(Z_t | Z_{t-1}) = p(Z_2 | Z_1) \quad \text{for } t = 2, \dots, n$$

The log-likelihood on the observed data is given by

$$\log(p(\mathbf{x}|\theta)) = \log\left(\sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}|\theta)\right) = \log\left(\sum_{\mathbf{z}} p(\mathbf{z}|\theta)p(\mathbf{x}|\mathbf{z}, \theta)\right)$$

which is very hard to optimize due to the summation inside the log (generally not convex). Note that here $\mathbf{x}$ and $\mathbf{z}$ are the observed vectors of the sequences $\mathbf{X}$ and $\mathbf{Z}$, respectively. The *Baum-Welch Algorithm* was developed to solve this problem. It uses the EM algorithm and the Forward-Backward algorithm.

13.5.1 EM Algorithm for HMM (discrete case)

E-step: Under the current value of θ , denoted as $\theta^{(k)}$, find $p(\mathbf{z}|\mathbf{x}, \theta^{(k)})$, the distribution of the unobserved variables given the observed data and $\theta^{(k)}$. Then calculate:

$$\begin{aligned} g(\theta) &= \mathbf{E}_{\mathbf{Z}|\mathbf{x}, \theta^{(k)}} \log p(\mathbf{x}, \mathbf{Z}|\theta) \\ &= \sum_{\mathbf{z}} P(\mathbf{Z} = \mathbf{z}|\mathbf{x}, \theta^{(k)}) \log p(\mathbf{x}, \mathbf{z}|\theta) \end{aligned}$$

M-step: Re-estimate the parameter θ to maximize $g(\theta)$:

$$\theta^{(k+1)} = \arg \max_{\theta} g(\theta)$$

How to calculate $p(\mathbf{z}|\mathbf{x}, \theta^{(k)})$ for our HMM problem? E-step

We first write out the log-likelihood of the **complete data**:

$$\log p(\mathbf{z}, \mathbf{x}|\theta) = \log\left(p(z_1) \prod_{t=1}^{n-1} p(z_{t+1}|z_t, \dots, z_1, \theta) \prod_{t=1}^n p(x_t|z_t, \dots, z_1, \theta)\right)$$

Using the memoryless and stationarity assumptions, this can be simplified to

$$\log p(\mathbf{z}, \mathbf{x}|\theta) = \log w(z_1) + \sum_{t=1}^{n-1} \log A(z_t, z_{t+1}) + \sum_{t=1}^n \log B(z_t, x_t)$$

Note that $\theta = (w, A, B)$. We then try to integrate it over all possible values of $\mathbf{Z}$, based on a current iteration value $\theta^{(k)}$:

$$E_{\mathbf{Z}|\mathbf{x}, \theta^{(k)}} \left(\log p(\mathbf{x}, \mathbf{Z}|\theta) \right)$$

To calculate this expectation, we need the conditional distribution of $(\mathbf{Z}|\mathbf{x}, \theta^{(k)})$ which is just the following:

$$\begin{aligned}\gamma_t(i, j) &= P(Z_t = i, Z_{t+1} = j | \mathbf{X}, \theta^{(k)}) \\ \gamma_t(i) &= P(Z_t = i | \mathbf{X}, \theta^{(k)})\end{aligned}$$

Suppose we already have the γ_t values, the E-step is

$$\begin{aligned}E_{\mathbf{Z}|\mathbf{x}, \theta^{(k)}}(\log p(\mathbf{x}, \mathbf{Z} | \theta)) &= E_{\mathbf{Z}|\mathbf{x}, \theta^{(k)}}\left(\log w(Z_1) + \sum_{t=1}^{n-1} \log A(Z_t, Z_{t+1}) + \sum_{t=1}^n \log B(Z_t, x_t)\right) \\ &= \sum_{i=1}^{m_z} \gamma_1(i) \log w(i) + \sum_{t=1}^{n-1} \sum_{i,j=1}^{m_z} \gamma_t(i, j) \log A(i, j) \\ &\quad + \sum_{t=1}^n \sum_{i=1}^{m_z} \gamma_t(i) \log B(i, x_t)\end{aligned}$$

If we can compute each $\gamma_t(i, j)$ and $\gamma_t(i)$, this step is done.

M-step

At the M-step, we update the parameters $\theta = (w, A, B)$

$$\begin{aligned}w^{(k+1)}(i) &= \gamma_1(i), \quad i = 1, \dots, m_z \\ A^{(k+1)}(i, j) &= \frac{\sum_{t=1}^{n-1} \gamma_t(i, j)}{\sum_{j'} \sum_{t=1}^{n-1} \gamma_t(i, j')}, \quad i, j = 1, \dots, m_z \\ B^{(k+1)}(i, l) &= \frac{\sum_{t: x_t=l} \gamma_t(i)}{\sum_{t=1}^n \gamma_t(i)}, \quad i = 1, \dots, m_z; \quad l = 1, \dots, m_x\end{aligned}$$

They are just the MLE estimators obtained by pooling and averaging a particular transition/emission event. For example, $B(i, \ell)$ is observing $X = \ell$ if the state is $Z = i$, so we go through all events with $Z = i$ in the entire chain, and average the events where $X = \ell$ is observed to get the probability.

13.5.2 Forward-Backward Algorithm for HMM

In both steps, we need to calculate the conditional probabilities:

$$\gamma_t(i, j) = P(Z_t = i, Z_{t+1} = j | \mathbf{x}, \theta^{(k)})$$

which is the conditional probability of transitioning from state i to state j at time point t given all the observed data x , and

$$\gamma_t(i) = P(Z_t = i | \mathbf{x}, \theta^{(k)})$$

the conditional probability of being at state i at time point t given all the observed data x .

We will use a **forward-backward algorithm** to calculate this.

With no risk of ambiguity, we omit $\theta^{(k)}$ from the notation, i.e., p is always $p_{\theta^{(k)}}$.

For $\gamma_t(i, j)$ by Bayes' theorem, we have:

$$\begin{aligned}
 \gamma_t(i, j) &= P(Z_t = i, Z_{t+1} = j | x) \\
 &\propto P(X_{1:t}, Z_t = i, Z_{t+1} = j, x_{t+1}, x_{(t+2):n}) \\
 &= \underbrace{P(X_{1:t}, Z_t = i)}_{\alpha_t(i)} \times \underbrace{P(Z_{t+1} = j | Z_t = i)}_{A(i, j)} \\
 &\quad \times \underbrace{P(x_{t+1} | Z_{t+1} = j)}_{B(j, x_{t+1})} \times \underbrace{P(x_{(t+2):n} | Z_{t+1} = j)}_{\beta_{t+1}(j)} \\
 &= \alpha_t(i) A(i, j) B(j, x_{t+1}) \beta_{t+1}(j)
 \end{aligned}$$

where $\alpha_t(i) = P(x_{1:t}, Z_t = i)$ is the forward probability of observing $x_{1:t}$ and having state i at time t and $\beta_{t+1}(j) = P(x_{(t+2):n} | Z_{t+1} = j)$ is the backward probability of observing $x_{(t+2):n}$ given state j at time t . Note that $\alpha_t(i)$ is a **joint** probability, and $A(i, j)$ is a **conditional** probability.

How to calculate $\alpha_t(i)$ and $\beta_{t+1}(j)$? We do this recursively starting from the two end points $t = 1$ and $t = n$.

Forward Probability

For the first time point $t = 1$:

$$\alpha_1(i) = P(x_1, Z_1 = i) = w(i)B(i, x_1)$$

For each t , we can then calculate the next time point $\alpha_{t+1}(i)$ using the information of $\alpha_t(i)$:

$$\begin{aligned}
 \alpha_{t+1}(i) &= P(x_1, \dots, x_{t+1}, Z_{t+1} = i) \\
 &= \sum_j P(X_1, \dots, X_{t+1}, Z_t = j, Z_{t+1} = i) \\
 &= \sum_j P(x_{1:t}, Z_t = j) p(Z_{t+1} = i | Z_t = j) P(x_{t+1} | Z_{t+1} = i) \\
 &= \sum_j \alpha_t(j) A(j, i) B(i, x_{t+1})
 \end{aligned}$$

Backward Probability

Define $\beta_t(i) = P(x_{(t+1):n} | Z_t = i)$. For the last time point $t = n$ there is no future observation, so we set

$$\beta_n(i) = 1.$$

Then we recurse backward. Using β_{t+1} we obtain β_t :

$$\begin{aligned} \beta_t(i) &= P(x_{(t+1):n} | Z_t = i) \\ &= \sum_j P(Z_{t+1} = j | Z_t = i) P(x_{t+1} | Z_{t+1} = j) P(x_{(t+2):n} | Z_{t+1} = j) \\ &= \sum_j A(i, j) B(j, x_{t+1}) \beta_{t+1}(j). \end{aligned}$$

Finally, the conditional probability $\gamma_t(i, j)$ needs to be normalized by the marginal probability to be a *proper* distribution:

$$\gamma_t(i, j) = \frac{\alpha_t(i) A(i, j) B(j, x_{t+1}) \beta_{t+1}(j)}{\sum_i \sum_j \alpha_t(i) A(i, j) B(j, x_{t+1}) \beta_{t+1}(j)}$$

Note that this is a joint probability, so we divide by the sum of all cases. This concludes the calculation of $\gamma_t(i, j)$

Similarly, we can calculate $\gamma_t(i)$ using Bayes' Theorem

$$\begin{aligned} \gamma_t(i) &= P(Z_t = i | \mathbf{x}) \\ &\propto P(Z_t = i, \mathbf{x}) \\ &= P(Z_t = i, \mathbf{x}_{1:t}) P(\mathbf{x}_{t+1:n} | Z_t = i, \mathbf{x}_{1:t}) \\ &= P(Z_t = i, \mathbf{x}_{1:t}) P(\mathbf{x}_{t+1:n} | Z_t = i) \\ &= \alpha_t(i) \beta_t(i) \end{aligned}$$

Hence, after normalization, this is a proper distribution

$$\gamma_t(i) = \frac{\alpha_t(i) \beta_t(i)}{\sum_i \alpha_t(i) \beta_t(i)}$$

This concludes all the components needed in the EM algorithm!

13.6 Inference on the Hidden States Z

There are several ways to find the **optimal** hidden state sequence Z given an observation sequence $\mathbf{x}$, depending on the definition of *optimality*.

One criterion is to choose the hidden states Z_t s that are **individually most likely at the time point t** , that is

$$\hat{Z}_t^* = \arg \max_i P(Z_t = i | \mathbf{x}, \hat{\theta}) = \arg \max_i \gamma_t(i)$$

Here we can plug in $\hat{\theta}$ from the aforementioned EM algorithm and simply get the argmax. Such a solution is optimal in the sense that it maximizes the expected number of correct states (by choosing the most likely state for each t). However, the resulting sequence may not be the most likely one and it may not even be a valid sequence.

For example, if Z has three states (instead of 2), and the transition probability $A(1, 3) = 0$ (impossible to transit from 1 to 3), but it is still possible to have

$$\hat{Z}_t^* = 1 \quad \text{and} \quad \hat{Z}_{t+1}^* = 3$$

An alternative approach is to find the most likely single sequence, i.e.,

$$\hat{\mathbf{Z}}^* = \arg \max_{z_{1:n}} P(\mathbf{Z} = \mathbf{z} | \mathbf{x}, \hat{\theta})$$

Note that maximizing this is the same as maximizing

$$P(\mathbf{Z} = \mathbf{z}, \mathbf{x} | \hat{\theta}) \text{ w.r.t. } z_{1:n}$$

Solving this with brute force will cost m_z^n tries, which is almost impossible. Hence, we need some sophisticated algorithm. A dynamic programming method called the Viterbi algorithm was proposed for this.

Define a maximizing sequence $z_{1:t}$ up to time t , which ends with $z_t = i$. The associated probability is

$$\mu_t(i) = \max_{z_{1:(t-1)}} P(\mathbf{Z}_{1:(t-1)} = \mathbf{z}_{1:(t-1)}, Z_t = i, \mathbf{x}_{1:t} | \hat{\theta})$$

which is the highest probability along a single path from 1 to t that ends up at state $Z_t = i$, given the observed sequence and the parameter estimation. Realizing that (with $\hat{\theta}$ omitted)

$$\begin{aligned} \mu_{t+1}(i) &= \max_{z_{1:t}} P(z_{1:t}, Z_{t+1} = i, x_{1:(t+1)} | \hat{\theta}) \\ &= \max_{z_{1:t}} \left\{ P(z_{1:(t-1)}, Z_t = z_t, \mathbf{x}_{1:t} | \hat{\theta}) P(Z_{t+1} | Z_t, \hat{\theta}) P(x_{t+1} | z_{t+1}, \hat{\theta}) \right\} \\ &= \left\{ \max_j \mu_t(j) A(j, i) \right\} B(i, x_{t+1}). \end{aligned}$$

To recover the sequence we also store the maximizing predecessor

$$\psi_{t+1}(i) = \arg \max_j \{ \mu_t(j) A(j, i) \}.$$

At time n set $\hat{Z}_n = \arg \max_i \mu_n(i)$, then trace backward $\hat{Z}_t = \psi_{t+1}(\hat{Z}_{t+1})$. This is the Viterbi path.